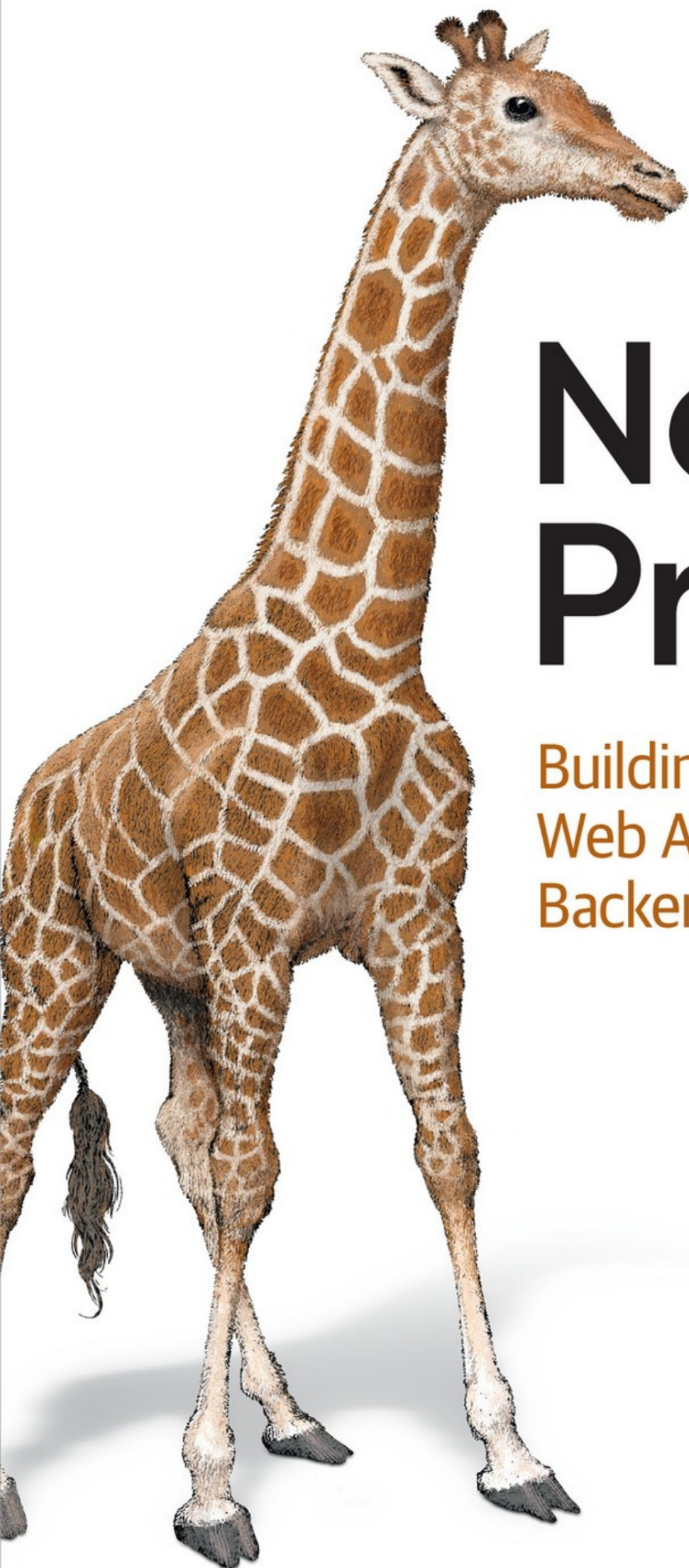


O'REILLY®



Node.js Projects

Building Real-World
Web Applications and
Backend APIs

Jonathan Wexler

Proyectos de Node.js

Creación de aplicaciones web y backend del mundo real
API

Jonathan Wexler

O'REILLY®

Proyectos de Node.js

por Jonathan Wexler

Copyright © 2025 Jon Wexler. Todos los derechos reservados.

Publicado por O'Reilly Media, Inc., 141 Stony Circle, Suite 195, Santa Rosa, CA 95401.

Los libros de O'Reilly se pueden comprar con fines educativos, comerciales o de ventas.

Uso promocional. También hay ediciones en línea disponibles para la mayoría de los títulos.

(<http://oreil.ly/com>). Para más información, contacte con nuestro

Departamento de ventas corporativas/institucionales: 800-998-9938 o corporativo@oreil.ly.com.

Editores de adquisiciones: Amanda Quinn y Aaron Black

Editor de desarrollo: Michele Cronin

Editor de producción: Gregory Hyman

Corrector de estilo: Dwight Ramsey

Correctora: Sonia Saruba

Indexador: Ellen Troutman-Zaig

Diseñador de portada: Susan Brown

Diseñador de interiores: David Futato

Ilustrador de la portada: Monica Kamsvaag

Ilustradora de interiores: Kate Dullea

Agosto de 2025: Primera edición

Historial de revisiones de la primera edición

- 31/07/2025: Primer lanzamiento

Consulte los detalles de la <https://www.oreil.ly/con/cataloger/ataisbncsp?isbn=9781098173142> para
versión de [httporeilly](https://www.oreil.ly).

El logotipo de O'Reilly es una marca registrada de O'Reilly Media, Inc.

Los proyectos de Nodejs, la imagen de portada y la imagen comercial relacionada son
marcas comerciales de O'Reilly Media, Inc.

Las opiniones expresadas en este trabajo son las del autor y no

representan las opiniones del editor. Si bien el editor y el autor

Han realizado esfuerzos de buena fe para garantizar que la información y

Las instrucciones contenidas en esta obra son exactas, el editor y el

El autor declina toda responsabilidad por errores u omisiones, incluidos

Sin limitación, responsabilidad por daños resultantes del uso de

o confianza en este trabajo. Uso de la información e instrucciones

contenido en este trabajo es bajo su propio riesgo. Si algún ejemplo de código o

Otra tecnología que esta obra contiene o describe está sujeta a la divulgación abierta.

licencias de origen o los derechos de propiedad intelectual de otros, es su

responsabilidad de garantizar que su uso de la misma cumpla con dichas

licencias y/o derechos.

978-1-098-17314-2

[LSI]

Prefacio

Cuando comencé a escribir este libro en 2022, vi cuánto había evolucionado la comunidad tecnológica desde "Aprende a programar con Node.js" (Manning). Si bien muchos conceptos fundamentales se mantuvieron, el enfoque de desarrollo había avanzado significativamente. Node.js había cumplido 10 años desde el lanzamiento inicial de Ryan Dahl, y internet estaba inundado de tutoriales sobre cómo crear "servidores web sencillos". Lo cierto es que la mayoría de los proyectos aún dependen de servidores web, pero las expectativas sobre cómo los construimos y usamos han cambiado. Hoy en día, un servidor no es solo un servidor; es la pieza central de aplicaciones complejas, resilientes y escalables. Asimismo, la forma en que usamos JavaScript se ha profundizado, abriendo un amplio abanico de herramientas y técnicas para explorar.

Proyectos Node.js está diseñado para mostrar estas nuevas técnicas y ayudarte a crecer como desarrollador, centrándote en cinco principios clave: un enfoque de aprendizaje práctico, aprendizaje modular, diversos casos de uso, desarrollo progresivo de habilidades y retroalimentación y gratificación inmediatas. Cada proyecto está diseñado para brindar experiencia práctica y práctica, permitiéndote aplicar Node.js en pasos modulares y fáciles de entender. Este enfoque no solo promueve el desarrollo gradual de habilidades, sino que también garantiza que cada capítulo te brinde una gratificante sensación de logro, reforzando tu crecimiento a medida que progresas en aplicaciones variadas y cada vez más desafiantes.

Los principiantes en programación comprenden rápidamente cómo ensamblar una aplicación, pero crear una aplicación va más allá del código: se trata de comprender la arquitectura y el diseño. Cada capítulo te pone en el papel de un ingeniero que toma decisiones reales. Creo que la experiencia práctica no consiste en copiar y pegar código, sino en desarrollar la mentalidad, las habilidades de resolución de problemas y la comunicación necesarias para crear productos impactantes.

El alcance de los desafíos del software varía: algunos proyectos se desarrollan durante meses, mientras que otros implican una resolución de problemas rápida y específica. Este libro ofrece un equilibrio, ofreciendo tanto ejercicios de programación breves como proyectos más grandes y complejos. Su estructura modular te anima a completar las secciones a tu propio ritmo, permitiéndote hacer pausas, cambiar de enfoque o profundizar en capítulos que se ajusten a tus intereses y nivel de habilidad.

JavaScript, y por extensión Node.js, ha experimentado un crecimiento increíble en los últimos 15 años. Esta evolución ha convertido a Node.js en una herramienta indispensable para crear desde aplicaciones web de alto rendimiento hasta sistemas de notificación en tiempo real y plataformas de streaming de vídeo. Aprender Node.js hoy en día significa comprender una amplia gama de posibles aplicaciones. Cada capítulo te ofrece no solo habilidades técnicas, sino también una mentalidad para abordar situaciones reales que probablemente encontrarás en el trabajo. De esta manera, desarrollarás la versatilidad para aplicar Node.js a diversos proyectos, mejorando tanto tu confianza como tu adaptabilidad.

Con tantos recursos gratuitos en línea, quizás te preguntes por qué elegir un libro sobre Node.js. Y en un mundo donde las herramientas de IA asumen cada vez más tareas de desarrollo, ¿por qué un ingeniero debería dedicar tiempo a un aprendizaje profundo y estructurado? Es cierto que aprender Node.js es más accesible que nunca, pero esa accesibilidad conlleva un desafío: encontrar recursos que simulen el proceso de pensamiento y la estructura del desarrollo real. Node.js Projects desarrolla habilidades de forma gradual, guiándote hacia logros tangibles con cada proyecto. Esta ruta estructurada está diseñada para ayudarte a ganar confianza y habilidades con cada capítulo.

En definitiva, no quería escribir un libro más sobre Node.js. Esta es una recopilación de las lecciones y técnicas más valiosas que he aprendido a medida que la industria ha evolucionado. Mi objetivo es ayudarte a lograr un crecimiento a largo plazo mediante un progreso gradual y satisfactorio.

Cada capítulo presenta conceptos que van más allá de Node.js y reflejan los estándares de la comunidad de desarrolladores actual. Ya sea que se trate de un

Si te toma una semana o un año completar estos proyectos, estoy seguro de que este viaje te convertirá en un ingeniero más fuerte y versátil.

Convenciones utilizadas en este libro

En este libro se utilizan las siguientes convenciones tipográficas:

Itálico

Indica nuevos términos, URL, direcciones de correo electrónico, nombres de archivos y extensiones de archivos.

Ancho constante

Se utiliza para listados de programas, así como dentro de párrafos para hacer referencia a elementos del programa como nombres de variables o funciones, bases de datos, tipos de datos, variables de entorno, declaraciones y palabras clave.

Cursiva de ancho constante

Muestra texto que debe reemplazarse con valores proporcionados por el usuario o por valores determinados por el contexto.

CONSEJO

Este elemento significa un consejo o sugerencia.

NOTA

Este elemento significa una nota general.

Uso de ejemplos de código

Hay material complementario disponible (ejemplos de código, ejercicios, etc.) para descargar en <https://oreil.ly/nodeprojects-code>.

Si tiene una pregunta técnica o un problema al utilizar el código ejemplos, envíe un correo electrónico a support@oreilly.com.

Este libro está aquí para ayudarte a realizar tu trabajo. En general, por ejemplo, Con este libro se ofrece código, puedes usarlo en tus programas y documentación. No necesita contactarnos para obtener permiso a menos que Estás reproduciendo una parte importante del código. Por ejemplo, Escribir un programa que utilice varios fragmentos de código de este libro. No requiere permiso. Vender o distribuir ejemplares de Los libros de O'Reilly requieren permiso. Responder a una pregunta de Citar este libro y citar el código de ejemplo no requiere permiso. Incorporando una cantidad significativa de código de ejemplo de Para incluir este libro en la documentación de su producto se requiere permiso.

Agradecemos, pero generalmente no exigimos, la atribución. Una atribución Generalmente incluye el título, el autor, la editorial y el ISBN. Por ejemplo: Proyectos Node.js de Jonathan Wexler (O'Reilly). Copyright 2025 Jon Wexler, 978-1-098-17314-2."

Si considera que su uso de ejemplos de código está fuera del uso legítimo o de la Si ha dado el permiso anterior, no dude en ponerse en contacto con nosotros en permisos@oreilly.com.

Aprendizaje en línea de O'Reilly

NOTA

Durante más de 40 años, **O'Reilly Media** Ha proporcionado tecnología y Capacitación empresarial, conocimiento y visión para ayudar a las empresas a tener éxito.

Nuestra red única de expertos e innovadores comparten sus conocimientos y experiencia a través de libros, artículos y nuestro aprendizaje en línea.

Plataforma. La plataforma de aprendizaje en línea de O'Reilly le ofrece acceso a demanda. acceso a cursos de formación en vivo, rutas de aprendizaje en profundidad, interactivos entornos de codificación y una vasta colección de texto y vídeo de O'Reilly y más de 200 editoriales. Para más información, visite <https://oreilly.com>.

Cómo contactarnos

Por favor, dirija sus comentarios y preguntas sobre este libro a la editor:

O'Reilly Media, Inc.

141 Stony Circle, Suite 195

Santa Rosa, CA 95401

800-889-8969 (en Estados Unidos o Canadá)

707-827-7019 (internacional o local)

707-829-0104 (fax)

soporte@oreilly.com

<https://oreilly.com/acerca/decontacto.html>

Tenemos una página web para este libro, donde enumeramos erratas, ejemplos, y cualquier información adicional. Puede acceder a esta página en

Proyectos de código de O'Reilly <https://oreillynode.org.uk/>

Para noticias e información sobre nuestros libros y cursos, visita <https://oreilly.com>.

Encuéntrenos en LinkedIn: <https://linkedinoreillymedia.com> /

Míranos en YouTube: <https://YouTube.com/oreillymedia>.

Agradecimientos Escribir

proyectos Node.js ha sido un viaje de largas noches, sesiones de depuración profunda y experimentos en el mundo real, y no habría sido posible sin el apoyo y el conocimiento de muchas personas.

Ante todo, gracias al equipo de O'Reilly por impulsar este libro desde su concepción hasta su lanzamiento. Su atenta guía, su experiencia editorial y su compromiso con la calidad contribuyeron a convertirlo en algo verdaderamente útil para desarrolladores de todos los niveles.

A los revisores técnicos y primeros lectores: su atención al detalle, sus generosos comentarios y su disposición a cuestionar las suposiciones hicieron que este libro fuera más preciso y preciso. Gracias por ayudarme a detectar errores, ejemplos poco claros y complejidades innecesarias antes de que se publicara.

A la comunidad de Node.js en general: sus contribuciones de código abierto, publicaciones de blog, discusiones en GitHub e hilos de Stack Overflow me brindaron inspiración, soluciones y, a veces, la seguridad de que no era el único con un problema. Este libro se basa en el ecosistema que han creado.

A los estudiantes, colegas y desarrolladores con los que he trabajado a lo largo de los años: muchas de las ideas y patrones de este libro surgieron de nuestros proyectos y conversaciones compartidos. Gracias por la colaboración y la curiosidad.

A mi familia y amigos: su aliento y paciencia durante los largos sprints de escritura me mantuvieron en marcha. Estoy profundamente agradecido por su apoyo.

Finalmente, a ti, lector: gracias por elegir este libro. Espero que te ayude a construir, romper, corregir y crear con confianza con Node.js.

Capítulo 1. Introducción y configuración

Este capítulo es tu guía para configurar un entorno de desarrollo para Node.js. Abarca las herramientas principales necesarias, como VS Code, Node.js y Fastify, junto con componentes opcionales para optimizar tu flujo de trabajo. Aunque herramientas como Fastify son más especializadas, puedes omitir su configuración por ahora y revisar esas secciones cuando se aborden en capítulos posteriores.

Encontrará varios métodos de instalación explicados, incluido el uso de una interfaz gráfica de usuario (GUI), herramientas de terceros o paquetes binarios. Las instalaciones binarias, aunque a veces están precompiladas, pueden requerir pasos adicionales de extracción y configuración, lo que las hace especialmente útiles para servidores sin interfaz gráfica. Aunque las instrucciones proporcionadas se adaptan a las herramientas recomendadas para este libro, le invitamos a utilizar alternativas que se ajusten a sus necesidades de desarrollo. El objetivo es proporcionarle una base sólida para empezar a codificar y ejecutar sus aplicaciones Node.js eficazmente.

NOTA

Por razones de economía, a lo largo del resto de este libro nos referiremos a Node.js simplemente como Node.

Antes de escribir su primera línea de código, es importante elegir un editor de texto que admita el desarrollo de JavaScript moderno y se integre bien con las herramientas de Node.

Instalación de VS Code

Visual Studio Code (VS Code) es un popular editor de texto de código abierto, Ampliamente recomendado para el desarrollo de Node debido a su robustez. Características, flexibilidad y amplias opciones de personalización. Esta sección lo guiará a través de la instalación de VS Code en macOS, Windows y Linux.

NOTA

Los editores de código impulsados por IA son cada vez más populares y uno Esta herramienta es **Cursor**, Un editor mejorado con IA creado en VS Code que puede Ayudar con sugerencias y explicaciones sobre el código. Aunque es útil, para principiantes Debería equilibrar su uso con el aprendizaje independiente de los conceptos básicos.

La forma más fácil de instalar VS Code es visitando el [sitio web de VS Code](#) [Página de descarga](#). La **Figura 1-1** muestra las opciones de instalación para cada Sistema operativo.

Download Visual Studio Code

Free and built on open source. Integrated Git, debugging and extensions.



↓ Windows

Windows 10, 11

User Installer

x64 Arm64

System

x64 Arm64

Installer

x64 Arm64

.zip

x64 Arm64

CLI

x64 Arm64



↓ .deb

Debian, Ubuntu

↓ .rpm

Red Hat, Fedora, SUSE

.deb

x64 Arm32 Arm64

.rpm

x64 Arm32 Arm64

.tar.gz

x64 Arm32 Arm64

Snap

Snap Store

CLI

x64 Arm32 Arm64



↓ Mac

macOS 10.15+

.zip

Intel chip Apple silicon Universal

CLI

Intel chip Apple silicon

Cifra 11. Página de instalación de VS Code

La imagen ofrece opciones para instalar VS Code en sistemas Mac, Windows y Linux. Las siguientes secciones explican cómo proceder en cada sistema operativo.

Instalación en Mac

Asegúrate de identificar el tipo de Mac que tienes (Intel o Apple Silicon).

1. Descargue el instalador apropiado para su Mac.
2. Abra el archivo descargado y arrastre el ícono de VS Code a la carpeta Aplicaciones .
3. Abra VS Code desde la carpeta Aplicaciones .

CONSEJO

Es posible que necesite permitir que la aplicación se ejecute en Preferencias del Sistema en Seguridad y privacidad si encuentra algún problema.

Instalación de Windows

Para instalar VS Code en una máquina Windows, siga estos pasos:

1. Descargue el instalador para Windows.
2. Abra el archivo descargado y siga el asistente de instalación. pasos.
3. Una vez instalado, puedes iniciar VS Code desde el menú Inicio.

NOTA

Durante la instalación, asegúrese de seleccionar la opción para agregar VS Code a su PATH para facilitar el acceso a la línea de comandos.

Instalación de Linux

La instalación en equipos Linux es un poco diferente, ya que requiere pasos desde la línea de comandos. Para la configuración estándar de un equipo Debian/Ubuntu, ejecute los siguientes comandos en la ventana de la línea de comandos:

```
sudo apt update sudo ❶  
apt install software-properties-common apt-transport-https wget wget -q https://  
❷  
packages.microsoft.com/keys/microsoft.asc -O- | \ sudo apt-key add - sudo add-apt-repository  
\"deb  
[arch=amd64] https:// ❸  
packages.microsoft.com/repos/vscode  
stable main" sudo apt  
update sudo apt install code  
❹ ❺ ❻
```

- ❶** Actualiza las listas de paquetes para garantizar que obtenga la última versión y dependencias.
- ❷** Instala las herramientas necesarias para agregar repositorios y manejar HTTPS.
- ❸** Descarga y agrega la clave GPG de Microsoft para verificar paquetes.
- ❹** Agrega el repositorio oficial de Microsoft VS Code.
- ❺** Actualiza nuevamente las listas de paquetes para incluir el nuevo repositorio de VS Code.
- ❻** Instala VS Code.

CONSEJO

Para otras distribuciones, consulte la [documentación oficial](#).

Para mejorar su experiencia de desarrollo, considere instalar extensiones de VS Code como Prettier para formatear, ESLint para detectar errores de manera temprana y npm Intellisense para autocompletar inteligente al importar paquetes.

Con su editor listo para usar, tomémonos un momento para comprender qué es Node y por qué es la tecnología central detrás de todo lo que construirá en este libro.

Entendiendo Node. Node es una

herramienta de código abierto, lo que significa que cualquiera puede ver, modificar y compartir su código gratuitamente. Además, es multiplataforma, por lo que funciona en diferentes sistemas operativos como Windows, macOS y Linux. Node es un entorno de ejecución, lo que significa que proporciona un entorno para ejecutar código JavaScript fuera del navegador; normalmente, JavaScript solo se usa dentro de los navegadores web para funciones como botones interactivos o animaciones. Con Node, los desarrolladores pueden usar JavaScript para crear aplicaciones del lado del servidor, como sitios web o API.

Node se basa en el motor JavaScript V8 de Chrome, una tecnología que convierte JavaScript en código máquina, el lenguaje que tu ordenador entiende. Este proceso permite que JavaScript se ejecute con mayor rapidez y eficiencia. Gracias a esta velocidad, Node se describe como ligero (no requiere muchos recursos) y eficiente (resulta muy productivo con mínima sobrecarga). Es especialmente ideal para crear aplicaciones escalables, es decir, programas que pueden gestionar más usuarios o datos sin ralentizarse.

Una de las características destacadas de Node es su arquitectura sin bloqueos y basada en eventos. En pocas palabras, no espera a que una tarea termine para comenzar otra. Imagina que doblas la ropa y hierves agua al mismo tiempo. En lugar de esperar junto a la estufa, doblas la ropa hasta que el agua hierva. De igual forma, Node alterna entre tareas, maximizando la productividad. Por eso Node es ideal para el tiempo real.

aplicaciones como aplicaciones de chat, juegos en línea o herramientas donde los usuarios colaboran en vivo: todo sigue respondiendo, incluso con muchos usuarios.

Por qué Node se destaca

Node se ha convertido en una opción popular para los desarrolladores debido a sus características y ventajas únicas:

Desarrollo unificado

Utiliza JavaScript tanto en el lado del cliente como del servidor, lo que elimina la necesidad de múltiples lenguajes de programación en su pila y simplifica los flujos de trabajo de desarrollo.

vasto ecosistema

Incluye npm, un repositorio de paquetes masivo con millones de herramientas prediseñadas, lo que hace que sea más rápido y fácil crear aplicaciones.

Adopción de la industria

Empresas líderes como Netflix, LinkedIn y PayPal confían en nosotros por su capacidad para gestionar alto tráfico y demandas en tiempo real.

Escalabilidad y eficiencia

Diseñado para manejar miles de conexiones simultáneas sin disminuir la velocidad, gracias a su arquitectura sin bloqueos y basada en eventos.

Operaciones fluidas

Node garantiza un flujo de solicitudes eficiente, incluso con cargas de trabajo elevadas. Imagine una autopista sin atascos.

¿Qué está pasando bajo el capó?

En el fondo, Node utiliza el motor V8, que agiliza JavaScript convirtiéndolo directamente en código máquina. La magia de gestionar miles de tareas simultáneas proviene del bucle de eventos de Node y la biblioteca Libuv. El bucle de eventos es como una recepcionista que gestiona varios clientes a la vez: mientras uno espera una respuesta (p. ej., la obtención de datos), el recepcionista se desplaza para ayudar a los demás. Libuv es como el equipo que trabaja entre bastidores y se encarga de las tareas que consumen mucho tiempo, como leer archivos de un disco o resolver nombres de dominio (búsquedas de DNS). Mientras Node se centra en cambiar rápidamente entre tareas, Libuv garantiza que estas operaciones más lentas se realicen en segundo plano sin bloquear el hilo principal. Piénselo como un chef en una cocina ajetreada: mientras el jefe de cocina sigue preparando platos rápidos, Libuv es el equipo que trabaja en la cocción lenta del guiso. Esto permite a Node seguir avanzando a la siguiente tarea sin atascarse esperando a que finalice una operación.

Instalando Node. Estás a

punto de instalar Node, una plataforma potente y versátil que te permite ejecutar código JavaScript en tu ordenador, sin necesidad de un navegador web. Con Node, puedes crear desde sitios web sencillos hasta aplicaciones complejas del lado del servidor, automatizar tareas y mucho más.

Al comenzar, tenga en cuenta que instalar Node es un proceso sencillo, pero hay algunas cosas que debe tener en cuenta.

Dependiendo de tu sistema operativo, podrías necesitar ajustar algunas configuraciones o instalar herramientas adicionales, como administradores de versiones o elementos esenciales de compilación. Sigue cada paso a continuación, incluyendo consejos para solucionar problemas comunes.

Para todas las instalaciones, visite la [página de descarga de Node](#), donde podrás elegir la instalación del Nodo para la configuración de tu máquina ([Figura 1-2](#)).

NOTA

nvm (Node Version Manager) le permite instalar y cambiar fácilmente entre múltiples versiones de Node, lo cual es ideal para probar proyectos en entornos. Obtenga más información visitando el [repositorio de GitHub del proyecto](#).

Download Node.js®

Download Node.js the way you want.

Package Manager Prebuilt Installer Prebuilt Binaries Source Code

Install Node.js v20.16.0 (LTS) on macOS using nvm

```
1 # installs nvm (Node Version Manager)
2 curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
3
4 # download and install Node.js (you may need to restart the terminal)
5 nvm install 20
6
7 # verifies the right Node.js version is in the environment
8 node -v # should print 'v20.16.0'
9
10 # verifies the right npm version is in the environment
11 npm -v # should print '10.8.1'
```

Bash

 Copy to clipboard

Cifra 12. Página de instalación del nodo

CONSEJO

Asegúrese de seleccionar la opción para instalar npm (Node Package Manager) Durante la instalación.

Instalación en Mac

Asegúrate de identificar el tipo de Mac que tienes (Intel o Apple Silicon).

Luego:

1. Descargue el instalador apropiado para su Mac. A partir de la
Al escribir este libro, debe elegir la versión de Node 22.16.0 o posterior.
2. Abra el archivo descargado y siga las instrucciones en pantalla.
Instrucciones para instalar Node.

NOTA

También puedes usar Homebrew para la instalación ejecutando `brew install node` en una nueva ventana de terminal.

Instalación de Windows

Asegúrese de usar un sistema Windows con privilegios administrativos para instalar el software. Luego:

1. Descargue el instalador para Windows. Al momento de escribir este libro, debería elegir la versión 22.16.0 o posterior de Node.
2. Abra el archivo descargado y siga las instrucciones en pantalla.
Instrucciones para instalar Node.

Instalación de Linux

La instalación en equipos Linux requiere pasos desde la línea de comandos. Para la configuración estándar de un equipo Debian/Ubuntu, ejecute los siguientes comandos en la ventana de la línea de comandos:

```
sudo apt update  
sudo apt install nodejs npm
```

❶

❷

- ❶ Actualiza las listas de paquetes para garantizar que obtenga la última versión de los paquetes.
- ❷ Instala Node y npm (Administrador de paquetes de Node).

NOTA

Para otras distribuciones, consulte la [documentación oficial](#).

Convertirse en desarrollador de Node. Para empezar,

necesitarás comprender los módulos principales (fs para el manejo de archivos, http para servidores y eventos para la lógica personalizada). Una habilidad fundamental es la programación asíncrona, que te permite realizar tareas en segundo plano sin bloquear la aplicación. Imagina que pides pizza mientras haces la tarea: las promesas y async/await son las actualizaciones de entrega que te avisan cuando llega la pizza, para que no tengas que dejar de trabajar.

También querrás aprender a crear API con frameworks como Express.js o Fastify, donde defines "rutas" como rutas en un mapa para determinar cómo responde tu aplicación a los usuarios. A partir de ahí, el siguiente paso es trabajar con bases de datos como MongoDB o PostgreSQL, lo que te permite almacenar y recuperar datos de forma eficiente.

Dominando el oficio. Para convertirte

en un experto, concéntrate en proyectos reales. Por ejemplo, crea una aplicación de chat para practicar la comunicación en tiempo real o un gestor de tareas con persistencia de datos para perfeccionar tus habilidades con bases de datos. Profundiza en la optimización del rendimiento mediante la creación de perfiles de tu aplicación, como identificar consultas lentas a la base de datos o un uso excesivo de memoria. Imagina ajustar...

Motor de automóvil para un rendimiento máximo donde cada pequeña mejora cuenta.

Manténgase conectado con la comunidad de Node a través de foros, contribuciones de código abierto y boletines informativos como Node Weekly. La clave para alcanzar la maestría reside en el aprendizaje continuo, la experimentación y la interacción con otros que comparten su pasión por el desarrollo.

Al comprender Node desde sus fundamentos hasta sus capacidades avanzadas, no solo obtendrá una herramienta de desarrollo, sino también un conjunto de habilidades versátiles muy valoradas en la industria. Para inspirar su experiencia, exploremos algunos conceptos avanzados que hacen de Node algo único y potente.

CONCEPTOS DE NODO AVANZADOS

Antes de profundizar en cómo Node maneja las tareas detrás de escena, es esencial comprender el mecanismo central de Node para administrar operaciones asincrónicas.

Bucle de

eventos. El bucle de eventos en Node es como un gerente de fábrica que divide las tareas en fases para garantizar que todo funcione de forma fluida y eficiente. Cada fase gestiona un tipo específico de trabajo:

Temporizadores

Para tareas programadas (por ejemplo, alarmas)

I/O Operaciones O

Para tareas de entrada/salida (por ejemplo, leer archivos)

Devoluciones de llamadas

Para respuestas que necesitan atención inmediata

El bucle de eventos procesa estas fases en orden, garantizando que las tareas se completen sin que una bloquee a otra.

Imagínate que diriges un restaurante:

- El chef prepara la comida (temporizadores).
- El servidor lo entrega (operaciones de E/S).
- El cajero procesa los pagos (devoluciones de llamadas).

Todo esto sucede sin problemas, manteniendo el flujo de trabajo fluido.

Flujos y buffers

Los flujos y los buffers en Node administran datos de manera eficiente en fragmentos, lo que facilita el manejo de archivos grandes o flujos de datos continuos:

Arroyos

Como tuberías de agua, entregando pequeñas cantidades manejables de datos a lo largo del tiempo

Buffers

Almacenar temporalmente los datos hasta que estén listos para usarse

Ver una película en línea utiliza streaming. El video se reproduce mientras se descarga. Si tuvieras que descargar el video completo antes de verlo, tardaría mucho más. Los streamings permiten gestionar tareas como la transmisión de video o la subida de archivos en tiempo real de forma más rápida y eficiente.

Escalabilidad con agrupamiento en clústeres

Node puede manejar cargas de trabajo más grandes mediante el uso de clústeres, que generan múltiples instancias de la aplicación para compartir la carga:

- Cada instancia se ejecuta independientemente pero comparte la misma lógica de aplicación.
- Un equilibrador de carga distribuye las solicitudes entrantes entre estas instancias.

Piense en la agrupación como si estuviera abriendo múltiples cajas registradoras en un supermercado concurrido:

- Cada contador (instancia de nodo) atiende a los clientes (solicitudes) de forma independiente.
- El administrador (balanceador de carga) dirige a cada cliente al siguiente mostrador disponible.

Esta configuración garantiza que se pueda atender a más usuarios simultáneamente, optimizando el rendimiento.

Para profundizar en la configuración de Node.js, la inicialización de proyectos y los patrones de sintaxis modernos, consulte el Apéndice A. Para obtener herramientas y prácticas adicionales que enriquecen su entorno de desarrollo, como formateadores, depuradores y el uso de Git, consulte el Apéndice B. El Apéndice C proporciona contexto y una guía paso a paso para trabajar con bases de datos y sistemas de colas, que se utilizan en capítulos posteriores. El Apéndice D explica cómo configurar y ejecutar sus proyectos con Docker para un entorno de desarrollo consistente. El Apéndice E explica cómo crear y configurar cuentas de terceros, como proveedores de nube y plataformas API, necesarias para algunos de los proyectos de este libro.

A continuación, se presenta una descripción general para ayudarle a comprender qué es Node y su importancia. Node se ha convertido en un componente fundamental del desarrollo web moderno gracias a su velocidad, escalabilidad y ecosistema próspero. Esto convierte a Node en la base ideal para explorar potentes frameworks web como Fastify.

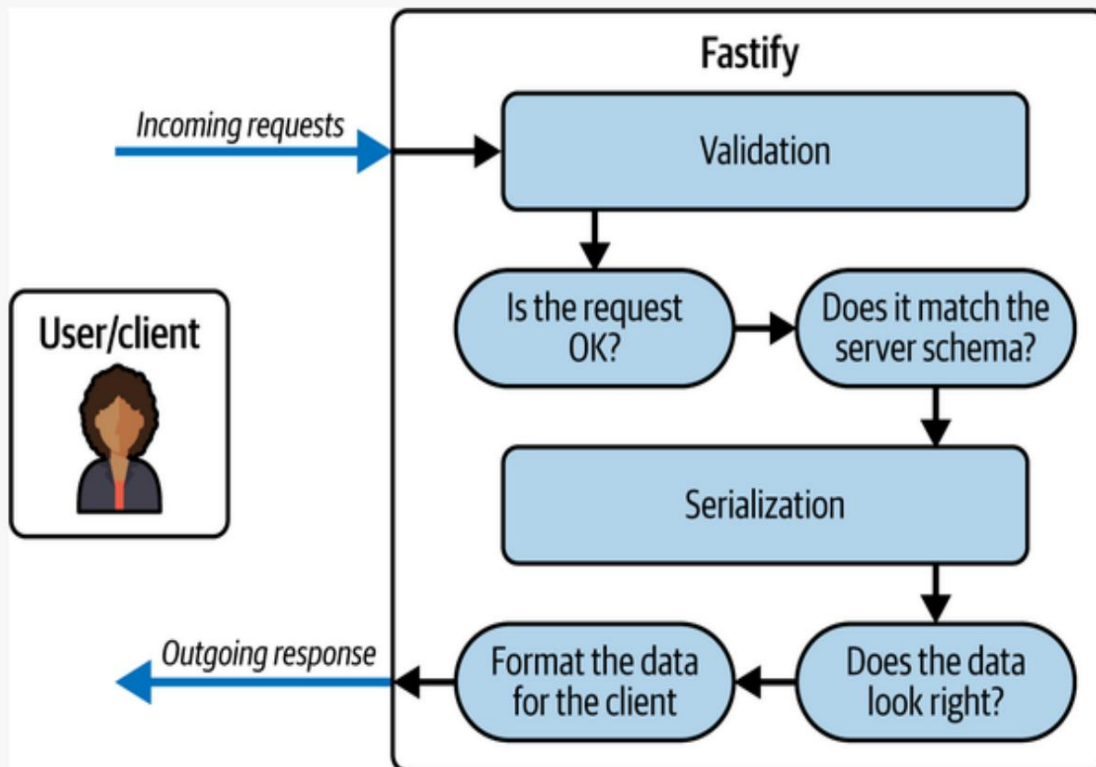
Uso de Fastify en este libro. Fastify es un

framework web Node de alto rendimiento y bajo consumo, diseñado para ser extremadamente rápido y ligero. Esto significa que puede realizar una gran cantidad de trabajo sin ralentizar ni consumir demasiados recursos del equipo.

Está optimizado para crear API y servicios web que requieren tiempos de respuesta rápidos, lo que lo hace ideal tanto para proyectos pequeños como para aplicaciones a gran escala. Fastify destaca en el procesamiento de solicitudes HTTP y la entrega de respuestas con una latencia mínima, lo cual es crucial para aplicaciones web donde los tiempos de respuesta rápidos mejoran la experiencia del usuario y la eficiencia del sistema. Además, Fastify puede gestionar un gran número de solicitudes por segundo (RPS), lo que lo convierte en una excelente opción para aplicaciones que necesitan atender a muchos usuarios simultáneamente, como API o microservicios.

BAJO EL CAPÓ

Fastify logra su alto rendimiento mediante una combinación de principios de diseño eficientes y decisiones arquitectónicas inteligentes (Figura 1-3). Utiliza el módulo HTTP de bajo nivel de Node, un módulo integrado. Característica escrita en C++ que maneja solicitudes entrantes y respuestas salientes de manera eficiente, sin necesidad de bibliotecas adicionales. Esto mantiene el núcleo de Fastify ligero y rápido. Fastify también emplea validación y serialización basadas en esquemas utilizando AJV, un Potente validador de esquemas JSON.



Cifra 13. Diagrama de la arquitectura de bajo consumo de Fastify

Esto significa que cuando se envían datos a su aplicación, Fastify garantiza que coincida con el formato esperado (validación) y cuando los datos se envían de vuelta al usuario y formatea la salida de manera eficiente (serialización). Precompilar estos procesos con antelación. Reduce aún más la carga de trabajo durante cada solicitud.

La arquitectura asíncrona y sin bloqueos de Fastify es otra clave para su escalabilidad. Node opera mediante un bucle de eventos, lo que permite que múltiples tareas (como gestionar solicitudes HTTP, consultar una base de datos o leer archivos) se ejecuten simultáneamente sin crear múltiples subprocesos. Esto significa que incluso si una operación es lenta, como esperar la respuesta de una base de datos, otras solicitudes pueden continuar procesándose sin demora. Fastify se basa en este comportamiento básico de Node para gestionar múltiples solicitudes simultáneas con facilidad.

Otra ventaja de Fastify es su sistema modular de complementos. Estos complementos son funciones reutilizables (como la autenticación o el registro) que se pueden añadir a la aplicación. El sistema de complementos de Fastify garantiza que estas funciones estén aisladas y no interfieran entre sí. Por ejemplo, si se añade una nueva función a una ruta, no afectará a otra involuntariamente. Esto hace que Fastify no solo sea escalable, sino también más fácil de mantener a medida que la aplicación crece.

Fastify también incluye Pino, una biblioteca de registro de alto rendimiento que registra eventos importantes de la aplicación sin sobrecargarla. Los registros son esenciales para la depuración y la monitorización, y el diseño optimizado de Pino garantiza que no ralenticen la aplicación. Para más información sobre Pino, consulte su [sitio web](#).

Este libro utiliza Fastify en lugar de Express.js debido a estas ventajas de rendimiento y sus funciones intuitivas para desarrolladores. Si bien Express es versátil y ampliamente utilizado, Fastify ofrece compatibilidad integrada con API basadas en JSON, un enfoque en la gestión de solicitudes de alta velocidad y optimizaciones que lo hacen especialmente adecuado para la creación de servicios web modernos. Con Fastify, adquirirá experiencia práctica con patrones de diseño eficientes, ideales para aplicaciones de alto rendimiento, especialmente aquellas que necesitan escalar eficazmente a medida que crecen.

La Tabla 1-1 compara Fastify con otros tres frameworks de Node populares: Express, Koa y Hapi. Cada columna ofrece una descripción rápida.

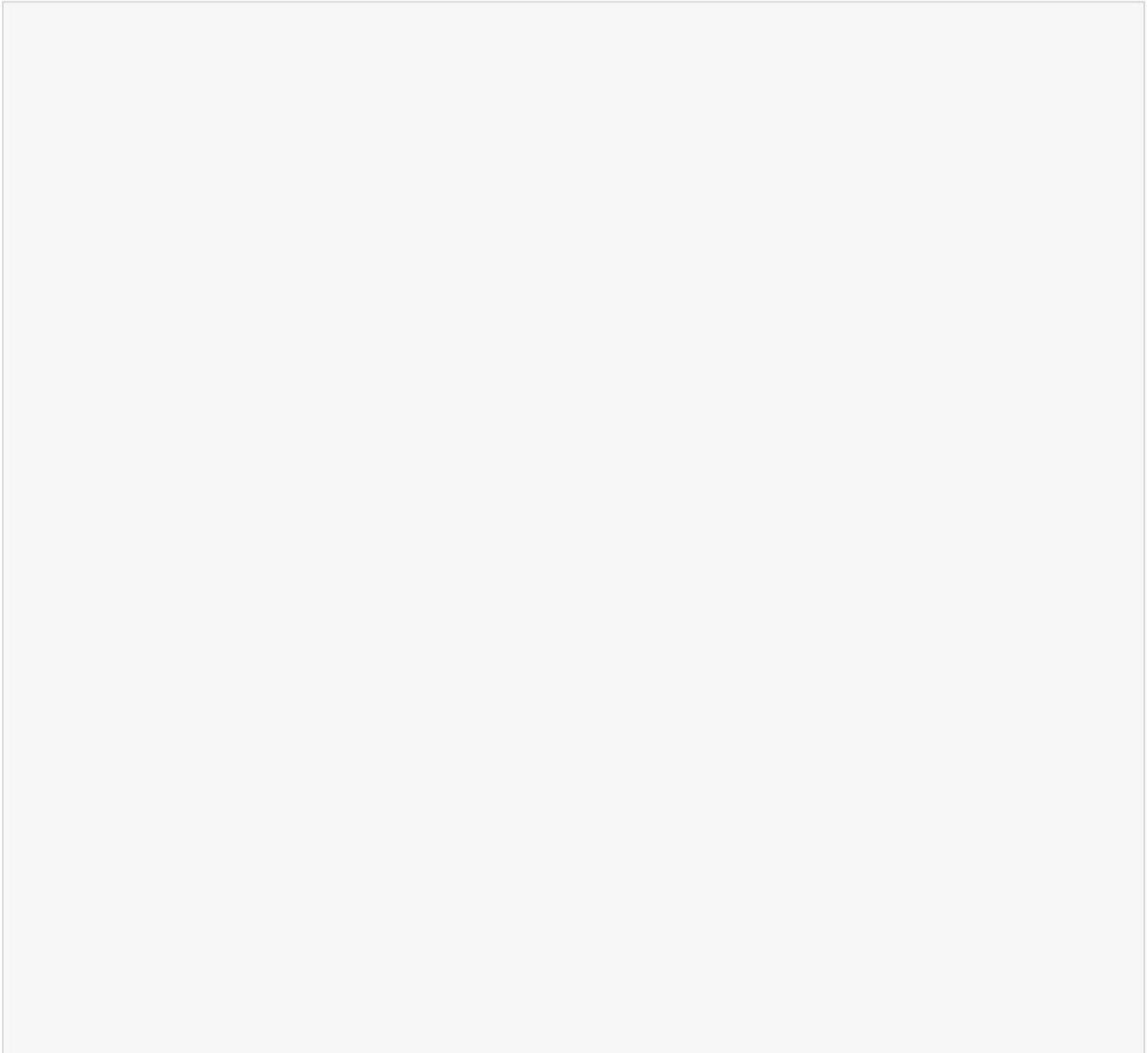
descripción general de las principales fortalezas y diferencias de cada marco, que puede utilizar para ayudar a decidir cuál es el más adecuado para su proyecto.

Mesa 11. Marcos de nodos

Rendimiento del marco		Basado en esquemas validación	Incorporado complementos	Idea caso
Fastify	Alto	Sí (JSON Esquema)	Limitado; ligero	Alto rendimiento API micr
Expresar	Medio	No (requiere adicional bibliotecas)	Extenso middleware ecosistema	General púrpura aplicación
Koa	Medio	No (requiere middleware)	Mínimo, modular	Doblar custo aplicación
Feliz	Medio-Alto Sí (Joi)		Complemento enriquecido ecosistema	Entrar leve aplicación API con necesidad

En resumen, Fastify destaca por su rendimiento y es ideal para API y microservicios de alta velocidad gracias a su validación basada en esquemas y su mínima sobrecarga. Express sigue siendo una opción versátil con un amplio ecosistema de complementos, ideal para aplicaciones web de propósito general.

A lo largo de este libro, Fastify servirá como marco fundamental para tus aplicaciones Node, ayudándote a crear servicios escalables, responsivos y de alto rendimiento. A medida que avances en los capítulos, aprenderás a aprovechar las ventajas de Fastify y a comprender mejor cómo crear aplicaciones web modernas y eficientes.



EJERCICIOS DEL CAPÍTULO

1. Verifique la configuración de Node y VS Code:

- a. Después de completar las instalaciones, confirme
Todo funciona correctamente al abrir la línea de comandos
y ejecutar `node -v` y `npm -v`. Debería ver los números de
versión de Node y npm, lo que indica que están instalados
correctamente.
- b. Inicie VS Code y abra una nueva carpeta llamada `hello-
node`.
- c. Dentro de la carpeta, crea un archivo llamado `index.js` con el
siguiente código: `console.log("Node is working!");`
- d. Navegue al directorio raíz de su proyecto en su línea de
comando y ejecute el archivo ejecutando `node index.js`.
- e. Deberías ver el mensaje ¡ El nodo está
funcionando! impreso en tu consola.

CONSEJO

Si algo no funciona, revise los pasos de instalación en este capítulo y
verifique las instrucciones específicas de su sistema operativo.

2. Explora la plantilla de inicio de Fastify:

- a. Empieza a usar Fastify generando un proyecto básico. En
la línea de comandos, ejecuta `npm init fastify@latest
fastify-test-app`

y siga las instrucciones para crear un nuevo proyecto. Este comando utiliza la CLI de Fastify para configurar un nuevo proyecto con todos los archivos y dependencias necesarios.

- b. Navegue al nuevo directorio del proyecto en su línea de comando y ejecute `npm install` para instalar las dependencias.
- c. Ejecute `npm run dev` para iniciar el servidor de desarrollo, abra su navegador y vaya a `httplocalhost` . Debería ver un mensaje JSON como `{ "root": true }`. Esto indica que su servidor Fastify funciona correctamente.

Estos ejercicios garantizan que su entorno de desarrollo esté listo y le permiten ejecutar código con Node y lanzar un servidor web basado en Fastify.

Resumen

En este capítulo, usted:

- Configure su entorno de desarrollo de Node instalando herramientas esenciales como VS Code y Node en macOS, Windows y Linux
- Aprendí el rol de Node como un entorno de ejecución de JavaScript rápido basado en eventos y construido sobre el motor V8
- Exploró los fundamentos de la arquitectura de Fastify para crear API y servicios web.

Capítulo 2. Aplicación práctica

Este capítulo cubre lo siguiente:

- Introducción a una aplicación Node
- Lectura de la entrada del usuario en la línea de comandos
- Escribir datos en un CSV

Tanto si eres nuevo en programación como si eres un ingeniero experimentado, probablemente hayas abierto este capítulo para empezar a usar Node. Muchos proyectos de Node abarcan miles de líneas de código, pero algunas de las aplicaciones más útiles y prácticas se pueden escribir en tan solo unas pocas líneas. Saltemos la parte en la que te sientes abrumado y pasemos a la aplicación práctica.

En este capítulo, verás un primer vistazo a lo que Node puede ofrecer de forma inmediata y autónoma. Aprenderás a crear un programa sencillo que se ejecuta en cualquier ordenador y ahorra tiempo y dinero a usuarios reales. Al final del capítulo, tendrás Node instalado en tu ordenador con un entorno de desarrollo listo para crear prácticamente cualquier cosa.

HERRAMIENTAS Y APLICACIONES UTILIZADAS EN ESTE CAPÍTULO

Antes de comenzar, asegúrese de instalar y configurar las herramientas y aplicaciones necesarias para este proyecto. Las instrucciones de instalación de Node, Fastify y VS Code se encuentran en el **Capítulo 1**, mientras que los pasos de inicialización del proyecto, como la configuración de la estructura de directorios, la configuración de package.json y el uso de sintaxis moderna, se describen en **el Apéndice A**. Una vez completado, regrese aquí para continuar. Desarrollar un proyecto desde cero le ayuda a comprender mejor cada componente, lo que le brinda mayor control y flexibilidad a medida que avanza.

Su mensaje Un agente

de viajes quiere contratarlo para convertir su **Rolodex** (tarjetas de información física) a algún formato digital. Acaban de comprar unas computadoras nuevas y quieren empezar a importar datos. No necesitan nada sofisticado como un sitio web o una aplicación móvil, solo un mensaje para crear un formato de tabla o valores separados por comas (CSV) de datos.

Por suerte, acabas de

empezar a desarrollar aplicaciones en Node y ves esta gran oportunidad para usar algunas de las bibliotecas predeterminadas de Node. Esta empresa solo quiere introducir manualmente datos que se puedan escribir en un archivo CSV. Así que empiezas a **diagramar** los requisitos del proyecto, y el resultado se muestra en **la Figura 2-1**. El diagrama detalla los siguientes pasos:

1. Las tarjetas de información física son recogidas por el usuario.
2. Abra la línea de comando e inicie la aplicación ejecutando `node index`.

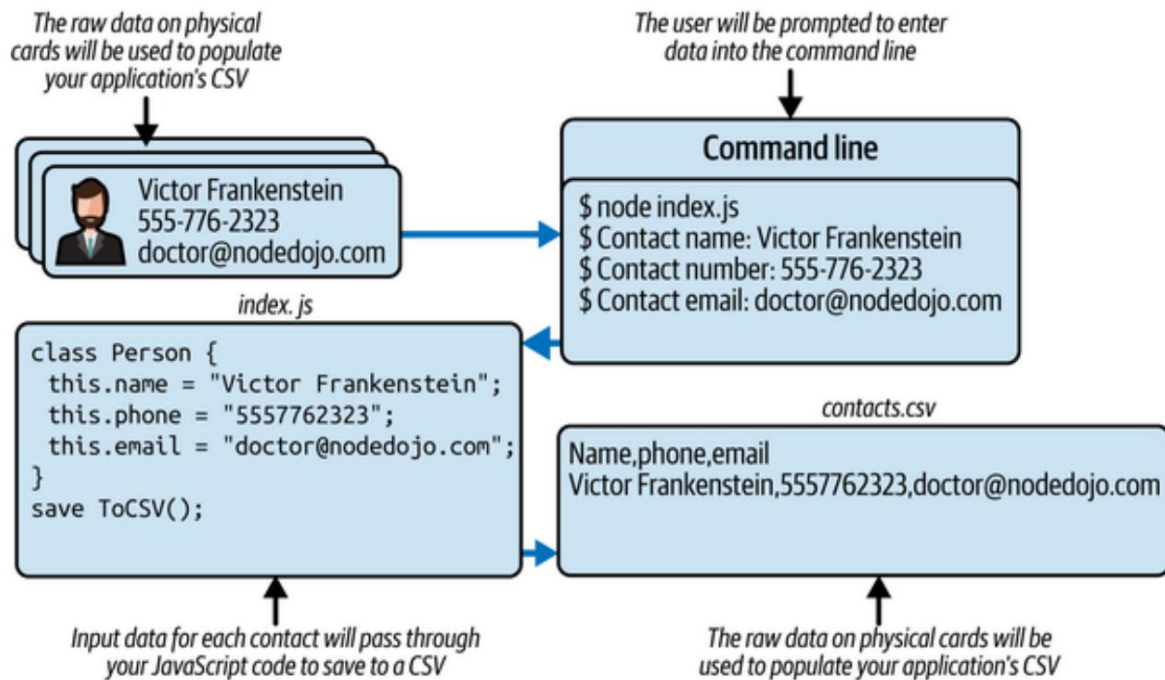
3. Ingrese el nombre, el número de teléfono y la dirección de correo electrónico en el indicador de línea de comandos.
4. Después de cada entrada, los datos de contacto se guardan a través de su aplicación en un archivo CSV en su computadora.

Cada contacto tiene su nombre completo, número de teléfono y dirección de correo electrónico en su tarjeta. Decides crear una aplicación sencilla de línea de comandos de Node para que los usuarios puedan introducir manualmente estos tres datos para cada tarjeta de contacto.

Al terminar, el usuario podrá abrir la línea de comandos, ejecutar `node index.js` y seguir las instrucciones para ingresar la información de contacto. Cada contacto se guardará en un archivo CSV como una cadena delimitada por comas.

NOTA

•CSV es un precursor de muchas de las tablas de bases de datos convencionales que se utilizan actualmente. Si bien no es ideal para almacenar datos a largo plazo, es una excelente manera de visualizar filas de datos para su visualización y procesamiento sin una base de datos.



Cifra 21. Diagramar el plan del proyecto y el flujo de información

Obtenga programación

Con Node configurado, comience a configurar su proyecto. Elija un Ubicación donde desea almacenar el código de su proyecto para este capítulo y ejecute `mkdir csv_app` en su línea de comando para crear el proyecto carpeta.

CONSEJO

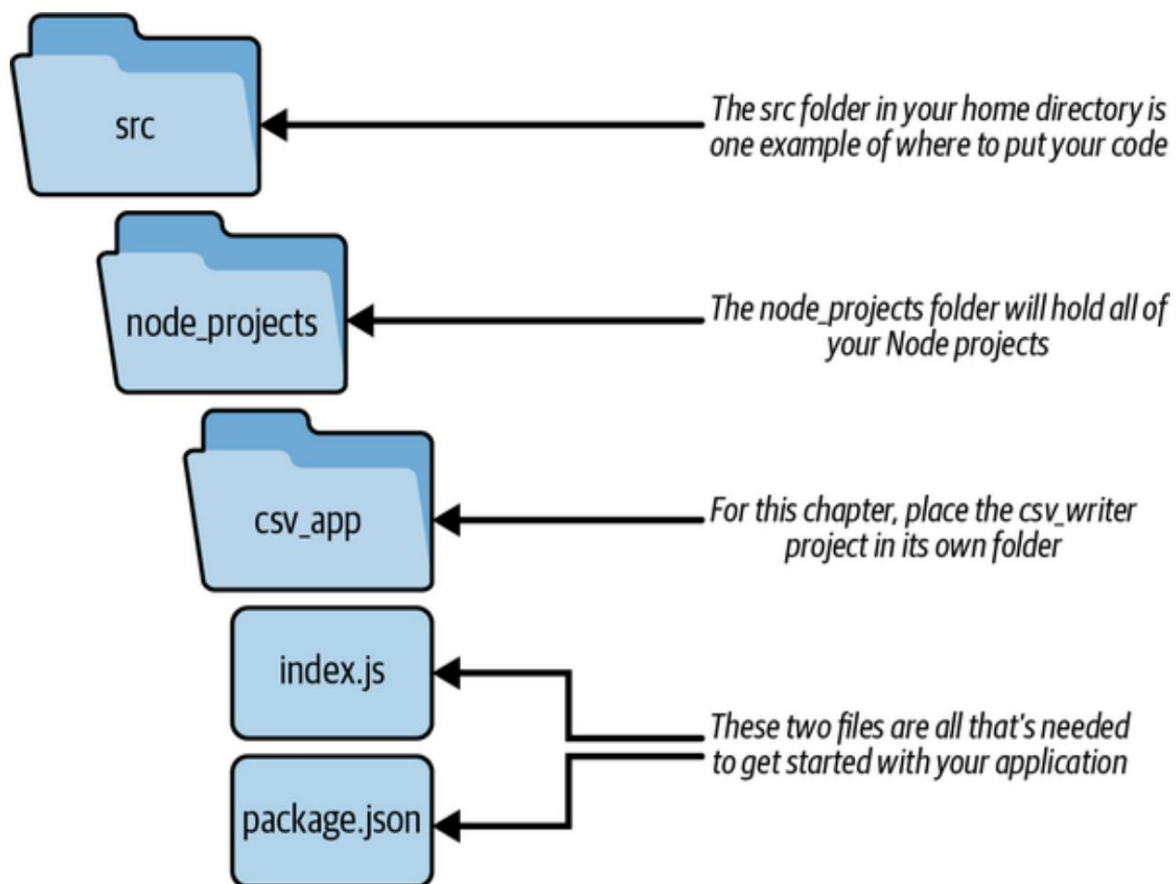
Para mantener sus proyectos de codificación separados de otros trabajos en su computadora, puedes dedicar un directorio para la codificación. Crear un directorio Llamado `src` en el nivel raíz del directorio de usuario de su computadora es una buena NOMBRE DE USUARIO. Esa ubicación es `Usersrc` para Mac (`~ src`), y `C:\Users\user\src` para computadoras Windows.

A continuación, seguirás los pasos para inicializar tu aplicación Node. Al finalizar, La estructura del directorio de su proyecto debería verse como la **Figura 2-2**.

NOTA

Estos pasos también se describen en [el Apéndice B](#).

Dentro de la carpeta `csv_app`, ejecute `npm init` para iniciar el proceso de creando un nuevo archivo de configuración de la aplicación Node. Verá un `prompt` y puede completar sus respuestas como en [el Ejemplo 2-1](#).



Cifra 22. Estructura del directorio del proyecto

Ejemplo de 21. Solicitud de `npm init`

nombre de paquete: (`csv_app`)

1

versión: (`1.0.0`)

Descripción: Una aplicación para escribir información de contacto en un `csv` archivo.

punto de entrada: (`index.js`)

2

comando de prueba:

repositorio git:

Palabras clave:

autor: Jon Wexler

licencia: (ISC)

❶ Este es el nombre de tu proyecto.

❷ index.js es donde comenzará su aplicación.

Este proceso ha creado un nuevo archivo para usted: package.json

(Ejemplo 2-2). Este es el archivo de configuración de su aplicación, donde

Le indicará a tu computadora cómo ejecutar tu aplicación Node.

¿QUÉ ES PACKAGE.JSON?

Cada proyecto de Node tiene un archivo especial llamado package.json. Piensa como manual de instrucciones y kit de herramientas de su aplicación. Explica qué necesita tu aplicación para funcionar, qué tareas puede realizar e incluso detalles sobre el proyecto en sí. Si eres nuevo en Node, comprender esto...

El archivo es crucial, por lo tanto, vamos a desglosarlo.

¿Por qué es importante package.json? Imagina que estás horneando un pastel.

Necesitarás ingredientes, herramientas e instrucciones paso a paso.

De manera similar, una aplicación Node necesita bibliotecas específicas (ingredientes), configuraciones (herramientas) y scripts (pasos) para trabajar. El

El archivo package.json organiza todo esto en un solo lugar, lo que lo hace fácil. para compartir su proyecto con otros o configurarlo en una nueva computadora.

Las partes clave de package.json incluyen lo siguiente:

Dependencias

Estos son los “ingredientes” que necesita tu aplicación: terceros bibliotecas o paquetes de los que depende. Por ejemplo, si tu aplicación usa Google Maps, debes incluir el paquete @google/maps aquí. Esto les dice a los demás (y a sus computadoras) qué instalar para que funcionen. La aplicación funciona.

Guiones

Estos son comandos que automatizan tareas para su aplicación, como iniciarlo, probarlo o construirlo. Por ejemplo, la mayoría El script común es npm start, que inicia la aplicación. Puedes También agregue scripts personalizados como npm run build para preparar archivos para su implementación.

Versión

Esto rastrea la versión de la aplicación, lo cual es útil a medida que realiza actualizaciones. Las versiones siguen un formato como 1.0.0, donde

Los números representan cambios importantes, actualizaciones menores o pequeñas correcciones (llamadas "parches").

Nombre y descripción

Estos campos describen su proyecto. El campo de nombre proporciona su La aplicación tiene un identificador único y la descripción explica qué es. hace.

Licencia

Esto les informa a otros los términos para usar, compartir o modificar su información. aplicación. Por ejemplo, podría permitir el uso abierto con un MIT licencia o establecer términos más estrictos.

Si está trabajando en equipo o compartiendo su código, packagejson Garantiza que todos estén en sintonía. Automatiza la configuración y las listas. bibliotecas necesarias y le ahorra tener que explicar manualmente cómo hacerlo Ejecuta tu aplicación.

Para obtener más información, visita la [documentación de Node](#).

El primer paso es agregar "type": "module" a su packagejson.

Esto habilita la sintaxis del módulo ES6 (importación/exportación) en su Node.

Proyecto. A partir de Node v13.2.0, compatibilidad estable con módulos ES6.

Se introdujo, haciéndolos cada vez más populares en los proyectos Node.

Muchos desarrolladores ahora prefieren los módulos ES6 en lugar del antiguo CommonJS

Sintaxis (require/module.exports) para un uso más limpio y moderno. código.

Ejemplo { 22. Contenido del paquete.json

```
"nombre": "csv_app",  
"versión": "1.0.0",  
"tipo": "módulo",  
"description": "Una aplicación para escribir información de contacto a un  
archivo csv.",
```

```

    "principal": "index.js",
    "guiones": {
      "prueba": "echo \"Error: no se especificó ninguna prueba\" && exit 1"
    },
    "autor": "Jon Wexler",
    "licencia": "ISC"
  }
}

```

1 Esta adición permite la sintaxis de importación de módulos en su aplicación.

A continuación, crea el punto de entrada a tu aplicación, un archivo titulado `index.js`. Aquí se ubicará el núcleo de tu aplicación.

Para este proyecto, te das cuenta de que Node viene preempaquetado con todo lo que necesitas ya. Puedes usar el módulo `fs`, una biblioteca que ayuda a su aplicación a interactuar con el sistema operativo de su computadora. Sistema de archivos: para crear el archivo CSV. Dentro de `index.js`, agrega la función `writeFileSync` del módulo `fs` escribiendo `import { writeFileSync } from 'fs'`.

Ahora tienes acceso a funciones que pueden crear archivos en tu computadora. Ahora puedes usar la función `writeFileSync` para probar que su aplicación Node puede crear (o sobrescribir) archivos correctamente y añádeles texto. Agrega el código del **Ejemplo 2-3** a `index.js`. Aquí . Estás creando una variable llamada `contenido` con algún marcador de posición. texto. Luego, dentro de un bloque `try/catch`, ejecuta `writeFileSync`, una función sincrónica y de bloqueo para crear un nuevo archivo llamado `test.txt` dentro del directorio de su proyecto. Si el archivo se crea con su contenido agregado, verá un mensaje registrado en su línea de comandos con el texto ¡Éxito! De lo contrario, si se produce un error, vea el seguimiento de la pila y el contenido de ese error registrado en su ventana de línea de comandos.

NOTA

Con la sintaxis de importación del módulo ES6 puedes utilizar la desestructuración de tarea. En lugar de importar una biblioteca completa, puedes importar las funciones o módulos que necesitas solo mediante la desestructuración. Para más información, lee [las páginas de referencia de JavaScript de Mozilla](#).

Ejemplo de 23. Contenido de index.js

```
import { writeFileSync } from "fs"; ❶

const content = "¡Prueba el contenido!"; ❷

try { ❸
  writeFileSync("./test.txt", content); ❹
  console.log("¡Éxito!"); ❺
} catch (err) { ❻
  console.error(err);
}
```

- ❶ Desestructura el módulo fs para importar su writeFileSync función.
- ❷ Asigna la variable de contenido a una cadena.
- ❸ Agrega un bloque try/catch para finalizar su llamada.
- ❹ Escribe en un nuevo archivo llamado test.txt.
- ❺ Registrar el éxito al escribir el archivo.
- ❻ Registra un error si writeFileSync falla.

Estás listo para probar esto navegando a la carpeta de tu proyecto csvapp dentro de la ventana de tu terminal y ejecutando node index.js. Ahora, Comprueba si se creó un nuevo archivo llamado test.txt. Si es así, ábrelo.

¡Para ver el contenido de la prueba! Ya está listo para pasar al siguiente nivel.

Paso y adapte esta aplicación para manejar la entrada del usuario y escribir en un archivo CSV.

Traduciendo la entrada del usuario a CSV. Ahora

que tu aplicación está configurada para guardar contenido en un archivo, comienza a escribir la lógica para aceptar la entrada del usuario en la línea de comandos. Node incluye un módulo llamado `readline` que realiza precisamente eso. Al importar la función `createInterface`, puedes asignar la entrada y salida estándar de la aplicación a la funcionalidad `readline`, como se muestra en el **Ejemplo 2-4**. Al igual que el módulo `fs`, no se requieren instalaciones adicionales, salvo la instalación predeterminada de Node en tu ordenador.

En este código, `process.stdin` y `process.stdout` son formas en que el proceso de su aplicación Node transmite datos hacia y desde la línea de comandos, lo que permite la interacción con el usuario directamente a través de la terminal.

Ejemplo 24. Asignación de entrada y salida a `readline` en `index.js`

```
de importación { createInterface } desde "readline";
```

❶

```
const readline = createInterface({ entrada:
  proceso.stdin, salida:
  proceso.stdout });
```

❷

❶ Desestructura el módulo `readline` para importar `createInterface`.

❷ Llame a `createInterface()` con `stdin` y `stdout` para configurar la entrada y salida de usuario basadas en terminal para su aplicación Node.

A continuación, utilice esta asignación mediante la función de pregunta en su interfaz de `readline`. Esta función recibe un mensaje o mensaje que mostrará al usuario en la línea de comandos y devolverá la entrada del usuario como valor de retorno.

NOTA

Dado que esta función es asíncrona, puedes encapsular su valor de retorno en una promesa para usar la sintaxis `async/await` de ES6. Si necesitas repasar las promesas de JavaScript, visita las páginas [de referencia de JavaScript de Mozilla](#) .

Cree una función llamada `readLineAsync` que espere a que el usuario responda y presione Enter antes de que se resuelva el valor de la cadena ([Ejemplo 2-5](#)). De esta manera, su función `readLineAsync` personalizada se resolverá con una respuesta que contenga la entrada del usuario sin interrumpir la aplicación Node.

Ejemplo 5.2 Función `readLineAsync` envuelta en promesa
index.js

```
const readLineAsync = (mensaje) => ❶  
  nueva Promesa((resolver) => readline.pregunta(mensaje, resolver));  
                                ❷
```

- ❶ Declara una función asíncrona `readLineAsync` que toma un mensaje como entrada.
- ❷ Envuelve `readline.question` en una Promesa para habilitar el uso `async/await`, resolviéndose con la entrada del usuario.

USANDO PROMISIFY PARA ENVOLVER TU SINCRÓNICO FUNCIONES

Usar `promisify`, desde el módulo util integrado de Node, es una técnica potente para convertir funciones tradicionales basadas en devoluciones de llamada en funciones basadas en promesas, lo que permite un código asíncrono más sencillo y limpio con `async/await`. En muchas API de Node, especialmente las más antiguas, las funciones asíncronas se basan en una devolución de llamada donde el primer argumento es un error (si lo hay) y el segundo es el resultado. Este enfoque puede generar un problema de devolución de llamada y código menos legible. Con `promisify`, puedes transformar estas funciones de devolución de llamada en promesas, simplificando tu flujo asíncrono y reduciendo la complejidad de la gestión de errores.

Por ejemplo, usar `promisify` para convertir una función como `fs.readFile` o `readline.question` en una promesa permite integrar estas funciones sin problemas en código JavaScript moderno. Esto facilita el mantenimiento del código, ya que permite usar `async/await` en lugar de devoluciones de llamadas profundamente anidadas, lo que resulta en una estructura más limpia y síncrona. `Promisify` es especialmente útil cuando se trabaja con módulos de Node heredados o se desea actualizar el código base gradualmente para usar patrones más modernos sin una refactorización significativa. Consulta [el Ejemplo 2-6](#) para saber cómo usar `promisify` con `readline.question`.

Este código configura una CLI mediante el módulo `readline` de Node, lo que permite la entrada asíncronica del usuario con la ayuda de `promisify`. Las partes clave incluyen la conversión del método de pregunta basado en devoluciones de llamada en una promesa (mediante `promisify`), la habilitación del uso de `async/await` para solicitar al usuario su nombre y el cierre de la interfaz una vez finalizada la interacción.

Ejemplo en 26. Función readLineAsync envuelta en promesa
index.js

```

import { promisify } de 'util'; ❶
...
constante readLineAsync =
prometer(readline.question).bind(rl); ❷
(async () => {
  intentar {
    const name = await readLineAsync('¿Cuál es tu nombre?'); ❸
    console.log(`¡Hola, ${name}!`);
  } atrapar (err) {
    console.error('Error:', err.message); ❹
  } finalmente {
    readline.close(); ❺
  }
})();
...

```

- ❶ Importe la función promisify desde el módulo util .
- ❷ El método readLineAsync está envuelto con promisify para convertirlo en una función basada en promesas.
- ❸ Solicitar entrada al usuario mediante async/await.
- ❹ Agregue un bloque try/catch para manejar cualquier error potencial que ocurrir.
- ❺ Agregar un bloque finally garantiza que la interfaz readline sea cerrado.

Con estas funciones en su lugar, todo lo que necesita es llamar a readLineAsync para que su aplicación comience a recuperar la entrada del usuario. Porque su El mensaje tiene tres valores de datos específicos para guardar, puede crear una clase para encapsular esos datos para cada contacto. Como se ve en el [Ejemplo 2-7](#), dentro del índice.js importa appendFileSync desde el módulo fs ,

que creará y anexará un nombre de archivo determinado. Luego, se define una clase Person que toma nombre, número y correo electrónico como argumentos en su constructor. Finalmente, se agrega un método saveToCSV a la clase Person para guardar la información de cada contacto en un formato delimitado por comas, compatible con CSV, en un archivo llamado contactscsv.

Ejemplo de 27. Definición de la clase Persona en index.js

```
importación { appendFileSync } desde "fs"; ❶

class Persona ❷
{ constructor(nombre = "", número = "", correo electrónico = "") { ❸
    este.nombre = nombre;
    este.número = número; este.correo
    electrónico = correo electrónico;
  }
  saveToCSV() { const ❹
    content = `${este.nombre},${
este.número},${este.correo electrónico}\n`; ❺
    try
    { appendFileSync("./contacts.csv", content); console.log(` $ ❻
    {this.name} ¡Guardado!`); } catch (err)
    { console.error(err);
  }
}
}
```

❶ Desestructura el módulo fs para importar appendFileSync.

❷ Define la clase Persona .

❸ Define un constructor con nombre, número y correo electrónico.

❹ Agregue un método saveToCSV para guardar la información de contacto en CSV.

❺ Utilice la interpolación de cadenas para separar la información de cada contacto con comas.

❻

Guarde y agregue la cadena de contenido en un archivo llamado `contactscsv` en el directorio de su proyecto.

CONSEJO

Si desea asegurarse de que el archivo `contactscsv` exista antes de intentar Añadirlo, también puedes importar la función `existsSync` desde el sistema de archivos módulo y verifique si el archivo existe antes de agregarlo usando `existsSync("./contacts.csv")`. De esta manera, puede evitar errores si El archivo no existe.

El último paso es crear una nueva instancia de un objeto `Persona` para cada nuevo contacto que estás ingresando manualmente en tu aplicación. Para ello, Crea una función `startApp` asíncrona que utiliza un bucle `while` controlado por una variable `shouldContinue`. Inicialmente establecida como verdadera, esta La variable determina si el bucle continúa.

Dentro del bucle, la función recopila la entrada del usuario para el nombre, número y correo electrónico utilizando la función `readLineAsync`. Cada entrada se recopila en secuencia, lo que garantiza que la función espere al usuario proporcionar cada valor antes de continuar.

Una vez que se hayan recopilado todos los valores necesarios, se crea un nuevo objeto `Persona`. se instancia con los valores de entrada y el método `saveToCSV()` es Se llamó a la instancia de persona para guardar los datos en un archivo. Después de guardarlos, Se le solicita al usuario que decida si desea continuar ingresando más datos escribiendo y. Si el usuario escribe y, el bucle continúa; de lo contrario, `shouldContinue` se establece en falso y la interfaz `readline` es cerrado, finalizando la aplicación (Ejemplo 2-8).

Ejemplo de recopilación de entrada del usuario dentro de `startApp` en `index.js`

```
const startApp = async () => {
  deje que shouldContinue = verdadero;
  mientras (deberíaContinuar) {
```

1

2

```

const name = await readLineAsync(" Nombre del contacto: ");
const number = await readLineAsync(" Número de contacto: ");
const email = await readLineAsync(" Correo electrónico de contacto: ");

const persona = nueva Persona(nombre, numero, email);
persona.saveToCSV();

const respuesta = await readLineAsync("¿Continuar? [y a
continuar]: ");
shouldContinue = respuesta.toLowerCase() === "y";
}
readline.close();
};

```

1

Declara una función startApp asincrónica para administrar la
Flujo de la aplicación.

2

Utiliza un bucle while controlado por la variable shouldContinue
para indicaciones repetidas.

3

Crea una instancia de Persona con el nombre, número y
entradas de correo electrónico .

4

Pregunta al usuario si desea continuar ingresando datos.

5

Las actualizaciones deben continuar según la respuesta del usuario (y a
continuar).

6

Cierra la interfaz de lectura , finalizando la aplicación.

Luego agregue startApp() en la parte inferior de indexjs parainiciar la aplicación
Al ejecutar el archivo, el archivo index.js final debería verse como el del Ejemplo 2.
9.

Ejemplo 29. Index.js completo

```

de importación { appendFileSync } desde "fs";
importar { createInterface } de "readline";

```

```

constante readline = crearInterfaz({

```

```

    entrada: proceso.stdin, salida:
    proceso.stdout, });

```

```

const readLineAsync = (mensaje) =>
    nueva Promesa((resolver) => readline.pregunta(mensaje, resolver));

```

```

class Persona

```

```

    { constructor(nombre = "", número = "", correo electrónico = "") {
        este.nombre = nombre;
        este.número = número; este.correo
        electrónico = correo electrónico; }

```

```

        saveToCSV() { const
            content = `${este.nombre},${
{este.número},${este.correo electrónico}\n`;
            try
                { appendFileSync("./contacts.csv", content); console.log(`$
                {this.name} ¡Guardado!`); } catch (err)
                { console.error(err);
            }
        }
    }
}

```

```

const startApp = async () => { let
    shouldContinue = true; mientras
    (shouldContinue) {
        const name = await readLineAsync(" Nombre del contacto: "); const
        number = await readLineAsync(" Número de contacto: "); const email = await
        readLineAsync(" Correo electrónico del contacto: ");

        const persona = new Persona(nombre, número, correo
        electrónico); persona.saveToCSV();

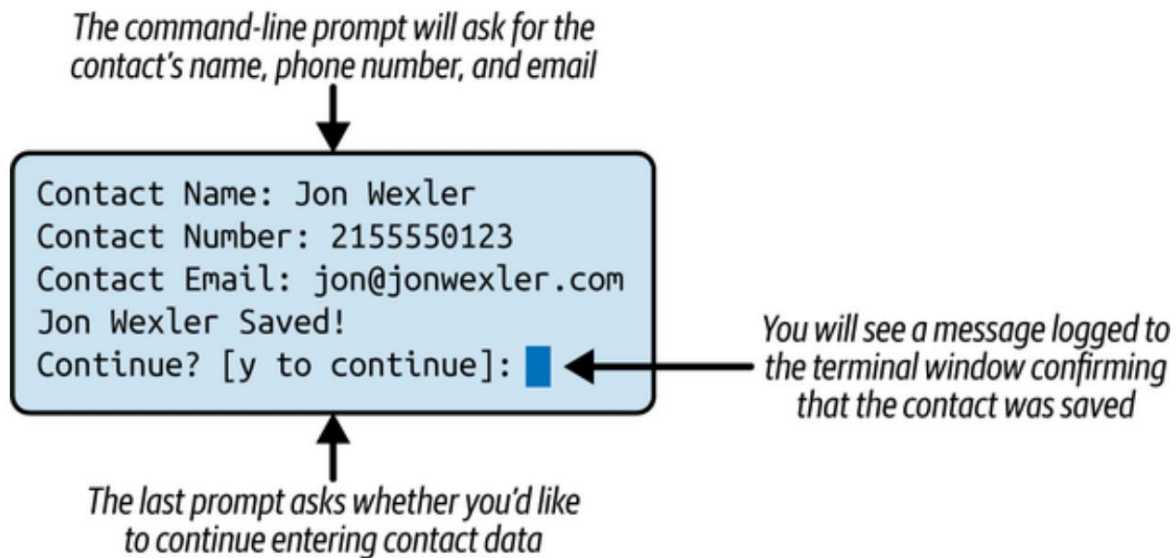
        const respuesta = await readLineAsync("¿Continuar? [y para continuar]: ");
        shouldContinue =
            response.toLowerCase() === "y";

    } readline.close(); };

```

```
iniciarAplicación();
```

En la carpeta del proyecto en su línea de comandos, ejecute `node index` para iniciar viendo indicaciones de texto, como se ve en la Figura 2-3.



Cifra 23. Indicaciones de la línea de comandos para la entrada del usuario

Cuando haya terminado de ingresar todos los datos del contacto, podrá:

Ver que la información se ha guardado en un archivo llamado `contactscsv`.

en la misma carpeta. Cada línea de ese archivo debe estar delimitada por comas.

Pareciendo `Jon Wexler,2155550123,Jon@jonwexler.com`.

Esto debería ser justo lo que la agencia de viajes necesita para convertir su...

Tarjetas de contacto físicas en un archivo CSV que pueden usar en muchas otras formas. En la siguiente sección, explorará cómo las bibliotecas de terceros pueden Simplifique aún más su código.

Trabajar con paquetes externos

No se necesitó mucho código para crear una aplicación que fuera tan funcional

Y tan efectivo como `csv_app`. La buena noticia es que tu trabajo está hecho.

¡Hay aún mejores noticias! Si bien Node ofrece módulos integrados para...

Variedad de tareas, hay muchas bibliotecas externas (paquetes npm)
Puedes instalarlo para reducir aún más el código escrito.

Para mejorar la legibilidad del código escrito hasta ahora, puedes instalar
Los paquetes de solicitud y escritor CSV ejecutando npm install
"csv-writer@^1.6.0" "prompt@^1.3.0" en la carpeta de su proyecto
en la línea de comandos. Este comando también listará estos dos
paquetes en su archivo packagejson .

NOTA

Consulte el sitio web de npm para obtener más información sobre [las indicaciones](#). y [escritor csv](#).

En su archivo indexjs , importe el mensaje y úselo para reemplazar su
Llamadas readLineAsync , como se muestra en [el Ejemplo 2-10](#). Comience por inicializar
aviso con prompt.start() y configurando prompt.message
a una cadena vacía para eliminar prefijos innecesarios en la terminal.
Ahora puedes eliminar la función readLineAsync , readline
asignaciones de interfaz e importaciones de módulos readline desde su código.

prompt.get se utiliza para recopilar la entrada del usuario para múltiples indicaciones
a la vez, devolviendo un objeto donde corresponde el nombre de cada solicitud
a su respuesta. Estas respuestas se pasan directamente a un nuevo
Constructor de personas .

Después de guardar los valores de la persona en un archivo CSV usando saveToCSV
método, el programa solicita al usuario que decida si desea
continuar. La respuesta del usuario se desestructura y se asigna a la
nuevamente variable, que determina si el programa recursivamente
se llama a sí mismo para recoger entradas o salidas adicionales.

Ejemplo 210. Reemplazo de readLineAsync con prompt en index.js

```

importar mensaje de "prompt";
prompt.start();
prompt.message = "";
...
const startApp = async () => { const ❶
  questions = [ { name: ❷
    "nombre", description: "Nombre del contacto" }, { name:
    "número", description: "Número de contacto" }, { name: "correo
    electrónico", description: "Correo electrónico de contacto" }, ];

  const respuestas = await prompt.get(preguntas); const ❸
  persona = new Persona(respuestas.nombre,
respuestas.numero, respuestas.correo ❹
  electrónico); esperar persona.saveToCSV();

  const { otra vez } = await prompt.get([ { nombre:
    "otra vez", descripción: "¿Continuar? [y para continuar]" }, ]);

❺

  si (again.toLowerCase() === "y") esperar startApp(); ❻

```

- ❶** Define la función startApp como asíncrona para permitir el uso de await .
- ❷** Crea una serie de preguntas para recopilar información sobre nombre, número y correo electrónico .
- ❸** Solicita al usuario que formule preguntas y recopila las respuestas.
- ❹** Crea una instancia de un objeto Persona con las respuestas recopiladas.
- ❺** Pregunta al usuario si desea continuar agregando contactos.
- ❻** Llama recursivamente a startApp si el usuario opta por continuar; de lo contrario, el programa finaliza.

De manera similar a cómo el paquete de aviso externo desplazó al Módulo readline , csv-writer reemplaza la necesidad de su sistema de archivos. El módulo importa y define un enfoque más estructurado para escribir su CSV incluyendo un encabezado, como se muestra en el **Ejemplo 2-11**. Coloque este código en la parte superior de su archivo index.js , justo debajo del mensaje de solicitud de importación y la inicialización.

Ejemplo 211. Importación y configuración de csv-writer en index.js

```
de importación { createObjectCsvWriter } desde "csv-writer"; ❶
...
constante csvWriter = crearObjetoCsvWriter({ ❷
  ruta: "./contactos.csv",
  añadir: verdadero,
  encabezado: [
    { id: "nombre", título: "NOMBRE" },
    { id: "número", título: "NÚMERO" },
    { id: "correo electrónico", título: "CORREO ELECTRÓNICO" },
  ],
});
```

❶ Utilice una importación con nombre para incorporar createObjectCsvWriter desde escritor de csv.

❷ Configurar csvWriter para escribir en contactscsv, agregarlo al archivo, e incluir encabezados.

Finalmente, modifica tu método saveToCSV en la clase Person a

Utilice csvWriter.writeRecords en **su lugar (Ejemplo 2-12)**.

Ejemplo 2-12. Actualice saveToCSV en la clase Person para usar csvWriter.writeRecords en index.js

```
...
async guardar en CSV() {
  intentar {
    const { nombre, número, correo electrónico } = this; ❶
    await csvWriter.writeRecords([ { nombre, número, correo electrónico } ]);
  } ❷
  console.log(` ${name} ¡Guardado!`);
}
```

```

    } catch (err)
    { console.error(err);
    }
}
...

```

❶

Desestructura las variables de instancia de Persona .

❷

Utilice csvWriter.writeRecords para escribir una fila de valores en su archivo CSV.

Con estos dos cambios implementados, su nuevo archivo indexjs debería verse como [el Ejemplo 2-13](#).

Ejemplo Usar de paquetes externos en index.js [import](#)

```

{ createObjectCsvWriter } from "csv-writer"; import prompt from
"prompt";

```

```

prompt.start();
prompt.message = "";

```

```

constante csvWriter = crearObjetoCsvWriter({
  ruta: "./contacts.csv", agregar:
  verdadero,
  encabezado:
    [ { id: "nombre", título: "NOMBRE" }, { id: "número",
    título: "NÚMERO" }, { id: "correo electrónico", título:
    "CORREO ELECTRÓNICO" }, ], });

```

clase Persona

```

{ constructor(nombre = "", número = "", correo electrónico = "") {
  este.nombre = nombre;
  este.número = número; este.correo
  electrónico = correo
  electrónico; }

```

```

async saveToCSV() { try
  { const
    { nombre, número, correo electrónico } = this;

```

```

        await csvWriter.writeRecords([ { nombre, número, correo electrónico
    }]);
    console.log(` ${name} ¡Guardado!`);
  } catch (err) {
    console.error("Error al guardar el contacto:", err);
  }
}
}
}

```

```

const startApp = async () => { const
  questions = [ { name:
    "nombre", description: "Nombre del contacto" }, { name:
    "número", description: "Número de contacto" }, { name: "correo
    electrónico", description: "Correo electrónico de contacto" }, ];

```

```

    const respuestas = await prompt.get(preguntas); const persona
    = new Persona(respuestas.nombre,
    respuestas.numero, respuestas.correo
    electrónico); esperar persona.saveToCSV();

```

```

    const { otra vez } = await prompt.get([ { nombre:
    "otra vez", descripción: "¿Continuar? [y para continuar]" }, ]);

```

```

    si (again.toLowerCase() === "y") esperar startApp(); };

```

```

iniciarAplicación();

```

Ahora, al ejecutar node index, el comportamiento de la aplicación debería ser exactamente igual que en la sección anterior. Esta vez, el archivo csv debería incluir los encabezados al principio. Este es un excelente ejemplo de cómo usar Node de forma inmediata para resolver un problema real y, luego, refactorizar y mejorar el código utilizando paquetes externos creados por la activa comunidad en línea de Node.

EJERCICIOS DEL CAPÍTULO

1. Agregue validación de entrada para correo electrónico y número de teléfono:
 - a. Actualice su aplicación CLI para que solo acepte direcciones de correo electrónico y números de teléfono válidos.
 - b. Utilice una expresión regular para comprobar que el correo electrónico contiene un formato válido como `ejemplo@dominio.com` y que el número de teléfono contiene solo dígitos.
 - c. Si la entrada no es válida, muestra un mensaje de error y solicita al usuario que ingrese el valor nuevamente.
 - d. Después de implementar esto, intente ejecutar la aplicación e intente ingresar datos incorrectos (por ejemplo, `abc@`, `123abc456`) para asegurarse de que vuelva a preguntar correctamente.

CONSEJO

Para una verificación básica de correo electrónico, puedes usar `\\S+@\\S+\\.\\S+/,` ¡pero siéntete libre de mejorarlo!

2. Incluya una marca de tiempo para cada entrada de contacto:
 - a. Modifique la clase `Persona` o el código de su generador de CSV para incluir un campo `createdAt`.
 - b. Cuando el usuario envía un contacto, automáticamente agrega la fecha y hora actuales en formato ISO usando `new Date().toISOString()`.
 - c. Actualice su lógica CSV para incluir un encabezado `CREATED_AT` y escriba este nuevo campo junto con el nombre, número y correo electrónico del contacto.

d. Ejecute su aplicación, ingrese algunos contactos y confirme que cada fila ahora incluye la marca de tiempo en la columna final.

Estos ejercicios le ayudarán a consolidar su comprensión de la validación de entrada, el formato y el enriquecimiento de datos, todas tareas comunes en aplicaciones Node del mundo real.

Resumen

En este capítulo, usted:

- Se desarrolló una aplicación Node para automatizar la conversión de datos de contacto de tarjetas físicas a un formato CSV.
- Aprendí a leer la entrada del usuario desde la línea de comandos utilizando bibliotecas Node integradas y paquetes externos.
- Información de contacto almacenada de manera eficiente en un archivo CSV con encabezados estructurados utilizando el paquete `csv-writer`.
- Mejore su proyecto integrando bibliotecas externas, mejorando tanto la legibilidad del código como la funcionalidad.
- Estableció un flujo de trabajo fundamental para crear herramientas de línea de comandos que resuelvan problemas del mundo real con un código mínimo.

Capítulo 3. Creación de un servidor web de Node

Este capítulo cubre lo siguiente:

- Usando Node como interfaz web
- Creación de una aplicación web Node con Fastify
- Sirviendo páginas estáticas con contenido dinámico

Node se centra en el uso de JavaScript en el servidor. JavaScript ya es un lenguaje asíncrono por naturaleza, pero no fue hasta 2009 que se utilizó fuera de los navegadores web estándar. A medida que la dependencia de internet crecía a nivel mundial, las empresas demandaban nuevas estrategias de desarrollo innovadoras que también consideraran las habilidades disponibles para el mercado. A partir de entonces, JavaScript despegó para el desarrollo frontend y backend, estableciendo nuevos patrones de diseño de aplicaciones con el bucle de eventos de un solo hilo de Node.

En este capítulo, explorarás el caso de uso más común de Node, una aplicación web, y la importancia del bucle de eventos. Al finalizar este capítulo, podrás usar Fastify, el framework de proyectos más popular de Node, para crear tanto servidores web sencillos como aplicaciones más complejas.

HERRAMIENTAS Y APLICACIONES UTILIZADAS EN ESTE CAPÍTULO

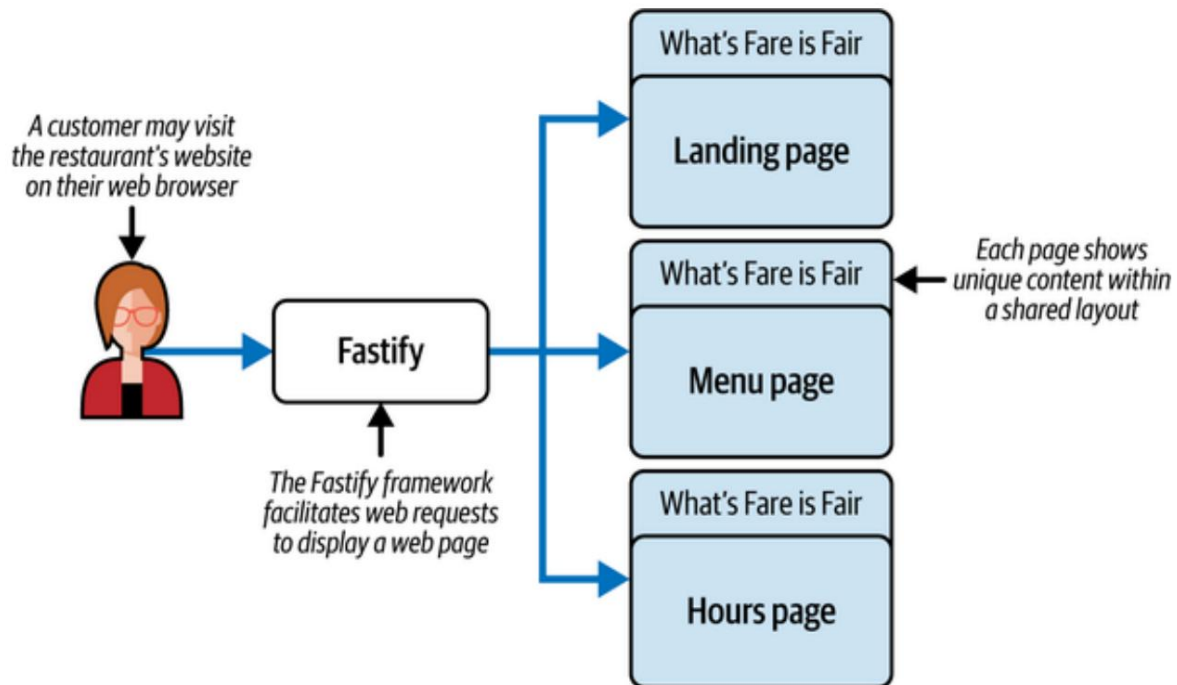
Antes de comenzar, asegúrese de instalar y configurar las herramientas y aplicaciones necesarias para este proyecto. Las instrucciones de instalación de Node.js, Fastify y VS Code se encuentran en **el Capítulo 1**, mientras que los pasos de inicialización del proyecto, como la configuración de la estructura de directorios, la configuración de package.json y el uso de sintaxis moderna, se describen en **el Apéndice A**. Una vez completado, regrese aquí para continuar. Desarrollar un proyecto desde cero le ayuda a comprender mejor cada componente, brindándole mayor control y flexibilidad a medida que avanza.

Los **dueños** de un

restaurante local, What's Fare is Fair, han decidido invertir en una aplicación web propia para ofrecer un mejor servicio a sus clientes. Se han puesto en contacto contigo, un ingeniero de software entusiasta, para que les ayudes a crear una aplicación ligera que pueda mostrar su página de inicio, su menú estático y su horario de atención.

Tras una cuidadosa

reflexión, se da cuenta de que esta empresa solo necesita un servidor web sencillo para ofrecer contenido estático a sus clientes. En este caso, solo necesita dar soporte a tres páginas web. Como ingeniero de Node con experiencia, sabe que puede usar el módulo http integrado, pero le resulta más flexible trabajar con una biblioteca externa popular llamada Fastify. Antes de empezar a programar, diagrama los requisitos del proyecto y el resultado, como se muestra en **la Figura 3-1**.



Cifra 31. Plano del proyecto

UNA PALABRA SOBRE LOS SERVIDORES WEB

El entorno de ejecución de Node es lo suficientemente versátil como para crear aplicaciones que se comunican a través de Internet utilizando estándares protocolos, siendo HTTP el más común. Si bien el módulo http integrado de Node permite crear servidores web, configurar incluso...

Un sitio web básico puede requerir código y configuración importantes.

Para agilizar este proceso, los marcos como Fastify proporcionan una solución más completa. Fastify no solo simplifica la

Implementación de http para manejar solicitudes web y respuestas, sino que también introduce un enfoque estructurado para organizar los archivos de su aplicación, integrando aplicaciones de terceros paquetes y crear aplicaciones web de forma más eficiente. De hecho,

Muchos otros marcos de Node se basan en Fastify como base

Para obtener más información sobre Fastify, consulte sus herramientas y funciones adicionales. y sus capacidades, visite el [sitio web de Fastify](#).

A medida que los clientes visitan el sitio web del restaurante, su aplicación Fastify los dirigirá eficientemente a las páginas solicitadas. Gracias al bucle de eventos de un solo subproceso de Node, Fastify gestiona las solicitudes entrantes de forma asíncrona. Cada solicitud representa el intento de un cliente de acceder a una página específica mediante una URL. A pesar de funcionar en un solo subproceso, la arquitectura sin bloqueo de Node le permite gestionar múltiples solicitudes simultáneamente, delegando tareas (como consultas a bases de datos o lecturas de archivos) en segundo plano. Esto significa que su aplicación puede procesar rápidamente la solicitud de cada cliente sin interrupciones, siempre que ninguna operación pesada que consuma mucha CPU (como calcular el número 50 de Fibonacci) bloquee el bucle de eventos. Gracias a este enfoque, Fastify garantiza un rendimiento rápido y escalable incluso bajo carga. Visualice cómo el bucle de eventos de Node gestiona y prioriza las s

BLOQUEO DEL BUCLE DE EVENTOS

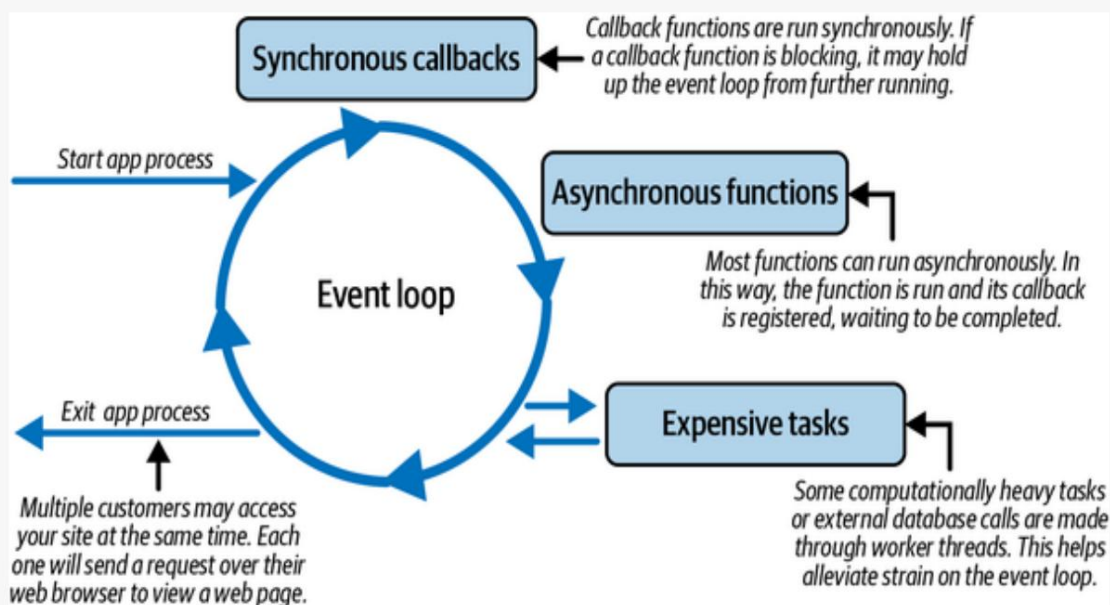
El bucle de eventos de Node es el corazón de cómo funciona cada aplicación de Node.

se ejecuta. Dado que JavaScript en Node opera en un solo hilo, es

Es crucial permitir que este hilo maneje tantas tareas como sea posible.

Sin demoras. La Figura 3-2 ilustra algunas formas comunes en que

El bucle de eventos se puede bloquear.



Cifra 32. Cómo bloquear el bucle de eventos

Aunque Node utiliza un solo hilo, puede crear hilos adicionales.

subprocesos de un "grupo de subprocesos", a menudo denominados "subprocesos de trabajo".

A estos subprocesos de trabajo generalmente se les asignan tareas que consumen más recursos, como operaciones del sistema de archivos y consultas de bases de datos.

(E/S) o funciones criptográficas. El propósito de estos trabajadores

Los subprocesos sirven para descargar tareas pesadas del bucle de eventos principal.

Permitiéndole seguir respondiendo. Sin embargo, si el bucle de eventos en sí es...

ocupado manejando tareas complejas o de ejecución lenta, su aplicación

El rendimiento puede verse afectado.

El hilo principal es responsable de coordinar el trabajo asíncronico.

tareas y procesar sus devoluciones de llamadas cuando estén listas.

Sin embargo, si esas devoluciones de llamadas incluyen operaciones como bucles anidados,

Al procesar grandes conjuntos de datos u otros cálculos que requieren un uso intensivo de la CPU, el bucle de eventos se bloquea. Esto significa que no puede procesar otras tareas entrantes hasta que finalice la actual, lo que provoca respuestas más lentas.

En el contexto de un servidor web, cada ciclo del bucle de eventos corresponde a la gestión de las solicitudes de los clientes. Si el bucle de eventos se bloquea por la solicitud de un cliente (como generar una página web con procesamiento complejo), los demás clientes tendrán que esperar hasta que se complete la tarea. Al construir servidores web, es importante diferenciar entre las tareas que se ejecutan en tiempo constante (operaciones rápidas) y las que se ejecutan en tiempo exponencial (como bucles anidados o algoritmos con un uso intensivo de la CPU) y que podrían bloquear el bucle de eventos.

Para obtener más información, visite el [sitio web de Node](#).

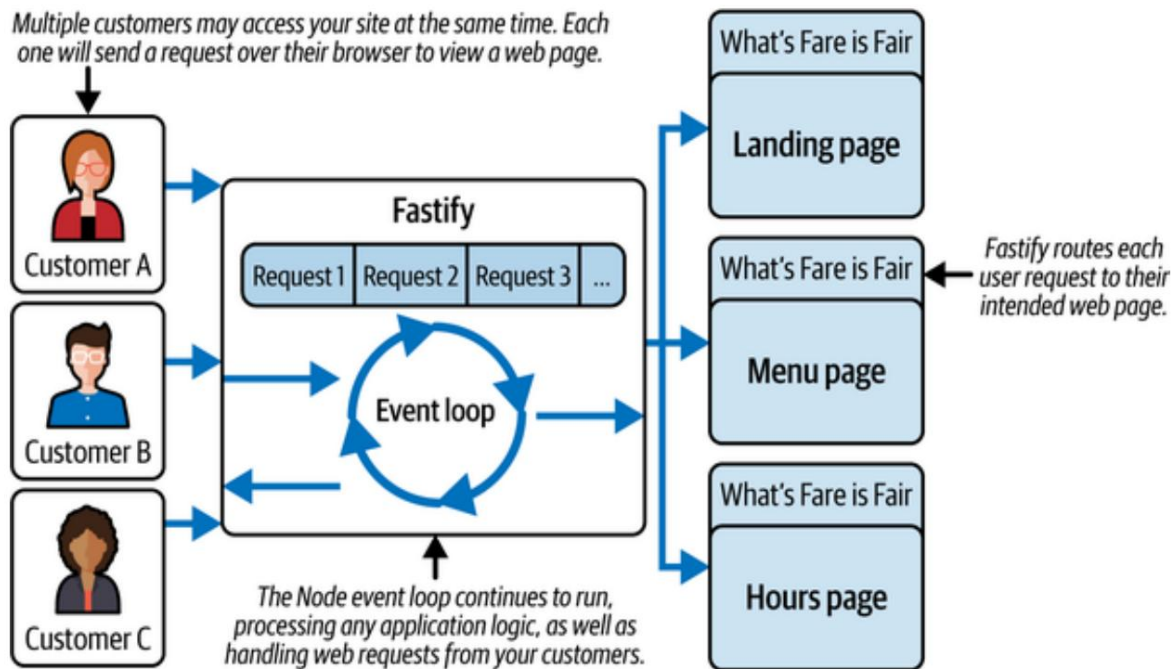
La **Figura 3-3** ilustra cómo se procesan las solicitudes web en su aplicación web con Fastify. Al igual que en **la Figura 3-1**, los clientes acceden al sitio web del restaurante sin ninguna previsibilidad en cuanto al número de visitantes ni la frecuencia con la que llegan las solicitudes. Sin embargo, cada solicitud se procesa individualmente al entrar en la aplicación. El bucle de eventos de Node encola eficientemente las solicitudes entrantes, asignando respuestas y gestionando simultáneamente las nuevas.

Gracias al sistema de colas, el bucle de eventos gestiona cada solicitud en cuanto los recursos están disponibles. Por ejemplo, cuando un cliente solicita el menú del restaurante, Fastify lo redirige a una página web estática que muestra el horario de atención, lo que garantiza que todas las solicitudes se gestionen sin problemas, incluso con cargas variables.

NOTA

Para obtener más información sobre el bucle de eventos de Node, consulte la [página de documentación oficial](#) o [las páginas de referencia de JavaScript de Mozilla](#).

Una vez que se tiene el panorama general, es hora de comenzar a programar.



Cifra 33. Flujo de trabajo de desarrollo

Construyendo el esqueleto de la aplicación

Si bien este proyecto puede ser relativamente sencillo, a menudo resulta útil dividir las tareas en segmentos más manejables. Para empezar, crear un esqueleto de aplicación que contenga el marco Fastify y parte de la lógica de su aplicación.

Trabajar con Fastify

Inicialice una nueva aplicación Node creando una nueva carpeta llamada `restaurant_web_server`, ingresando a la carpeta en su comando línea y ejecutando `npm init` (Ejemplo 3-1).

Ejemplo 31. Solicitud de `npm init`

Nombre del paquete: (restaurant_web_server) **1**
 versión: (1.0.0)

Descripción: Una aplicación web para un restaurante local.
 punto de entrada: (index.js) **2**

Comando de prueba:

repositorio git: palabras

clave: autor:

Jon Wexler licencia: (ISC)

- ❶ Este es el nombre de tu proyecto.
- ❷ index.js es donde comenzará tu aplicación.

Tras ejecutar el mensaje de solicitud, el archivo package.json aparecerá en la carpeta del proyecto. Este archivo contiene tanto la configuración general de la aplicación como los módulos dependientes.

A continuación, ejecute `npm install fastify@^4.28.1` para instalar el paquete fastify . Necesitará una conexión a internet, ya que al ejecutar este comando se obtendrá el contenido del paquete fastify del **registro de npm**. y añádelos a la carpeta node_modules en la raíz de tu proyecto. A diferencia de los módulos fs y http preinstalados con Node, el módulo fastify no se incluye en la instalación inicial. En su lugar, Fastify se incluye en un paquete llamado fastify que puede descargarse e instalarse por separado a través de npm, la herramienta de registro de gestión de paquetes de Node.

NOTA

Tanto Node como Fastify son proyectos respaldados por la Fundación OpenJS. Para más información sobre proyectos JavaScript de código abierto de OpenJS, visite el **sitio web de la Fundación OpenJS**.

Después de instalar Fastify , notarás que Fastify se agrega a tu archivo paquete json en una sección llamada dependencias, como se ve en el **Ejemplo 3-2**.

NOTA

Hay una variedad de formas de escribir comandos npm, algunas más cortas y Otros son más explícitos en su redacción. Para saber más sobre NPM Para abreviaturas y banderas de la línea de comandos, consulte la [documentación de npm](#).

Ejemplo de dependencias de paquetes en package.json

```
"dependencies": {  
  "fastify": "^4.28.1"  
},
```

1

fastify v^4.28.1 aparece como la única dependencia.

NOTA

El uso de ^ en el control de versiones de paquetes npm significa que su aplicación...
Asegúrese de que esta versión, o cualquier versión compatible del paquete con
Se instalarán actualizaciones menores o parches en su aplicación. El ~ antes
El número de versión significa que solo se instalarán actualizaciones de parches, pero no
Cambios menores en la versión. Para más información sobre packagejson y cómo...
El control de versiones funciona, consulte la [documentación de npm](#).

Con fastify instalado, crea un archivo llamado index.js en la raíz nivel de la carpeta de tu proyecto. A continuación, importa Fastify a tu aplicación en la primera línea agregando import Fastify from 'fastify'.

NOTA

A partir de Node v12, las importaciones de módulos ES6 se admiten de forma nativa, pero son No está habilitado por defecto. Debe agregar "type": "module" a su archivo packagejson para usar la sintaxis de importación . Alternativamente, si no lo hace Si desea modificar packagejson, puede usar la extensión de archivo mjs para archivos individuales

Como se muestra en el **Ejemplo 3-3**, luego escribe `const app = Fastify()` para crear una nueva instancia de una aplicación Fastify y asignarla a una variable llamada `aplicación`. También se le asigna otra variable llamada `puerto` . Número de puerto de desarrollo 3000.

NOTA

En desarrollo, puedes usar casi cualquier número de puerto para probar tu código. Los puertos como el 80, el 443 y el 22 suelen estar reservados para fines específicos: el 80 para tráfico web normal, 443 para tráfico web seguro (SSL) y 22 para SSH Conexiones. Sin embargo, el puerto 3000 se ha convertido en una opción predeterminada popular entre ingenieros de software para el desarrollo.

Su objeto de aplicación tiene funciones para gestionar las solicitudes web entrantes. Agregue `app.get` en `"/"`, que escucha las solicitudes HTTP GET a su Página de inicio de la aplicación web. La función de devolución de llamada `app.get` procesa el solicitud y devuelve directamente una respuesta de texto sin formato usando "Bienvenido" ¡Lo que es justo es justo!". Esta respuesta se envía de vuelta a la el navegador web del cliente cuando visita la página de inicio de su aplicación. El El código se simplifica mediante el uso de una declaración de retorno .

NOTA

El archivo `app.get` de Fastify se nombra según el tipo de solicitud HTTP. Las solicitudes más comunes son GET, POST, PUT y DELETE. Para familiarizarse con estos métodos de solicitud, consulte las páginas de referencia [de JavaScript de Mozilla](#).

Después de definir las rutas, se llama `await app.listen({ port })` para iniciar el servidor. Una vez en ejecución, se registra un mensaje de confirmación mediante `console.log(...)`.

Ejemplo 33. Configurando su aplicación Fastify en `index.js`

```
de importación Fastify desde "fastify"; ❶
const app = Fastify(); const ❷
port = 3000; ❸

app.get("/", async (solicitud, respuesta) => { ❹
  devuelve "¡Bienvenido a What's Fare is Fair!"; }); ❺

await app.listen({ puerto }); ❻
console.log(` El servidor web está escuchando en
http://localhost:${puerto} `); Importe el ❼
```

- ❶ módulo Fastify para crear un servidor.
- ❷ Cree una instancia de Fastify, que actúa como su servidor web.
- ❸ Defina el puerto en el que se ejecutará la aplicación (predeterminado 3000).
- ❹ Registra una ruta que escuche solicitudes HTTP GET en la URL raíz ("/").
- ❺ Responda con un mensaje de texto simple utilizando la declaración de retorno .
- ❻

Inicie el servidor y vincúlelo al puerto definido usando `await app.listen({ port })`.

- 7 Registre un mensaje de confirmación en la consola, incluida la dirección del servidor, una vez que se ejecute correctamente.

CONSEJO

Si no utiliza `process.exit(1)` cuando se produce un error durante el inicio, su aplicación podría seguir ejecutándose sin funcionar correctamente, lo que podría causar un comportamiento impredecible. Salir del proceso garantiza que el error se detecte a tiempo y se gestione adecuadamente.

Una vez que su aplicación se inicia correctamente, Fastify se encarga de gestionar las solicitudes entrantes y el envío de respuestas. Comprender cómo Fastify estructura su ciclo de solicitud-respuesta es clave para crear rutas eficientes y fiables.

NOTA

En Fastify, se gestionan las solicitudes mediante "solicitud" y se envían las respuestas mediante "respuesta". La función está marcada como asíncrona para funcionar con el enrutamiento basado en promesas de Fastify. Fastify serializa automáticamente el valor devuelto como el cuerpo de la respuesta. Con el objeto "respuesta", se pueden configurar encabezados, códigos de estado e incluso transmitir respuestas.

Ahora, puede iniciar su aplicación ejecutando "node index" en la ventana de línea de comandos de su proyecto. Debería ver una declaración registrada que indica: " El servidor web está escuchando en `http://localhost:3000`".

NOTA

localhost es un nombre de host especial que hace referencia a su propia computadora. Se utiliza para pruebas y desarrollo sin enviar datos a través de Internet. Está asignado a direcciones IP reservadas como 127.0.0.1 (IPv4) y ::1 (IPv6) a través del archivo de hosts del sistema, evitando las búsquedas DNS. Cuando se utiliza localhost, su computadora envía la solicitud de vuelta a sí misma a través de un adaptador de red virtual llamado loopback.

Esto significa que puedes abrir tu navegador web favorito y visitar `http://localhost:3000` para ver el texto en la Figura 3-4.

Your web browser's address bar should point to localhost:3000 or https://127.0.0.1:3000



The home page displays a plain-text welcome message

Cifra 34. Visualización de la respuesta de su servidor web en su navegador web

Una vez que hayas decidido la base de tu aplicación, es hora de agregar Un poco de estilo al sitio What's Fare is Fair.

Agregar rutas y datos

Con su aplicación en ejecución, continúe para agregar más rutas y Contexto del sitio web de tu restaurante. Ya agregaste una ruta: GET solicitud a la página de inicio (/). Ahora puedes agregar dos rutas más para la página del menú y la página de horas de funcionamiento, como se muestra en el Ejemplo 3-4. Cada `app.get` proporciona una nueva ruta en la que se cargan sus páginas web. accesible.

Ejemplo 34. Añadiendo dos rutas más en `index.js`

```
app.get("/menu", async (solicitud, respuesta) => {
  devolver "TODO: Página del menú";
});
```

❶

```
app.get("/horas", async (solicitud, respuesta) => {
  devolver "TODO: Página de horas";
});
```

❷

❶ Una ruta Fastify para solicitudes GET a la ruta /menu

❷ Una ruta Fastify para solicitudes GET a la ruta /horas

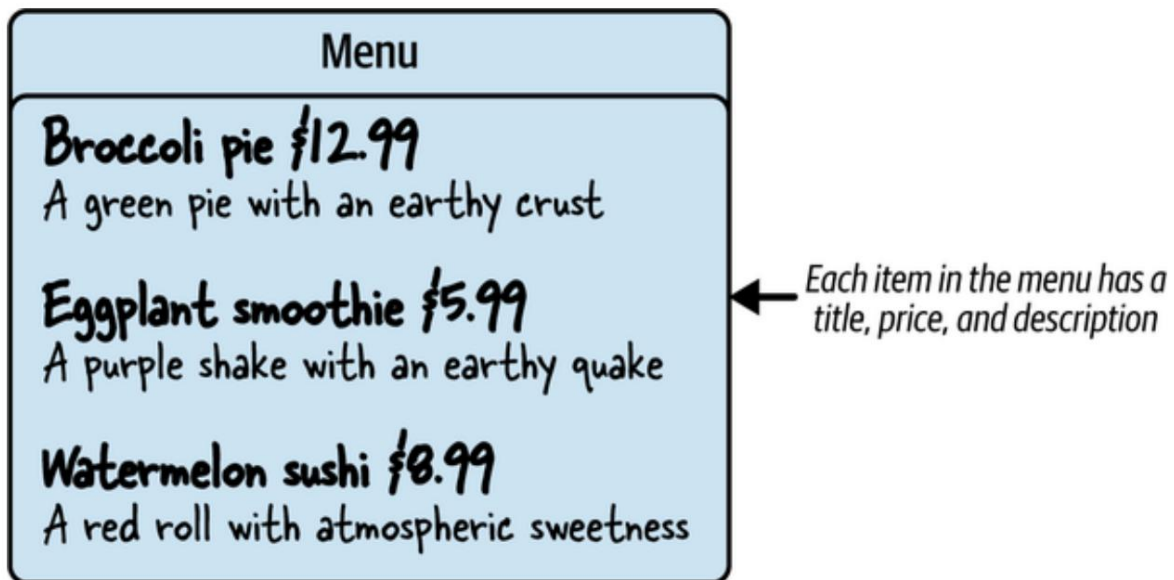
Puede detener su servidor Node presionando Ctrl+C en el comando línea de su proyecto en ejecución. Con los nuevos cambios implementados, puede Reinicie su proyecto ejecutando node index. Ahora, cuando...

navegue hasta `http://host:3000/`

`mensajes` `:3000/ horas`, verás que el texto cambia a TODO

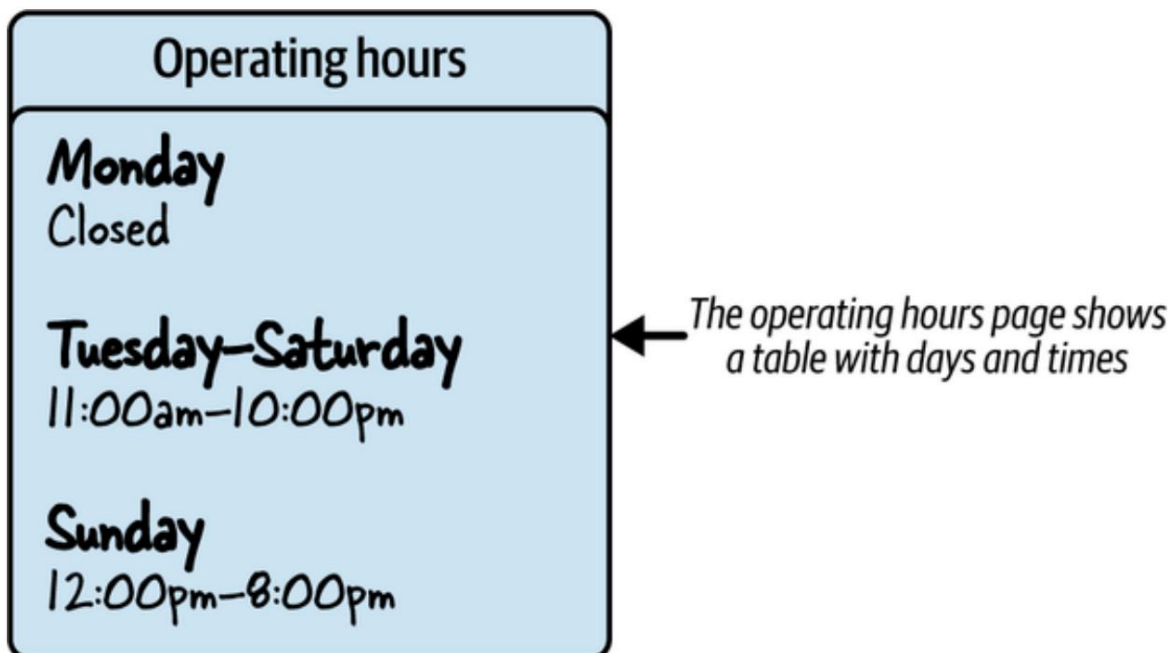
`httplocalhost` .

Este es un buen comienzo, pero necesitarás completar algunos datos significativos. aquí. Su contacto en What's Fare is Fair le proporciona fotografías de Su menú (Figura 3-5). Esta imagen proporciona una idea de la estructura de los datos del menú del restaurante. Por ejemplo, cada artículo tiene un Título, precio y descripción.



Cifra 35. Elementos del menú

De igual forma, el restaurante ofrece una visualización de su horario de funcionamiento, como se muestra en la [Figura 3-6](#). Aquí, se observa que ciertos días comparten el mismo horario de funcionamiento, mientras que un día tiene horarios diferentes y uno Día en que el restaurante está cerrado. Poder examinar esta información Antes de crear sus páginas web puede ayudarle a diseñar su proyecto en una manera eficiente



Cifra 36. Horario de atención

Con estas dos referencias de datos, puedes convertir el menú y lista de horas en módulos de datos compatibles con JavaScript. Primero, crea un carpeta llamada `datos` en el directorio de su proyecto, donde agregará un `menuItems.js` y un archivo `OperatingHours.js`. Desde estos archivos, se utiliza la sintaxis de exportación predeterminada de ES6 para exportar todo el contenido de los archivos para su uso en otros módulos, como se muestra en los Ejemplos 3-5 y 3-6.

Ejemplo de datos de menú en `menuItems.js`

```
exportación predeterminada [ ❶
{
  nombre: "Pastel de brócoli",
  Descripción: "Un pastel verde con una corteza terrosa",
  costo: 12,99,
},
{
  Nombre: "Batido de berenjena",
  Descripción: "Un batido morado con un temblor terroso",
  costo: 5,99,
},
{
  Nombre: "Sushi de sandía",
  Descripción: "Un panecillo rojo con una dulzura atmosférica",
  costo: 8,99,
},
];
```

❶ Exportar una serie de elementos de menú para utilizarlos en otras partes del proyecto.

Transformando los datos de un menú físico a este formato digital tipo JSON

La estructura le permitirá mostrar sistemáticamente lo relevante de forma más sencilla.

Los platos del menú se ofrecen a los clientes del restaurante. Debido al horario de atención son los mismos todos los días excepto lunes y domingo, lo normal

Las horas se pueden enumerar como valores predeterminados en `OperatingHours.js`.

Ejemplo de datos de horas en `OperatingHours.js`

```
exportar predeterminado { ❶
  Horas predeterminadas: {
    abierto: 11,
    cerrado: 22,
```

```

    },
    lunes: {
      abierto: nulo,
      cerrado: nulo,
    },
    domingo: {
      abierto: 12,
      cerrado: 20,
    },
  },
};

```

- ❶ Exportar un objeto con horas de funcionamiento y valores predeterminados para casos especiales.

Para utilizar estos datos, importe los dos módulos relativos en el parte superior de `index.js` (Ejemplo 3-7).

Ejemplo 37. Importación de módulos de datos a `index.js`

```

de importación de OperatingHours desde "../data/operatingHours.js";
importar menuItems desde "../data/menuItems.js";

```

- ❶ Importar módulos personalizados desde su directorio de datos relativo.

Para probar que estos valores se están cargando correctamente, reemplace el declaraciones de retorno en las rutas `/menu` y `/hours` con

`responder.enviar(menuItems)` y

`reply.send(operatingHours)`, respectivamente. Al hacerlo, debería

Reemplace el texto estático que vio anteriormente al cargar su web

Página con datos más significativos proporcionados por el restaurante. Este paso

En el proceso le ayuda a validar que los datos que espera son

fluyendo adecuadamente hacia las páginas web a las que desea llegar.

Reinicie su servidor Node y visite `http://localhost/menu` y `:3000/`

`http://localhost/hours:3000/`. Su resultado para la página del menú en su

El navegador web debería verse como la Figura 3-7.

```
[{"name":"Broccoli Pie","description":"A green pie with an earthy crust","cost":12.99}, {"name":"Eggplant Smoothie","description":"A purple shake with an earthy quake","cost":5.99}, {"name":"Watermelon Sushi","description":"A red roll with atmospheric sweetness","cost":8.99}]
```

← Your menu data displays in JSON format

Cifra 37. Datos del menú

Con estos datos mostrados en el navegador, el siguiente paso es formatearlo para que sea más atractivo visualmente.

Creando tu interfaz de

usuario. Puedes crear una interfaz de usuario con un framework frontend como React.js, Vue.js o Angular.js. Sin embargo, para simplificar la aplicación, configurarás la renderización del lado del servidor (SSR) mediante plantillas de JavaScript incrustado (EJS) con Fastify.

NOTA

Existen diversos motores de plantillas, como EJS y Pug, que funcionan bien con Node y Fastify. Consulta los [sitios web de EJS](#) y [Pug](#) para obtener más información sobre estas herramientas.

En la línea de comandos, dirígete a la carpeta de tu proyecto y ejecuta `npm install @fastify/view@^9.1.0 ejs@3.1.6`. Esto instala el paquete `ejs`, que facilita la conversión de contenido HTML con datos dinámicos a páginas HTML estáticas, y el complemento de Fastify `@fastify/view`, que te permite usar EJS como motor de plantillas en tu proyecto Fastify.

NOTA

El complemento `@fastify/view` permite usar diversos motores de plantillas con Fastify. Este paquete es un complemento independiente de Fastify para mantener la filosofía principal del framework: ser ligero, modular y de alto rendimiento. Para más información sobre cómo usar este complemento, visite el [sitio web de Fastify](#).

A continuación, configure Fastify para usar el motor de plantillas `ejs` (**Ejemplo 3-8**). Una vez configurado, puede usar el método `reply.view` de Fastify para renderizar páginas con plantillas HTML y EJS.

Ejemplo 38. Actualización de rutas de Fastify para renderizar archivos EJS en `index.js`

```
importar ejs desde 'ejs';           ❶
importar fastifyView desde '@fastify/view';
...
app.register(fastifyView, { motor: ❷
  { ejs:
    ejs, }, });

app.get("/", (req, reply) =>
  { reply.view("views/index.ejs", { name: "Lo que es justo es justo" }); }); ❸

app.get("/menu", (req, responder) => ❹
  { responder.view("views/menu.ejs", { menuItems }); });

app.get('/horas', (req, responder) => { const ❺
  días = [ "lunes",
    "martes",
    "miércoles",
    "jueves",
    "viernes",
    "sábado",
```



```

    "domingo",
  ];
  responder.view("vistas/horas.ejs", { horasoperativas, dias });
});

```

```

app.listen({ puerto: 3000 }, (err, dirección) => {
  si (err) lanzar err;
  console.log(`Servidor ejecutándose en ${address}`);
});

```

- ❶ Importe el motor de plantillas EJS y el complemento de visualización Fastify.
- ❷ Establezca EJS como su motor de plantillas.
- ❸ Utilice `reply.view` para mostrar la página `indexejs` en sus páginas carpeta.
- ❹ Utilice `reply.view` para mostrar la página `menuejs` en sus páginas carpeta, pasando datos de `menuItems`.
- ❺ Utilice `reply.view` para mostrar la página `hoursejs` en sus páginas Carpeta que pasa datos de horas y días de funcionamiento.

Para representar correctamente estas páginas, debe crear una carpeta llamada `vistas` en el nivel raíz de su proyecto y agregue tres nuevos archivos: `indexejs`, `menuejs` y `hoursejs`. Estos archivos se ubicarán en el Motor de plantillas EJS en Fastify cuando se realiza una solicitud al Ruta correspondiente. Para completar el proceso, puede completar estos archivos con una combinación de HTML y EJS. El ejemplo 3-9 muestra un ejemplo de su página de destino, `indexejs`.

Ejemplo 39. Contenido de la página de destino en `index.ejs`

```

<!DOCTYPE html> ❶
<html lang="es">

<cabeza>
  <meta charset="UTF-8">
  <title>Restaurante</title>
</cabeza>

```

```
< cuerpo >
  Bienvenido a <%= name %></h1>
</ cuerpo >
```

```
</html>
```

- ❶ Estructura básica de HTML5 con el contenido principal en la etiqueta del cuerpo
- ❷ Texto para mostrar los datos del nombre dinámico de su archivo index.js

NOTA

EJS usa `<%= %>` para mostrar contenido dentro del HTML. En el **Ejemplo 3-10** Utilice esta sintaxis para mostrar el nombre de la empresa. Si desea ejecutar JavaScript en la página sin imprimir nada, omita el `=`.

Cuando reinicie su proyecto y visite el navegador `http://host local :3000`, su debería verse como la **Figura 3-8**.

Welcome to What's Fare is Fair

← Notice the slight change in font and presentation

Cifra 38. Representación del navegador de index.js

Continúa agregando la misma estructura HTML a `menu.js` y `menu.ejs` modificando solo las etiquetas del cuerpo de cada uno, su

Contiene un bucle `for` que itera sobre cada elemento del menú. Desde allí, mostrar el nombre, la descripción y el costo de cada artículo (**Ejemplo 3-10**).

Ejemplo 310. Contenido de la página del menú en `menu.ejs`

Nuestro Menú

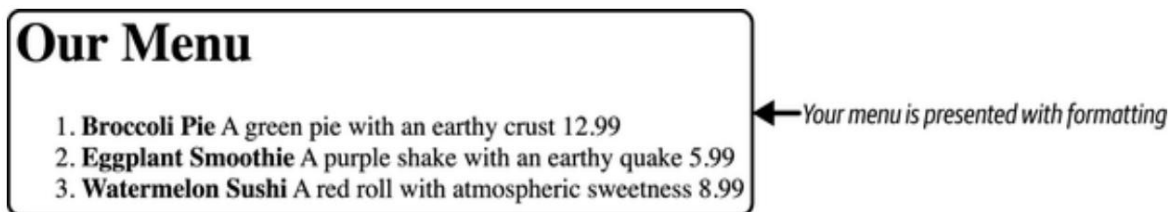
```
<ol>
  <% para (dejar elemento de menuitems) { %>
    <li>
      <strong><%= nombre_del_artículo %></strong>
      <span><%= artículo.descripcion %>
      <span><%= artículo.costos %>
    </li>
```

```
<% }%>
```

```
</ol>
```

- ❶ Recorra los valores de los elementos del menú y asigne a cada objeto un valor elemento con cada iteración.
- ❷ Mostrar el elemento del menú en negrita.
- ❸ Muestra la descripción y el costo junto al nombre.

Con este código en su lugar, reinicie su servidor de nodo y navegue a <http://localhost:3000/> en su navegador web para ver una página que parece como la Figura 3-9.



Cifra 39. Representación del menú ejs en el navegador

De manera similar, en `hours.ejs`, itera sobre cada día en tu matriz de días .

Desde allí, determina si tienes datos para ese día.

mostrar, o simplemente usar los valores predeterminados previamente definidos. Se utiliza una condición `if-else` aquí en EJS para separar la UI para los días que Tiene horario para exhibir y el lunes, cuando el restaurante está cerrado. (Ejemplo 3-11).

Ejemplo 311. Contenido de la página de horas en `hours.ejs`

Nuestro horario

```

❶ <% for(let día de días) { %>
❷   <% const hoursObj = horasOperando[día] ||
horasdeoperación['horaspredeterminadas'] %>
❸   <section style="display: inline-flex; flex-direction:
column; relleno: 5px;">
❹     <h2><%= día.toUpperCase() %></h2>
❺     <div>
❻       <% if (hoursObj.abierto) {%>
Abierto: <%= hoursObj.open %></p>

```

```
        Cerrado: <%= hoursObj.closed %></p> <% } else  
{%>  
        <p>CERRADO</p>  
    }  
    p> <% }  
%> </div>  
</section>
```

- ❶ <% }%> Mostrar el nombre de la página.
- ❷ Recorrer los días, asignando cada valor al día en cada iteración.
- ❸ Define una variable hoursObj para contener los datos de las horas de apertura y cierre.
- ❹ Muestra el nombre del día completamente en mayúsculas.
- ❺ Comprueba si el día dispone de datos de horario de apertura.
- ❻ Mostrar el horario de apertura y cierre.
- ❼ Mostrar el mensaje cuando se cierra.

Con esta última página completa, reinicia tu servidor de nodo y navega a <http://localhost:3000/hours> en tu navegador web para ver una página que se parece a la Figura 3-10.

Our Hours					
MONDAY	TUESDAY	WEDNESDAY	THURSDAY	FRIDAY	SATURDAY
CLOSED	Open: 11	Open: 11	Open: 11	Open: 11	Open: 11
	Closed: 22	Closed: 22	Closed: 22	Closed: 22	Closed: 22
SUNDAY					
Open: 12					
Closed: 20					

Operating hours display in your browser as a structured table

Cifra 310. Representación de hours en el navegador

Es cierto que estas páginas no son visualmente atractivas, aunque...
 Mostrar la información necesaria para el restaurante. Esta es la etapa donde podría considerar mejorar la interfaz de usuario con HTML, CSS y JavaScript del lado del cliente. Si bien estas adiciones están fuera del alcance de un Proyecto Fastify del lado del servidor, puedes integrarlas fácilmente. Para aprender...
 Más información sobre cómo servir activos estáticos como hojas de estilo e imágenes en Fastify, visita el [repositorio de GitHub fastify-static](#). A continuación, explorarás cómo Agregar un poco de estilo básico puede mejorar la apariencia de su sitio web.
 solicitud.

Mejorando la interfaz de usuario

La creación de una aplicación de pila completa generalmente implica trabajar tanto en el frontend y backend. Como ingeniero de Node, tu enfoque principal será A menudo están en el backend. Por lo tanto, no hay un requisito estricto para estar Dominio de HTML o CSS, o sus bibliotecas y frameworks relacionados.
 Lo mejor es dedicar tiempo a desarrollar la interfaz de usuario o asignar el trabajo a Alguien con experiencia en frontend.

Para agregar rápidamente una biblioteca CSS a Fastify, comience creando una biblioteca pública carpeta en la raíz de tu proyecto. Luego, necesitas instalar el `@fastify/static` ejecutando `npm install @fastify/static@^7.0.4` en el nivel raíz de su proyecto en el línea de comandos.

A continuación, registre el complemento `@fastify/static` en su archivo `index.js` para Servir recursos estáticos desde el directorio público ([Ejemplo 3-12](#)). Una vez configurado Arriba, puedes agregar cualquier archivo CSS , imágenes u otro contenido estático a tu carpeta pública y acceda a esos recursos directamente en sus archivos EJS. Luego necesitarás hacer referencia a tus archivos CSS en la etiqueta `<head>` de sus archivos EJS. Por ejemplo, `<link rel="stylesheet" href="public/stylesheets/style.css" />` se vincularía a un `stylecss` en la carpeta `publicstylesheets` /

Ejemplo 312. Cómo agregar activos estáticos a su aplicación Fastify en `index.js`

```
...
import fastifyStatic desde '@fastify/static';           ❶
import { join } desde "ruta";                          ❷
constante publicPath = join(proceso.cwd(), "público");  ❸
...

aplicación.register(fastifyStatic, {                   ❹
  raíz: rutaPública,                                  ❺
  prefijo: '/público/',                                ❻
});
...

```

❶ Importe el complemento `@fastify/static` , que se utiliza para servir archivos estáticos como CSS, imágenes y otros activos en Fastify.

❷ Importar unión desde la ruta, utilizada para construir una plataforma cruzada Ruta de archivo compatible con la carpeta pública .

❸ Crea una ruta absoluta al directorio público en la raíz de tu proyecto.

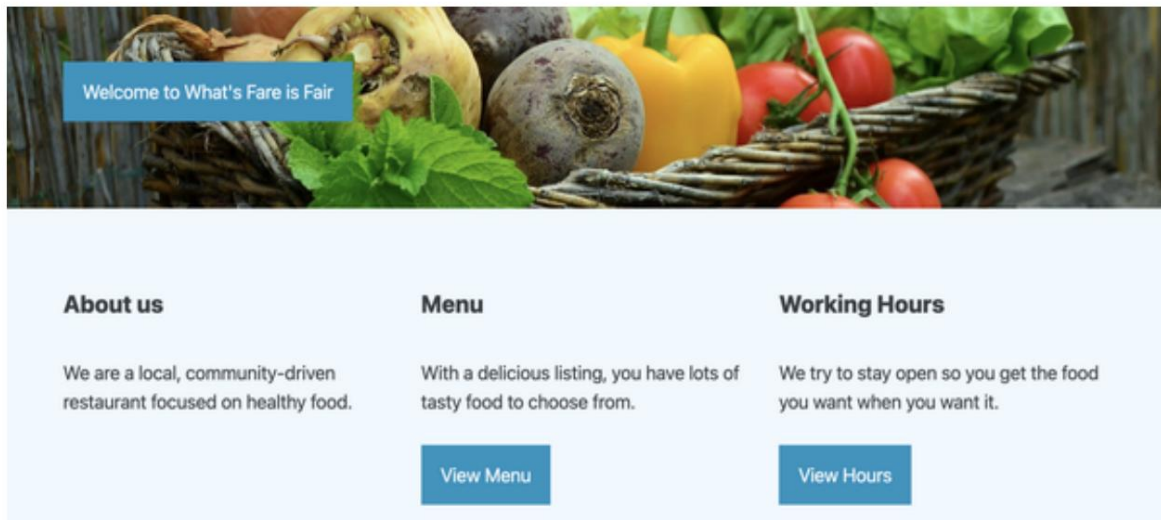
❹

Registre el complemento fastifyStatic para habilitar la entrega de mensajes estáticos archivos.

- 5 Establezca el directorio publicPath desde el que se guardarán los archivos estáticos. servido.
- 6 Establezca el prefijo de URL para archivos estáticos en /public/.

Mire las figuras 3-11, 3-12 y 3-13 para ver cómo se suma

Las hojas de estilo pueden mejorar la estética de las páginas que usted crea.



Cifra 311. Representación del navegador de indexejs con estilo

La página de destino se puede diseñar con el diseño que prefieras. Fastify

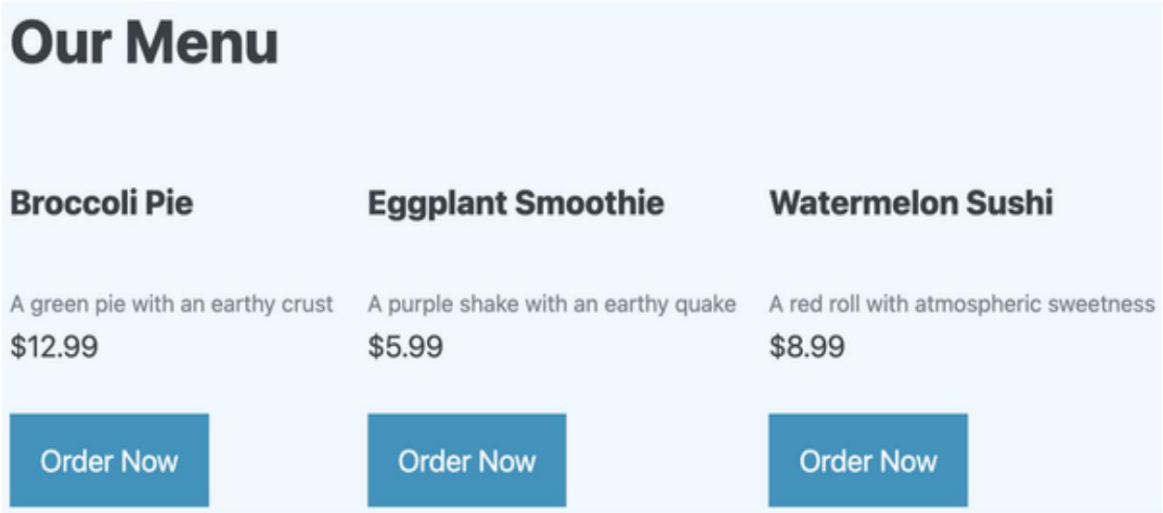
Admite una amplia gama de motores de plantillas, lo que permite una creación flexible y

Diseños personalizables. Motores populares como EJS, Pug y

Los manillares se pueden integrar fácilmente con Fastify mediante complementos. Para

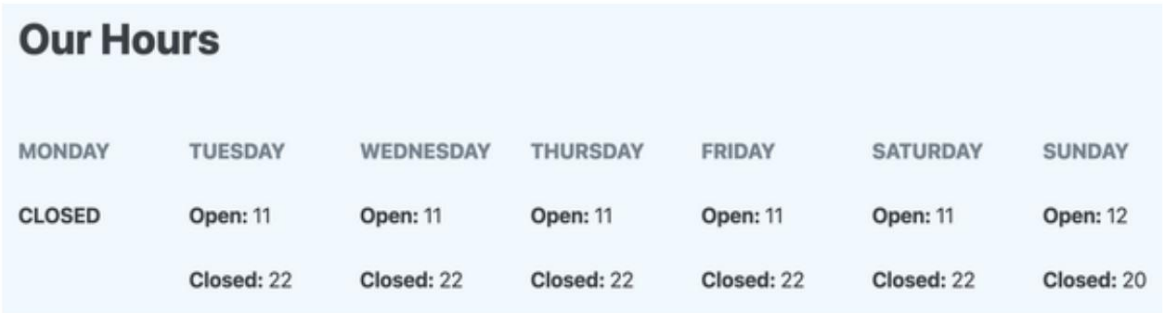
Para obtener más información sobre el uso de motores de plantillas con Fastify, visite el sitio web

[Repositorio de GitHub desde el punto de vista.](#)



Cifra 312. Representación del navegador de menuejs con estilo

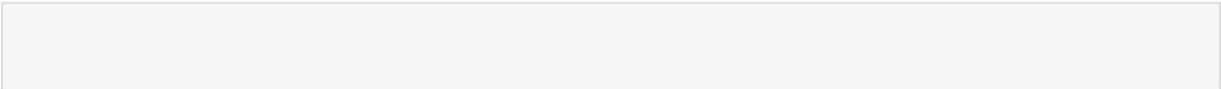
La interfaz de usuario del menú se siente inmediatamente más familiar con una imagen que...
Indica que el cliente puede realizar un pedido en línea. Compras en línea
se puede agregar a una aplicación de Node como esta. Por ahora, es simplemente un elemento visual que...
Puede animar al restaurante a invertir más tiempo para construir un futuro más sostenible.
Aplicación robusta.



Cifra 313. Representación del navegador de hoursejs con estilo

El horario de atención se puede mostrar de diversas maneras. En
En la Figura 3-13, se agrega un estilo de cuadro flexible para garantizar que todos los días y horas estén
espaciados sin superposición.

Si tienes tiempo para dedicarte a mejorar la UI de un sitio web de Node
aplicación, o si puede trabajar junto con un desarrollador frontend, puede encontrar
el resultado final más agradable para su cliente.



EJERCICIOS DEL CAPÍTULO

1. Agrega una nueva página a tu sitio:

- a. Cree un nuevo archivo de plantilla EJS llamado `about.ejs` dentro de su carpeta de vistas .
- b. Agregue un encabezado y un párrafo corto que describa La historia o misión del restaurante.
- c. En su archivo `index.js` , defina una nueva ruta GET (`/about`) que representa la vista `about.ejs` .
- d. Reinicie su servidor Node y navegue hasta `http://localhost:3000/` A punto de confirmar que funciona.

CONSEJO

Reutilice la estructura HTML de sus otros archivos `ejs` para consistencia.

2. Mejora la página de horas con el estado de hoy:

- a. Actualice el controlador de ruta `/hours` para determinar el día actual usando `new Date().getDay()` y asignarlo a su matriz de días .
- b. Pase una variable adicional llamada `today` a su Plantilla EJS.
- c. En la `about.ejs`, resalta el día actual usando un `if` declaración de horas y aplicar una clase o estilo CSS especial Para que destaque visualmente.

Estos ejercicios refuerzan cómo pasar y renderizar datos dinámicos.

Desde su servidor a las plantillas EJS: habilidades clave en la web del lado del servidor desarrollo.

Resumen

En este capítulo, usted:

- Creé un servidor web Fastify y aprendí cómo el bucle de eventos de Node maneja solicitudes asíncronas y evita bloqueos.
- Configuró rutas para servir contenido dinámico y organizó su proyecto utilizando módulos de datos separados.
- Se implementó la representación del lado del servidor con EJS y se mejoró el diseño visual de las páginas web utilizando activos estáticos.
- Complementos Fastify integrados para mejorar la funcionalidad y optimizar su aplicación.

Capítulo 4. Cree un administrador de contraseñas local seguro

Este capítulo cubre lo siguiente:

- Conceptos de hash de datos
- Trabajando con Bcrypt
- Guardar datos en una colección de MongoDB

Desarrollar aplicaciones en el servidor ofrece ventajas inmediatas en comparación con hacerlo en el cliente. Una de ellas es un mayor control sobre la seguridad de los datos.

El ingeniero de servidores suele ser responsable de proteger los datos en la base de datos, determinando qué datos puede ver el cliente y quién puede verlos. Por esta razón, existen numerosos paquetes de hash en el registro de npm que se pueden usar con Node para ocultar datos confidenciales a cualquier persona que no sea su propietario original.

NOTA

Aunque comúnmente se expande como "Node Package Manager", npm no es oficialmente un acrónimo. Sus creadores han declarado que originalmente significaba "npm is not an accord" (npm no es un acrónimo).

En este capítulo, crearás un gestor de contraseñas utilizando el paquete de hash bcrypt y MongoDB para almacenamiento persistente. Empezarás por comprender qué sucede en segundo plano con el hash y cómo puedes usar este mecanismo para crear una herramienta de productividad eficaz.

Más adelante, presentará el almacenamiento de documentos con MongoDB para conservar sus datos en hash para acceso futuro.

HERRAMIENTAS Y APLICACIONES UTILIZADAS EN ESTE CAPÍTULO

Antes de comenzar, asegúrese de instalar y configurar las herramientas y aplicaciones necesarias para este proyecto. Las instrucciones de instalación de Node.js, Fastify y VS Code se encuentran en **el Capítulo 1**, mientras que los pasos de inicialización del proyecto, como la configuración de la estructura de directorios, la configuración de package.json y el uso de sintaxis moderna, se describen en **el Apéndice A**. Las instrucciones de instalación de MongoDB están disponibles en **el Apéndice C**. Una vez completado, regrese aquí para continuar. Desarrollar un proyecto desde cero le ayuda a comprender mejor cada componente, lo que le brinda mayor control y flexibilidad a medida que avanza.

Tu mensaje: Tu

startup de blockchain, Crypto Spies, está creciendo. Con cada nuevo servicio que usa tu empresa, te resulta más difícil controlar tus contraseñas y aún no confías en empresas externas para que las administren. Decides dedicar unas horas a crear tu propio gestor de contraseñas. De esta forma, puedes acceder rápidamente a tus contraseñas desde tu gestor casero, que se ejecuta en tu ordenador.

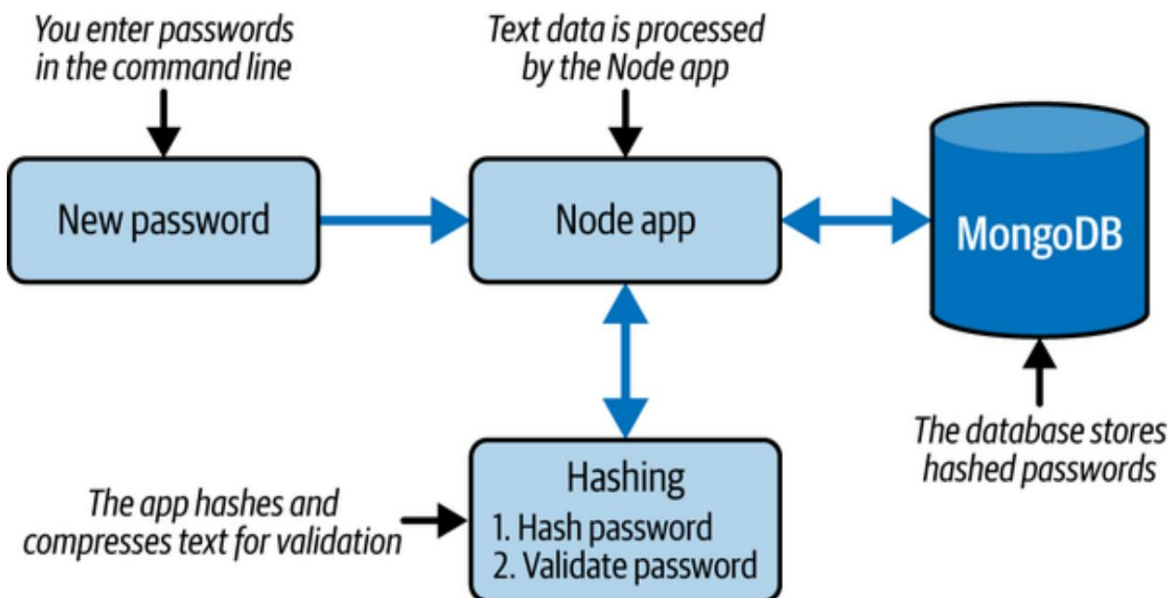
Ha decidido que

quiere que esta aplicación de Node se ejecute en su propio equipo, que solo permita el acceso mediante una contraseña cifrada y que guarde sus contraseñas personales en una base **de** datos. Para que esta aplicación funcione rápidamente, elige una biblioteca de hash existente para cifrar sus contraseñas y MongoDB para almacenarlas.

Antes de comenzar a programar, diagrama los requisitos del proyecto y su resultado.

La **Figura 4-1** muestra el flujo de información de su aplicación completa. Su aplicación almacenará tanto una contraseña maestra en hash como contraseñas en texto plano. Los siguientes pasos detallan cómo debería funcionar la aplicación:

1. Para comenzar, se escribe una contraseña maestra en la línea de comandos y enviado a su aplicación Node.
2. La lógica de su aplicación crea un hash de esa contraseña y la guarda en su base de datos.
3. La próxima vez que acceda a su aplicación, escriba su contraseña maestra, que será validada con su contraseña en hash.
4. Si la contraseña ingresada coincide con su contraseña maestra, puede optar por guardar las contraseñas personales o ver una lista de contraseñas guardadas.



A medida que escribes nuevas contraseñas para guardarlas en tu base de datos, el texto de la contraseña se introduce en tu aplicación Node. A partir de ahí, la lógica de la aplicación crea un hash de tu contraseña y la guarda en tu base de datos. Para recuperar esa lista de contraseñas, debes volver a escribir una contraseña maestra que solo tú conozcas.

Ahora es el momento de empezar a codificar.

Creación de un gestor de línea de comandos local. Es recomendable crear una versión simple de la aplicación e ir añadiendo funciones gradualmente. Para la primera versión, se planea crear una aplicación Node con la lógica para generar un hash de la contraseña principal y almacenar la lista de contraseñas restantes en memoria (esto significa que la lista se elimina al cerrar el proceso de la aplicación).

NOTA

Antes de incorporar una base de datos para guardar sus datos a largo plazo, su computadora puede guardar los datos temporalmente en la memoria. Esto significa que los datos de su aplicación solo se conservan mientras esta se ejecuta y la computadora está encendida.

Comience creando una nueva carpeta de proyecto llamada `passwordmanager`. Esta carpeta puede ser la que se creará donde planea guardar proyectos de Node en su computadora. Acceda a esta carpeta en la línea de comandos y ejecute `npm init` para inicializar el proyecto de Node. El mensaje de inicialización debería ser similar al **del Ejemplo 4-1**.

del paquete 41. Solicitud de `npm init` Nombre

de ejemplo : (password_manager) Versión: (1.0.0) ❶

Descripción: Una

aplicación de Node para almacenar contraseñas Punto de entrada:

(index.js) Comando de prueba: ❷

Repositorio git:

Palabras

clave: Autor: Jon Wexler

Licencia: (ISC)

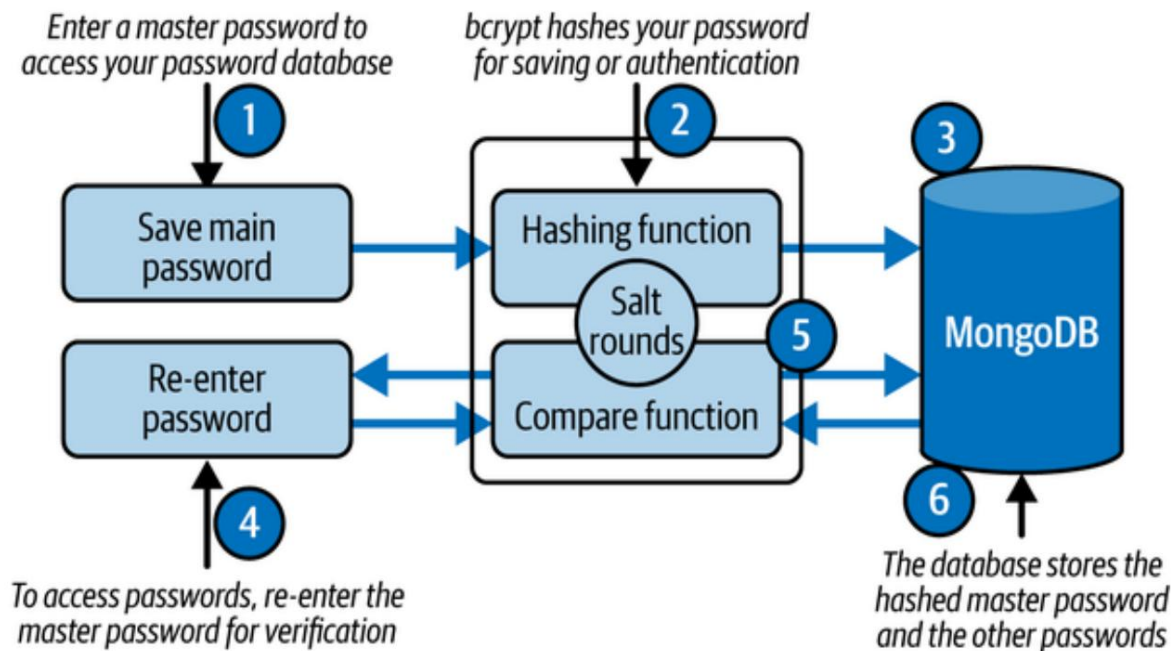
❶ Este es el nombre de tu proyecto.

❷ index.js es donde comenzará su aplicación.

A continuación, ejecute `npm install bcrypt@^5.0.1` para instalar el paquete `bcrypt`. La función `bcrypt.hashSync` es una de las muchas que puede usar para aplicar un hash a su contraseña. Este proceso consta de dos pasos: aplicar un hash a su contraseña y validar una contraseña de texto plano con respecto a la contraseña hasheada. La Figura 4-2 muestra cómo la función `hash` es un procedimiento unidireccional. De esta forma, es muy difícil aplicar ingeniería inversa a la contraseña original a partir del valor hasheado.

Primero, escribe la contraseña principal que usarás para acceder a tus demás contraseñas. La función `hash` de `Bcrypt` utiliza una sal (texto generado aleatoriamente) para mezclar el texto de tu contraseña un número de veces igual al valor de tus rondas de sal. La contraseña hash resultante se almacena en tu base de datos. Más tarde, al volver a escribir tu contraseña para acceder a tu administrador, el texto introducido se vuelve a hash y se compara con el hash de la contraseña almacenada. La función de comparación de `Bcrypt` comparará tu texto plano con la contraseña hash. Si tu contraseña coincide con la contraseña hash de tu base de datos, estás autorizado.

De esta manera, `bcrypt` no revierte un hash de contraseña, sino que vuelve a codificar una contraseña reescrita y compara el resultado con el hash de contraseña en la base de datos.



Cifra 42. Proceso de hash con bcrypt

Ahora, crea tu índice. Aquí archivo js en el nivel raíz del directorio de su proyecto. es donde residirá la mayor parte de la lógica de tu aplicación. Puedes probar Algunas de las funciones de bcrypt agregando el código del [Ejemplo 4-2](#) a index.js.

Ejemplo 42. Probando bcrypt en index.js

```
de importación bcrypt desde "bcrypt"; ❶
constante contraseña = "prueba1234"; ❷
const hash = bcrypt.hashSync(contraseña, 10); ❸
console.log(`Mi contraseña en hash es: ${hash}`); ❹
```

- ❶ Importe el paquete bcrypt .
- ❷ Definir una contraseña de prueba.
- ❸ Utilice la función hashSync para codificar su contraseña, con 10 sal rondas (un factor de costo de $2^{10} = 1024$ iteraciones).
- ❹ Envíe su contraseña en formato hash a su consola.

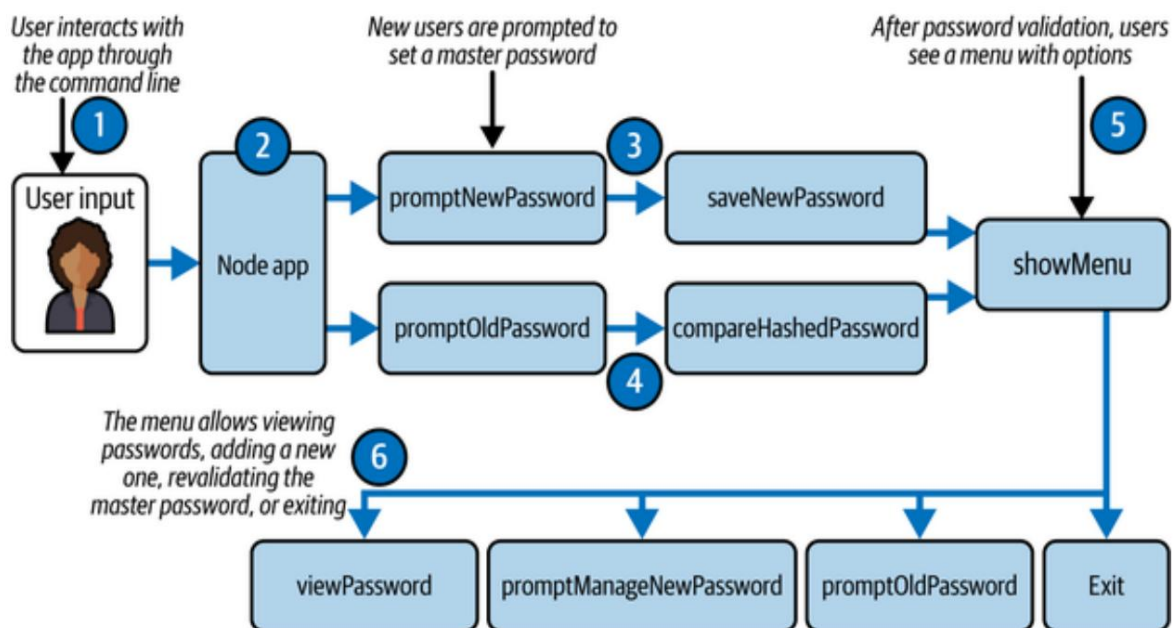
Con este código, puede ejecutar el índice del nodo en la raíz de su proyecto desde la ventana de línea de comandos. El resultado debería ser similar a " Mi contraseña con hash es: \$2b\$10\$/mLO.3etH.eG1.tsFY" (con un valor hash diferente, por supuesto).

NOTA

Si no ve una declaración registrada en su ventana de línea de comandos, verifique que su archivo index.js se haya guardado en el mismo directorio desde el cual está ejecutando la aplicación.

Con el caso de prueba funcionando, se crean las funciones necesarias para facilitar el guardado de una nueva contraseña con hash. La Figura 4-3 muestra el flujo lógico según los nombres de las funciones que se utilizarán. Para comenzar, la aplicación ejecuta una función de solicitud para permitir la interacción del usuario en la línea de comandos. A continuación, se comprueba si ya existe un hash de contraseña maestra almacenado. Si existe, se ejecuta `promptOldPassword` para solicitar al usuario que vuelva a escribir su contraseña. De lo contrario, ejecute `promptNewPassword` para solicitar al usuario que escriba una nueva contraseña maestra por primera vez. Cuando el usuario escriba su nueva contraseña, la función `saveNewPassword` guardará el hash resultante en la base de datos.

Si el usuario escribe su contraseña maestra, se compara su entrada con el hash de la contraseña almacenada mediante `compareHashedPassword`. Si se valida la contraseña, se muestra un menú de opciones mediante la función `showMenu`. En este menú, el usuario puede ver su lista de contraseñas (`viewPasswords`), añadir una nueva contraseña a la lista (`promptManageNewPassword`), volver a verificar su contraseña hash o salir de la aplicación.



Cifra 43. Diagrama de flujo lógico del código

El código para esta lógica se puede escribir una función a la vez. Primero, instale el paquete `prompt-sync` ejecutando `npm install prompt-sync@^4.2.0` en el nivel raíz de su proyecto en su línea de comandos. Luego, agregue las importaciones `bcrypt` y `prompt-sync` a su archivo `index.js`. Además, agregue un objeto JavaScript con contraseñas. clave asignada a un objeto vacío para representar su base de datos. Como usted Agregue nuevas contraseñas para guardar, este objeto se completará. (Ejemplo 4-3).

NOTA

En otros capítulos se utiliza el paquete `prompt`, que proporciona una La sintaxis para solicitar al usuario en funciones asíncronas es diferente a la de `prompt-sync`. En este caso, se bloquean las interacciones con la aplicación hasta que... Las indicaciones se responden debido a su naturaleza sincrónica.

Ejemplo de 43. Agregue las importaciones del módulo y la base de datos simulada en la parte superior de `index.js`

importación `bcrypt` desde `"bcrypt"`;
importar `promptModule` desde `"prompt-sync"`;

1

```
const prompt = promptModule();  
const mockDB = { contraseñas: {} };  
...
```

1

Importar paquetes bcrypt y prompt-sync .

2

Crea una instancia del mensaje para utilizar su funcionalidad async-await.

3

Define un objeto para representar la base de datos local.

Con las importaciones realizadas, puede crear su primera función, `saveNewPassword`, que toma una contraseña en texto plano, "password", como argumento y utiliza la función `hashSync` de `Bcrypt` para convertir el texto en un valor hash. El valor resultante se almacena en la base de datos simulada, `mockDB`. Le informa al usuario que la contraseña se ha guardado mediante un mensaje de registro y luego llama a la función `showMenu` , que escribirá en breve (Ejemplo 4-4).

Ejemplo 44. Agregue la función `saveNewPassword` en `index.js`

...

```
const saveNewPassword = (contraseña) => {  
  mockDB.hash = bcrypt.hashSync(password, 10);  
  console.log("¡La contraseña ha sido guardada!");  
  showMenu(); };  
...
```

1

Codifique la contraseña en texto simple y guarde el hash de la contraseña en la clave hash en su base de datos local.

2

Imprimir un mensaje en la consola.

3

Llama a la función `showMenu` .

Después de agregar `saveNewPassword` , creará una función llamada `compareHashedPassword`, como se muestra en el Ejemplo 4-5. En esta función, acepta un argumento de contraseña en texto plano , que es

Comparado con el hash de la contraseña almacenado en mockDB. El valor resultante es verdadero o falso.

Ejemplo 45. Agregue la función compareHashedPassword en index.js

```
...
const compareHashedPassword = async (contraseña) => {
  await bcrypt.compare(contraseña, mockDB.hash);
}
```

1

Define una función personalizada para comparar una contraseña de texto simple con una contraseña en hash.

2

Compare la contraseña de entrada con el valor en su base de datos local.

Las dos siguientes funciones solicitarán al usuario que escriba una nueva contraseña o que vuelva a escribir la anterior (Ejemplo 4-6). promptNewPassword registra un mensaje en la consola de línea de comandos para que el usuario escriba su contraseña maestra. La contraseña introducida se guarda posteriormente en la función saveNewPassword. Mientras tanto, promptOldPassword solicita al usuario que vuelva a escribir su contraseña maestra anterior. El texto introducido se valida, determinando si el usuario puede ver el menú ejecutando showMenu. Si la contraseña es incorrecta, se le solicita nuevamente hasta que ingrese la contraseña correcta.

Ejemplo 46. Solicitar al usuario que escriba contraseñas en index.js

...

```
constante promptNewPassword = () => {
  const respuesta = prompt("Ingrese una contraseña principal: ");
  return saveNewPassword(respuesta);
}
```

```
const promptOldPassword = async () => {
  deje que se verifique = falso;
  mientras (! verificado)
  { const respuesta = prompt("Ingrese su contraseña: "); const
    resultado = await compareHashedPassword(respuesta);
  }
```

```

    if (resultado)
    { console.log("Contraseña verificada.");
      verificado = verdadero; ❸
      showMenu(); } ❹
    else
    { console.log("Contraseña incorrecta. Inténtalo de nuevo."); ❺
    }

  }
};
...

```

- ❶ Solicitar al usuario que escriba una nueva contraseña maestra.
- ❷ Guarde la entrada del usuario utilizando saveNewPassword.
- ❸ Define una bandera para rastrear si la contraseña ha sido verificada.
- ❹ Solicitar al usuario que vuelva a escribir su contraseña maestra existente.
- ❺ Compare la entrada con la contraseña cifrada almacenada.
- ❻ Establezca el indicador de verificación en verdadero una vez que se valide la contraseña.
- ❼ Mostrar el menú si la contraseña es correcta.
- ❽ Mostrar un error y volver a intentarlo si la contraseña es incorrecta.

Hasta ahora, ha añadido funciones para facilitar las interacciones iniciales y la autenticación del usuario. **El ejemplo 4-7** añade código para mostrar un menú de opciones una vez autenticado. showMenu registra cuatro opciones para que el usuario seleccione. La primera opción ejecuta viewPasswords para mostrarle todas sus contraseñas guardadas.

La segunda opción ejecuta promptManageNewPassword para que el usuario guarde una nueva contraseña en su base de datos. La tercera opción vuelve a ejecutar promptOldPassword, lo que permite al usuario revalidar su contraseña maestra. Finalmente, el usuario puede salir de la aplicación seleccionando la opción 4. Si no se selecciona ninguna de las cuatro opciones, se le notificará y se le pedirá que la seleccione de nuevo.

Ejemplo 47. Construyendo la función showMenu en index.js

```

...
const showMenu = async () =>
{ console.log(`
  1. Ver contraseñas 2.
  Administrar nueva contraseña
  3. Verificar contraseña
  4. Salir`); ❶
const response = prompt(">");

if (respuesta === "1") viewPasswords(); else if ❷
(respuesta === "2") promptManageNewPassword(); else if (respuesta
=== "3") promptOldPassword(); else if (respuesta === "4")
process.exit(); else { console.log(`Esa es una respuesta
❸
inválida.`); showMenu();

} };
```

...

- ❶** Pídale al usuario cuatro opciones para seleccionar.
- ❷** Después de seleccionar un valor del 1 al 4, el usuario podrá ver sus contraseñas, agregar una nueva, verificar su contraseña principal o salir de la aplicación.
- ❸** Si no se selecciona ninguna opción válida, se le pregunta nuevamente al usuario.

Con el menú listo para mostrar, solo necesita agregar las funciones para ver las contraseñas almacenadas y para ver y guardar las contraseñas en memoria.

Agregue el código del **Ejemplo 4-8**, donde viewPasswords desestructura sus contraseñas de la base de datos simulada.

Con sus contraseñas como pares clave-valor, registra ambas en su consola para cada contraseña almacenada. Luego, vuelve a mostrar el menú, que solicita al usuario que realice otra selección. promptManageNewPassword es la función que solicita al usuario que escriba la fuente de su contraseña (en realidad, una aplicación o...)

Nombre del sitio web para el que se almacena la contraseña. A continuación, se le solicita al usuario la contraseña que desea guardar. El par fuente-contraseña se guarda en su mockDB y, de nuevo, se ejecuta showMenu para mostrar los elementos del menú.

Example 48. Agregar las funciones viewPasswords y promptManageNewPassword en index.js

```
...
constante viewPasswords = () => {
  const { contraseñas } = mockDB; ❶
  Object.entries(contraseñas).forEach(([clave, valor], índice) => {
    console.log(`❷ ${índice + 1}. ❸ ${clave} => ${valor}`); }); showMenu(); };

const promptManageNewPassword = () =>
{ const source = prompt("Ingrese el nombre para la contraseña: ❹");
  const password = prompt("Ingrese la contraseña para guardar: ");

  mockDB.passwords[source] = password; ❺
  console.log(`❻ ¡La contraseña para ${source} ha sido guardada!`);
  showMenu(); };

```

- ❶ Desestructura las contraseñas de tu mockDB.
- ❷ Itere a través de las contraseñas en su base de datos local y regístrelas en su consola.
- ❸ Llame a showMenu para mostrar las opciones del menú.
- ❹ Solicita al usuario que agregue una nueva contraseña y un nombre de fuente para administrar.
- ❺ Guarde el par de fuente y contraseña en mockDB.
- ❻ Llame a showMenu para mostrar las opciones del menú.

Su aplicación está lista para ejecutarse. El último paso es agregar el código en

Ejemplo 4-9. Aquí, se verifica que mockDB tenga un valor hash existente . Si uno no existe, se le solicita al usuario que cree uno a través de promptNewPassword. De lo contrario, se le pedirá al usuario que vuelva a escribirla. su contraseña maestra a través de promptOldPassword.

Ejemplo 49. Determine el punto de entrada para su aplicación en index.js

...

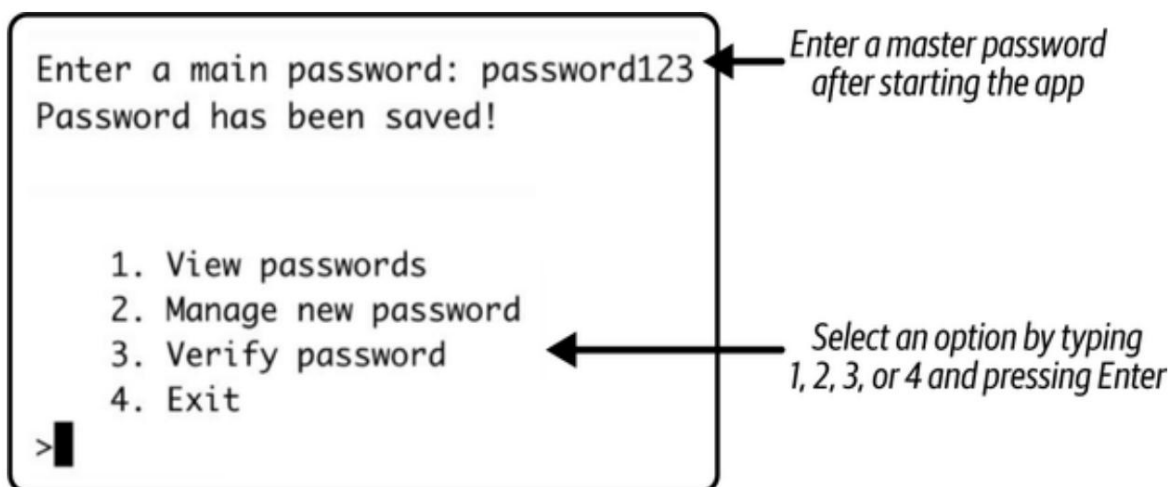
```
si (!mockDB.hash) promptNewPassword();  
de lo contrario promptOldPassword();
```

1

Comprueba si tienes una contraseña local guardada o si necesitas
Escriba una nueva contraseña principal.

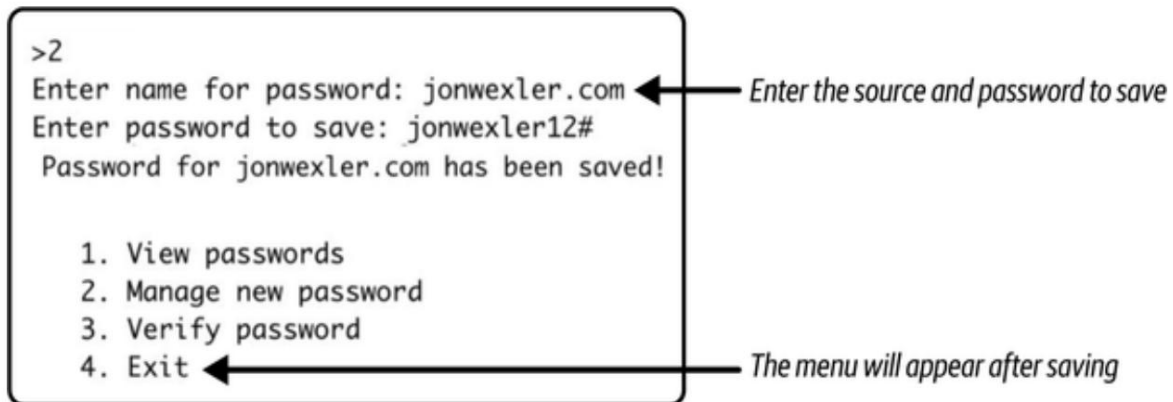
Con este código en su lugar, tiene la mayor parte de la lógica que necesita para ejecutar el Administrador de contraseñas. La única desventaja es la base de datos local. almacena temporalmente sus contraseñas administradas mientras la aplicación está en uso en ejecución. Debido a que la base de datos local es solo un objeto en memoria, Se eliminará cada vez que inicies tu aplicación.

Para probar esto, vaya al nivel raíz de la carpeta de su proyecto en su comando línea y ejecutar el índice del nodo. Se le pedirá que escriba un nuevo contraseña como en la **Figura 4-4**. Después de escribir su contraseña, será procesado por bcrypt y verá un menú de elementos para elegir.



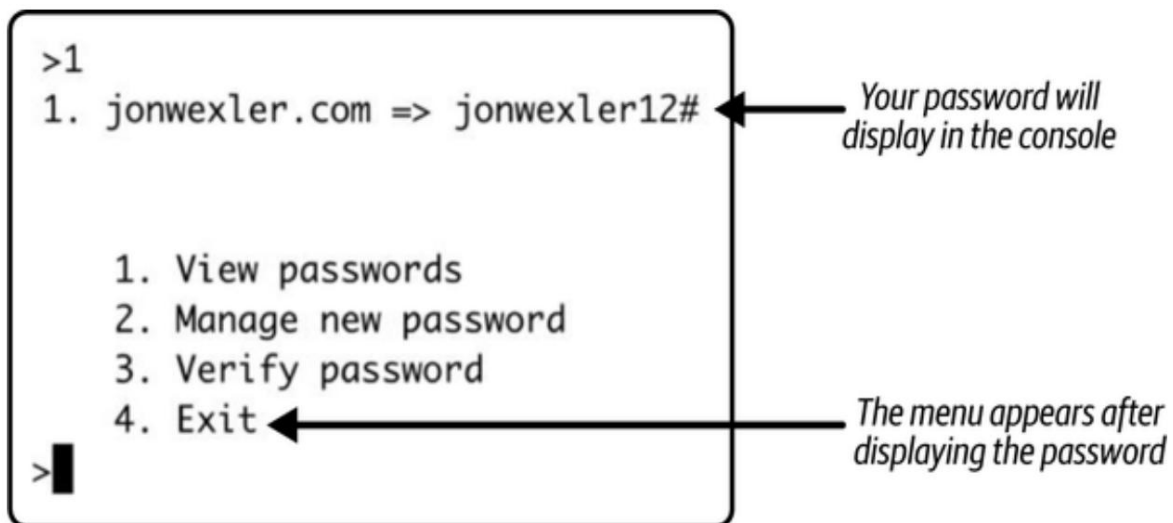
Cifra 44. Escribe tu contraseña principal para acceder al menú de tu aplicación

Desde aquí puede seleccionar 2 y presionar Enter para agregar una nueva contraseña para administrar. Intente escribir una fuente como jonwexler.com y una contraseña, como se ve en [la Figura 4-5](#).



Cifra 45. Guardar una nueva contraseña para administrar

Después de presionar Enter, esta contraseña se guarda en su memoria. objeto. Deberías ver aparecer los elementos del menú original. Selecciona 1 y presione Enter para ver la lista de contraseñas que ahora contienen su Contraseña de jonwexler.com ([Figura 4-6](#)).



Cifra 46. Seleccionar para ver todas las contraseñas administradas

También puedes probar tu contraseña principal (la primera contraseña que escribiste) cuando iniciaste la aplicación) seleccionando 3 y presionando Enter. Si Si escribe la contraseña original incorrecta, verá una declaración de registro que le informará

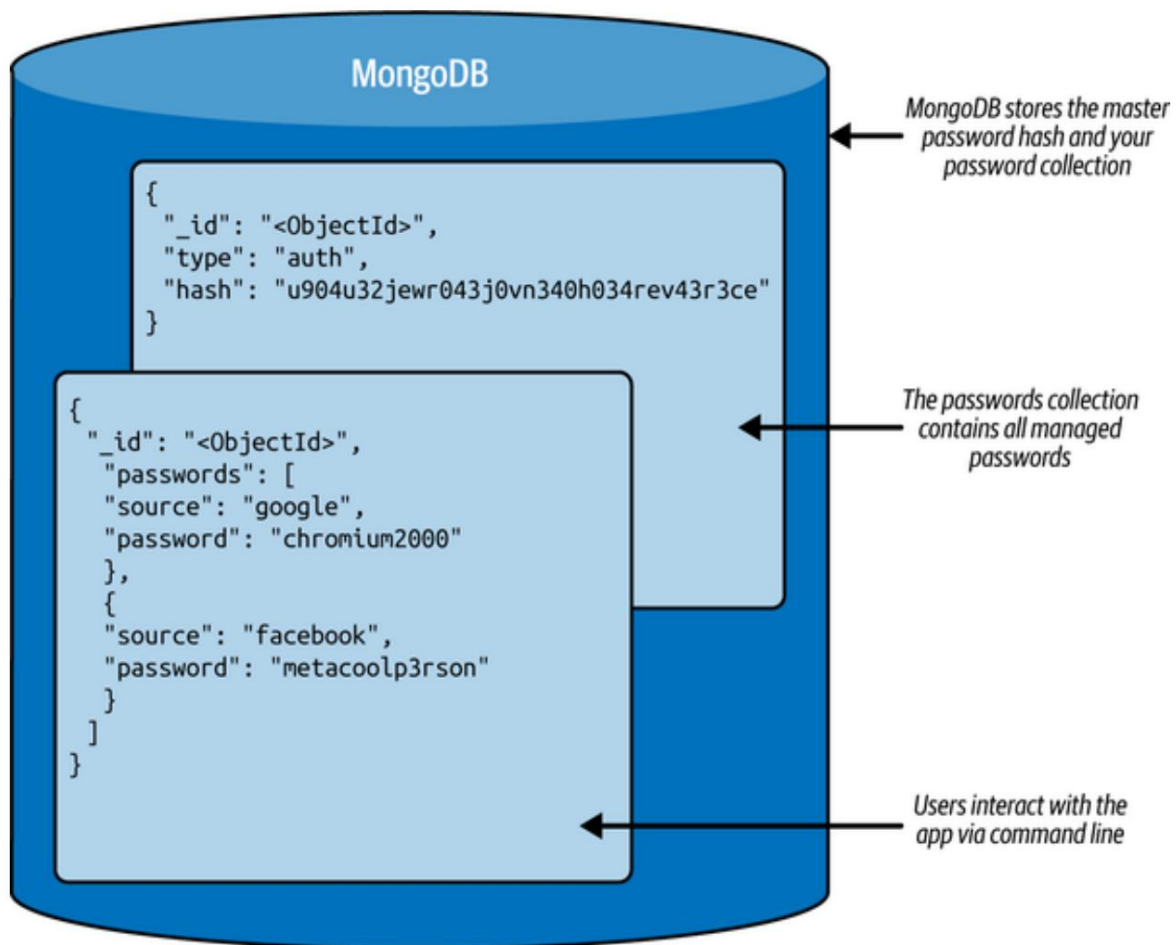
Sabe que la contraseña es incorrecta. De lo contrario, se le indicará que la contraseña coincide con la contraseña cifrada y se le devolverá a la menú.

Ahora puede salir de la aplicación de forma segura escribiendo 4 y pulsando Intro. Este paso finaliza el proceso de Node de forma segura y cierra la aplicación de línea de comandos. El siguiente paso es agregar una base de datos persistente para evitar que se borren las contraseñas cada vez que se ejecuta la aplicación.

Guardado de contraseñas con MongoDB. Tras completar la

mayor parte de la lógica de su aplicación Node, el siguiente paso es implementar una forma de guardar los datos de la aplicación cuando esta deje de ejecutarse. MongoDB es una base de datos que puede usar para almacenar esta información. MongoDB es un gestor de bases de datos orientado a documentos, lo que significa que gestiona bases de datos NoSQL no relacionales. Dado que su aplicación está diseñada para almacenar su propia colección de contraseñas, el uso de colecciones de MongoDB es adecuado para este proyecto.

En la **Figura 4-7** se ve un diagrama con un ejemplo de cómo se podrían almacenar los datos. Esta estructura es similar a la Notación de Objetos JavaScript (JSON), lo que facilita el trabajo con JavaScript en el backend. Observe que en esta figura se almacena el valor hash de la contraseña principal. A continuación, se obtiene una lista de contraseñas que asignan un nombre de origen a una contraseña de texto plano. Además, MongoDB asignará un ObjectId a los nuevos elementos de datos dentro de una colección.



Cifra 47. Diagrama de cómo guardar contraseñas en MongoDB

NOTA

Para esta sección, deberá asegurarse de que MongoDB esté instalado correctamente.

Visite [el Apéndice C](#) para conocer los pasos de instalación.

Vaya al nivel raíz de su proyecto en el símbolo del sistema y ejecute npm instalar mongodb@^6.8.0.

Una vez instalado, el paquete mongodb le proporcionará su Node aplicación las herramientas que necesita para conectarse a su base de datos y comenzar añadiendo datos. Por esta razón, ya no necesita el almacenamiento temporal en memoria, mockDB, mencionado anteriormente en este capítulo. En su lugar,

Usa MongoClient para configurar una nueva conexión a tu servidor local de MongoDB. Tu servidor de desarrollo debería estar ejecutándose en mongodb://localhost:27017 en tu ordenador. Por último, configura una base de datos llamada passwordManager para conectarte.

En index.js, agregue el código del [Ejemplo 4-10](#).

Variables 410. Importar mongodb y definir la conexión a la base de datos de ejemplo

```
...
importar { MongoClient } de "mongodb"; const dbUrl      ❶
= "mongodb://localhost:27017"; const cliente = nuevo      ❷
MongoClient(dbUrl); dejar hasPasswords = falso;          ❸
dejar passwordsCollection,                                ❹
authCollection; const dbName = "passwordManager";        ❺
...
```

- ❶ Importe la clase MongoClient del paquete mongodb .
- ❷ Defina la dbUrl para conectarse a tu servidor MongoDB local.
- ❸ Cree una nueva instancia de cliente MongoDB utilizando la URL de conexión.
- ❹ Declare una bandera hasPasswords para rastrear si ya existe una contraseña maestra.
- ❺ Establezca el nombre de la base de datos en passwordManager.

A continuación, crea una función asíncrona para establecer la conexión de tu aplicación con la base de datos. En [el Ejemplo 4-11](#) encontrarás el código que necesitas agregar para conectarte a la base de datos. La función client.connect intentará iniciar una conexión con tu servidor local de MongoDB. A continuación, client.db(dbName) se conectará a una base de datos con el nombre asignado a dbName. En tu caso, tendrás dos colecciones de MongoDB en la base de datos: authCollection , que se encarga de almacenar el hash de tu contraseña, y passwordsCollection , que almacena la lista de...

Contraseñas. Declare estas variables al principio de index.js. Una vez conectada, esta función buscará un hash de contraseña existente ejecutando `authCollection.findOne({ "type": "auth" })`. `hasPasswords = !!hashedPassword` convierte el resultado de la búsqueda en un valor booleano. Finalmente, se configuran `passwordsCollection` y `authCollection`.

NOTA

La razón por la que se necesita una función asíncrona es que la conexión a la base de datos MongoDB es una operación de E/S asíncrona. Al conectarse a una base de datos y ejecutar un comando, el tiempo de ejecución puede variar. Usar `async-await` nos permite escribir este código de forma que parezca síncrono, pausando la ejecución en cada espera hasta recibir una respuesta de la base de datos, sin bloquear el resto de la aplicación.

Ejemplo 4-11. Función principal para inicializar la base de datos

```
...
const main = async () => { try { await ❶

  client.connect(); ❷
  console.log("Conectado correctamente al servidor"); const db =
  client.db(dbName); authCollection = ❸
  db.collection("auth"); passwordsCollection = ❹
  db.collection("contraseñas"); const hashedPassword = await
  authCollection.findOne({ type: "auth" }); hasPasswords = !!hashedPassword; }
catch (error) { ❺

  console.error("Error al conectar con la base de datos:", error); ❷
}
process.exit(1);

}};
```

❶ Define una función principal para inicializar tu base de datos.

- ② Llame a `client.connect` para establecer una conexión con su servidor de base de datos.
- ③ Cree o conéctese a una base de datos con el nombre `passwordManager`.
- ④ Busque o cree una colección de bases de datos llamada `authCollection` y una llamada `passwordsCollection`.
- ⑤ Verifique si existía una contraseña con hash con el tipo de autenticación en su colección `authCollection`.
- ⑥ Asigne `hasPasswords` al valor booleano de la búsqueda resultante en la base de datos.
- ⑦ Detecte cualquier error y salga del proceso si hay un problema al conectarse a la base de datos.

En la parte inferior de `indexjs`, agregue el código del **Ejemplo 4-12** para llamar al Función principal y comienza a procesar tu aplicación. Este código primero ejecuta la función principal, que configura la conexión a la base de datos y comprueba si ya existe una contraseña principal. Según el resultado, solicita al usuario que cree una nueva contraseña principal si no existe (`promptNewPassword`) o solicita la contraseña existente para verificar el acceso (`promptOldPassword`).

Ejemplo 4-12. Llamar a `main` para configurar colecciones de MongoDB

...

```
esperar principal(); ①  
si (!tieneContraseñas) solicitarNuevaContraseña(); de lo ②  
contrario solicitarAntiguaContraseña();
```

- ① Llamada principal, utilizada para asignar las colecciones `passwordsCollection` y `authCollection`.

②

Solicitar al usuario que cree o verifique una contraseña maestra según si existe una en la base de datos.

Ahora puede reiniciar su aplicación Node saliendo de cualquier aplicación en ejecución y escribiendo " node index". Si su aplicación se conectó correctamente a la base de datos, debería ver el mensaje "Conectado correctamente al servidor" registrado en su línea de comandos.

NOTA

Después de guardar las contraseñas, si desea eliminar la base de datos de contraseñas y comenzar desde cero, siempre puede agregar `await passwordsCollection.deleteMany({})` o `await authCollection.deleteMany({})` para eliminar sus contraseñas o la contraseña hash principal, respectivamente.

Con la base de datos conectada, debe modificar parte de la lógica de su aplicación para gestionar la lectura y escritura en sus colecciones de MongoDB. Cambie `saveNewPassword` para que sea una función asíncrona . Dentro de esa función, asigne una nueva variable ``hash`` al resultado de ``bcrypt.hashSync`` y guárdelo en la base de datos con ``await authCollection.insertOne({ "type": "auth", hash })``). Esto guardará el hash de la contraseña en la colección ``authCollection`` de su base de datos.

A continuación, cambie `compareHashedPassword` a `async`, por lo que la primera línea será `const { hash } = await authCollection.findOne({ "type": "auth" })`. Esta línea buscará en su `authCollection` una contraseña con hash y la enviará a la función de comparación de `bcrypt` .

CONSEJO

Es posible que desee comprobar si existe un hash de contraseña antes de ejecutar la función de comparación . Si no existe, puede devolver "false" para indicar que la contraseña no es válida.

Las últimas tres funciones que se modificarán se encuentran en **el Ejemplo 4-13**.

En este caso, viewPasswords se modifica para extraer todas las contraseñas (por origen y valor) de la colección de contraseñas. showMenu se mantendrá igual, pero al igual que las demás funciones, se volverá asíncrona y, para facilitar la lectura, utilizará una sentencia switch/case . En esta función, se añade "await" antes de cada llamada, ya que ahora realizan operaciones de E/S. Por último, promptManageNewPassword utiliza la función findOneAndUpdate de MongoDB para añadir una nueva entrada de contraseña si no existe, o sobrescribir y actualizar una entrada de contraseña si existe un valor anterior. Las opciones "returnDocument" y "upsert" indican a la función que sobrescriba el valor modificado y devuelva una copia del valor modificado al finalizar la operación de guardado.

Ejemplo 4-13. Agregar llamadas de base de datos a funciones en index.js

...

```
const viewPasswords = async () => { const
  passwords = await
passwordsCollection.find({}).toArray();
  passwords.forEach(({ fuente, contraseña }, índice) => {
    console.log(` ${índice + 1}. ${fuente} => ${contraseña}`); }); showMenu(); }
```

```
const showMenu = async () =>
{ console.log(` 1.
  Ver contraseñas 2.
  Administrar nueva contraseña
  3. Verificar contraseña 4.
  Salir`);
```



```

constante respuesta = prompt(">");

switch (respuesta) { caso "1":
  await
    viewPasswords(); break; caso "2": ❸
  await

  promptManageNewPassword(); break; caso "3": await

  promptOldPassword(); break; caso "4":

  process.exit();
  predeterminado:

  console.log("Esa es una respuesta inválida."); await showMenu();

}};

constante promptManageNewPassword = async () => {
  const source = prompt("Ingrese el nombre para la contraseña: "); const password =
  prompt("Ingrese la contraseña para guardar: "); await
  passwordsCollection.findOneAndUpdate( { source }, { $set:
    { password } }, {

    returnDocument: "después", upsert:
    verdadero,

    ❹
  } ); console.log(` ¡ Se ha guardado la contraseña para ${source} !`); showMenu(); };

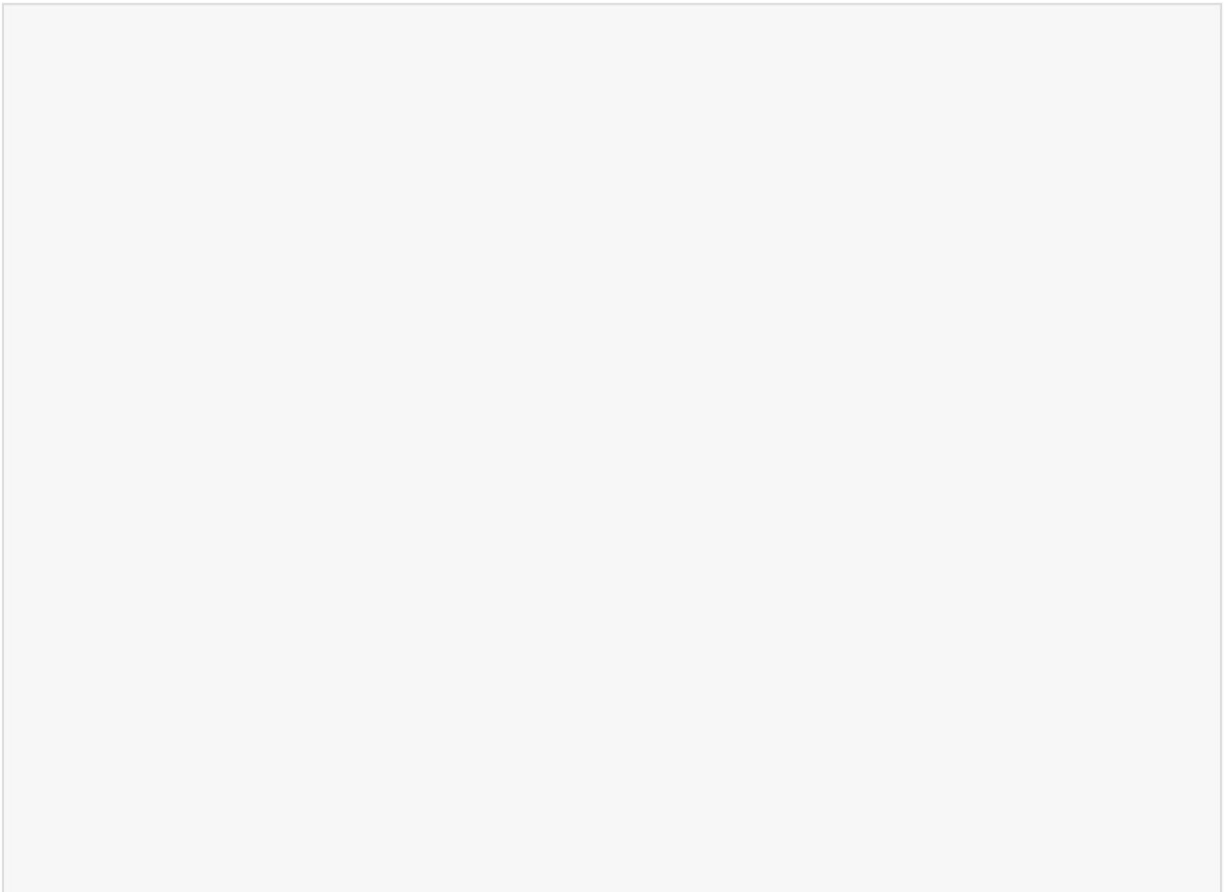
```

- ❶ Consultar todas las contraseñas de passwordCollection.
- ❷ Itere a través de las contraseñas y regístrelas en la consola.
- ❸ Cuando la entrada del usuario sea “1”, ejecute la función viewPasswords .
- ❹

Utilice `findOneAndUpdate` para buscar una contraseña existente que coincida con su fuente y luego configure la nueva contraseña.

Con este código, tendrá una base de datos completamente funcional para su aplicación de gestión de contraseñas. Cierre cualquier aplicación de Node que se estuviera ejecutando y reiníciela ejecutando `node index`. Las indicaciones que ve en la línea de comandos no deberían cambiar. Solo que, esta vez, los valores que escriba se conservarán incluso al cerrar la aplicación.

Una vez completada esta aplicación, podrá ejecutarla localmente y agregar o recuperar contraseñas protegidas con su contraseña principal encriptada. A continuación, podría añadir un cliente con una interfaz de usuario para visualizar sus datos de contraseñas o configurar su base de datos en la nube para que sus contraseñas sean persistentes entre computadoras.



EJERCICIOS DEL CAPÍTULO

1. Haga que el proceso de hash sea configurable:
 - a. Actualice su función `saveNewPassword` para utilizar una cantidad configurable de rondas de sal en lugar de un valor codificado (por ejemplo, 10).
 - b. Solicitar al usuario que ingrese la cantidad de sal.
rondas cuando crean su contraseña maestra.
Este valor controla qué tan costosa es computacionalmente la operación hash.
 - c. Guarde las rodajas de sal elegidas junto con contraseña en formato hash en su `authCollection`, por ejemplo:
`{ type: "auth", hash, saltRounds }`.
 - d. Al verificar la contraseña más tarde, recupere las rondas de sal almacenadas y reutilícelas al comparar la contraseña ingresada con el hash almacenado.

CONSEJO

Aumentar las rondas de sal mejora la seguridad pero reduce el rendimiento. Intente experimentar con diferentes valores (por ejemplo, 8, 12, 15) y observe el impacto en el tiempo de ejecución.

2. Permitir la recuperación de contraseña por nombre de origen:
 - a. Agregue una nueva opción CLI a su menú principal: 5.
Buscar contraseña por fuente.
 - b. Cuando se selecciona, solicita al usuario que ingrese un nombre de fuente (por ejemplo, gmail).

c. Consulte la colección de contraseñas para esa fuente específica e imprima su nombre de usuario y contraseña correspondientes.

d. Si no se encuentra ninguna coincidencia, muestre un mensaje útil como "No se ha guardado la contraseña para esa fuente".

Estos ejercicios fortalecen su comprensión de la configuración de hash seguro y presentan patrones de consulta de bases de datos del mundo real basados en la entrada del usuario.

Resumen

En este capítulo, usted:

- Cree su propia aplicación de gestión de contraseñas
- Se implementó una lógica de hash de contraseña segura utilizando el paquete bcrypt
- Configurar una colección de bases de datos MongoDB para contraseñas
- Nodo configurado para usar el paquete mongodb con la entrada del usuario

Capítulo 5. Feed de agregación de contenido

Este capítulo cubre lo siguiente:

- Análisis y visualización de datos de fuentes RSS
- Obtener contenido de múltiples fuentes mediante API modernas
- Creación de un lector de feeds en tiempo real usando Node
- Permitir que la entrada del usuario amplíe los resultados agregados

Los paneles son la piedra angular de la visualización de datos moderna, ya que permiten a los usuarios realizar un seguimiento de métricas, monitorear tendencias y mantenerse informados en tiempo real. Si bien los paneles suelen depender de API para obtener y mostrar datos, una de las herramientas originales para la agregación de contenido fue la fuente RSS (Really Simple Syndication). RSS organiza el contenido, generalmente en formato XML, en fuentes estructuradas que presentan fragmentos de texto, titulares y las últimas actualizaciones de diversas fuentes. Esto permitió a los usuarios evitar la necesidad de visitar manualmente varios sitios web y, en su lugar, ver actualizaciones seleccionadas en un único lector personalizado.

Aunque la popularidad de los canales RSS ha disminuido, su arquitectura subyacente sigue siendo muy relevante. Combinados con las API modernas, proporcionan un marco potente para crear plataformas versátiles de agregación de contenido que pueden extraer datos de diversas fuentes.

En este capítulo, irás más allá de RSS para crear un agregador de contenido que integra tanto feeds RSS como API. Aprenderás a obtener y procesar datos en formatos XML y JSON, a normalizarlos y combinarlos en una estructura unificada, y a servir el contenido agregado mediante una línea de comandos o un cliente web. Al finalizar, tendrás un...

Agregador capaz de entregar datos relevantes en tiempo real desde múltiples fuentes, todo en un solo lugar.

HERRAMIENTAS Y APLICACIONES UTILIZADAS EN ESTE CAPÍTULO

Antes de comenzar, asegúrese de instalar y configurar las herramientas y aplicaciones necesarias para este proyecto. Las instrucciones de instalación de Node.js, Fastify y VS Code se encuentran en [el Capítulo 1](#), mientras que los pasos de inicialización del proyecto, como la configuración de la estructura de directorios, la configuración de package.json y el uso de sintaxis moderna, se describen en [el Apéndice A](#). Una vez completado, regrese aquí para continuar. Desarrollar un proyecto desde cero le ayuda a comprender mejor cada componente, brindándole mayor control y flexibilidad a medida que avanza.

Tu mensaje: A tus

compañeros de trabajo les apasiona la comida y les encanta hablar de las últimas tendencias, recetas e innovaciones. Sin embargo, con el contenido disperso en sitios web, blogs y redes sociales, mantenerse informado se ha vuelto un desafío.

Aunque compartir enlaces en el chat grupal ayuda, sabes que hay una mejor manera de mantener a todos al tanto.

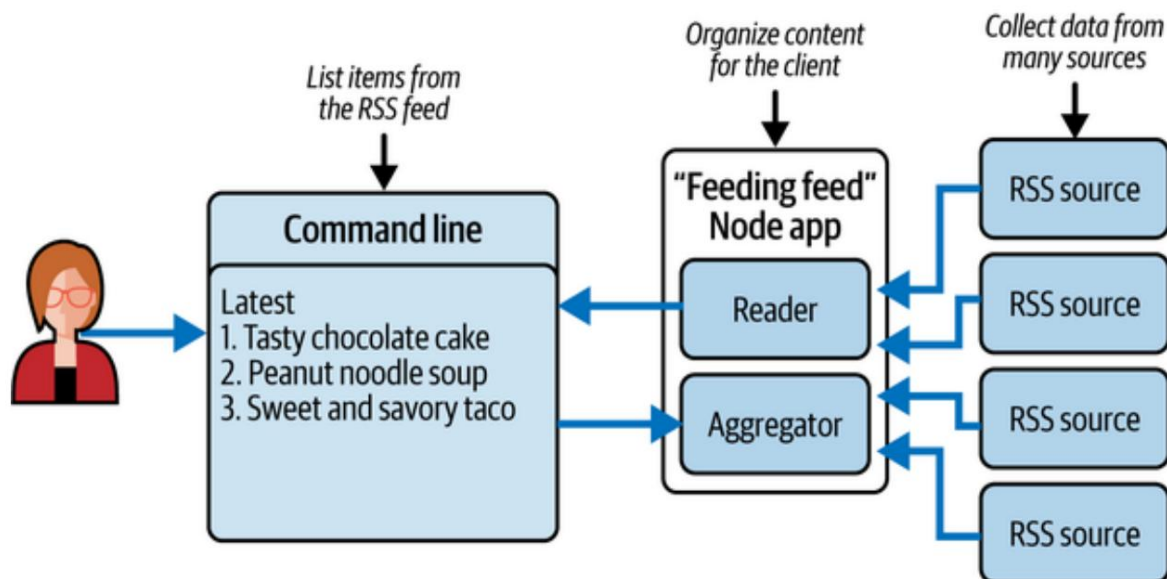
Decides crear un agregador de contenido unificado para tu empresa, que recopila los últimos artículos, publicaciones en redes sociales y actualizaciones del sector relacionados con la alimentación, tanto desde feeds RSS como API. Ya sea que se trate de la popularidad de la carne vegana o del nuevo food truck más popular de la ciudad, tu agregador garantizará que todos reciban actualizaciones frescas y relevantes en un solo lugar. Con este proyecto, crearás un centro centralizado para todo lo relacionado con la alimentación, combinando lo mejor de RSS y la integración de API modernas en un feed de "Feeding".

Obtener planificación

El objetivo de este proyecto es crear una aplicación que agregue contenido nuevo y relevante de múltiples fuentes para un grupo grande de personas. Aunque existen aplicaciones populares de agregación de contenido, diseñarás tu propia aplicación de Node para combinar datos de fuentes RSS y API modernas. Comenzarás experimentando con los paquetes RSS y API disponibles en npm. A partir de ahí, seguirás los requisitos de diseño del proyecto, como se muestra en **la Figura 5-1**.

En este diagrama, puede ver el flujo de lógica e información. Desde el cliente (cualquier computadora o dispositivo con conexión de red), se realiza una solicitud a su aplicación Node para obtener el contenido agregado más reciente. Su aplicación procesa datos de múltiples fuentes, incluyendo feeds RSS y API, normalizándolos y analizándolos en un formato unificado.

A continuación, devuelve una lista resumida de resultados que puede mostrarse en el cliente de línea de comandos o en una interfaz web. Este diseño garantiza un flujo continuo de actualizaciones en tiempo real desde diversas fuentes de contenido.



Cifra 51. Plan de proyecto para un lector y agregador RSS de aplicaciones

Antes de empezar a programar, conviene repasar el funcionamiento de RSS y comprender el tipo de datos que encontrará. RSS funciona extrayendo contenido de diversas fuentes web, a menudo de sitios de noticias.

o blogs que desean brindar a su audiencia una experiencia rápida y sencilla. Acceso a los principales titulares o actualizaciones. Por ejemplo, podrías usar el Fuente RSS de recetas Bon Appétit para tu aplicación, disponible en

<https://www.bonappetit.com/feed/recipe-rss>. Puedes ver /

contenido del feed sin procesar haciendo clic en el enlace en su navegador web, como

se muestra en la Figura 5-2.

Los canales RSS suelen utilizar XML (lenguaje de marcado extensible), un Formato de datos estructurados con etiquetas similares a HTML. La estructura XML Ayuda a organizar los datos del feed en secciones, lo que facilita su análisis.

y uso. El archivo XML de una fuente RSS generalmente comienza con una etiqueta rss para define el tipo de documento, seguido de una etiqueta de canal que contiene metadatos sobre el feed, como el nombre, el enlace y el idioma del

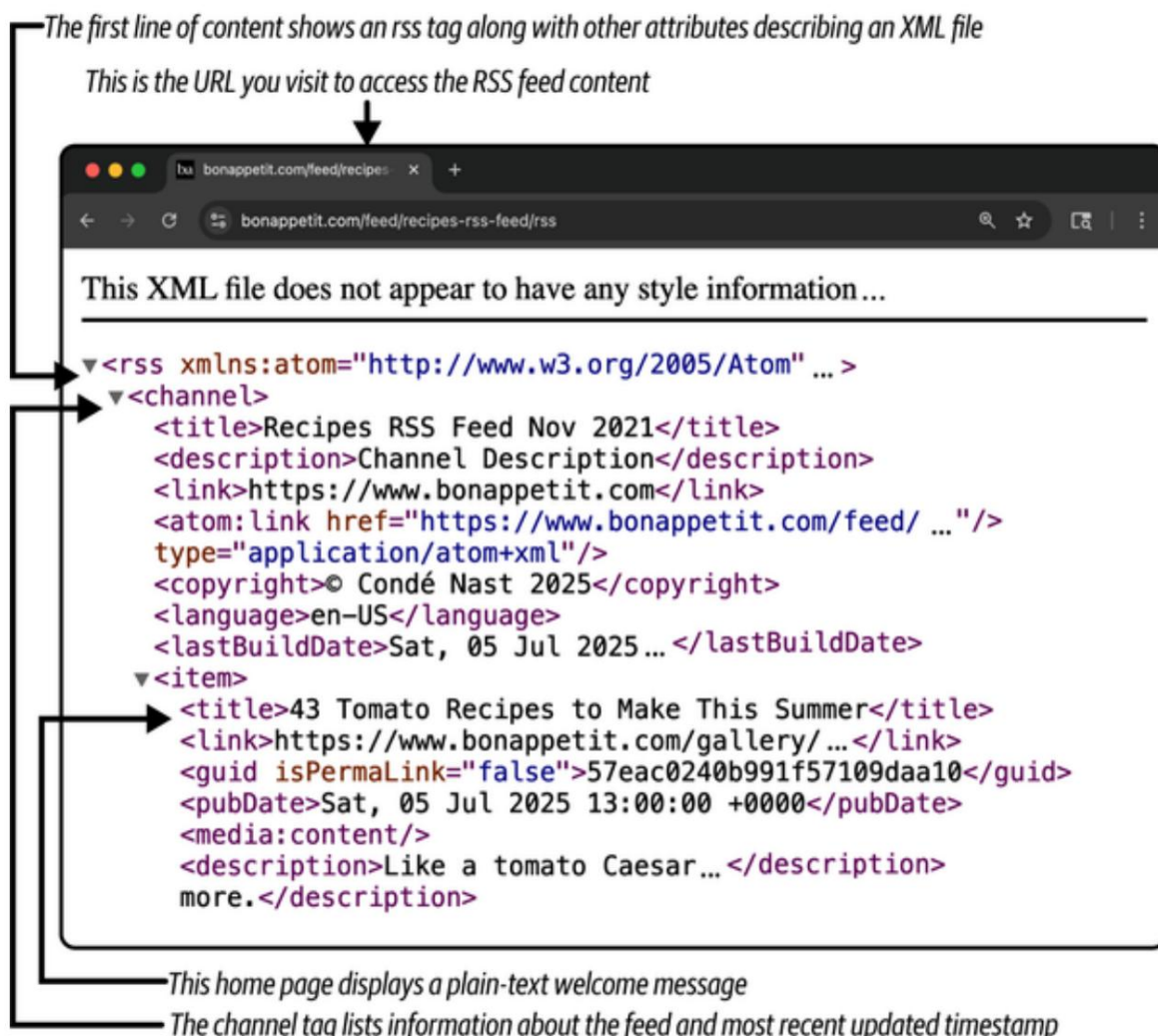
Fuente. Dentro de la etiqueta del canal, encontrarás los elementos de contenido principales,

Cada artículo está envuelto en una etiqueta. Cada artículo contiene detalles como un título, Descripción y fecha de publicación. Su tarea será extraerlos.

elementos y reformatearlos para mostrarlos en tu aplicación.

NOTA

Si bien XML es el formato tradicional para las fuentes RSS, es solo uno de muchos Formatos de datos utilizados en aplicaciones web modernas. Las API a menudo proporcionan datos en JSON, que es más compacto y ampliamente utilizado. Este proyecto se centrará sobre el análisis de XML para RSS y al mismo tiempo la exploración de API basadas en JSON, Permitiéndole combinar y presentar datos de múltiples fuentes sin problemas.



Cifra 52. Resultados de la fuente RSS de recetas de Bon Appétit en el navegador

El primer paso para crear una aplicación que pueda utilizar datos XML es

Llamar a la URL de la fuente RSS directamente desde la aplicación Node. Obtener

Comenzó creando una carpeta `food_feeds_rss_app` y navegando

A la carpeta del proyecto en la línea de comandos. Desde aquí, ejecute `npm init`.

para inicializar la aplicación Node con las configuraciones predeterminadas, como se muestra en Ejemplo 5-1.

Ejemplo { 51. Configuraciones de inicio de npm para su aplicación

```

"nombre": "aplicación rss de feeds de alimentos", ❶
"versión": "1.0.0",
"descripción": "",
"principal": "index.js",

```

```
"scripts": { "test":
  "echo \"Error: no se especificó ninguna prueba\" && exit 1" }, "author": "Jon
  Wexler", "license": "ISC"
```

```
}
```



Agregue el nombre de su proyecto para reflejar el tipo de fuentes RSS que admitirá.

Con la aplicación inicializada, añada "type": "module" a tu archivo paquetes JSON . Esto te permitirá usar la sintaxis de importación para externos. A continuación, crea un archivo index.js que sirva como punto de entrada para tu aplicación. Añade el código del **Ejemplo 5-2** a index.js. Este código utiliza la API Fetch para realizar una solicitud a la URL de la fuente RSS de Bon Appétit y acceder a sus datos XML. Dado que se espera una respuesta, la llamada fetch se encapsula en una función asíncrona llamada main. Asigna la variable URL al punto final de la fuente RSS de Bon Appétit . Luego, asigna la respuesta de tu llamada de búsqueda a respuesta. Ahora puede utilizar la llamada a la función response.text() para extraer los datos XML para enviarlos a su consola.

NOTA

A partir de Node v18.0.0, la API de búsqueda está disponible de forma nativa y se puede usar sin instalar paquetes externos. Para versiones anteriores de Node, deberá instalar el paquete node-fetch ejecutando npm install node-fetch en el directorio de su proyecto.

MÁS INFORMACIÓN SOBRE LA API FETCH

JavaScript ofrece múltiples maneras de acceder al contenido mediante HTTP. La interfaz XMLHttpRequest, la base de la gestión de solicitudes AJAX en navegador, impulsa la mayoría de las solicitudes AJAX, mientras que el JavaScript del lado del servidor incluye la biblioteca http integrada, preinstalada en Node. Con el tiempo, muchos paquetes externos han desarrollado el módulo http para ofrecer soluciones más robustas e intuitivas para gestionar solicitudes HTTP.

La API Fetch se introdujo en JavaScript para proporcionar una interfaz moderna y flexible que permitiera obtener recursos de la web. Su principal ventaja reside en sus objetos de solicitud y respuesta, que encapsulan funcionalidades comunes para generar y procesar solicitudes HTTP de forma asíncrona. Fetch también se integra a la perfección con la sintaxis Promise y async/await de JavaScript, lo que facilita la gestión de operaciones asíncronas para los desarrolladores.

Debido a su popularidad y amplio uso, Fetch se añadió a Node en 2022, a partir de la versión 18.0.0. Esta incorporación permite a los desarrolladores usar Fetch de forma nativa en el servidor sin depender de paquetes externos. Su versatilidad y diseño moderno lo convierten en una herramienta esencial para crear aplicaciones que requieren comunicación HTTP.

Obtenga más información sobre la API Fetch en la [documentación oficial](#).

Ejemplo 52. Ejemplo de obtención de una fuente RSS mediante fetch en index.js

```
const main = async () => {  
  const url = "https://www.bonappetit.com/feed/recetas-rss-feed/rss";  
  const respuesta =  
    await fetch(url);  
  console.log(await  
    respuesta.text());  
}
```

```
} main(); ④
```

- ① Define una función asíncrona para encapsular la lógica de la solicitud HTTP.
- ② Realice una solicitud GET para la URL proporcionada.
- ③ Utilice el texto de respuesta() para leer el cuerpo de la respuesta como una cadena y registrarlo en la consola.
- ④ Llama a tu función principal .

Guarda tu archivo, accede a la carpeta de tu proyecto en la línea de comandos y ejecuta `node index`. Deberías ver el mismo resultado XML que al acceder a la URL en un navegador web, pero esta vez se imprime en la ventana de la línea de comandos. Con este resultado de texto, puedes analizar cada línea y extraer la información necesaria. Afortunadamente, existen paquetes externos que facilitan este proceso. En la siguiente sección, implementarás el paquete `rss-parser` .

Lectura y análisis de un feed. Tu aplicación

"Feeding feed" está diseñada para recopilar datos y mostrarlos a tus usuarios de forma intuitiva. El XML sin procesar no es particularmente fácil ni interesante de leer. La comunidad tecnológica lo reconoce y, sin duda, existen numerosas bibliotecas externas que puedes instalar y que pueden ayudarte. Una de estas bibliotecas se encuentra en el paquete `rss-parser` . Este paquete abarca tanto la obtención de un feed como el análisis de su contenido XML. Puedes instalar este paquete accediendo a la raíz de tu proyecto en la línea de comandos y ejecutando el comando `npm install rss-parser@^3.13.0`.

Una vez instalado el paquete, notará que su archivo `package.json` agregó una nueva dependencia y se creó una carpeta llamada `node_modules` en la raíz de su proyecto. A continuación, importe el analizador RSS.

paquete en tu aplicación añadiendo `import Parser from 'rss-parser'`; al principio de tu archivo `index.js`. En la siguiente línea, instancie la clase `Parser` agregando `const parser = new Analizador()`;

NOTA

El analizador en mayúsculas representa la clase `Parser` del analizador `rss` paquete, mientras que el analizador en minúsculas es la instancia de esa clase que crear para usar en su aplicación.

Ahora tienes un objeto analizador que puedes usar en lugar de tu `Fetch Código API`. Reemplace el contenido de su función principal con el código. En el [ejemplo 5-3](#), este código implementa `parser.parseURL` función que obtiene el contenido de la URL de su fuente RSS y preparándolos en un formato estructurado. Luego tendrás acceso a la Título y elementos del feed. Al final, solo registras lo que quieres mostrar. De ese feed. En este caso, se trata del título y el enlace del artículo.

Ejemplo 53. Obtener y analizar una fuente RSS mediante `rss-parser` en `index.js`

```
...
const url = "https://www.bonappetit.com/feed/recetas-rss-feed/rss";

const {título, elementos} = await parser.parseURL(url);           ❶
console.log(título);                                             ❷
const resultados = items.map(({título, enlace}) => ({título,
enlace})); ❸
console.tabla(resultados);                                       ❹
...
```

❶ Obtener y analizar el XML de la fuente RSS y desestructurarlo
Respuesta para acceder al título y los artículos.

❷ Imprima el título del feed.

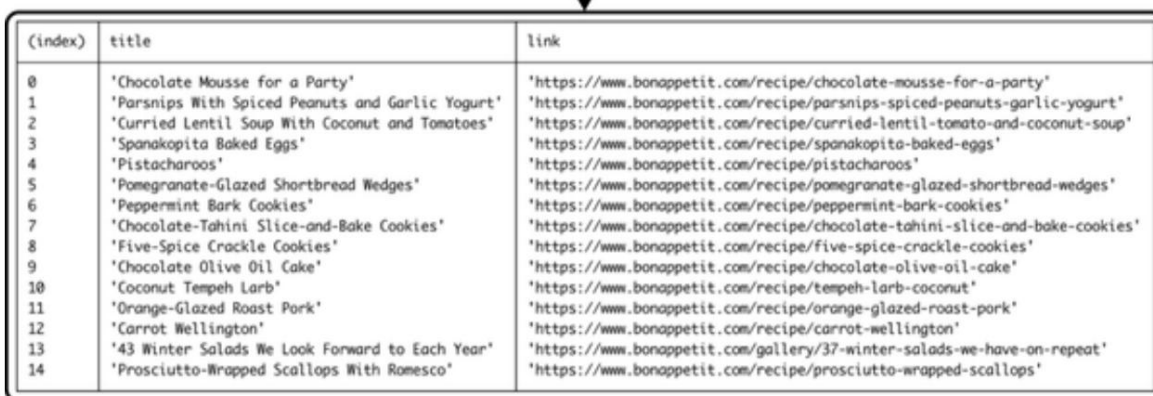
- 3 Extraiga solo el título y el enlace de cada elemento y asigne esa nueva matriz a los resultados.
- 4 Imprima el contenido del feed en su consola en un formato de tabla estructurada.

CONSEJO

console.log es por lejos la función de registro y depuración más utilizada. Sin embargo, existen otros tipos de registro que puedes usar, como console.table, que imprime tu contenido en un formato más legible que la función anterior. Obtén más información sobre console.table en [las páginas de referencia de las API web de Mozilla](#).

Después de agregar el código del analizador RSS , guarde el archivo, navegue a la raíz del proyecto en la línea de comandos y ejecute el comando ` node index ` . El resultado debería ser similar al de [la Figura 5-3](#).

Each entry represents an RSS item



(index)	title	link
0	'Chocolate Mousse for a Party'	'https://www.bonappetit.com/recipe/chocolate-mousse-for-a-party'
1	'Parsnips With Spiced Peanuts and Garlic Yogurt'	'https://www.bonappetit.com/recipe/parsnips-spiced-peanuts-garlic-yogurt'
2	'Curried Lentil Soup With Coconut and Tomatoes'	'https://www.bonappetit.com/recipe/curried-lentil-tomato-and-coconut-soup'
3	'Spanakopita Baked Eggs'	'https://www.bonappetit.com/recipe/spanakopita-baked-eggs'
4	'Pistacharoos'	'https://www.bonappetit.com/recipe/pistacharoos'
5	'Pomegranate-Glazed Shortbread Wedges'	'https://www.bonappetit.com/recipe/pomegranate-glazed-shortbread-wedges'
6	'Peppermint Bark Cookies'	'https://www.bonappetit.com/recipe/peppermint-bark-cookies'
7	'Chocolate-Tahini Slice-and-Bake Cookies'	'https://www.bonappetit.com/recipe/chocolate-tahini-slice-and-bake-cookies'
8	'Five-Spice Crackle Cookies'	'https://www.bonappetit.com/recipe/five-spice-crackle-cookies'
9	'Chocolate Olive Oil Cake'	'https://www.bonappetit.com/recipe/chocolate-olive-oil-cake'
10	'Coconut Tempeh Larb'	'https://www.bonappetit.com/recipe/tempeh-larb-coconut'
11	'Orange-Glazed Roast Pork'	'https://www.bonappetit.com/recipe/orange-glazed-roast-pork'
12	'Carrot Wellington'	'https://www.bonappetit.com/recipe/carrot-wellington'
13	'43 Winter Salads We Look Forward to Each Year'	'https://www.bonappetit.com/gallery/37-winter-salads-we-have-on-repeat'
14	'Prosciutto-Wrapped Scallops With Romesco'	'https://www.bonappetit.com/recipe/prosciutto-wrapped-scallops'

Cifra 53. Salida de consola para una tabla de elementos de la fuente RSS

Esta salida muestra los títulos de las recetas y sus URL correspondientes. Esta es una excelente manera de resumir el contenido de la fuente RSS de recetas de Bon Appétit , aunque esta lista es estática y procesa el contenido solo cuando se ejecuta la aplicación. Dado que esta fuente recibe actualizaciones,

Sería ideal que su aplicación Node reflejara esas actualizaciones en tiempo real. Para obtener nuevas actualizaciones cada dos segundos, cambie la llamada a `main()` `setInterval` mantendrá `js` a `setInterval(main, 2000)`. Al final de `index`, su proceso de Node ejecutándose indefinidamente, procesando una nueva solicitud de URL, analizando y registrando cada dos segundos (dos mil milisegundos). Para que esto sea más evidente en su consola, agregue `console.clear()`; en `index.js` justo encima de la línea `console.table` para limpiar la consola con cada intervalo. Además, agregue `console.log('Last updated ', (new Date()).toUTCString() );` justo debajo del registro de la tabla para imprimir una marca de tiempo actualizada. Ahora, al ejecutar su aplicación, aunque no vea que el contenido del feed cambie inmediatamente, notará que la marca de tiempo actualizada cambia con cada intervalo.

Este lector RSS de línea de comandos es una excelente manera de tener las últimas actualizaciones de tus feeds RSS favoritos ejecutándose en tu ordenador. En la siguiente sección, agregarás más feeds externos y crearás tu propio agregador para mostrar solo el contenido más relevante.

Creando un Agregador. Con tu aplicación

Node imprimiendo correctamente el contenido de las fuentes RSS en tu consola, quizás te preguntes cómo puedes ampliar la herramienta para que sea más práctica. Después de todo, tu objetivo es recopilar recetas específicas que se ajusten a las preferencias dietéticas de tus compañeros de trabajo. La buena noticia es que tu aplicación está diseñada para gestionar más contenido. Con la lógica establecida para obtener una fuente RSS, puedes agregar más URL de fuentes para llamar.

NOTA

Las fuentes RSS pueden dejar de funcionar si los editores las cambian o las interrumpen. Si una fuente ya no se actualiza, busca una nueva URL o una fuente alternativa.

Para probar la obtención de datos desde varias URL, puede utilizar Budget Bytes y el subreddit /r/Recipes. Ambos feeds

Ofrecer contenido variado en diferentes momentos, lo que lo convierte en un desafío mayor.

Para analizar. Para incorporar estos feeds adicionales, agregue

alimentar <https://www.budgetbytes.com/category/recipes/> y <https://www.reddit.com/r/recipes/>

al <https://www.budgetbytes.com/category/recipes/> a la lista de URL para explorar en

en la parte superior de index.js (Ejemplo 5-4). La constante URL se mostrará más adelante.

Se utiliza para recorrer cada URL y recopilar su XML correspondiente.

respuesta.

Ejemplo 54. Definición de la lista de URL para leer en index.js

```
const urls = [
  "https://www.bonappetit.com/feed/recetas-rss-feed/rss",
  "https://www.budgetbytes.com/category/recipes/feed/",
  "https://www.reddit.com/r/recipes/.rss"
];
...
```

1 Asignar una lista de cadenas de URL a las URL.

Con esta lista en su lugar, ahora puede modificar la función principal mediante Recorriendo cada URL para obtener el contenido del feed (Ejemplo 5-5). Primero,

Asignar una constante feedItems a una matriz vacía: aquí es donde se encuentra su

Los elementos del feed se almacenarán. A continuación, se iterarán las URL.

matriz utilizando la función de mapa , que visitará cada URL y ejecutará el

Función parser.parseURL para devolver una Promesa en su lugar. En el

En la siguiente línea, utiliza Promise.all que espera todas las solicitudes

a URL externas para regresar con respuestas antes de completar. Cada

La respuesta se almacenará en una matriz de respuestas . Por último, se utiliza un

Función de agregación e impresión personalizada para filtrar los datos

respuestas y registre el resultado deseado, respectivamente.

Ejemplo 55. Definición de la lista de URL para leer en index.js

```
const main = async () => {
  constante feedItems = [];
  constante awaitableRequests = urls.map(url =>
```



```

parser.parseURL(url)); const ❷
  respuestas = await Promise.all(awaitableRequests); agregado(respuestas, ❸
  feedItems); imprimir(feedItems); ❹
} ❺

```

- ❶ Define una matriz feedItems para almacenar elementos de la fuente RSS.
- ❷ Ejecute parser.parseURL en cada URL, lo que devuelve un objeto Promise para eventualmente devolver una respuesta desde el punto final RSS externo.
- ❸ Recopila respuestas esperando que todas las Promesas completen sus solicitudes.
- ❹ Pase la matriz de respuestas y feedItems a una función de agregación personalizada para combinar los resultados de la fuente RSS.
- ❺ Pase la matriz feedItems resultante a una función de impresión personalizada para registrarla en su consola.

Antes de volver a ejecutar la aplicación, debe definir las funciones de agregación e impresión . Agregue el código del **Ejemplo 5-6** debajo de la función principal . En la función de agregación , se recopilan todos los datos de la fuente de cada fuente externa y, para este proyecto, solo se conservan los elementos que contienen recetas con verduras. Primero, se recorre el array de respuestas y se examinan solo los elementos dentro de cada respuesta XML. Luego, un bucle interno visita cada elemento y desestructura únicamente el título y el enlace , ya que estos son los únicos datos que le interesan en este proyecto. Con acceso al título de cada elemento, se comprueba si este incluye la cadena "veg". Si se cumple esta condición, se agrega un objeto con el título y el enlace al array feedItems .

En su función de impresión , acepta feedItems como argumento.

A continuación, borre la consola de registros anteriores utilizando console.clear.

Imprima sus feedItems en su consola usando console.table y luego registre su última hora de actualización generando un nuevo objeto Fecha y convirtiéndolo en una cadena legible para humanos.

Ejemplo 56. Definición de las funciones de agregación e impresión en index.js

```
...
const added = (respuestas, feedItems) => { for (let {items} de respuestas) {
  para (let {título, enlace} de elementos) { if (title.toLowerCase().includes('veg')) { feedItems.push({título, enlace});
  }
}
} devolver feedItems;

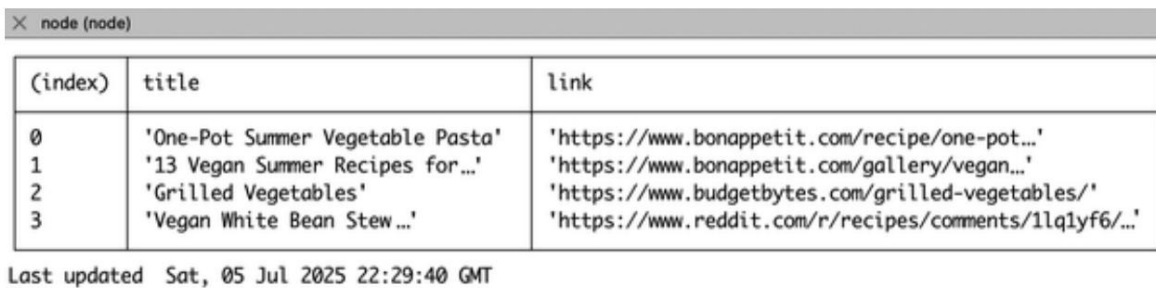
const print = feedItems => { console.clear();
  console.table(feedItems);
  console.log('Última actualización ', (new Date()).toUTCString());
}
```

- 1 Pase sus respuestas RSS y la matriz feedItems a la función de agregación .
- 2 Recorra cada respuesta del feed y acceda a su matriz de elementos .
- 3 Recorra cada elemento y extraiga solo el título y el enlace.
- 4 Si el título contiene la subcadena “veg” (sin distinguir entre mayúsculas y minúsculas), agregue el elemento a feedItems.
- 5 Devuelve la matriz feedItems filtrada.
- 6 Borrar registros anteriores de la consola.
- 7

Imprima los elementos filtrados como una tabla.

B Muestra la hora de la última actualización en formato UTC.

Ahora, reinicie su aplicación. Notará que esta vez hay menos resultados registrados en su consola (**Figura 5-4**), pero los elementos mostrados provienen de diversas fuentes, todos con títulos que indican alguna receta de verduras o vegetariana. Puede modificar la condición de agregación a su gusto centrándose en otras palabras clave o incluso examinando datos distintos del título y el enlace utilizados en este ejemplo.



A screenshot of a Node.js console window titled 'node (node)'. It displays a table with three columns: '(index)', 'title', and 'link'. The table contains four rows of data. Below the table, it shows the last updated time: 'Last updated Sat, 05 Jul 2025 22:29:40 GMT'.

(index)	title	link
0	'One-Pot Summer Vegetable Pasta'	'https://www.bonappetit.com/recipe/one-pot...'
1	'13 Vegan Summer Recipes for...'	'https://www.bonappetit.com/gallery/vegan...'
2	'Grilled Vegetables'	'https://www.budgetbytes.com/grilled-vegetables/'
3	'Vegan White Bean Stew ...'	'https://www.reddit.com/r/recipes/comments/1lq1yf6/...'

Last updated Sat, 05 Jul 2025 22:29:40 GMT

Cifra 54. Salida de consola para una tabla agregada de elementos de la fuente RSS

Finalmente, al compartir este agregador con tus colegas, podrán añadir cualquier URL de fuente RSS adicional para aumentar la cantidad de resultados relevantes. Antes de finalizar este proyecto, decides añadir una función más: añadir elementos personalizados al feed.

Añadiendo elementos personalizados a su agregador. La mayoría de los agregadores RSS recopilan únicamente los resultados de fuentes externas y los agregan según reglas definidas. No es frecuente tener la oportunidad de modificar la lista agregada resultante con contenido que no se encuentra en ningún otro lugar. El cliente de este proyecto ha sido la consola de línea de comandos. Sin embargo, es posible convertir esta aplicación en una herramienta accesible desde la web con la ayuda de un framework web. Con un framework web, puede publicar el agregador y permitir que cualquier persona lea los elementos de la fuente resultante en su...

propio navegador. Sin embargo, no necesita depender de la web ni de un navegador, para diseñar una aplicación Node independiente.

Para recopilar la información ingresada por el usuario, instale el paquete `prompt-sync` ejecutando `npm install prompt-sync@^4.2.0` en el nivel raíz de

tu proyecto en la línea de comandos. Luego, en `indexpromptModule`, agregue importación desde `'prompt-sync'`; al principio del archivo,

seguido de `const prompt = promptModule({sigint:`

`verdadero});` para crear una instancia de la función de solicitud con una configuración `sigint` que te permite salir de tu aplicación. Por último, agrega `const customItems =`

`[]`; para definir una matriz para los elementos de su feed personalizado. A continuación, modifique la función de impresión agregando el código del [Ejemplo 5-7](#) en la parte superior de esa función. La función de solicitud mostrará `Agregar elemento:` en su consola y esperar una respuesta escrita. Cuando se presiona la tecla `Enter`

Al presionarlo, la entrada se guarda en una constante de resolución. La entrada del usuario debe estar en el formato: `título + + enlace.`, Entonces, el título y el enlace son

Se extrae dividiendo la cadena de entrada resultante. El nuevo elemento personalizado...

El objeto se agrega a su matriz de constantes globales `customItems`.

Ejemplo 57. Modificar la función de impresión para aceptar la entrada del usuario en `index.js`

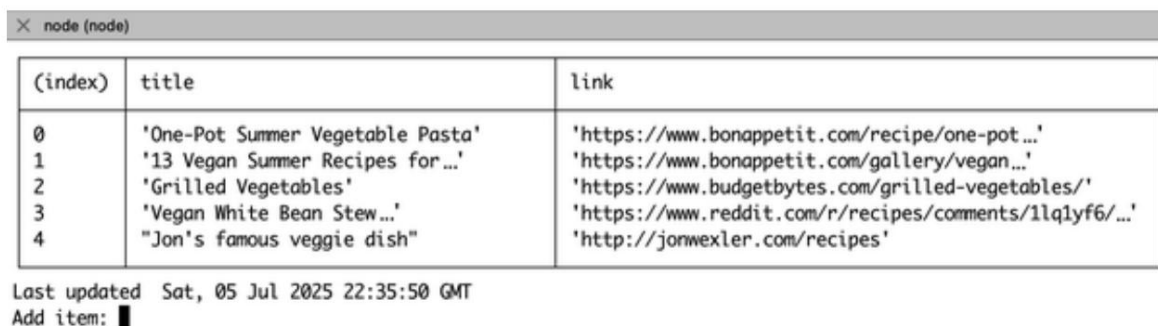
```
const res = prompt('Agregar elemento: '); ❶
const [título, enlace] = res.split(',');    ❷
si (![título, enlace].incluye(indefinido))
customItems.push({título, enlace});        ❸
...
```

❶ Solicita al usuario que agregue un nuevo título y enlace al elemento del feed.

❷ Divida la cadena de entrada con un delimitador de coma para desestructurar una Título y enlace.

❸ Agregue el objeto de elemento resultante a `customItems` si ni el título o falta el enlace.

Finalmente, modifica tu declaración de registro para incluir la matriz `customItems` reemplazando esa línea con `console.table(feedItems.concat(customItems));`. Al reiniciar la aplicación, deberías ver un mensaje solicitando un elemento de feed personalizado. Si no tienes nada que añadir, simplemente pulsa Intro. De lo contrario, puedes añadir una receta personalizada, como el famoso plato vegetariano de Jon (<http://jonwexler.com/recipes>) , y pulsar Intro para añadirla a tu feed de elementos agregados. El resultado debería ser similar al de la Figura 5-5 .



The screenshot shows a Node.js console window with a table of feed items. The table has three columns: (index), title, and link. Below the table, it shows the last updated time and a prompt to add an item.

(index)	title	link
0	'One-Pot Summer Vegetable Pasta'	'https://www.bonappetit.com/recipe/one-pot...'
1	'13 Vegan Summer Recipes for...'	'https://www.bonappetit.com/gallery/vegan...'
2	'Grilled Vegetables'	'https://www.budgetbytes.com/grilled-vegetables/'
3	'Vegan White Bean Stew...'	'https://www.reddit.com/r/recipes/comments/1lqlyf6/...'
4	'Jon's famous veggie dish'	'http://jonwexler.com/recipes'

Last updated Sat, 05 Jul 2025 22:35:50 GMT
Add item: █

Cifra 55. Salida de consola para una tabla agregada con elementos de feed personalizados

Ahora tienes una aplicación que puedes compartir con otros en tu oficina. Puedes usar el agregador para recopilar recetas relevantes cada dos segundos (o en el intervalo que elijas). También puedes agregar enlaces personalizados que no se encuentran en tus fuentes externas. Cuando los usuarios de tu aplicación publiquen sus resultados, todos podrán beneficiarse de tu nueva colección agregada de enlaces de recetas de acceso rápido. Puedes seguir desarrollando la aplicación para que sea accesible en una red compartida o crear un framework web con una base de datos integrada en tu aplicación para que los clientes web puedan acceder a los datos del feed.

EJERCICIOS DEL CAPÍTULO

1. Agregue un filtro de palabras clave configurable a su agregador:
 - a. Modifique el agregador para que los usuarios puedan especificar un palabra clave en tiempo de ejecución (por ejemplo, “vegano”, “chile”, “ensalada”) en lugar de codificar 'veg'.
 - b. Utilice prompt-sync para solicitar una palabra clave antes de que se ejecute main() y almacenar esa palabra clave en una variable global.
 - c. Actualice su función added() para filtrar títulos según si incluyen la palabra clave especificada (sin distinguir entre mayúsculas y minúsculas).
 - d. Ejecute su aplicación y pruebe cómo cambia el feed al utilizar diferentes palabras clave.

CONSEJO

Si desea hacerlo aún más dinámico, considere permitir que los usuarios cambien la palabra clave en vivo durante cada ciclo de actualización.

2. Muestra qué tan recientemente se publicó cada elemento del feed:
 - a. Actualice la función agregada() para que también incluya el campo pubDate (si está disponible) de cada elemento.
 - b. En la función print() , calcule la antigüedad de cada elemento en minutos u horas comparando pubDate con new Date().
 - c. Muestra la antigüedad de cada elemento de tu tabla en una nueva columna como Antigüedad o Publicado.

d. Si a un elemento le falta una fecha de publicación, mostrar Desconocido
o --.

Estos ejercicios le dan a su agregador más control y transparencia:
los usuarios pueden filtrar los resultados según sus intereses y ver qué tan
actualizado es cada elemento, tal como en un lector de noticias profesional.

Resumen

En este capítulo, usted:

- Creó un agregador de feeds RSS usando Node y rss-parser
- Se recuperaron y analizaron datos de feeds de fuentes externas utilizando Llamadas API
- Se agregó soporte para elementos personalizados enviados por el usuario a través de la línea de comando
- Se muestra contenido agregado en un formato de tabla legible
- Mejore su agregador con funciones como filtrado de palabras clave y seguimiento del tiempo de publicación.

Capítulo 6. API de la biblioteca

Este capítulo cubre lo siguiente:

- Construyendo una API con Fastify.js
- Creación de puntos finales REST
- Conexión a una base de datos relacional

Aunque la mayoría de las personas están familiarizadas con internet a través de su navegador web, la mayor parte de la actividad y la transferencia de datos se realiza en segundo plano. Algunos datos, como los horarios de trenes en tiempo real, están disponibles no solo como una aplicación web independiente, sino como un recurso que otros pueden usar e implementar en sus propias aplicaciones. Este recurso se denomina Interfaz de Programación de Aplicaciones (API) y permite a sus usuarios ver todos o algunos de los datos disponibles pertenecientes a un entorno restringido (como la autoridad ferroviaria). Algunas API permiten añadir, modificar y eliminar datos, especialmente si se trata de datos propios o si usted es la autoridad sobre ese recurso.

En este capítulo, crearás una API RESTful, lo que significa que permitirá el acceso y la modificación de datos mediante un protocolo estándar. También conectarás la API a una base de datos, a la que podrás añadir nueva información y desde la que podrás acceder a registros antiguos. Finalmente, adquirirás las habilidades necesarias para crear una API para prácticamente cualquier tipo de datos.

HERRAMIENTAS Y APLICACIONES UTILIZADAS EN ESTE CAPÍTULO

Antes de comenzar, asegúrese de instalar y configurar las herramientas y aplicaciones necesarias para este proyecto. Las instrucciones de instalación de Node.js, Fastify y VS Code se encuentran en [el Capítulo 1](#), mientras que los pasos de inicialización del proyecto, como la configuración de la estructura de directorios, la configuración de package.json y el uso de sintaxis moderna, se describen en [el Apéndice A](#). Para una explicación más detallada de los conceptos de SQLite, consulte [el Apéndice C](#). Una vez completado, vuelva aquí para continuar. Desarrollar un proyecto desde cero le ayuda a comprender mejor cada componente, lo que le brinda mayor control y flexibilidad a medida que avanza.

[Su propuesta](#): Su

municipio está otorgando subvenciones a personas que puedan modernizar la biblioteca pública. Su prioridad es crear un sistema que permita al público solicitar libros que aún no se encuentran disponibles o que tienen poca disponibilidad en la biblioteca. Ya se han otorgado subvenciones a grupos que diseñan los clientes móviles y web, pero aún se necesita a alguien que desarrolle la API de backend. Usted está familiarizado con Node y Fastify y decide aceptar el reto.

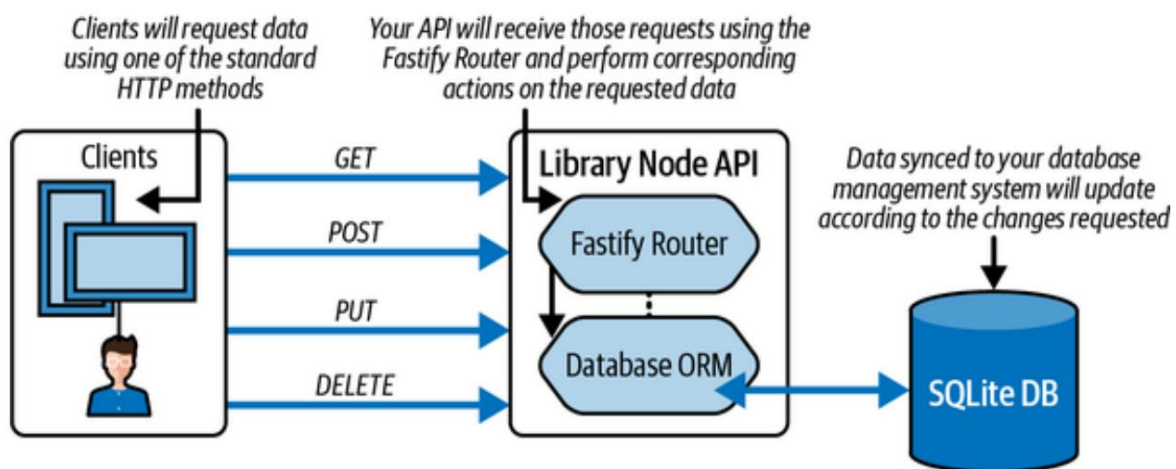
[Obtener planificación](#)

El objetivo de este proyecto es crear una API a la que las aplicaciones móviles y web puedan conectarse para ver, agregar, actualizar o eliminar registros de libros. Al igual que con las URL de un sitio web, creará una API que proporcione puntos finales (URI) accesibles para otros clientes. Para comenzar, creará una aplicación Node con el framework Fastify y gradualmente añadirá componentes para facilitar la interacción con los datos de la biblioteca y conservarlos en una base de datos. Seguirá los requisitos de diseño del proyecto, como se muestra en [la Figura 6-1](#).

NOTA

Los identificadores uniformes de recursos (URI) son un superconjunto de puntos finales que se utilizan para acceder a recursos o datos. En este caso, se utiliza una URI para obtener datos de una API. Los localizadores uniformes de recursos (URL) son un subconjunto de los URI, Generalmente asociado con el acceso a una dirección web.

Este diagrama muestra cuatro comportamientos correspondientes a acciones HTTP (GET, PUT, POST y DELETE), que deberás implementar en el API. Cuando se realiza una solicitud a la API, su aplicación debe distinguir entre estos cuatro tipos de solicitud y ejecutar la lógica de la aplicación relevante sobre los datos solicitados. En otras palabras, si alguien quiere enviar un recomendación de libro, no borre accidentalmente un libro diferente de tu base de datos. Al final del desarrollo, podrás probar tu API en su navegador web, línea de comandos o aplicación de terceros como el cartero.



MÉTODOS HTTP

El Protocolo de Transferencia de Hipertexto (HTTP) es un protocolo de Internet utilizado por sitios web y aplicaciones para transmitir datos a través de la web. HTTP ofrece una arquitectura y una semántica para enviar paquetes de datos de una dirección IP a otra. Parte de esta arquitectura incluye métodos definidos para solicitar datos.

Existen casi 40 métodos HTTP disponibles, aunque solo unos pocos representan la mayoría de las solicitudes realizadas en internet. Los siguientes métodos HTTP son los que utilizará en este libro:

CONSEGUIR

Este método de solicitud se utiliza cuando alguien visita una página de destino o una página web con contenido estático. Representa la mayoría de las solicitudes en internet y es el más sencillo de implementar en la API.

CORREO

Este método de solicitud se utiliza siempre que se envían datos a un servidor; por ejemplo, cuando un usuario inicia sesión en su cuenta. Estas solicitudes son necesarias para que la mayoría de los datos ingresen a una base de datos y permitan que la información persista entre sesiones web.

PONER

Este método de solicitud es similar a POST , ya que envía datos en el cuerpo de la solicitud. Sin embargo, en la práctica, se utiliza para actualizar o modificar datos existentes en el servidor. Estas solicitudes se distinguen de los datos POST , generalmente, por un ID que coincide con un registro en la base de datos del servidor.

BORRAR

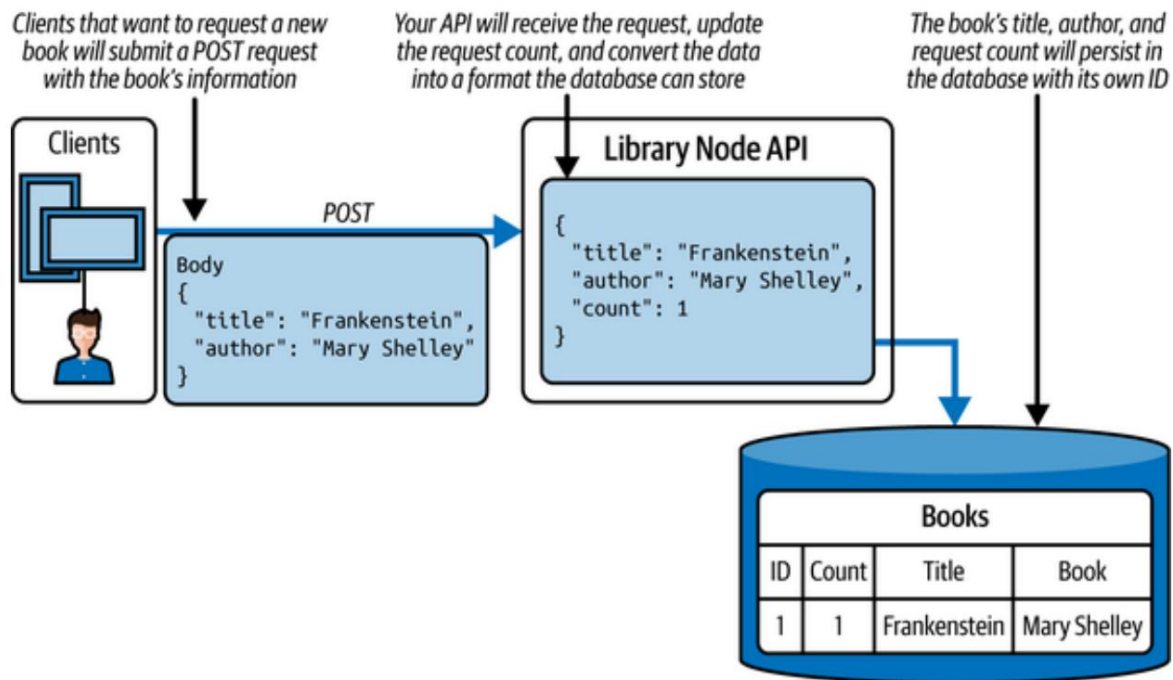
Este método de solicitud envía una clave principal al servidor, indicando que un registro debe eliminarse. La naturaleza de una solicitud DELETE es similar a la de una solicitud GET, pero a diferencia de esta última, busca cambiar el estado de los datos en el servidor.

Si bien existen muchos otros métodos de solicitud en HTTP, estos cuatro son suficientes para comenzar a desarrollar una API. Para más información sobre los métodos HTTP, visite las páginas de referencia [HTTP de Mozilla](#).

Dado que la biblioteca desea que su API dé visibilidad a los libros populares y solicitados, los datos que proporcione deben contener suficiente información para identificar dichos libros y su nivel de interés. Al configurar su base de datos, deberá almacenar el título y el autor del libro, así como un recuento de las solicitudes recibidas. De esta forma, los clientes móviles y web que utilicen su API podrán notificar a la biblioteca y a sus usuarios las solicitudes más populares.

La **Figura 6-2** muestra el flujo de datos durante una solicitud POST para un nuevo libro. Esta solicitud contiene el título y el autor del libro. Si el libro ya existe en la base de datos, el número de solicitudes aumenta.

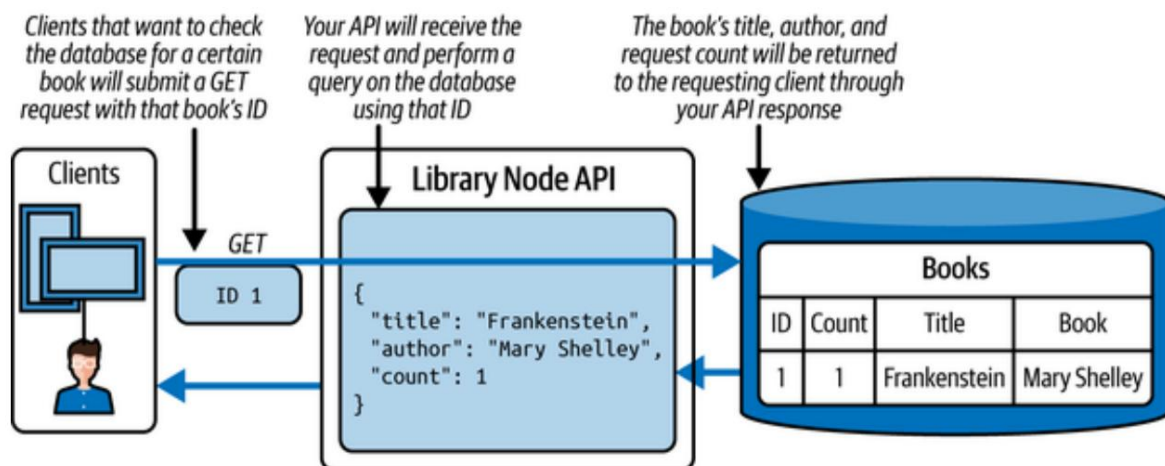
De lo contrario, el registro de ese libro se agrega a la base de datos por primera vez con su propio ID de serie.



Cifra 62. Flujo de datos durante una solicitud POST

Una vez que hay algunos datos persistentes (almacenados), los clientes de la biblioteca pueden enviar una solicitud GET usando el ID persistente de ese libro (Figura 6-3). El

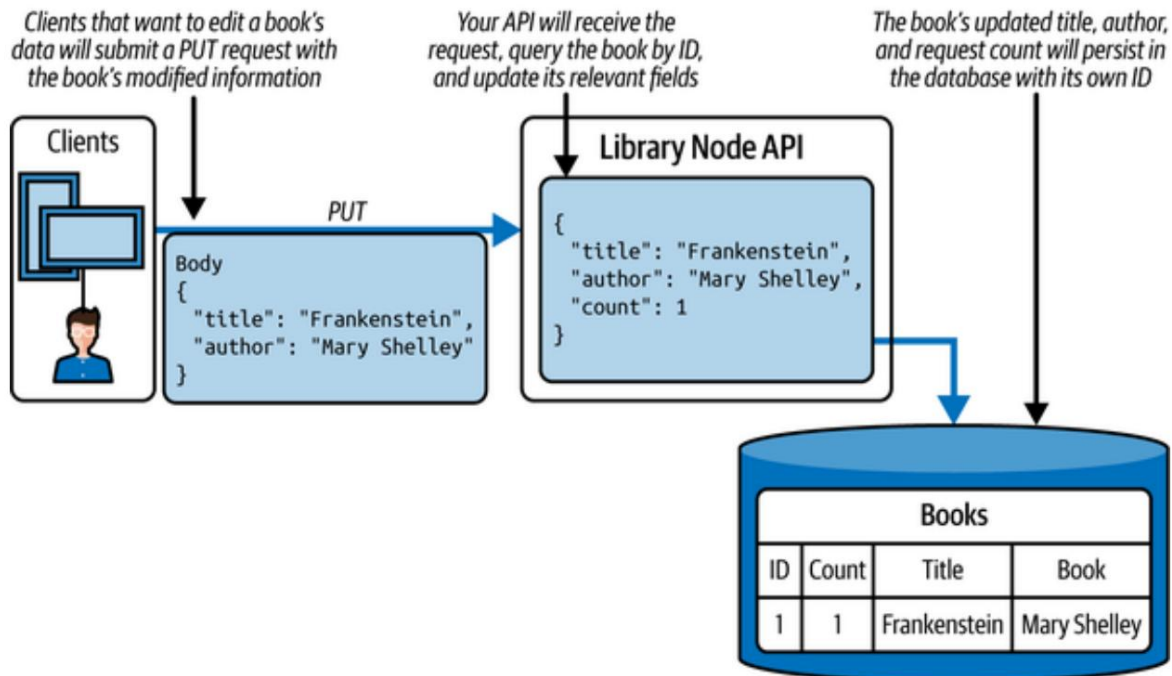
La solicitud GET solo necesita enviar un parámetro de ID, pero a su vez recibe todos los datos del libro.



Cifra 63. Flujo de datos durante una solicitud GET

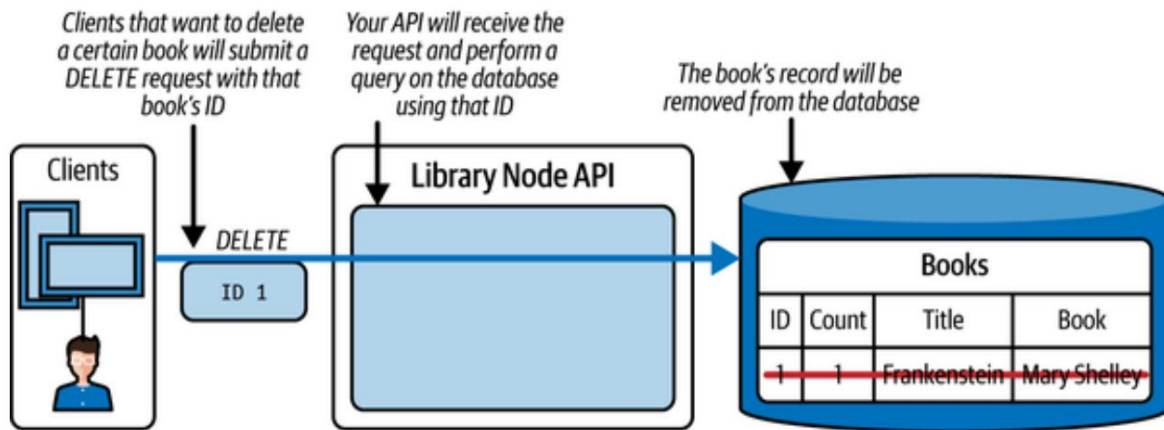
Si un administrador de biblioteca detecta un error en los datos del libro, puede modificar el título o el autor y enviar una solicitud PUT. En la Figura 6-4, una

La solicitud PUT se envía con información actualizada del libro, como corregir un título o nombre de autor. Esos nuevos datos se procesan dentro su API y se guarda en la base de datos con el mismo ID de registro.



Cifra 64. Flujo de datos durante una solicitud PUT

Por último, si la biblioteca obtiene suficientes copias de un determinado libro para satisfacer Sus usuarios pueden enviar una solicitud de ELIMINACIÓN para eliminar ese libro de la base de datos. La Figura 6-5 muestra que una solicitud DELETE solo Necesita el ID del libro como parámetro. A partir de ahí, la API puede realizar una consulta de eliminación en la base de datos.



Cifra 65. Flujo de datos durante una solicitud DELETE

Con la arquitectura y las estructuras de datos definidas, estás listo para Comienza a crear tu API con Node y Fastify. En la siguiente sección,... Establecer los componentes básicos para una aplicación Node típica.

Obtenga programación con un diseño de API

Para comenzar a desarrollar su aplicación, cree una nueva carpeta llamada `library_api`. Acceda a la carpeta de su proyecto desde la línea de comandos. y ejecuta `npm init`. Este comando inicializa tu aplicación Node.

Puede presionar Enter durante los pasos de inicialización. El resultado de estos Los pasos son la creación de un archivo llamado `package.json`. Este archivo indica su aplicación Node de cualquier configuración o script necesario para funcionar.

A continuación, crea un archivo llamado `index.js` dentro de la carpeta de tu proyecto. Este archivo... actúa como punto de entrada para su aplicación.

PASOS ADICIONALES PARA MEJORAR SU APLICACIÓN

Este libro te anima a crear cada proyecto desde cero. Por ello, los pasos de instalación para Node.js, Git y VS Code se proporcionan en [el Capítulo 1](#), mientras que los detalles de inicialización del proyecto (como la estructura de directorios, la configuración de paquetes y la sintaxis moderna) se tratan en [el Apéndice A](#). Para continuar creando tu aplicación, completa los pasos de esas secciones antes de continuar.

Para configurar tu aplicación como API, instala Fastify accediendo a la carpeta de tu proyecto desde la línea de comandos y ejecutando el comando `npm install fastify@^5.4.0`. Este comando instala Fastify y lo añade como una dependencia de paquete en tu archivo `package.json`. Con Fastify instalado, importas su módulo e inicializas una nueva instancia de la aplicación Fastify (instanciación), llamada `app`, como se muestra en [el Ejemplo 6-1](#). Durante el desarrollo de la aplicación, defines el puerto de la API como 3000 y usas la función `app.listen` para configurar tu API para que escuche las solicitudes en ese puerto.

escuchar ⁶¹. Creando una instancia de tu aplicación Fastify y comenzando a solicitudes de ejemplo

```

en index.js import Fastify from 'fastify'; const 1
app = Fastify(); const PORT = 2
3000; 3

try
  { await app.listen({ puerto: PUERTO }); 4
    console.log(`Escuchando en http://localhost:${PUERTO}`); } catch (err)
{ console.error(err);
  proceso.exit(1);
}

```

¹ Importar el módulo Fastify.

² Cree una instancia de Fastify para crear su aplicación API.

- 3 Asigne la constante PORT a 3000.
- 4 Utilice la función await de nivel superior para iniciar el servidor Fastify sin necesidad de una función contenedora.

Tu aplicación ya está lista para aceptar cualquier solicitud HTTP. Por ahora, puedes probarlo ejecutando `npm start`. Verás que la consola de comandos imprime una instrucción "Listening at http://localhost:3000" para indicar que tu aplicación se está ejecutando.

NOTA

Con el paquete nodemon instalado, solo necesita ejecutar `npm start` una vez durante el desarrollo de su aplicación. Cada cambio posterior que realice en el proyecto reiniciará automáticamente el proceso de Node y reflejará los cambios inmediatamente. Consulte el [Apéndice B](#) para obtener más información sobre la instalación de nodemon.

Tu aplicación está lista para procesar solicitudes. Sin embargo, para que sea compatible con una API, debes indicarle a Fastify el tipo de datos que tu aplicación espera recibir. Fastify te permite registrar plugins o usar el análisis integrado, que procesa las solicitudes entrantes antes de que puedas ver su contenido.

Las funciones de middleware, como su nombre indica, se ubican entre la solicitud que recibe la aplicación y la solicitud que procesa la lógica personalizada de la aplicación. En este caso, se utiliza el análisis de cuerpo JSON integrado de Fastify, que gestiona las solicitudes entrantes con datos JSON. Dado que esta API espera recibir y servir datos JSON, es necesario garantizar que esta capacidad de análisis esté habilitada.

De forma similar, registrar el complemento `@fastify/formbody` permite que la API de Fastify analice y comprenda las solicitudes con codificación URL. Esto ayuda a descifrar las cadenas modificadas y codificadas para...

Mayor eficiencia en la web. Fastify analiza automáticamente el estándar.
estructuras de datos anidadas cuando se utiliza este complemento.

Ejecute el comando `npm install @fastify/formbody@^8.0.2`
en su línea de comando y agregue el código del **Ejemplo 6-2** debajo de su
Definición de PORT en `index.js`.

Ejemplo: **6-2** Como añadir compatibilidad con análisis codificado en JSON y URL
su aplicación Fastify en `index.js`

...

`importar formbody desde '@fastify/formbody';` **1**
`esperar aplicación.register(formbody);` **2**

...

- 1** Importe el complemento Fastify que permite el análisis de formularios codificados en URL.
- 2** Registre el complemento para agregar soporte para analizar solicitudes
`application/x-www-form-urlencoded`.

Para completar la primera etapa de creación de su API, agregue el código en
Ejemplo 6-3 justo debajo de las funciones de middleware. En este bloque,
`app.get` define una ruta que solo escucha solicitudes GET.

`"/` indica que su aplicación está escuchando las solicitudes realizadas al
Punto final URI predeterminado: `httplocalhost :3000/`. Esto significa que si el HTTP

La solicitud utiliza el método GET y apunta a la URI predeterminada, la

Se ejecuta la función de devolución de llamada proporcionada. Fastify proporciona su devolución de llamada.

función con un objeto de solicitud (petición) y respuesta (respuesta) como

parámetros.(objetos de solicitud El objeto de solicitud se utiliza para examinar

el contenido de la solicitud, mientras que el objeto de respuesta le permite asignar
valores y datos del paquete en la respuesta al cliente.

NOTA

Según algunas convenciones, las variables que no se utilizan en una función, pero que están definidas, tienen un guion bajo en su nombre. Esto ayuda al ingeniero a saber que no debe esperar que esa variable tenga ningún comportamiento en la función. Por eso, el argumento de solicitud en este ejemplo se llama `_request`.

Una vez procesada la solicitud, se utiliza la función `reply.send` para responder al cliente con un JSON estructurado que contiene una clave llamada `mensaje` y el valor `"ok"`. Esta respuesta indica al cliente que el servidor API funciona correctamente.

En el **Ejemplo 6-3**, `app.setNotFoundHandler` configura una función para gestionar todas las demás solicitudes que no gestiona la ruta `GET`. Esto se denomina middleware de gestión de errores general. En esta función, el objeto de error, `e`, es el primer argumento que se pasa. De esta forma, si se produce un error, la aplicación no solo se bloqueará, sino que devolverá un código de estado 500 (error interno del servidor).

Ejemplo 63. Agregar una ruta GET en `index.js`

```
...
aplicación.get("/", async (_solicitud, responder) => { ❶
  { responder.send({ mensaje: "ok" }); }); ❷

app.setNotFoundHandler((solicitud, respuesta) => { ❸
  const { mensaje, código de estado } = solicitud.error || {}; ❹
  responder.estado(código de estado || 500).enviar({ mensaje }); }); ❺
...

```

- ❶ Agregue una solicitud GET para el punto final URI principal.
- ❷ Responder con un mensaje JSON.
- ❸ Defina el middleware de manejo de errores con un argumento de error.

- ④ Desestructura el mensaje, el seguimiento de la pila y el código de estado del error.
- ⑤ Responda con el mensaje JSON de error y el código de estado correspondiente.

Guarde los cambios y acceda a `http://host.local:3000` en su navegador web. Debería ver el mensaje `{ message: "ok" }` impreso en la pantalla.

NOTA

Si no ve un mensaje en su navegador, es posible que la URL o el número de puerto se hayan ingresado incorrectamente. Además, asegúrese de que su servidor esté funcionando. En caso de un error, es posible que el servidor finalice el proceso y espere a que se solucione el problema antes de reiniciarse.

En la siguiente sección agregarás las otras rutas necesarias para soportar la API de tu biblioteca.

Añadiendo rutas y acciones a tu app. Tu app se está ejecutando, pero no puede diferenciar entre tipos de solicitud. Además, necesitarás la capacidad de crear, leer, actualizar y eliminar (CRUD) tus datos. Estas cuatro acciones se asignan a los cuatro métodos HTTP principales compatibles con las rutas de Fastify.

A partir de este punto, te referirás a las funciones de devolución de llamada dentro de tus rutas Fastify como tus acciones CRUD.

Antes de agregar más código a `index.js`, es importante mantener y organizar la estructura de su proyecto. Hasta ahora, tiene un archivo JavaScript en su proyecto. Al final de esta sección, la estructura de directorios de su proyecto debería verse como la **del Ejemplo 6-4**. Ahora, presentará...

una nueva carpeta para separar la lógica de enrutamiento del resto de la aplicación lógica.

NOTA

La carpeta `node_modules` se genera automáticamente y aparece dentro de su carpeta de proyecto cada vez que instale un nuevo paquete externo.

Biblioteca 64. Diseño de la estructura del directorio de enrutamiento

de ejemplo -api ❶

- |
- index.js
- rutas ❷
 - |
 - index.js
 - booksRouter.js
- paquete.json
- módulos_de_nodo

❶ El nivel raíz del árbol de directorios de su proyecto contiene cuatro elementos secundarios.

❷ La carpeta de rutas almacena las rutas para manejar información relacionada con los libros. actas.

Navegue a la carpeta de su proyecto en su línea de comando y cree un Nueva carpeta llamada rutas. Dentro de esa carpeta, cree dos archivos nuevos. `index.js` y `booksRouter.js`. Dentro del código de `booksRouter` del archivo `js` agregar el **Ejemplo 6-5**.

Este código introduce una función del complemento Fastify. La función contiene Lógica del marco de Fastify para gestionar todo tipo de solicitudes de Internet. Ya ha utilizado la instancia de la aplicación Fastify para crear una ruta GET en `index.js`. Ahora, definirás tus rutas de forma más explícita dentro un complemento reutilizable llamado `booksRouter`.

Este enrutador personalizado se llama `booksRouter` porque se usa únicamente para definir rutas relacionadas con la creación, lectura, actualización o eliminación de registros de libros. La primera de estas rutas es la ruta `GET`, que se registra mediante `fastify.get()`. Dentro de esta función, `"/:id"` indica que se puede pasar un valor al endpoint, y Fastify debe traducir ese parámetro como una variable con el nombre `id`.

De esta manera, cuando un cliente solicita el registro del libro con el ID 42, puede usar el entero 42 para consultar su base de datos y encontrar un libro con la clave principal correspondiente. A continuación, desestructura el valor del ID del objeto `params` de la solicitud.

Como aún no hay una base de datos configurada, se devuelve el ID al cliente en una estructura JSON. Se encapsula la respuesta en un bloque `try/catch`; en caso de que algo falle, se podrán registrar los errores en el servidor de API. Si algo no funciona como se espera, se llama a `reply.send(err)` para devolver el error al cliente mediante el sistema de gestión de errores integrado de Fastify.

Ejemplo 65. Agregar una ruta `GET` para libros en `booksRouter.js` [función](#)

```
asíncrona booksRouter(fastify, _opts) {
  fastify.get("/:id", async (solicitud, respuesta) => { const { id } =
    solicitud.params; try { const libro = { id };

    responder.send(libro); } catch
      (e) { console.error("
Ocurrió un error:",
      e.message); responder.send(e);

  } });
}
```

❶ Define una función de complemento de Fastify que tome la instancia de Fastify y los parámetros opcionales.

❷ Registra una ruta `GET` con un parámetro `:id` dinámico.

❸

Desestructurar el valor de identificación de los parámetros de ruta de la solicitud.

- ④ Crea un objeto de libro con el ID indicado.
- ⑤ Envía el objeto de libro como una respuesta JSON al cliente.
- ⑥ Registre cualquier error detectado en la consola del servidor.
- ⑦ Responda con el error utilizando el mecanismo de manejo de errores de Fastify.

Para probar esto, agregue exportar libros predeterminados de Router al final de su archivo `booksRouter.js` para permitir que se acceda a este módulo en otra parte de su aplicación. A continuación, abra el archivo `index.js` dentro de su carpeta de rutas del proyecto y agregue el código en el **Ejemplo 6-6**. Similar a El código en `booksRouter.js`, este `complemento.js` registra el `booksRouter` de índice con la instancia de Fastify. Sin embargo, este archivo no define nuevas rutas, sino que simplemente organiza las rutas existentes bajo un espacio de nombres. `fastify.register(booksRouter, { prefijo: '/books' })` indica al servidor que maneje todas las solicitudes con un Ruta URI de `/books` usando las rutas de `booksRouter`. Después de este cambio, Se esperaría que se enviara una solicitud GET a `http://localhost:3000/books/42` para un libro con ID 42.

Ejemplo: Registrar las rutas de su aplicación en `routes/index.js`

```

importar booksRouter desde "./booksRouter.js"; ①

función asíncrona rutas(fastify, _opts) { ②
  fastify.register(booksRouter, { prefijo: "/books" }); ③
}

exportar rutas predeterminadas; ④
```

- ① Importe el complemento `booksRouter` desde `booksRouter.js`.
- ② Defina una función del complemento Fastify para agrupar y registrar todos tus rutas.

- 3 Registre booksRouter en el espacio de nombres /books .
- 4 Exporte el complemento para que pueda usarse en el archivo de su aplicación principal.

Con el complemento de rutas configurado, importe este complemento en indexjs en el nivel raíz de su proyecto agregando rutas de importación desde `'./routes/index.js'`; Luego, registra este complemento con tu Instancia de Fastify agregando `app.register(routes, { prefix: "/api" })`; justo encima del bloque de gestión de errores en indexjs. Esto El nuevo código define un espacio de nombres adicional llamado /api. Este será El cambio de espacio de nombres final y le permitirá realizar solicitudes GET a la ruta `/api/books/:id` .

RUTAS DE DESCANSO

Este proyecto utiliza el enrutamiento para dirigir las solicitudes entrantes a través de tu aplicación. Una ruta es simplemente una forma de llegar desde un punto final URI específico a la lógica de tu aplicación. Puedes crear cualquier tipo de ruta, con los nombres que desees y tantos parámetros dinámicos como desees. Sin embargo, la forma en que diseñes tu estructura de enrutamiento tiene efectos secundarios y consecuencias para quienes usan tu API.

Por esta razón, este proyecto utiliza la Transferencia de Estado Representacional (REST) como convención para estructurar sus rutas. REST proporciona una configuración estándar de puntos finales URI que permite a sus usuarios saber qué tipo de recurso pueden esperar recibir a cambio.

Por ejemplo, si busca un libro específico en la base de datos, su punto final podría ser: `/Frankenstein/`

`database_books/return_a_book/`. Si bien esta ruta incluye la mayor parte de la información necesaria para obtener los detalles del libro, no sigue las convenciones RESTful y probablemente difiera en estructura de otras rutas en su API, lo que dificulta su uso consistente.

Una API RESTful permite a sus usuarios comprender rápida y fácilmente qué parte de la ruta se refiere al nombre del recurso y qué partes incluyen los datos necesarios para una consulta a la base de datos. De esta forma, una ruta como `/books` puede usarse tanto para solicitudes GET como POST, y el servidor entiende que cada solicitud para el mismo recurso: `books` se gestiona con una lógica diferente. Además, una ruta como `/books/:id` se añade al nombre del recurso, pero proporciona un parámetro dinámico: `id`. Esta estructura estándar facilita el uso y el diseño de una API para todos los involucrados. Para más información sobre el enrutamiento RESTful, visita la página [del glosario REST de Mozilla](#).

Ahora, reinicie su aplicación si aún no se está ejecutando y navegue hasta <http://localhost:3000/> en su navegador web. Debería

Vea { "id": "42" } impreso en su ventana. Con su ruta GET

trabajando, es hora de agregar las rutas para POST, PUT y DELETE.

Convenientemente, sus rutas PUT y DELETE se ven idénticas a sus GET

Ruta. Las tres requieren un parámetro de identificación . Duplica la ruta GET dos veces.

pero cambia uno de los métodos de ruta de los duplicados a fastify.put y

El otro para fastify.delete. Esta adición debería ser suficiente para probar esas rutas

Para la ruta POST , agregue el código del **Ejemplo 6-7** justo encima de su exportar la línea predeterminada de booksRouter en la parte inferior de librosRouter.js. .

En esta ruta, fastify.post se utiliza para que Fastify escuche POST

solicita específicamente. Dentro de la acción, se desestructura el título.

y el autor del cuerpo de la solicitud y devolverlos al cliente en

Formato JSON. El cuerpo de una solicitud es normalmente donde encontrará

Solicitar datos al publicar para crear o cambiar información en el servidor.

Ejemplo 67. Agregar una ruta POST para libros en booksRouter.js

```
...
fastify.post("/", async (solicitud, respuesta) => {
  const { título, autor } = solicitud.cuerpo;
  intentar {
    const libro = { título, autor };
    responder.enviar(libro);
  } captura (e) {
    console.error("Error ocurrido:", e.message);
    responder.enviar(e);
  }
});
...
```

1 Registra la ruta POST para libros.

2

Desestructurar el título y el autor del cuerpo de la solicitud.

- 3 Asigna el título y el autor a una constante, libro, y devuelve el valor JSON al cliente.

Con estos últimos cambios, es hora de probar otras rutas que no sean GET. Para probar estas rutas, abra una nueva ventana de línea de comandos y ejecute un comando cURL contra su servidor API.

NOTA

La URL de cliente (cURL) es una herramienta de línea de comandos para transferir datos a través de la red. Dado que ya no solo solicita ver datos, puede usar este enfoque para enviarlos a su servidor directamente desde la línea de comandos.

1. Pruebe la ruta GET nuevamente ejecutando:

```
rizo http://localhost:3000/api/books/42
```

El texto resultante en la consola de línea de comandos debería leerse `{"id": "42"}`.

2. Envíe una solicitud POST ingresando:

```
rizo -X POST -d \  
'título=Frankenstein&autor=Mary Shelley' http://\  
localhost:3000/api/books/
```

Este comando usa `-X` para especificar un método POST y `-d` para enviar los datos del cuerpo de la solicitud. El resultado debería ser similar a `{"title": "Frankenstein", "author": "Mary Shelley"}`.

3. A continuación, intente una solicitud PUT ejecutando:

```
curl -X PUT -d \  
  'título=Frankenstein&autor=Mary Shelley'  
http://localhost:3000/api/libros/42
```

Esta solicitud contiene un ID y datos en el cuerpo. Su respuesta debería mostrar {"id":"42"}.

4. Por último, ejecuta:

```
curl -X ELIMINAR http://localhost:3000/api/books/42
```

para ver la misma respuesta {"id":"42"} para una solicitud DELETE .

Ahora que las cuatro rutas son accesibles, es hora de la última pieza del rompecabezas: el almacenamiento persistente en una base de datos.

Conectar una base de datos a tu aplicación El almacenamiento de datos

puede ser sencillo o convertirse en un problema complejo, según la arquitectura de tu aplicación. Gracias a la popularidad de Node, prácticamente cualquier tipo de almacén de datos y sistema de gestión de bases de datos puede usarse con JavaScript. Para este proyecto, puedes elegir entre usar una base de datos NoSQL como MongoDB o una base de datos SQL (relacional).

A pesar de no haber introducido aún otros tipos de datos aparte de los títulos y autores de los libros, decide guardar sus datos en tablas de bases de datos relacionales. Considera que, con el tiempo, este proyecto podría incorporar la enorme cantidad de información que se encuentra en otras partes del sistema de la biblioteca, por lo que una base de datos relacional podría ser la opción adecuada.

NOTA

No hay una elección incorrecta al elegir una base de datos. En esta etapa del desarrollo del proyecto, todas las opciones de bases de datos populares funcionarán correctamente.

Aunque haya limitado su decisión a una base de datos SQL, hay muchos sistemas de gestión de bases de datos diferentes para elegir.

Decides comparar el uso de una base de datos SQLite y una base de datos PostgreSQL.

COMPARACIÓN DE SQLITE Y POSTGRESQL

SQLite y PostgreSQL son dos de los sistemas de gestión de bases de datos relacionales (SGBDR) más utilizados. Si bien SQLite es capaz de gestionar datos en la mayoría de los casos, como su nombre indica, este SGBDR es ligero y requiere mucha menos configuración que otros sistemas. SQLite se considera una base de datos integrada porque no requiere un servidor adicional para conectarse a ella. Por lo tanto, SQLite es la opción preferida para desarrollar rápidamente una aplicación para demostración, o incluso para su uso a largo plazo en una aplicación con tráfico mínimo.

PostgreSQL, al igual que SQLite, es de código abierto y admite una estructura relacional entre elementos de datos. PostgreSQL añade una capa adicional al admitir el mapeo objeto-relacional, lo que facilita el almacenamiento persistente de más tipos de datos en el código de la aplicación.

PostgreSQL se ejecuta en un servidor separado, lo que agrega más sobrecarga al proceso de desarrollo general.

Para obtener más información, lea el artículo [“SQLite vs PostgreSQL: 8 diferencias críticas”](#).

En general, si bien ambas bases de datos son suficientes para este proyecto, descubrirá que SQLite hará que su aplicación funcione más rápido.

Comienza a incorporar SQLite instalando su npm más reciente paquete y ejecutar el comando npm install

sqlite3@^5.1.7. Ahora que tiene un RDBMS instalado,

Podría simplemente conectarse a la base de datos y comenzar a ejecutar consultas SQL. buscar, guardar, modificar y destruir datos. Pero ¿qué tiene de divertido?

¿Desarrollar una API de Node si no puedes hacerlo todo puramente en JavaScript?

Instale otro paquete llamado sequelize ejecutando npm

Instale sequelize@^6.37.7 en su línea de comando.

sequelize es un mapeador relacional de objetos (ORM) entre

Objetos JavaScript y bases de datos SQL. Con este paquete instalado,

Puedes definir clases de JavaScript con sequelize y tenerlas

Asignar automáticamente funciones a sus consultas SQL correspondientes. Por lo tanto

No se necesitan conocimientos de SQL para este proyecto (o, realmente, ninguno en este libro).

Antes de continuar, revise el directorio de su proyecto. En este último...

En esta sección agregarás dos subcarpetas más, como se muestra en [el Ejemplo 6-8](#).

Crea la primera carpeta, modelos, y dentro de ella crea un archivo llamado

bookjsEste archivo contiene el código que Sequelize necesita para mapear su

Datos del libro a la base de datos. A continuación, cree la carpeta db . Dentro de esta

carpeta crea un archivo llamado configjs, que contendrá todos los

configuraciones necesarias para configurar su base de datos. Después de agregar todas las

cambios requeridos, su base de datos vivirá dentro de la carpeta de la aplicación

en un archivo llamado databasesqlite.

Ejemplo 68. Estructura del directorio del proyecto con una base de datos

API de biblioteca

❶

```
|
- index.js
- rutas
  |
  - index.js
  - booksRouter.js
- modelos ❷
  |
  - libro.js
```

```
-db ❸
  |
  - config.js
  - base de datos.sqlite
- paquete.json
- módulos_de_nodo
```

❶ La raíz de su proyecto contiene el archivo de su aplicación principal, carpetas para Rutas, modelos y configuración de bases de datos, además de sus packagejson y dependencias.

❷ La carpeta de modelos almacena todas las clases de datos asignadas entre su aplicación y la base de datos.

❸ La carpeta db contiene la lógica de conexión de su base de datos y la Archivo de base de datos SQLite.

Abra su archivo config.js y agregue el código del **Ejemplo 6-9**. En este código, Importa Sequelize e instancia una nueva conexión de base de datos utilizando SQLite, definiendo la ubicación de almacenamiento dentro de la carpeta db de su proyecto. Luego, autentica la conexión a la base de datos.

a través de db.authenticate(). Si la conexión es exitosa, Reciba una declaración registrada que lo indique. De lo contrario, registrará el error. que ocurrió al intentar conectar. Por suerte, no hay problemas adicionales. servidor para ejecutarse con SQLite, por lo que no debería haber muchos problemas. Solucione el problema en este paso. Al final del archivo, exporta ambos Sequelize la clase y la instancia de base de datos .

Ejemplo 69. Configurar la configuración de la base de datos en config.js

de importación { Sequelize } desde "sequelize"; ❶

```
constante db = nueva Sequelize({ ❷
  dialecto: "sqlite",
  almacenamiento: "./db/database.sqlite",
});
```

```
intentar {
  esperar db.authenticate(); ❸
```

```

    console.log(" Se ha establecido la conexión
con éxito."); } catch
(error) {
    console.error("No se puede conectar a la base de datos:", error); }

```

④

```

exportar predeterminado { ⑤
    Secuelar,

```

```

db, };

```

①

Importe la clase Sequelize del paquete sequelize .

②

Cree una instancia de una nueva conexión de base de datos Sequelize, db, usando SQLite.

③

Espere una conexión a la base de datos con db.authenticate().

④

Registra un mensaje de error si la base de datos no puede conectarse.

⑤

Exporte tanto la clase Sequelize como la instancia de la base de datos.

La base de datos está casi lista para iniciarse, pero primero necesita algunos datos para mapear en tu aplicación. Agrega el código del [Ejemplo 6-10](#) a bookjs.

En este archivo, se importan las configuraciones de la base de datos y se desestructuran los valores de Sequelize y db . A continuación, se utiliza la función db.define para crear un modelo de Sequelize llamado Book. Este nombre de modelo se asigna posteriormente en la base de datos SQLite para crear una tabla correspondiente con el mismo nombre. Los campos de este modelo reflejan los datos que la biblioteca desea almacenar:

- Un título como una cadena (título)
- Nombre del autor como cadena (autor)
- Número de solicitudes realizadas para el libro como un entero (count)

Estos campos son todo lo que se necesita para guardar innumerables registros de libros en su base de datos (aunque la estará contando). Book.sync se iniciará Sincronizar con la base de datos y configurar una tabla llamada Libros. Al final del archivo, exporta el modelo para usarlo nuevamente en tu booksRouter.js archivo.

CONSEJO

Pasar la opción `{force: true}` a `Book.sync` garantiza que con Cada vez que se inicia la aplicación, la función de sincronización intenta crear una tabla nueva. Si se produjeron cambios desde la última ejecución. Esto es útil para el desarrollo. Si no desea llenar su base de datos con demasiados registros de prueba.

Ejemplo de definición de un modelo de libro en book.js

importar configuración desde `'../db/config.js';` ❶

`const {Sequelize, db} = config;` ❷

`const Libro = db.define('Libro', {` ❸

 título: { ❹

 tipo: Sequelize.STRING,

 único: verdadero

 },

 autor: { ❺

 tipo: Sequelize.STRING

 },

 contar: { ❻

 tipo: Sequelize.INTEGER,

 valor predeterminado: 0

 },

}, {});

`Libro.sync();` ❼

`exportar libro predeterminado ;` ❽

❶

Importar todas las configuraciones desde config.js.

❷

Desestructura Sequelize y db desde tu módulo de configuración.

- 3 Define el modelo de libro con Sequelize.
- 4 Defina un campo de título como una cadena que no puede tener un valor duplicado en la base de datos.
- 5 Define un campo de autor como una cadena.
- 6 Define un campo de conteo como un entero que se incrementará con cada solicitud POST .
- 7 Sincronizar el modelo de libro con la base de datos SQLite.
- 8 Exportar el modelo de libro .

Con tu modelo de Sequelize configurado, deberás revisar las acciones CRUD que creaste previamente en `booksRouter.js`. Estas acciones actualmente devuelven los datos que reciben. Ahora que tienes acceso a una base de datos, puedes agregar la lógica necesaria para el procesamiento de datos en tu API.

Primero, importe su modelo `Book` a `booksRouter.js` añadiendo `"import Book from '../models/book.js';"` al principio del archivo. Esto le da acceso al objeto ORM `Book` y le permite crear, leer, actualizar y eliminar datos de `Book`. Cambie los valores asignados a las variables `book` y `books` en cada ruta para usar el resultado de sus consultas a la base de datos ([Ejemplo 6-11](#)).

El primer cambio realiza una llamada a `Book.findById`, donde el ID de los parámetros de la solicitud se pasa como clave principal para buscar en la base de datos un registro de libro coincidente. Se utiliza la palabra clave `"await"`, ya que se realiza una llamada asíncrona a la base de datos y se debe esperar el resultado antes de continuar. El siguiente cambio se realiza en la solicitud POST , `app.post`, donde se utiliza la función `Book.create` y se pasa el título y el autor obtenidos anteriormente.

Desde el cuerpo de la solicitud. Se espera a que la función de creación se complete y devuelva al cliente el registro resultante. De forma similar, `Book.update` también acepta el título y el autor como parámetros, pero esta vez reflejan los valores modificados de título y autor. Un segundo parámetro en esta solicitud PUT utiliza una clave WHERE para identificar el registro que se va a actualizar mediante su clave principal: `id`. Por último, `Book.destroy` utiliza la clave WHERE para buscar un registro por el `id` especificado en la solicitud DELETE y elimina el registro coincidente de la base de datos.

Para obtener más información sobre los tipos de consulta de modelos y la API de Sequelize, visita las [referencias de la API de Sequelize](#).

Ejemplo de actualización de rutas con el modelo Book en `booksRouter.js`

```

...
// OBTENER /libros/:id
const libro = await Libro.findByPk(id);           ❶
...
// POST /libros/ const
libro = await Libro.create({título, autor});      ❷
...
// PUT /libros/:id const
libro = await Libro.update({título, autor}, { donde: { id }, });  ❸

...
// ELIMINAR /libros/:id const
libro = await Libro.destroy({ donde: { id } });   ❹

```

- ❶ Ejecuta una consulta para encontrar un libro por su clave principal
- ❷ Ejecuta una consulta para crear un nuevo registro de libro con un campo de título y autor
- ❸

Ejecuta una consulta para encontrar un libro por su clave principal y actualiza los campos de autor y título

4

Ejecuta una consulta para encontrar un libro por su clave principal y eliminarlo de la base de datos

NOTA

La respuesta de las rutas PUT y DELETE no es el registro actualizado o eliminado, sino la cantidad de registros afectados por la consulta.

Esto se debe a que estas acciones no devuelven el registro actualizado o eliminado de forma predeterminada.

Ahora pruebe sus cambios, solo que esta vez los resultados son diferentes, ya que cada comando genera una acción en la base de datos. Observe el campo id que se devuelve en algunas respuestas. Observe también los campos updatedAt y createdAt que Sequelize agrega automáticamente para registrar cuándo se han introducido o modificado los datos en la base de datos. Vuelva a la línea de comandos, abra una nueva ventana y ejecute los siguientes comandos cURL:

1. Envíe una solicitud POST introduciendo `curl -X POST -d 'title=Frankenstein&author=Mary Shelly' http://localhost:3000/api/books/`. Este comando usa `-X` para especificar un método POST y `-d` para enviar los datos del cuerpo de la solicitud. El resultado debería ser similar a `{"count":0,"id":1,"title":"Frankenstein","author":"Mary Shelly","updatedAt":"2025-07-13T21:37:53.372Z","createdAt":"2025-07-13T21:37:53.372Z"}`.
2. Pruebe la ruta GET nuevamente ejecutando `curl http://localhost:3000/api/books/1`. El resultado

El texto en la consola de línea de comandos debería ser
`{"id":1,"title":"Frankenstein","author":"Mar y Shelly","requests":null,"createdAt":"2025-07-13T21:37:53.372Z","updatedAt":"2025-07-13T21:37:53.372Z"}`. Este registro se guardó en la base de datos en la última solicitud. Ahora se puede recuperar por ID.

3. A continuación, ejecute una solicitud PUT ejecutando `curl -X PUT -d 'title=Frankenstein&author=Mary Shelley' http://localhost:3000/api/books/1`. Esta solicitud contiene un ID y datos en el cuerpo. Su respuesta debería mostrar `[1]` para indicar que se modificó un registro. Puede ejecutar la solicitud GET de nuevo para ver el valor actualizado.
4. Por último, ejecute `curl -X DELETE http://localhost:3000/api/books/1` para obtener `1` como respuesta a una solicitud DELETE que indica que se eliminó un registro. Al ejecutar la solicitud GET de nuevo, debería devolver un valor nulo, ya que el registro ya no existe.

NOTA

Si ejecuta la solicitud POST una segunda vez con los mismos datos, obtendrá un error de validación en la consola de su servidor. Esto es previsible por diseño, ya que su modelo Book tiene un criterio de validación que establece que los libros nuevos deben tener títulos únicos.

Su API ya está configurada para gestionar nuevas solicitudes entrantes para crear, leer, actualizar y eliminar registros de libros. Si decide ampliar su API, puede añadir nuevos modelos o modificar la lógica de sus acciones existentes.

EJERCICIOS DEL CAPÍTULO

1. Cuente las solicitudes de libros existentes antes de crear uno nuevo entrada:
 - a. Actualice su ruta POST `/api/books` para verificar si ya existe un libro con el mismo título en la base de datos.
 - b. Si el libro existe, incremente su campo de conteo en 1 y guarde la actualización en lugar de crear un nuevo registro.
 - c. Si el libro no existe, cree una nueva entrada de libro con el recuento inicializado en 1.
 - d. Pruebe sus cambios utilizando múltiples solicitudes POST para el mismo título y asegúrese de que el recuento aumente correctamente.

CONSEJO

Utilice `Book.findOne({ where: { title } })` para verificar si hay duplicados, luego llame a `.save()` después de modificar el recuento.

2. Admite la inclusión de todos los libros en la base de datos:
 - a. Agregue una nueva ruta GET `/api/books` que devuelva Todos los libros en la base de datos.
 - b. Utilice `Book.findAll()` para obtener la lista completa de registros.
 - c. Responda al cliente con una matriz JSON de todos los libros almacenados.

d. Pruebe su punto final en un navegador o con curl `http://localhost:3000/api/books`.

Esto permite que la biblioteca vea todas las solicitudes en un solo lugar y, opcionalmente, las ordene o filtre en el lado del cliente.

Resumen En

este capítulo, creó una API completamente funcional con Node y Fastify. Además, agregó una base de datos SQL y usó la biblioteca Sequelize para conservar los datos procesados en la lógica de su API. Con las habilidades aprendidas en este capítulo, ahora podrá:

- Diseñar y organizar API RESTful personalizadas
- Cree una API que se conecte a cualquier base de datos SQL usando Sequelize
- Amplíe su API con nuevos puntos finales y modelos de recursos

Capítulo 7. Análisis de sentimientos del procesador de lenguaje natural

Este capítulo cubre lo siguiente:

- Procesamiento de texto mediante bibliotecas impulsadas por aprendizaje automático
- Envolviendo un modelo de aprendizaje automático en un servicio Node
- Creación de una aplicación de línea de comandos interactiva

En este capítulo, creará una aplicación de análisis de sentimientos utilizando Node y técnicas de procesamiento del lenguaje natural (NLP) para interpretar el tono emocional del texto cotidiano y adquirir experiencia práctica trabajando con aprendizaje automático (ML) en JavaScript.

Con estas técnicas, aprenderás las técnicas de un ingeniero de ML y crearás tu propia aplicación Node basada en ML. El ML ha ido ganando importancia gracias a su uso en tecnología en prácticamente todos los sectores.

Como subconjunto de la inteligencia artificial (IA), los equipos de ML, desde startups hasta grandes empresas, se esfuerzan por ofrecer una experiencia que se acerque lo más posible a lo que se espera de un experto humano. Si eres nuevo en ML, solo necesitas saber que se realizan muchos cálculos matemáticos y estadísticos con datos del mundo real para proporcionarte una función de JavaScript que puede tomar una entrada y ofrecer un resultado, como una puntuación de sentimiento, basado en patrones aprendidos durante el entrenamiento con conjuntos de datos etiquetados. Si estás familiarizado con el funcionamiento del ML, es probable que conozcas los diversos modelos que se utilizan actualmente. Un modelo de ML es lo que los científicos de datos entrenarán para construir una función que se generalice bien a nuevas entradas desconocidas.

Desde ayudar a predecir su próxima compra hasta recomendaciones de películas y reconocimiento facial, prácticamente no hay límites para el uso de los modelos de aprendizaje automático. Históricamente, la tecnología ha dependido de la inteligencia artificial.

Condiciones (básicamente , sentencias if) o reglas para extraer resultados concluyentes. Gradualmente, lenguajes como Python, que ahora domina el mundo del aprendizaje automático (ML), comenzaron a ofrecer nuevas herramientas y soporte de desarrollo para la creación de nuevos modelos. Un par de décadas después, los ingenieros de JavaScript pueden usar Node para crear modelos comparables. Esto significa que también se pueden crear modelos de ML para reconocer objetos en una imagen, crear aplicaciones de precios dinámicos o de pronóstico del mercado de valores, o usar NLP para analizar una cadena de texto de diversas maneras.

HERRAMIENTAS Y APLICACIONES UTILIZADAS EN ESTE CAPÍTULO

Antes de comenzar, asegúrese de instalar y configurar las herramientas y aplicaciones necesarias para este proyecto. Las instrucciones de instalación de Node.js, Fastify y VS Code se encuentran en [el Capítulo 1](#), mientras que los pasos de inicialización del proyecto, como la configuración de la estructura de directorios, la configuración de package.json y el uso de sintaxis moderna, se describen en [el Apéndice A](#). Para una explicación más detallada de los conceptos de SQLite, consulte [el Apéndice C](#). Una vez completado, vuelva aquí para continuar. Desarrollar un proyecto desde cero le ayuda a comprender mejor cada componente, lo que le brinda mayor control y flexibilidad a medida que avanza.

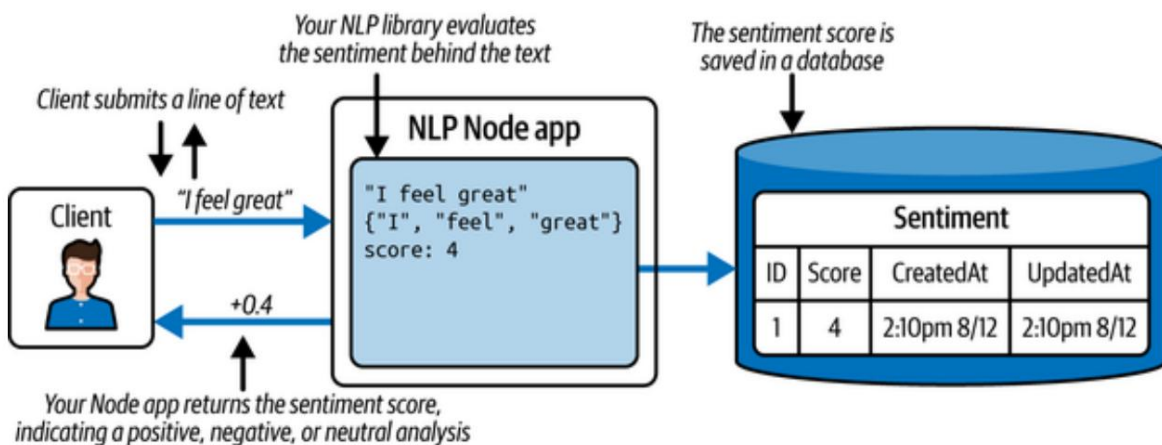
Su sugerencia. Una

clínica de terapia local, Woes Disposed, anima a sus clientes a escribir una frase en sus diarios personales cada hora. Los terapeutas descubren que la autorreflexión a lo largo del día puede ayudar a enriquecer la perspectiva de la vida. También creen que si el cliente puede recibir retroalimentación inmediata sobre si su entrada en el diario fue positiva o negativa, le ayudará a monitorear su estado de ánimo a lo largo del día y a impulsar su desarrollo. En definitiva, esta práctica busca que...

Construir una aplicación para analizar el texto de una revista y visualizar su análisis de sentimientos.

Obtener planificación EI

objetivo de este proyecto es crear una aplicación que pueda mostrar si una cadena de entrada dada es una declaración positiva o negativa. La aplicación podría usarse finalmente a través de un cliente web, pero puede existir con una CLI para comenzar. A medida que se escriben cadenas (digamos de una a tres oraciones) en la línea de comandos, su aplicación debe descomponer los elementos de la oración, analizar la disposición de las palabras y devolver una puntuación numérica que represente el sentimiento del texto. Usted sabe que hay paquetes npm que pueden ayudarlo a hacer precisamente eso, y su experiencia con JavaScript del lado del servidor facilita la implementación de esta lógica en una aplicación Node. Dibuja un diagrama que detalla cómo su aplicación manejará el flujo de datos desde el cliente y de regreso en la Figura 7-1.



Cifra 71. Plan de proyecto para una aplicación Node que analiza el sentimiento del texto

Debido a que los lenguajes de programación tradicionales no son adecuados para interpretar texto en lenguaje natural, necesitará la ayuda de modelos ML.

Podrías entrenar tu propio modelo de aprendizaje automático de PNL, aunque eso implicaría obtener cientos de miles, si no cientos de millones, de muestras de texto etiquetadas por humanos como positivas o negativas, y ajustar continuamente diversos parámetros hasta que analice correctamente las nuevas oraciones. Afortunadamente, este trabajo ya lo ha realizado el...

Comunidad de código abierto que le ofrece modelos ya entrenados y probados que puede utilizar en textos en inglés comunes.

La PNL tiene como objetivo mejorar la capacidad de una computadora para comprender texto humano. A veces, esto puede ser útil para corregir la sintaxis y la gramática, identificar oraciones similares y, en tu caso, medir el sentimiento del texto. Sin embargo, el lenguaje humano es complejo.

El inglés, en particular, presenta muchas irregularidades, conjugaciones verbales y múltiples significados para una misma palabra, además de que la puntuación y las palabras mal escritas pueden afectar los resultados de un modelo de PNL. Por ello, los modelos de PNL ofrecen herramientas adicionales para depurar el texto antes de analizarlo.

LIMPIEZA DEL TEXTO DE ENTRADA PARA EL ANÁLISIS DE PLN

El inglés escrito puede tener múltiples interpretaciones para el lector.

Por ejemplo, "No he sido feliz hasta ahora" podría implicar que no te sientes feliz al tener una cita o que nunca has sido feliz en general. Esto suele ser confuso para los intérpretes humanos y, por lo tanto, un desafío para las computadoras.

Aunque las oraciones confusas pueden seguir representando un obstáculo para los modelos de PLN, existen algunos métodos que ayudan a reducir una oración compleja a su estructura lingüística fundamental. Estos pasos de preprocesamiento se aplican comúnmente antes de pasar la entrada a muchos procesos de PLN tradicionales, especialmente para tareas como el análisis de sentimientos:

1. Corrección ortográfica

Como era de esperar, se utiliza la corrección ortográfica para garantizar que cualquier error tipográfico o palabra mal escrita se corrija antes del análisis. Después de todo, no tiene sentido intentar analizar el sentimiento de un texto que ni siquiera un humano puede comprender. Corregir "Tengo una pregunta" por "Tengo una pregunta" lo facilita mucho.

Para obtener más información sobre la corrección ortográfica, consulte la [edición en línea de Introducción a la recuperación de información \(Manning et al., Cambridge University Press\)](#).

2. Eliminación de palabras vacías

La eliminación de palabras vacías consiste en eliminar palabras que no contribuyen a la comprensión general del texto. Palabras como "no" o "nunca" pueden tener un significado significativo y no deben eliminarse a ciegas. Para más información sobre las palabras vacías, consulte la [edición en línea de Introducción a la Recuperación de Información](#).

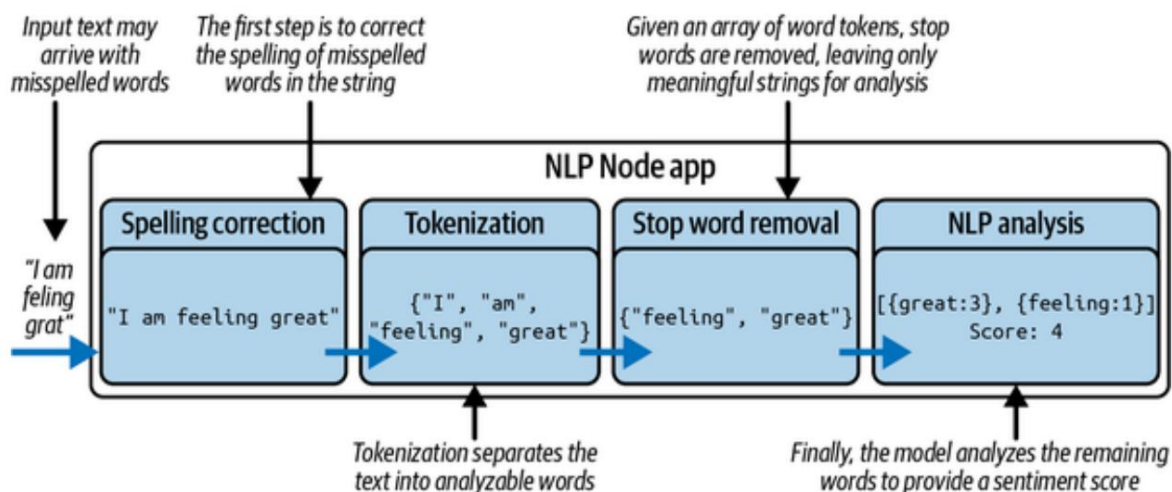
3. Derivación y lematización

La lematización reduce ciertas palabras a sus raíces. Por ejemplo, "goes" y "going" se lematizarán como "go". La lematización va un paso más allá al asignar palabras en contextos específicos a su lema. Por ejemplo, "am", "are" y "is" se asignarían a su lema, "be". De esta manera, el texto de entrada se reduce aún más a los fragmentos más importantes necesarios para el análisis. Este paso suele estar integrado en el modelo. Para más información sobre lematización y lematización, consulte la [edición en línea de Introducción a la Recuperación de Información](#).

La tokenización es uno de los primeros pasos de preprocesamiento que se realizan en el texto de entrada. Implica dividir una oración en partes más pequeñas, generalmente palabras o subpalabras, llamadas tokens. La elección de tokens puede variar según el modelo: algunos usan palabras completas, mientras que otros se basan en raíces o subpalabras para gestionar mejor la gramática y el vocabulario. Una vez tokenizado, el texto puede pasarse a un modelo de PLN entrenado para su posterior análisis. La [Figura 7-2](#) muestra un ejemplo de entrada, "¡Me siento genial!", que se somete a tres pasos de preprocesamiento (corrección ortográfica, tokenización y eliminación de palabras vacías) antes de realizar el análisis de sentimiento.

Primero, se corrige la ortografía del texto a "Me siento genial". A continuación, la oración se divide en tokens. Un tokenizador podría ser tan simple como llamar a `.split()` en una cadena, pero es más efectivo con una biblioteca de tokenización que tenga en cuenta la puntuación. Por último, se eliminan las palabras vacías de los tokens. Al final, quedan las palabras "feeling" y "great". Estas dos palabras se pueden analizar mediante un modelo de análisis de sentimientos de PLN para obtener una puntuación. En este ejemplo, "feeling" es bastante neutral, con una puntuación de 1.

Mientras que "genial" es una palabra positiva y obtiene una puntuación de 3. En conjunto, el análisis de sentimiento para esta cadena es 4.



Cifra 2:7 Pasos de preprocesamiento de PNL en una cadena de texto

Su aplicación debe seguir la misma estructura general al manejar Entradas de diario. Para empezar, decides probar cada paso del preprocesamiento en el código para garantizar que sus módulos npm de terceros funcionen como esperado.

Obtenga programación con procesamiento de cadenas Paquetes

Para comenzar a desarrollar su aplicación, cree una nueva carpeta llamada `sentiment_journal`. Navega a la carpeta de tu proyecto en tu Línea de comandos y ejecute `npm init`. Este comando inicializa su Aplicación de nodo. Puede presionar Enter durante los pasos de inicialización. El resultado de estos pasos es la creación de un archivo llamado `package.json`. Este archivo le indicará a su aplicación Node cualquier configuración o script necesario para operar.

A continuación, crea un archivo llamado `index.js` dentro de la carpeta de tu proyecto. Este archivo... Actuará como punto de entrada para su aplicación.

Antes de comenzar con el análisis de sentimientos, debe asegurarse de que El texto de entrada está escrito correctamente. Para comprobarlo, utilice el texto de ejemplo "Yo ¡Me siento muy bien!" Tu objetivo es crear una función que tome este texto como parámetro y genera la misma oración con todas las palabras escritas

correctamente. Para ello, necesitará un paquete npm que le ayude a identificar Palabras mal escritas y su corrección. Navega a tu proyecto.

nivel raíz de la carpeta en su línea de comandos y ejecute `npm install`

`corrector ortográfico@^3.7.1`. El corrector ortográfico es un paquete popular que

Ofrece funciones como `isMisspelled`, para comprobar si una palabra tiene alguna errores tipográficos y obtener correcciones para corregir errores ortográficos para ofrecerlos correctamente alternativas escritas

Agregue el código del **Ejemplo 7-1** a `index.js` para comenzar a probar este paquete.

En este fragmento de código, importa la clase `SpellChecker` desde

`corrector ortográfico` y usar su función `getCorrectionsForMisspelling`

Función en el texto de muestra `grat` para obtener una matriz de posibles errores ortográficos. correcciones.

Ejemplo de prueba de corrección ortográfica en `index.js`

```
importar el corrector ortográfico desde 'corrector ortográfico'; ❶
```

```
opciones constantes =
```

```
SpellChecker.getCorrectionsForMisspelling('grat'); ❷
```

```
console.log(opciones); ❸
```

❶ Importar módulo corrector ortográfico .

❷ Obtenga una lista de opciones de corrección ortográfica para la cadena de muestra utilizando la función `getCorrectionsForMisspelling` .

❸ Mostrar la lista de ortografías sugeridas.

Puede navegar a la carpeta del proyecto en su línea de comando y comenzar su aplicación para obtener un registro de todas las ortografías alternativas para `grat`.

Convenientemente, `SpellChecker` devuelve la ortografía corregida en el Orden de mayor probabilidad. Esto es particularmente útil en este ejemplo.

porque estas buscando corregir la cadena `grat` con la palabra

Genial, que es la primera opción en el array de palabras corregidas.

El **ejemplo 7-2** muestra el resultado después de ejecutar su aplicación.

Ejemplo 72. Salida de la línea de comandos para las opciones de corrección ortográfica

```
[ ❶
    'genial', 'agarrar', 'conceder', 'cabra', 'gris',
    'mocoso', 'graduado', 'gran', 'rejilla', 'fraternidad',
    'rata', 'grit', 'gram', 'graft', 'ghat', 'gat', 'drat',
    'gnat', 'grata', 'prat', 'groat', 'gras', 'grot', 'erat',
    'gmat'
```

```
]
```

```
❶
```

Registra las palabras corregidas en la ventana de la línea de comandos.

Dado que puede confiar en los resultados de esta biblioteca, generalmente puede elegir el primer resultado del array. Este es un buen punto de partida para probar una sola palabra.

Ahora, reemplaza las dos últimas líneas de `indexjs` con el código del [Ejemplo 7-3](#). Este código se basa en la lógica de corrección ortográfica que escribiste anteriormente. Esta vez, el código comienza con la cadena de entrada de ejemplo y encapsula la lógica de corrección ortográfica en una función llamada `correctSpelling`. En esta función, la entrada, `inputString`, se divide en palabras separadas y se define previamente un array para almacenar todas las palabras escritas correctamente. Un bucle `for` itera sobre cada palabra y utiliza la función `isMisspelled` para comprobar si la palabra está escrita correctamente.

Si la palabra está mal escrita, se pasa a la función `getCorrectionsForMisspelling`, que devuelve una matriz de alternativas correctamente escritas para la palabra de entrada. Como sabe que el primer elemento de esa matriz es la mejor opción de reemplazo, elige esa palabra (el índice 0) y la añade a la matriz de correcciones. Si la palabra no está mal escrita, se añade a la matriz de correcciones tal cual. Al final del bucle, la matriz de correcciones combina las palabras en una oración y devuelve el resultado. De esta manera, al llamar a la función `correctSpelling` con su `inputString`, el texto resultante se registrará inmediatamente en la consola de línea de comandos.

Ejemplo 73. Agregar una función de corrección ortográfica a index.js

```

const inputString = '¡Me siento genial!'; ❶

const correctSpelling = inputString => { const palabras ❷
  = inputString.split(' '); const correcciones = []; para ❸
  (let palabra de palabras) { si ❹
    (SpellChecker.isMisspelled(palabra))
    { const opciones = ❺

    Corrector ortográfico.getCorrectionsForMisspelling(palabra); ❻
      correcciones.push(opciones[0]); } else ❼

      { correcciones.push(palabra);
      }

    } devolver correcciones.join(' '); ❽
  } console.log(correctSpelling(inputString)); ❾

```

- ❶ Define tu cadena de entrada de muestra inicial.
- ❷ Defina una función correctSpelling que tome una cadena de entrada como parámetro.
- ❸ Separe las palabras en su cadena de entrada y asígnelas a una variable llamada palabras.
- ❹ Define una variable, correcciones, para almacenar todas las palabras escritas correctamente.
- ❺ Compruebe si cada palabra está escrita correctamente utilizando SpellChecker.isMisspelled.
- ❻ Encuentre todas las alternativas ortográficas para la palabra mal escrita utilizando SpellChecker.getCorrectionsForMisspelling.
- ❼ Añade la primera opción de la lista de palabras corregidas a tu matriz de correcciones . También puedes añadir la palabra original si no está mal escrita.

- 8 Combine las palabras en correcciones para reformatear el texto de entrada escrito correctamente.
- 9 Registre la cadena devuelta desde `correctSpelling` en su consola de línea de comandos.

Pruébalo volviendo a ejecutar la aplicación. La salida de la consola debería mostrar "Me siento genial". Es un buen comienzo, ya que garantiza que se revise la ortografía del texto introducido antes de llegar al modelo de análisis.

El siguiente paso es tokenizar la cadena corregida. Para gestionar este ejemplo y todas las demás cadenas posteriores, se utiliza una biblioteca compatible con la tokenización.

Instala el paquete `natural` ejecutando el comando `npm install natural@^8.1.0`. Este paquete incluye numerosas funciones de soporte de NLP, como tokenización, lematización e incluso análisis de sentimiento. Puedes importar `natural` añadiendo `"import natural from 'natural'"` al principio de tu archivo `index.js`. Luego, instancia un nuevo tokenizador `"natural.WordTokenizer"` añadiendo `"const tokenizer = new natural.WordTokenizer()"` debajo de la línea de importación. Crea una función llamada `"tokenizeInput"` para encapsular la nueva instancia del tokenizador y dividir la cadena de entrada en tokens, como se muestra en el [Ejemplo 7-4](#), que pasa la cadena de entrada corregida al tokenizador. Comienza aplicando la tokenización de `natural` para dividir la oración en palabras individuales, lo que facilita que el análisis de sentimiento posterior procese el texto con precisión.

Ejemplo 7-4 Agregar una función de tokenización a `index.js` `const`

```
tokenizeInput = inputString => {
  devolver tokenizador.tokenize(inputString);
}
```

- 1 Defina una nueva función `tokenizeInput` que tome su cadena de entrada como parámetro.

- ② Devuelve los tokens que representan tu cadena de entrada, utilizando la función `tokenize natural` .

Actualice su declaración de registro para separar cada llamada a las funciones `tokenizeInput` y `correctSpelling` , registrando solo el resultado final (Ejemplo 7-5). Ahora, el valor devuelto por `correctSpelling` pasará a la función `tokenizeInput` .

Vuelva a ejecutar su aplicación y verá que el resultado cambia a una matriz de tokens: `['I', 'am', 'feeling', 'great']` .

Ejemplo 5.7 Separar las llamadas a funciones del paso de preprocesamiento en `index.js`

```
const correctedSpelling = correctSpelling(inputString); const tokens ①
= tokenizeInput(correctedSpelling); console.log(tokens); ②
```

Asigna el resultado de ③

- ① `correctSpelling` a la variable `correctedSpelling`.

- ② Pase el valor `correctedSpelling` a `tokenizeInput` y asigne el valor devuelto a `tokens`.

- ③ Registra la matriz de tokens en tu consola.

Con una matriz de tokens, ahora puede realizar los dos últimos pasos de preprocesamiento antes del análisis de sentimiento: la lematización y la eliminación de palabras vacías. El paquete `natural` incluye una función de lematización, por lo que no es necesario importar una biblioteca adicional para ello.

La función de lematización se puede aplicar a una sola palabra a la vez, por lo que deberá crear una nueva función, `stemWords`, que tome sus tokens como parámetro y los recorra para devolver sus raíces, como se muestra en el Ejemplo 7-6. En esta función, define una nueva matriz vacía, `stems`, donde agregará cada nueva palabra lematizada que procese. Recorre sus tokens y pasa cada token a la

Función `natural.PorterStemmer.stem`, donde la palabra se descompone en su raíz mediante un algoritmo especial. Cada raíz generada se inserta en la matriz de raíces y, finalmente, se devuelve al invocador de la función.

NOTA

Cada algoritmo de PLN se deriva de un algoritmo probado, investigado y desarrollado por ingenieros lingüísticos informáticos. El algoritmo de lematización de Porter es uno de los más populares para lematizar palabras en inglés. Dado que existen muchos idiomas compatibles, cada uno con su gramática y sintaxis únicas, es posible que encuentre un algoritmo diferente más adecuado para su caso de uso. Puede obtener más información sobre el algoritmo de lematización de Porter [en línea](#).

Ejemplo 76. Agregar una función de derivación a `index.js`

```
const stemWords = tokens => { const ❶
  stems = []; for (let token ❷
  de tokens) { const stem = ❸
    natural.PorterStemmer.stem(token); stems.push(stem); ❹
  } ❺
  devuelve los tallos; ❻
}
```

- ❶ Define la función `stemWords` que toma `tokens` como parámetro de entrada.
- ❷ Define una matriz de tallos vacía.
- ❸ Recorra cada token que se procesará para obtener su raíz.
- ❹ Procesar la raíz de la palabra utilizando el algoritmo de derivación de Porter dentro del paquete `natural`.
- ❺ Añade el tallo a tu matriz de tallos.

6 Devuelve la matriz de tallos.

Ahora puede actualizar su declaración de registro definiendo primero una nueva variable, `stemmedWords`, que será igual a la matriz devuelta después de pasar sus tokens a `stemWords`: `const stems = stemWords(tokens)`. Luego, registre los tallos. Su salida debería mostrar ['I', 'am', 'feel', 'great'], lo que indica que feeling se originó en feel.

El último paso es eliminar las palabras vacías, como "yo" o "soy". Para ello, instalará un nuevo paquete, "stopword", ejecutando el comando `npm install stopwords@^3.1.5` en la raíz de la carpeta de su proyecto, desde la línea de comandos. A continuación, añada "stopword" a su proyecto añadiendo `"import { removeStopwords } from 'stopword'"` al principio de `index.js`. Esto importará específicamente la función "removeStopwords" que necesitará para este proyecto.

Ahora puedes ir a tu declaración de registro al final de `index.js` y agregar una nueva variable, `const removedStopWords = removeStopwords(stems)`, que pasa tu array de stems a la función `removeStopwords`, devolviendo solo las palabras necesarias para el análisis. Vuelve a ejecutar la aplicación para ver ['feel', 'great'] registrado en tu consola.

Puede parecer que son muchos pasos para construir, solo para reducir la oración a dos palabras, pero es un proceso necesario para sacar el máximo provecho de su modelo de análisis de sentimientos. En la siguiente sección, aplicará estos resultados a su analizador de sentimientos.

Análisis de sentimientos. En la sección anterior, se aseguró de que cualquier texto de entrada cumpla con los requisitos adecuados para el análisis de sentimientos. Ya ha instalado...

El paquete natural , que incluye una clase de análisis de sentimientos llamada SentimentAnalyzer. Además, se puede instanciar con un algoritmo de lematización. Esto significa que, en este caso, no es necesario realizar la lematización ni la eliminación de palabras vacías. Para probar la función del analizador, elimine el código que tenía después de definir la variable tokens y añada el código del **Ejemplo 7-7**.

Al añadir este código, se desestructuran SentimentAnalyzer (la clase analizadora) y PorterStemmer (la clase derivada) del paquete natural . A continuación, se instancia un nuevo objeto SentimentAnalyzer configurado para realizar lo siguiente:

- Analizar inglés
- Palabras raíz utilizando el algoritmo de lematización de Porter
- Utilice un conjunto de vocabulario específico identificado como afinn

Con este nuevo analizador configurado, se llama a la función getSentiment proporcionada pasando el array tokens. El resultado es una puntuación de análisis de sentimiento que se puede registrar en la consola.

Ejemplo 77. Análisis de sentimientos con tokens en index.js

```
...
const { SentimentAnalyzer, PorterStemmer } = natural; const 1
analizador = new SentimentAnalyzer("Inglés", PorterStemmer,
"afinn"); const sentimentResults 2
= analizador.getSentiment(tokens); console.log(sentimentResults); 3
4
```

1 Importe los módulos SentimentAnalyzer y PorterStemmer desde natural.

2 Cree una instancia de un nuevo analizador que incluya una configuración de derivación.

3

Analice el sentimiento de su cadena de entrada ejecutando `getSentiment` en sus tokens.

4 Registra la puntuación de sentimiento resultante en tu consola.

Intente volver a ejecutar su aplicación para obtener un puntaje de salida impreso en su consola. Su puntuación debe ser 1. Esta puntuación indica un sentimiento general positivo.

Puedes probar esto más a fondo cambiando la cadena "grat" en tu entrada a "bad". La puntuación resultante debería ser -0,5, lo que indica un sentimiento negativo. De esta manera, cuanto mayor sea el número positivo, más positiva será la afirmación, y cuanto más negativa sea la puntuación, más negativa será la afirmación.

Ahora que tienes un analizador de sentimientos en funcionamiento, te resultará más útil analizar más que solo texto de muestra estático. Para capturar la entrada del usuario en la línea de comandos, instala `prompt`, un paquete que solicita al usuario la entrada de texto y guarda la respuesta. Ejecuta `npm install prompt@^1.3.0` en la raíz del directorio de tu proyecto, desde la línea de comandos. A continuación, añade las líneas del **Ejemplo 7-8** al principio de `index.js`. En estas líneas de código, importa el paquete `prompt` y crea una instancia de `prompt`. También configura el `prompt` iniciándolo con la configuración predeterminada y eliminando el prefijo de mensaje predeterminado, para que el usuario vea un `prompt` limpio al escribir el texto de su diario.

NOTA

Al iniciar la aplicación, se te pedirá que escribas lo que quieras. Para salir de la aplicación y del mensaje, pulsa `Comando+C`.

Ejemplo 78. Añadiendo un mensaje a `index.js`

```
...
importar mensaje de 'prompt';      ❶
prompt.start({});                  ❷
prompt.message = "";               ❸
...
```

- ❶ Importe el mensaje a su proyecto.
- ❷ Configure el mensaje para permitir salir manualmente de su aplicación durante un mensaje.
- ❸ Borrar el mensaje de aviso.

Para probar con un mensaje, deberá eliminar la línea que define la variable `inputString` de ejemplo . Luego, agregue código para esperar la entrada del usuario. Como no hay forma de saber cuánto tarda el usuario en responder, deberá usar `"await"`, que devuelve una promesa de JavaScript que finalmente se completa con los datos de respuesta del usuario.

`prompt.get` toma como argumento un array que contiene los elementos que se solicitan al usuario. En este ejemplo, se pregunta "¿Cómo se siente?" y se asigna la respuesta a `inputString`. El resto de la lógica existente para el análisis de sentimientos puede permanecer igual. El último cambio consiste en encapsular el código en un bloque `try-catch` . Esto garantiza que, si algo falla mientras se espera una respuesta, se pueda detectar el error y registrarlo en la consola. Por último, todo este bloque de código debe encapsularse en una función asíncrona para usar la lógica `await` .

Aquí se utiliza una función anónima que se invoca inmediatamente para que el código se ejecute al iniciar la aplicación. El ejemplo 7-9 contiene todo el código necesario para la función asíncrona encapsulada en `indexjs`.

CONSEJO

Las expresiones de función de invocación inmediata (IIFE) son funciones que se ejecutan en cuanto se definen en la aplicación. La expresión es, en efecto, una función entre paréntesis, seguida de otro paréntesis para llamarla. Las IIFE son prácticas para iniciar una aplicación o ejecutar código de configuración importante lo antes posible.

Ejemplo 79. Agregar una función para solicitarle al usuario una entrada en index.js

...

```
(async () => { try
  { const
    {inputString} = await prompt.get([{ nombre: 'inputString',
      descripción: '¿Cómo te sientes?', }]); const correctedSpelling =
      correctSpelling(inputString); const tokens = tokenizeInput(correctedSpelling);
      const { SentimentAnalyzer, PorterStemmer } = natural; const
      analizador = new SentimentAnalyzer("Inglés",
        PorterStemmer, "afinn"); const
        sentimentResults = analyzer.getSentiment(tokens); console.log(sentimentResults); }
        catch (e) { console.log(`Ocurrió un error: $
          {e.message}`);
        }}());
```

- ➊ Define un IIFE para encapsular la lógica de tu aplicación.
- ➋ Envuelva su código en un bloque try/catch para manejar posibles errores.
- ➌ Espere la respuesta del usuario a una solicitud utilizando una descripción y asignando el valor de respuesta a inputString.
- ➍ Detecta cualquier error que ocurra mientras se ejecuta tu código y regístralo en tu consola.

Ahora tu código debería funcionar como antes. Solo que esta vez, se te pedirá que respondas " ¿Cómo te sientes?" en la línea de comandos antes de ejecutar cualquier código de análisis de sentimientos. Intenta reiniciar la aplicación y responde con "¡Tengo un día estupendo y me siento genial!". La consola de la línea de comandos debería registrar una puntuación de 1. Tu afirmación más positiva obtuvo una puntuación ligeramente superior a la del ejemplo anterior. En la siguiente sección, unirá todo esto conectando la aplicación a una base de datos y a una visualización de sus datos compatible con la línea de comandos.

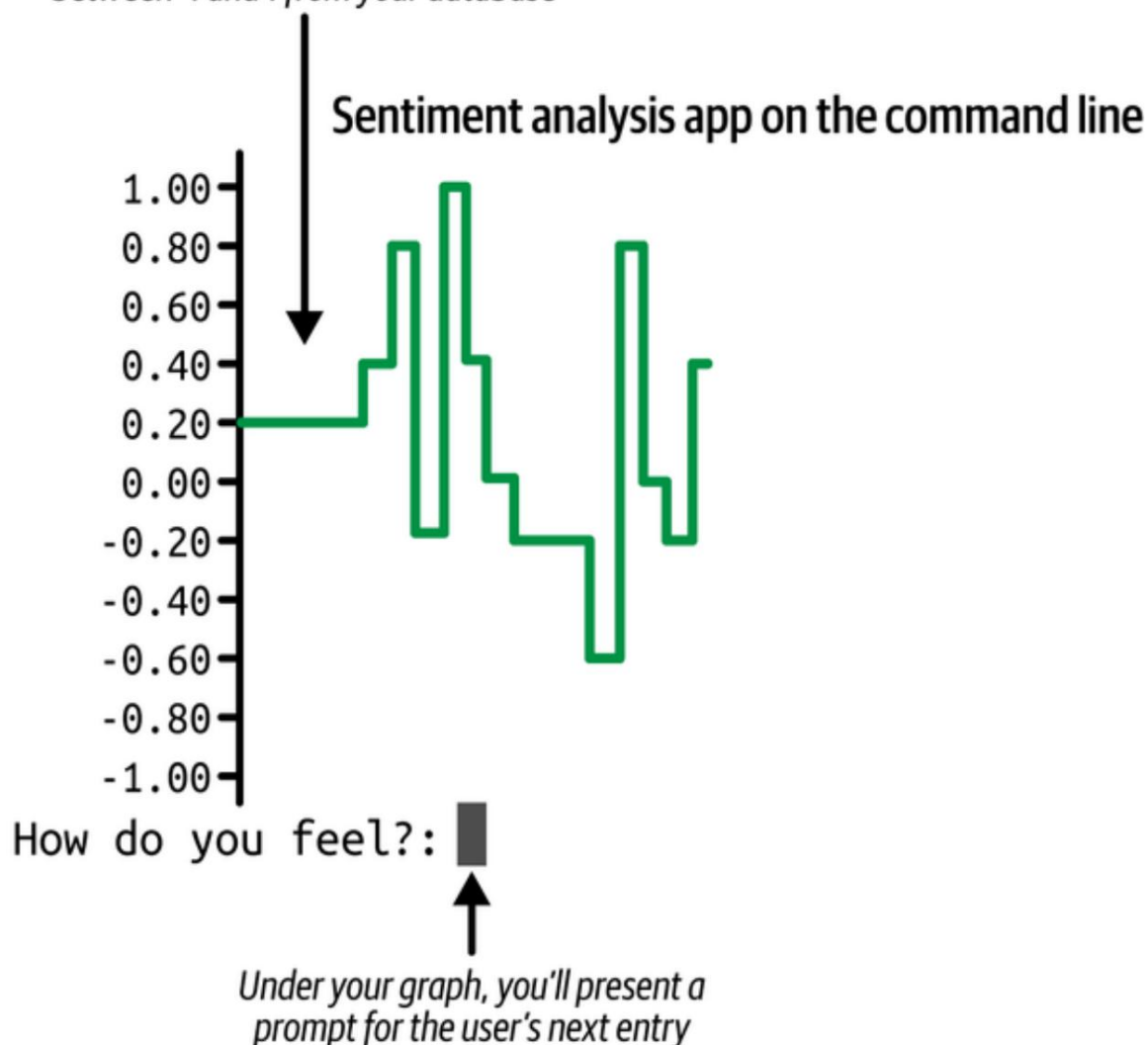
Conexión de una base de datos y una visualización. En este punto,

ya has creado una aplicación que puede calcular el sentimiento de una sola entrada del diario cada vez que la ejecutas. Idealmente, el usuario no solo recibiría una puntuación por cada entrada, sino que también podría compararla con entradas anteriores para seguir la evolución general de su estado de ánimo.

Ahora que ya tienes la mayor parte de la lógica necesaria para analizar texto, puedes centrarte en organizar tu código y añadir compatibilidad para guardar datos y mostrar resultados. En lugar de simplemente mostrar la puntuación de sentimiento en la consola, crearás un gráfico para mostrar las tendencias de sentimiento en una serie de publicaciones, como se muestra en **la Figura 7-3**.

Al final de esta sección, su aplicación permitirá a los usuarios escribir continuamente entradas de diario, guardar sus puntajes de sentimiento en una base de datos y mostrar el gráfico en verde durante un cambio positivo en el puntaje, o en rojo durante un cambio negativo.

*A graph displays a series of sentiment scores
between -1 and 1 from your database*



Cifra 73. Gráfico de consola que muestra una serie de puntuaciones de sentimiento

Puede comenzar configurando la base de datos y el esquema de la base de datos para Guardando puntuaciones de sentimiento. En la raíz de la carpeta del proyecto, ejecute el comando `npm install sqlite3@^5.1.7`

`sequelize@^6.37.7` para instalar el administrador de bases de datos SQLite paquete, junto con el mapeador relacional de objetos de base de datos (ORM), Sequelize. Estos dos paquetes te permitirán definir la estructura. desea guardar y conectarse a su base de datos.

NOTA

Aunque hay muchas opciones de bases de datos para elegir, necesitarás...

Implementar un sistema sencillo de gestión de bases de datos SQLite. Para más información

Para obtener información sobre cómo elegir entre tipos de bases de datos, vuelva a visitar [“Comparación SQLite y PostgreSQL”](#).

Cree un nuevo archivo llamado `db.js` y agregue el código del [Ejemplo 7-10](#). Este código importará las clases `Sequelize` y `DataTypes` desde su Módulo `Sequelize`. Se utiliza la clase `Sequelize` para instanciar una nueva conexión a la base de datos llamada `db`. Esta base de datos está configurada para utilizar SQLite y guarde los datos persistentes en un archivo llamado `journal.sqlite` en la carpeta de tu proyecto. Luego defines un nuevo modelo de `Sequelize` llamado `SentimentScore` para conservar las puntuaciones de sentimiento en un campo llamado `Puntuación`. (El modelo también se exporta para su uso en otros módulos). Por último, espera que la base de datos establezca una conexión y se registre su modelo cuando se inicia la aplicación.

Ejemplo 7-10. Configuración de su modelo y conexión a la base de datos en `db.js`

```

importar { Sequelize, DataTypes } de 'sequelize'; ❶

constante db = nueva Sequelize({ ❷
  dialecto: 'sqlite',
  almacenamiento: './journal.sqlite'
});
exportar const SentimentScore = db.define('SentimentScore', {
  ❸
  puntuación: DataTypes.DECIMAL,
});

esperar SentimentScore.sync(); ❹
```

❶ Importar las clases `Sequelize` y `DataTypes` desde `sequelize`.

❷

Cree una nueva configuración de base de datos con Sequelize, utilizando un Base de datos SQLite almacenada como journalsqlite en la carpeta de su proyecto.

3

Defina un modelo SentimentScore Sequelize que guarde una puntuación campo como valor decimal.

4

Registre su modelo SentimentScore con SQLite base de datos.

NOTA

No verá aparecer el archivo journalsqlite en su carpeta hasta que realice uso de este archivo en la lógica principal de su aplicación indexjs .

Con este modelo configurado, ahora puede guardar puntuaciones de sentimiento a través de El modelo SentimentScore . Antes de crear el archivo indexjs ,
Crea una nueva clase para todo tu preprocesamiento y sentimiento.
Lógica de análisis. En la carpeta de tu proyecto, crea un nuevo archivo llamado sentimentJournaljs. Este archivo funcionará como un módulo independiente de su archivo indexjs y contiene la mayor parte de la lógica agregada en secciones anteriores.

NOTA

Cuando se definen modelos Sequelize, se construyen con funciones especiales Para permitir interacciones con la base de datos. Por ejemplo,
SentimentScore.create le permitirá agregar un nuevo Registro de SentimentScore en la base de datos, y
SentimentScore.findAll consulta la base de datos para todos los datos existentes. registros. Para obtener más información sobre las funciones del modelo Sequelize, consulte [Sequelize referencias API](#).

Anteriormente en este capítulo usaste el paquete natural para construir cada paso del preprocesamiento de PNL. Porque no te preocupas demasiado.

Para personalizar los pasos de preprocesamiento, puede utilizar una biblioteca que Realiza todos los pasos por ti. En la raíz de tu proyecto, instala El paquete de sentimiento ejecutando el comando `npm install sentiment@^5.0.2`. Este paquete proporcionará todo el análisis. herramientas e incluso devolver el desglose de tokens junto con el Puntuación de sentimiento resultante. Continuarás usando el Paquete de corrector ortográfico . Agregue el código del **Ejemplo 7-11** al principio.

(Para obtener más información sobre el paquete `sentimentJournal` , visite el [sitio web de npm](#)).

Ejemplo 7-11. Añadiendo importación a `sentimentJournal.js`

```
importación Sentimiento desde 'sentimiento';           ❶
importar el corrector ortográfico desde 'corrector ortográfico';
importar { SentimentScore } desde './db.js';           ❷
```

❶ Importe los paquetes de corrector ortográfico y de sentimientos a su proyecto.

❷ Importe el modelo `SentimentScore` desde `db.js`.

Con estas bibliotecas listas para usar en su proyecto, puede comenzar a crear la clase `SentimentJournal` agregando el código del **Ejemplo 7-12**.

En este listado, se define la estructura de clase `SentimentJournal` .

Agrega campos para el motor de análisis de sentimientos, una matriz de puntuaciones (para ser cargado por la base de datos), y un campo para almacenar el diario del usuario entrada. Además, agrega un método de clase llamado `correctSpelling`, que utiliza efectivamente el mismo código de su Función `correctSpelling` anterior . Solo que, esta vez, el método se puede llamar solo en asociación con un `SentimentJournal` . Por último, exporta la clase para poder usarla en otras instancias. partes de tu proyecto.

Ejemplo 7-12. Creando la clase `SentimentJournal` en `sentimentJournal.js`

...

```

class SentimentJournal ❶
{ constructor () { ❷
  este.sentimiento = nuevo Sentimiento();
  este.puntuaciones = [0];
  este.entrada = "; }

  correctSpelling (inputString) { const ❸
    palabras = inputString.split(' '); const correcciones
    = []; para (let palabra de
    palabras) { si
      (SpellChecker.isMisspelled(palabra)) { const opciones
      =
SpellChecker.getCorrectionsForMisspelling(palabra);
      correcciones.push(opciones[0]); } de lo
      contrario
      { correcciones.push(palabra);
      }

    } devolver correcciones.join(' ');
  }

} exportar predeterminado SentimentJournal; ❹

```

❶ Define la clase SentimentJournal .

❷ Configure el constructor de clase para crear una instancia de un nuevo objeto de análisis de sentimientos, inicializar una matriz de puntajes a un puntaje inicial de 0 y definir un campo para almacenar la entrada del usuario.

❸ Defina un método de clase, correctSpelling, para tomar un parámetro inputString .

❹ Exporte la clase SentimentJournal para usarla en otros módulos.

Con esta estructura en su lugar, puede comenzar a agregar más métodos a esta clase para admitir el almacenamiento persistente y la funcionalidad original que ha creado.

Agregue los métodos de clase del **Ejemplo 7-13** a su clase `SentimentJournal` .

`saveScore` es un método de clase asíncrono que utiliza el modelo

`SentimentScore` y crea un nuevo registro mediante el parámetro de puntuación que se le pasa. El método asíncrono `fetchEntries` espera una consulta a la base de datos de hasta 100 registros recientes y asigna las puntuaciones de esos resultados al estado de puntuaciones de `SentimentJournals` . Utilice `results.map(({score}) => score)` porque los resultados iniciales contendrán más que solo los datos de puntuación, incluyendo los ID de registro y las marcas de tiempo. Esta lógica elimina los datos irrelevantes de los resultados.

Ejemplo: ~~Como~~⁷⁻¹³ agregar consultas de base de datos a su clase en `sentimentJournal.js`

```
...
async saveScore (puntuación) { ❶
  esperar SentimentScore.create({score}); }

async fetchEntries () { ❷
  const resultados = await SentimentScore.findAll({límite: 100}); if ❸
    (resultados.length) {
      este.scores = resultados.map(({score}) => puntuación); ❹
    }
  }
}
```

❶ Defina un método asíncrono , `saveScore`, para crear un nuevo registro de `SentimentScore` en su base de datos utilizando el parámetro de puntuación .

❷ Defina un método asíncrono , `fetchEntries`, para extraer todos los puntajes persistentes de la base de datos.

❸ Consulta los 100 registros de `SentimentScore` guardados más recientes .

❹

Si existen registros de `SentimentScore` , tome solo los valores de puntuación y asígneles al campo de matriz de puntuaciones .

Estos son los únicos dos métodos que necesitarás para guardar y recuperar datos de tu base de datos en este proyecto. A continuación, debes agregar la lógica de análisis.

Ahora que usa una nueva biblioteca de análisis de sentimientos, agregue el método del **Ejemplo 7-14** a su clase. Defina el método "analyze" para comprobar si hay una entrada para analizar. Si existe "this.entry" , ejecute "this.sentiment.analyze" , que utiliza la función "analyze" del paquete "sentimiento" para generar una puntuación para la entrada del diario proporcionada. Desestructura la puntuación de este resultado, divídala entre 10 y la fuerza dentro del rango de -1 y 1.

CONSEJO

Para garantizar que un número siempre se encuentre dentro de un rango determinado, puede obtener el `Math.max` de la variable número y el mínimo de su rango. Si su número es menor que el mínimo, devolverá ese valor mínimo. Luego, encierra el resultado de esa función en `Math.min`, con el otro valor como el máximo de tu rango. Si tu número es mayor que el mínimo, tomas el mínimo entre este y el máximo permitido. De esta manera, tu resultado siempre se mantiene dentro del rango establecido.

Con su puntuación normalizada establecida, puede guardarla en la base de datos como un nuevo registro pasándola a su método `saveScore` . También puede agregar la puntuación a la matriz de puntuaciones de la instancia de su clase para mantener el estado sin tener que volver a consultar la base de datos.

Ejemplo: **Como 7-14** agregar consultas de base de datos a su clase en `sentimentJournal.js`

```

...
async analizarSentimiento () { if (! ❶
  this.entry || this.entry === "") return; const { puntuación } = ❷
  this.sentimiento.analyze(this.entry); const puntuaciónNormalizada = ❸
  Math.min(Math.max(puntuación / 10, -1), 1); await ❹
  this.guardarPuntuación(puntuaciónNormalizada); ❺
  this.puntuaciones.push(puntuaciónNormalizada); ❻
}

```

- ...
- ❶ Define tu método analyzeSentiment .
 - ❷ Compruebe si el campo de entrada de la instancia de clase tiene un valor asignado.
 - ❸ Utilice la función de análisis del paquete de sentimientos para procesar la entrada y extraer la puntuación.
 - ❹ Calcular una puntuación normalizada entre -1 y 1.
 - ❺ Espere a que la puntuación se guarde en su base de datos.
 - ❻ Agregue la nueva puntuación al campo de matriz de puntuaciones de su instancia de clase.

Tu clase ya está lista para analizar el sentimiento de cualquier texto de entrada. Para obtenerlo, añadirás el módulo prompt añadiendo el código del **Ejemplo 7-15** al principio de sentimentJournal.js. Aquí, importas prompt, inicializas la biblioteca con prompt.start() y borras el mensaje del prompt.

Ejemplo: ⁷¹⁵Importación e inicialización del mensaje en sentimentJournal.js

```

...
importar mensaje desde 'prompt'; ❶
prompt.start();
prompt.message = "";

```

- ...
- ❶ Importe el paquete de solicitud e inicialice la biblioteca.

Con la biblioteca de indicaciones configurada, agregue la función del **Ejemplo 7-16** a su clase `SentimentJournal`. Este método espera la respuesta del usuario. respuesta a ¿Cómo te sientes? y asigna el valor resultante a respuesta. La variable de respuesta se pasa a través de la clase método `correctSpelling`, donde el texto resultante se asigna a El campo de entrada de la instancia de clase. Con este método, Debería ser capaz de recopilar respuestas de los usuarios y procesar una puntuación de análisis, Guárdalos en la base de datos y repítelos.

Ejemplo 7-16 Agregar un método de solicitud a su clase en `sentimentJournal.js`

```
...
async promptEntry () {
  const {respuesta} = await prompt.get([
    nombre: 'respuesta',
    Descripción: '¿Cómo te sientes?'
  ]);
  esta.entrada = esta.correctSpelling(respuesta);
}
...
```

- ❶ Defina un método `promptEntry` para `SentimentJournal`.
- ❷ Espere a que `prompt.get` devuelva la respuesta del usuario a su inmediato.
- ❸ Asignar la respuesta ortográfica corregida del usuario a esta entrada.

Para que este código funcione, deberá crear un índice usando `Archivo.js` y inicio estos métodos de clase. Agregue el código del **Ejemplo 7-17** a `index.js`.

En este archivo, importa su clase `SentimentJournal` y crea un Nueva instancia llamada `diario`. La variable `diario` permite... Accede a todos los métodos definidos en la clase. Para empezar, llama a `journal.fetchEntries` para cargar cualquier `SentimentScore` existente registros de su base de datos. Luego, ingresa un tiempo infinito

Bucle (verdadero) , donde cada ciclo solicita al usuario una nueva entrada de diario mediante `journal.promptEntry` y luego la analiza mediante `journal.analyzeSentiment`. Este bucle continúa indefinidamente, lo que permite a los usuarios enviar y procesar varias entradas en una sola sesión.

Ejemplo 7-17. Configura tu `SentimentJournal` en `index.js` con un de

```

bucle de importación SentimentJournal desde "../sentimentJournal.js"; ❶

const journal = new SentimentJournal(); await ❷
journal.fetchEntries(); ❸

mientras (verdadero) { ❹
    esperar diario.promptEntry(); esperar ❺
    diario.analyzeSentiment(); } ❻
  
```

- ❶ Importa la clase `SentimentJournal` desde tu módulo local.
- ❷ Crea una nueva instancia de `SentimentJournal`.
- ❸ Recupere cualquier entrada de diario guardada de su fuente de datos.
- ❹ Iniciar un bucle infinito para solicitar y analizar repetidamente nuevas entradas.
- ❺ Solicitar al usuario una entrada de diario.
- ❻ Analice el sentimiento de la entrada de diario más reciente.

Su índice El archivo `js` está listo para iniciar interacciones en la línea de comando. Antes de probar esto, debe agregar un elemento más a la salida: una visualización gráfica.

Para este proyecto puedes utilizar el paquete `asciichart` , Que puede generar una estructura gráfica usando caracteres ASCII. Ejecute el comando `npm install asciichart@^1.5.25` en la línea de comandos.

Solicitud para instalar asciichart y agregar importación asciichart desde 'asciichart' hasta la parte superior de su archivo sentimentJournal.js :

Puede definir configuraciones personalizadas en asciichart, desde color esquema para representar gráficamente las dimensiones. Porque quieres que el gráfico muestre Para datos entre -1 y 1 , agregue el código del **Ejemplo 7-18** debajo de su Las líneas de importación en `js`. Este objeto `chartConfig` sentimentJournal definen los valores mínimos y máximos del gráfico, así como un Altura de línea cómoda para mostrar los puntajes de sentimiento de este proyecto. Utilizará este conjunto de valores de configuración cuando configure la lógica del gráfico.

Ejemplo 7-18 Agregar un método de solicitud a su clase en sentimentJournal.js

```
...
constante chartConfig = { ❶
  mín.: -1,
  máximo: 1,
  altura: 10,
}
```

❶ Define las configuraciones para que tu gráfico muestre valores entre -1 y 1 , con una altura de línea de 10.

Para imprimir realmente el gráfico, agregue los dos métodos del **Ejemplo 7-19** a tu clase `SentimentJournal` en `sentimentJournal.js`. El método `setChartColor` verifica si tiene algún color reciente puntuaciones de sentimiento. Luego, visualiza la puntuación más reciente y la asigna a la variable `recentScore`.

Si el `recentScore` es negativo, cambia el color del gráfico a rojo en `chartConfig`. De lo contrario, se asigna el color del gráfico a verde, indicando una tendencia positiva en las puntuaciones. El método `printChart` realiza tres pasos:

- Borra la consola de línea de comandos de cualquier archivo impreso previamente. gráficos

- Establece el color del gráfico
- Imprime el gráfico usando `asciichart.plot` con los puntajes de su instancia `SentimentJournal` y las configuraciones `chartConfig` definidas previamente

719. Agregar métodos `asciichart` a su clase en `Example sentimentJournal.js`

```
...
setChartColor () { si (! ❶
  this.scores.length) devolver; const ❷
  puntuaciónReciente = this.scores[this.scores.length - 1]; si (puntuaciónReciente ❸
  < 0) { chartConfig.colors = ❹
    [asciichart.red] } de lo contrario { chartConfig.colors
    =
    [asciichart.green]
  }
}

printChart () ❺
{ console.clear(); ❻
  this.setChartColor(); ❼
  console.log(asciichart.plot([ this.scores ], chartConfig)); }
  ❽
```

- ❶ Declara el método `setChartColor`, responsable de establecer el color del gráfico en función de la puntuación más reciente.
- ❷ Comprueba si hay puntuaciones en `this.scores`. Si la matriz está vacía, la función finaliza para evitar errores.
- ❸ Recupera la puntuación más reciente de `this.scores` accediendo al último elemento de la matriz.
- ❹ Comprueba si la puntuación reciente es negativa, lo que determinará el color del gráfico.
- ❺

Declara el método `printChart` , que se encargará de mostrar el gráfico en la consola.

- ❖ Limpia la consola para proporcionar una vista limpia, eliminando cualquier salida anterior.
- ❖ Llama a `setChartColor` para actualizar el color del gráfico en función de la última puntuación antes de imprimir.
- ❖ Traza y registra el gráfico en la consola usando `asciichart.plot`, mostrando los puntajes con la configuración de color actual.

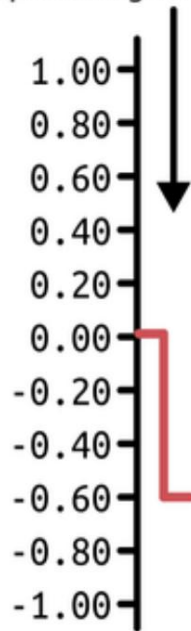
Con estos métodos finales implementados, su diario de opiniones está listo para analizar texto y mostrar los resultados de forma visualmente atractiva. En índice JavaScript, agregue `await journal.printChart()` como la primera línea del de su bucle `while` . Esto garantizará que se imprima un gráfico al iniciar la aplicación y que se obtengan los registros de opiniones anteriores.

NOTA

Cuando inicie la aplicación por primera vez, no habrá puntuaciones previas; solo la puntuación predeterminada de 0 establecida en el constructor de su clase.

Ahora, reinicia la aplicación y empieza a añadir entradas del diario. Al escribir " ¡Este día terrible es el peor!", verás que el gráfico se actualiza inmediatamente para mostrar una puntuación negativa (**Figura 7-4**).

The chart line falls to -0.6 in the color red after processing a negative sentence



How do you feel?: This terrible day is the worst! ■



This sentence containing a negative sentiment is given a negative score

Cifra 74. Salida de consola que muestra una puntuación de sentimiento negativa y una caída del gráfico en rojo después de ingresar una entrada de diario negativa

Tan pronto como agregue una nueva entrada, el gráfico se actualizará nuevamente. Si el sentimiento sigue siendo negativo, el gráfico permanecerá en rojo. De lo contrario, El gráfico mostrará una línea verde tan pronto como se haga una declaración positiva.

Pruébalo escribiendo ¡Este día increíble es excelente!.

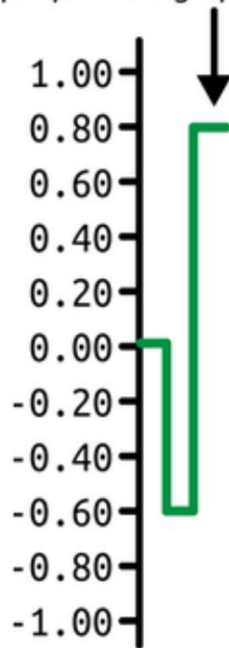
Observe el salto a 0,8 en el gráfico, como se ve en [la Figura 7-5](#).

CONSEJO

Si desea borrar su base de datos, es tan sencillo como eliminar el archivo journalsqlite en la carpeta de su proyecto.

Con tu aplicación efectivamente completa, puedes comenzar a demostrar su
Úsalo en la práctica terapéutica. Con la base de datos configurada, los clientes pueden
elegir agregar entradas de diario cuando lo deseen sin perderlas
Seguimiento de puntuaciones de sentimiento anteriores. Además, esta aplicación puede...
Extendido como una API, una aplicación web que los pacientes pueden usar en su
navegadores, o reconstruido para funcionar con cualquier interfaz.

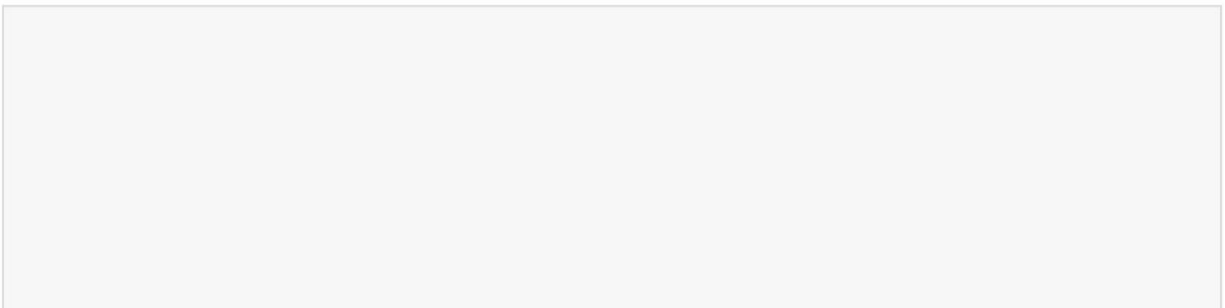
*The chart line climbs to 0.8 in the color green
after processing a positive sentence*



How do you feel?: This amazing day is excellent! ■

*This sentence containing a positive
sentiment is given a positive score*

Cifra 75. Salida de consola que muestra una puntuación de sentimiento positiva y un aumento del gráfico verde después de ingresar una entrada de diario positiva



EJERCICIOS DEL CAPÍTULO

1. Mejore la corrección ortográfica con lógica de respaldo:

a. Actualice su método `correctSpelling` para comprobar si

La función de relleno `SpellChecker.getCorrectionsForMisspelling` devuelve cualquier sugerencia.

b. Si devuelve una matriz vacía (es decir, no se encontró ninguna corrección), se vuelve al método de conservación para mantener la palabra original sin cambios.

c. Agregue el registro dentro del bucle para verificar qué palabras se corrigieron y cuáles se dejaron como estaban.

d. Pruébalo con una entrada como "me siento bien pero no genial" y confirme que su aplicación aún produce una salida utilizable.

CONSEJO

La biblioteca del corrector ortográfico no siempre tiene una solución para cada palabra, por lo que la lógica alternativa garantiza que su aplicación no se bloquee ni pierda tokens inesperadamente.

2. Agregue etiquetas de sentimiento a la salida del gráfico:

a. Modifique el método `printChart()` para imprimir un categoría de sentimiento etiquetada (por ejemplo, "Positivo", "Neutral", "Negativo") junto con cada nueva puntuación de sentimiento.

b. Utilice el valor final de puntuación normalizada para asignar una etiqueta:

- Puntuación > 0,3: "Positivo"
- Puntuación < -0,3: «Negativo»
- De lo contrario: "Neutral"

c. Muestre esta etiqueta claramente debajo del gráfico para que el usuario pueda interpretar el resultado de su última entrada de un vistazo.

d. Pruebe con varias entradas de diario para asegurarse de que cada etiqueta corresponda apropiadamente a la puntuación.

Estos ejercicios mejoran la resistencia y la claridad de su aplicación.

La función de respaldo ortográfico permite que el análisis se ejecute sin problemas y la retroalimentación del gráfico etiquetado ofrece una interpretación más legible para los humanos de la progresión del estado de ánimo del usuario.

Resumen

En este capítulo, usted:

- Aprendí a crear una aplicación Node en torno a un modelo ML
- Creó su propio flujo de preprocesamiento de texto
- Datos persistentes en una tabla SQL a través de un ORM
- Creó una aplicación de línea de comandos para rastrear puntuaciones de sentimiento

Capítulo 8. Correo de marketing

Este capítulo cubre lo siguiente:

- Envío de correos electrónicos desde una aplicación Node
- Configurar un programador para ejecutar tareas
- Verificación de cuentas y seguimiento de la interacción con el correo electrónico

En este capítulo, crearás una aplicación que envía correos electrónicos y gestiona la lógica para los usuarios receptores. En [el capítulo 1](#), aprendiste que HTTP es un protocolo de comunicación a través de internet. Aquí, explorarás algunos de los protocolos más utilizados por servidores y clientes de correo electrónico para facilitar la comunicación de otra manera. Así como Node puede utilizarse para proporcionar API y servidores web completos, también puede utilizarse para microtareas, como el envío de correos electrónicos.

A lo largo del capítulo, añadirás componentes adicionales para dar soporte a algunas de las funcionalidades más comunes de un servicio de email marketing. Comenzarás configurando un paquete conocido para el envío de correos electrónicos externos. Después, crearás una API para recopilar y verificar correos electrónicos. Finalmente, añadirás lógica para programar automáticamente correos electrónicos de marketing para grupos selectos de suscriptores de tu producto.

HERRAMIENTAS Y APLICACIONES UTILIZADAS EN ESTE CAPÍTULO

Antes de comenzar, asegúrese de instalar y configurar las herramientas y aplicaciones necesarias para este proyecto. Las instrucciones de instalación de Node.js, Fastify y VS Code se encuentran en **el Capítulo 1**, mientras que los pasos de inicialización del proyecto, como la configuración de la estructura de directorios, la configuración de package.json y el uso de sintaxis moderna, se describen en **el Apéndice A**. Una vez completado, regrese aquí para continuar. Desarrollar un proyecto desde cero le ayuda a comprender mejor cada componente, brindándole mayor control y flexibilidad a medida que avanza.

Su mensaje : Inn

Box, un gimnasio virtual, ha generado mucha expectación, ya que permitirá a los participantes aprender artes marciales mixtas y técnicas de boxeo desde casa. La startup ya ha recaudado fondos para su innovadora idea, pero quiere empezar a recopilar correos electrónicos de posibles participantes.

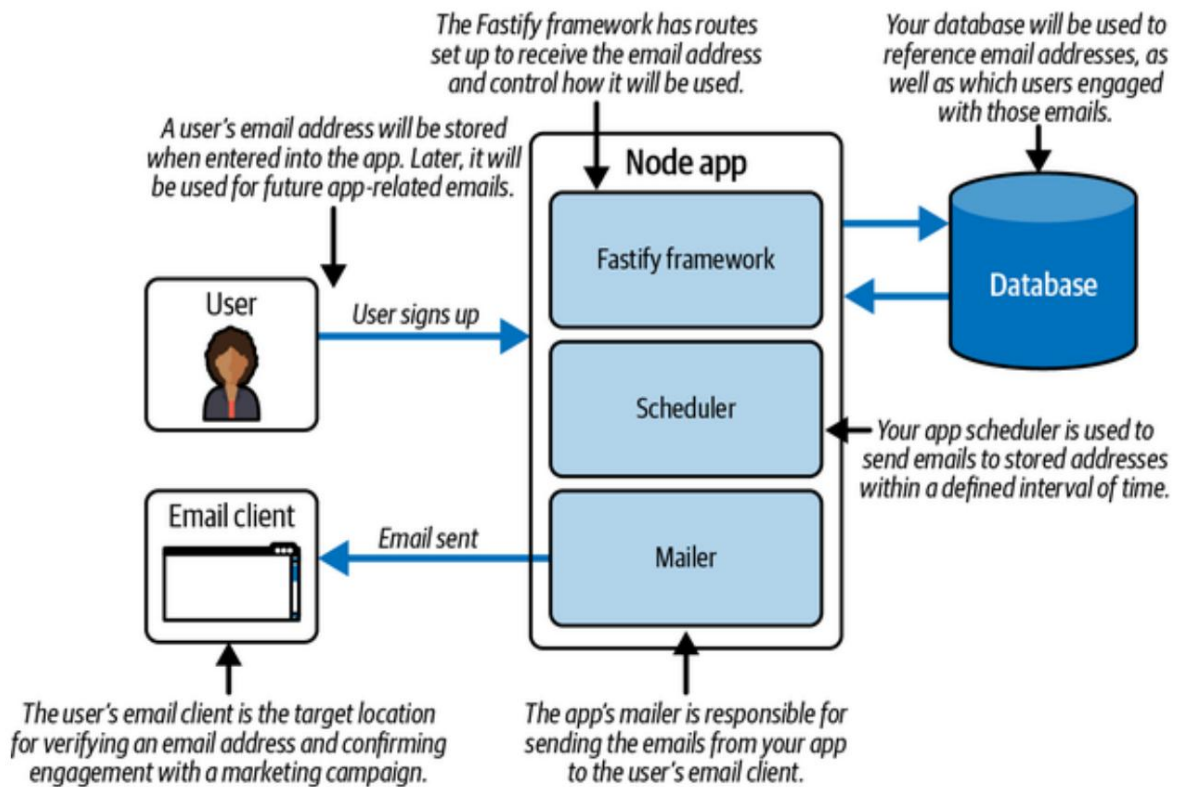
Te contrataron para crear una aplicación Node que envíe información de membresía, novedades del gimnasio y promociones. De esta forma, Inn Box puede empezar a conectar con su creciente audiencia.

Planifica tu proyecto.

Planeas crear una aplicación que recopile y verifique direcciones de correo electrónico, envíe correos electrónicos y determine qué campañas promocionales son las más efectivas. Al finalizar este proyecto, tu aplicación debería poder enviar correos electrónicos con contenido personalizable y almacenar toda la información de marketing relevante en una base de datos.

Comienza dibujando un diagrama que detalle el flujo de información de cada suscriptor a través de tu aplicación. En **la Figura 8-1**, estableces que un usuario (el suscriptor del gimnasio) se registrará con su dirección de correo electrónico.

Las direcciones de correo electrónico serán recopiladas por un punto final de API mediante el El marco de Fastify en tu aplicación Node. La dirección de correo electrónico es... registrado en tu base de datos. Crearás un servicio en tu aplicación para enviar Correos salientes. A medida que los usuarios reciben los correos en su correo personal Los clientes podrán interactuar con el contenido vinculado para verificar su cuenta o participar en promociones.



Cifra 81. Plan de proyecto para una aplicación de correo de marketing de Node

Los enlaces en los correos electrónicos salientes se rastrearán hasta la aplicación Node, lo que permitirá para guardar información sobre la interacción del usuario. Por último, Cree un servicio en su aplicación para programar y enviar periódicamente correos electrónicos automáticamente.

Profundizando en la lógica de la aplicación, nos centramos en tres partes cruciales:

Fastify

Este es el núcleo de cómo su aplicación maneja las entradas y salidas. interacciones con sus usuarios a través de la web.

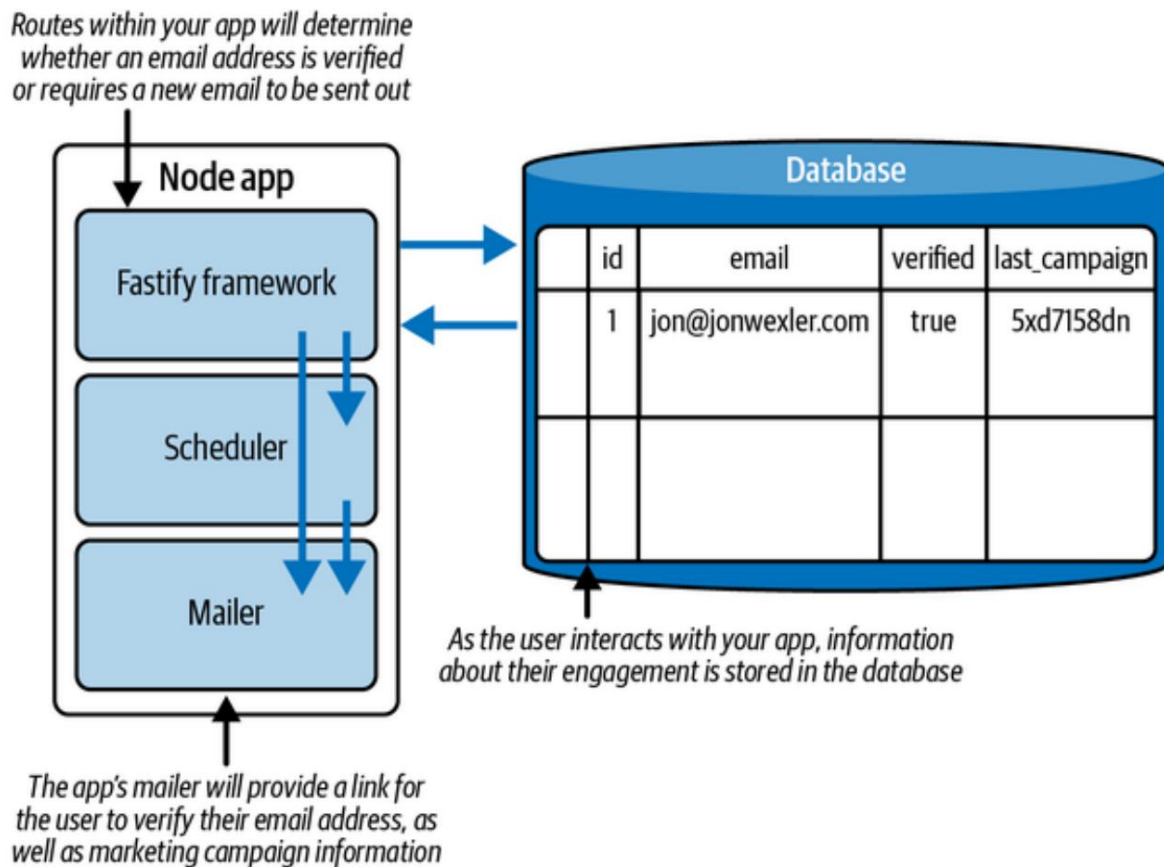
Programador

Aquí es donde definirás los detalles de la ejecución automática.
tareas, como cuándo enviar correos electrónicos.

Remitente

Esto es fundamental para poder enviar correos electrónicos con éxito.
desde su aplicación y llegan a las bandejas de entrada de sus usuarios.

A medida que los usuarios se registren en su aplicación, podrá almacenar datos como sus dirección de correo electrónico, estado de verificación y campañas de marketing que han realizado comprometido con (Figura 8-2). En conjunto, esta información proporciona Inn Caja con un valioso conjunto de datos de clientes potenciales, incluidos los verificados Direcciones de correo electrónico y comportamiento de participación, para guiar el alcance futuro y crecimiento.



Cifra 82. Diagrama que muestra cómo se guardan los datos en su base de datos

Con esta estructura en su lugar, estás listo para comenzar a crear una aplicación.

Para empezar **a programar**,

crea una carpeta llamada `marketing_mailer`. Accede a la carpeta de tu proyecto desde la línea de comandos y ejecuta `npm init`. Este comando inicializa tu aplicación de Node. Puedes presionar Enter durante los pasos de inicialización.

El resultado de estos pasos es la creación de un archivo llamado `package.json`. Este archivo le indicará a su aplicación Node las configuraciones o scripts necesarios para funcionar.

A continuación, crea un archivo llamado `index.js` dentro de la carpeta de tu proyecto. Este archivo servirá como punto de entrada para tu aplicación.

Con tu aplicación de Node inicializada, empieza por instalar el primer paquete npm: `nodemailer`. En el directorio raíz de tu proyecto, ejecuta `npm install nodemailer@^7.0.4` en la línea de comandos. El paquete `nodemailer` ofrece un conjunto completo de herramientas para configurar un servicio de correo para aplicaciones de Node. Para más información sobre cómo usar `nodemailer`, visita su **sitio web**.

Agregue la siguiente línea de importación en la parte superior de `index.js` para importar la función `createTransport` desde `nodemailer`:

```
importar { createTransport } desde "nodemailer";
```

Esta línea de importación desestructura la función `createTransport` del paquete `nodemailer`, lo que le permite usar directamente la función para crear una instancia de un objeto de correo.

A continuación, debe agregar configuraciones particulares a la función `createTransport` para comenzar a utilizar una instancia de `nodemailer` para enviar correo.

NOTA

Si desea explorar la posibilidad de crear su propio servidor de correo SMTP, pruebe los paquetes `smtp-server` y `mailparser` para configurar las opciones necesarias para enviar correo saliente y leer el correo entrante. Precaución: configurar un servidor de correo personalizado presenta muchos obstáculos en cuanto a seguridad y configuración. Como alternativa, puede crear su propio servidor de correo Haraka, que ofrece todo lo necesario para enviar y recibir correos electrónicos en Node. Para más información, visite el [sitio web de Haraka](#).

El [ejemplo 8-1](#) le muestra qué opciones son necesarias para configurar su objeto transportador.

nodemailer en sí no es un servidor de correo, sino que depende de un servidor existente para enviar sus correos electrónicos.

CONSEJO

Al configurar su transportador de correo, encontrará menos obstáculos al trabajar con las opciones de configuración de un servidor de correo conocido. Dado que su correo pasa por una serie de protocolos de seguridad en internet, un servidor de correo confiable probablemente garantizará su entrega correcta.

En este ejemplo, puede utilizar la cuenta de correo electrónico de la empresa o su cuenta de correo electrónico personal de Google para realizar la prueba.

CONFIGURACIÓN DE SU CORREO CON GOOGLE

Iniciar sesión en tu correo electrónico desde prácticamente cualquier lugar es cómodo, pero tiene sus inconvenientes. Por ejemplo, si planeas iniciar sesión en tu cuenta de Gmail en varios dispositivos, tendrás que escribir tu contraseña en cada uno, lo que podría provocar que tu contraseña principal se filtre o se use indebidamente. La contraseña de tu cliente de correo electrónico es sagrada y, por lo tanto, está más protegida gracias a los cambios implementados por Google y otras plataformas de correo electrónico.

Hay dos estrategias recomendadas para iniciar sesión en su cuenta de correo electrónico desde una aplicación potencialmente no segura:

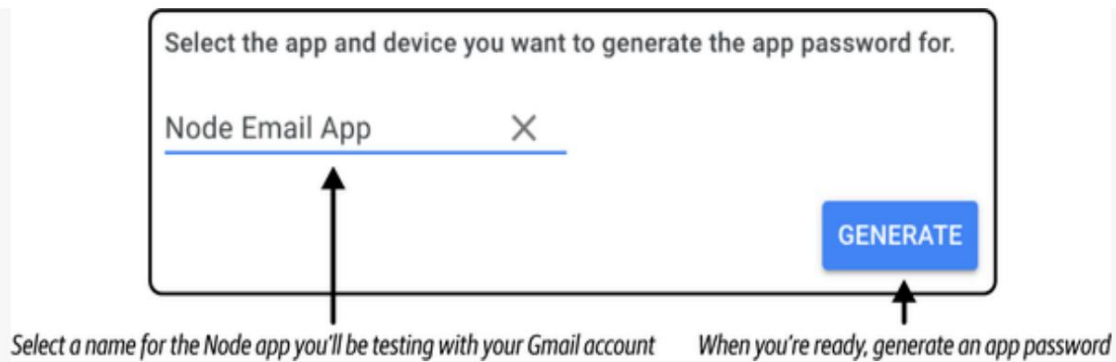
Iniciar sesión con Google

Este enfoque generalmente implica hacer clic en un enlace para iniciar sesión directamente con Google sin tener que escribir su contraseña directamente en la aplicación riesgosa.

Usar una contraseña de aplicación

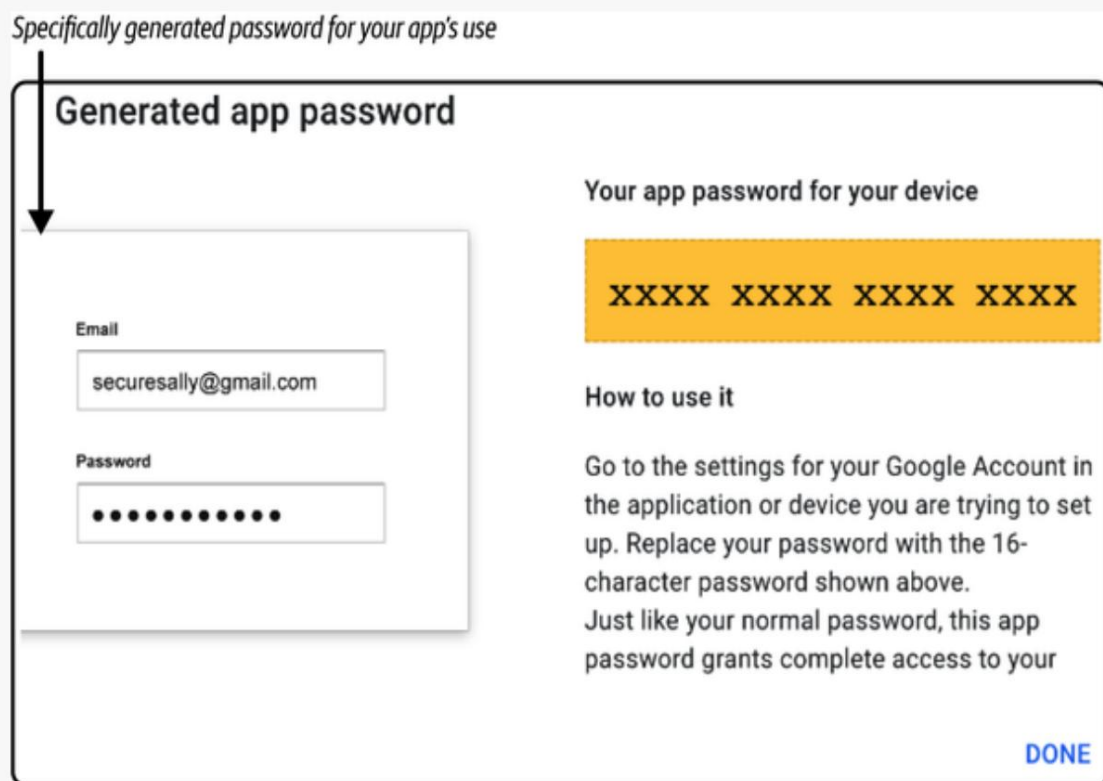
Las contraseñas de aplicaciones son contraseñas específicas generadas por Google para su uso en aplicaciones de terceros. Al usarlas, puede acceder a su correo electrónico a través de otro servicio sin revelar su contraseña principal.

Dado que está desarrollando una aplicación que no cumple con todos los estándares de seguridad necesarios para un servicio de correo electrónico, es posible que deba configurar una contraseña para usarla en el desarrollo. Para hacerlo con su cuenta de Gmail, siga **las instrucciones en línea de Google**. Desde aquí, se te guiará para administrar las contraseñas de tus aplicaciones y crear una nueva para la aplicación que estás creando (**Figura 8-3**). Puedes asignar cualquier nombre a la aplicación para identificar su asociación con la contraseña generada.



Cifra 83. Registrar una aplicación API de Gmail

Una vez que su aplicación esté registrada, se le solicitará un permiso especial. contraseña generada, como se ve en la Figura 8-4. En esta figura, las X Representa dónde se mostrará la contraseña de tu aplicación en tu sitio web. navegador.



Cifra 84. Recuperar una contraseña de token de API de Gmail

Cuando se complete este proceso, podrá utilizar la aplicación generada. contraseña junto con su dirección de correo electrónico de Gmail para iniciar sesión en su cuenta de tu proyecto.

Para la clave de servicio , ingrese un valor que coincida con su correo electrónico personal Servicio. En este caso, puede usar Gmail. La clave de autenticación se asigna a una objeto que contiene su usuario (dirección de correo electrónico) y contraseña (aplicación) contraseña) en [el Ejemplo 8-1](#).

Ejemplo 81. Configura tu transportador de correo en index.js

```
...
constante transportador = crearTransporte({ ❶
  servicio: "gmail", ❷
  aut: { ❸
    usuario: "jon@jonwexler.com",
    contraseña: "kdsnndsoonxxjsd",
  },
});
```

❶ Asignar la instancia createTransport al transporter.

❷ Añade un servicio de correo a tu configuración.

❸ Añade un correo electrónico y una contraseña de aplicación a tu configuración.

A continuación, configura un objeto con opciones para el contenido del correo electrónico.

[El ejemplo 8-2](#) muestra las claves de origen, destino, asunto y texto .

Indica que se enviará un correo electrónico de bienvenida desde la empresa Inn Box.

cuenta a su dirección de correo electrónico personal. El contenido del correo electrónico será Texto simple por ahora.

Ejemplo 82. Agregar el paquete nodemailer a index.js

```
...
constante mailOptions = { ❶
  de: "jon@innbox.jonwexler.com",
  a: "jon@jonwexler.com",
  Asunto: "¡Bienvenido a Inn Box!",
  texto: "Confirma tu correo electrónico",
};
```

❶ Asignar mailOptions para contener el contenido y los criterios del correo electrónico para un mensaje de bienvenida.

El último paso es crear una función asíncrona para enviar realmente el Correo electrónico. Agregue el código del **Ejemplo 8-3** al final de index.js. En este Código, crea una función `sendMail` que utiliza el Función `transporter.sendMail` , junto con sus opciones de correo, Para enviar un correo electrónico saliente. Esta llamada de función se encapsula en un try-catch para garantizar que la aplicación no falle en caso de que se genere una excepción. Durante el proceso de generación del correo electrónico. Al final del archivo, se llama `sendMail()` para ejecutar la función y enviar un correo electrónico tan pronto como

La aplicación se inicia.

Ejemplo 83. Agregar el paquete nodemailer a index.js

```
...
constante sendMail = async () => {           ❶
  intentar { ❷
    const info = await transporter.sendMail(mailOptions);           ❸
    console.log(`Correo electrónico enviado: ${info.response}`); ❹
  } captura (e) {
    console.log(`Ocurrió un error: ${e.message}`);           ❺
  }
};
```

`enviarCorreo();` ❻

- ❶ Define una función `sendMail` para enviar un correo electrónico.
- ❷ Utilice un bloque try-catch para manejar con elegancia cualquier error.
- ❸ Envíe un correo electrónico con `transporter.sendMail` y espere enviando para completar.
- ❹ Indique un correo electrónico enviado exitosamente en un registro en su consola.
- ❺ Detecta cualquier error y registra el mensaje en tu pantalla.
- ❻ Invocar la función `sendMail` .

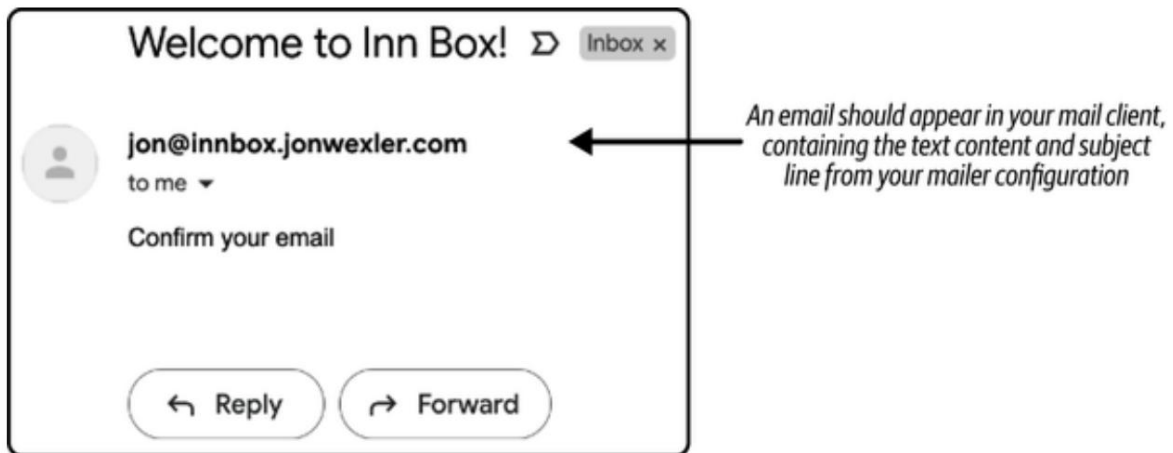
Navegue hasta el nivel raíz de su proyecto en su línea de comando y ejecute

Tu aplicación ingresando el índice del nodo. Después de un momento, verás el correo electrónico.
enviado: 250 2.0.0 OK 1662692125 m18-

20006b929a2bsm475205qkn.3 - gsmtip, o una variación similar de

texto. Ahora puede confirmar que se envió un correo electrónico a la dirección correcta.

dirección visitando directamente su cliente de correo electrónico. Allí encontrará una nueva
correo electrónico de Inn Box en su bandeja de entrada, como se muestra en la Figura 8-5.



Cifra 85. Mostrar el correo electrónico enviado correctamente en su cliente de correo

NOTA

Si no puede enviar el correo electrónico correctamente, intente solucionar el problema leyendo el mensaje de error que se muestra en la consola. Si el correo electrónico se envió... pero nunca lo recibió, puede intentar enviarlo a otra dirección de correo electrónico o revise su carpeta de spam.

Tenga en cuenta que el correo electrónico contiene solo texto sin formato: Confirme su correo electrónico. Debido a que no hay ningún enlace en este correo electrónico, tampoco hay forma de El usuario debe confirmar o verificar su cuenta con este ejemplo. Sin embargo, Puedes embellecer el correo electrónico reemplazando el texto simple por uno más enriquecido. Contenido HTML. Agregue el código del Ejemplo 8-4 sobre su correo. Definición de opciones en index.js. Este bloque de HTML mostrará...

el mismo texto, pero de forma más grande y audaz, utilizando las etiquetas de encabezado HTML h1 .

Por último, reemplace la clave y el valor de texto en mailOptions por html. En este caso, html es tanto la nueva clave como el nuevo valor representados por la variable html .

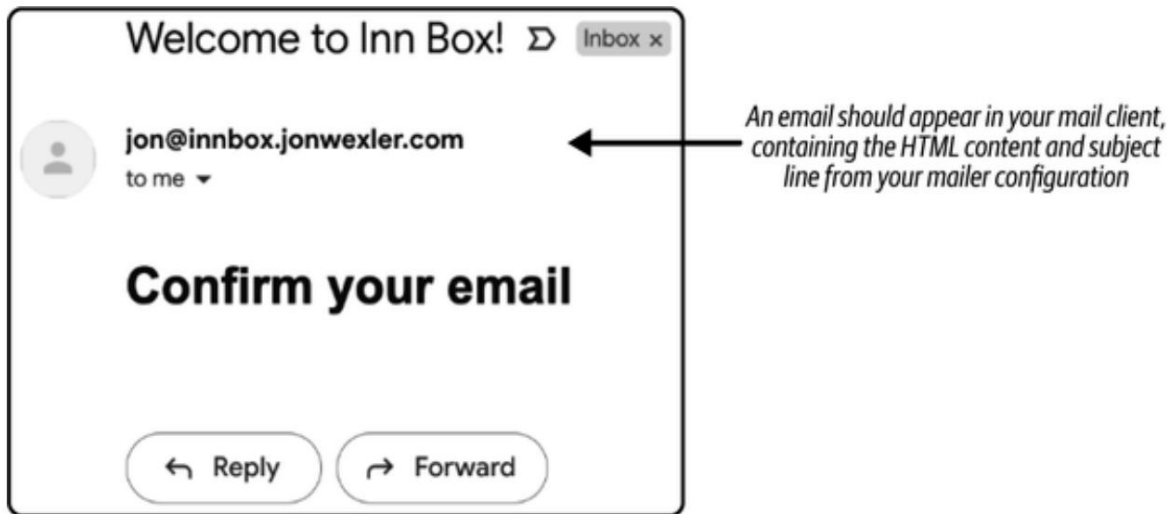
Ejemplo 84. Define un bloque de HTML para tu correo en index.js

```
...
const html = ❶
`<html>
  < cuerpo>
    < h1>
      Confirma tu correo
        electrónico
    < /h1> < /body> < /html>`;
...
❶
```

Definir contenido HTML y asignarlo a html.

Al cambiar las opciones de correo para usar HTML, vuelva a ejecutar la aplicación y observe el cambio en el próximo correo electrónico que reciba. Como se muestra en la Figura 8-6, verá el mismo texto que antes, pero mostrado en una etiqueta HTML. Desde aquí, puede agregar más estructura y estilos HTML para mejorar la apariencia del correo electrónico.

En la siguiente sección, presentaremos una API para recopilar direcciones de correo electrónico y enviar correos electrónicos dinámicamente en lugar de utilizar valores codificados.



Cifra 86. Muestra el correo electrónico HTML enviado correctamente en tu cliente de correo

Cómo agregar un marco para su servicio de correo

Has enviado correos electrónicos correctamente desde tu aplicación Node. Ahora necesitas... Esos correos electrónicos se envían a clientes potenciales. Para lograrlo, Necesitarás una API para recopilar correos electrónicos y enviarlos a tu sendMail. función. Antes de comenzar con la API, debe crear espacio en el Archivaindexjs moviendo su contenido actual. Cree una nueva carpeta llamada servicios en la raíz de tu proyecto. Dentro de esa carpeta, crea un archivo llamado correo.js y mover todo el código de indexjs a ese archivo.

Debido a que utilizará el contenido de mailerjs para enviar correos electrónicos, Querrás exportar la función sendMail . Primero, mueve el Definición de la variable mailOptions en la función sendMail y Agregue HTML y como parámetros de función. Esto le permitirá... Llamar dinámicamente a sendMail, pasando HTML personalizado y saliente direcciones de correo electrónico, como puede ver en el [Ejemplo 8-5](#).

NOTA

Puede agregar más parámetros para hacer que cualquier parte del contenido del correo electrónico sea visible. dinámico. O, si lo prefiere, puede elegir un asunto más genérico que
¡Funciona para todos los correos electrónicos, como el correo electrónico de Inn Box!

Ejemplo 5.8 Modificar y exportar sendMail en mailer.js

```
...
exportar const sendMail = async (a, html) => {
  constante mailOptions = {
    de: "jon@inbox.jonwexler.com",
    a,
    Asunto: "¡Correo electrónico de Inn Box!"
    html,
  };
  intentar {
    const info = await transporter.sendMail(mailOptions);
    console.log(`Correo electrónico enviado: ${info.response}`);
  } captura (e) {
    console.log(`Ocurrió un error: ${e.message}`);
  }
};
```

❶ Exportar la función sendMail para tomar html y enviarlo a (correo electrónico) argumentos.

❷ Intente enviar el correo electrónico utilizando el transportador y el registro configurados el resultado o mensaje de error.

Ahora tu correo ComEl módulo de servicio js está listo para ser utilizado en otros archivos. siguiente paso, puedes crear un módulo para contener el HTML de tu correo electrónico. Plantillas. Cree un nuevo archivo llamado mailTemplates.js en la raíz. de tu proyecto. En este archivo definirás las distintas plantillas de correo electrónico. Se utiliza para enviar correo saliente. Porque todo tu correo electrónico utilizará el Estructura HTML estándar, puede eliminar el bloque HTML de remitente.js y envuélvalo en una nueva función llamada htmlTemplate en

mailTemplates.js. Esta función toma un argumento de contenido y es
Se utiliza para generar una plantilla HTML con cualquier contenido que desees.

Agregue el código del **Ejemplo 8-6** para definir htmlTemplate y un
Función welcomeMail , que pasa texto envuelto en HTML como
contenido en su función htmlTemplate . Al pensar en nuevos correos electrónicos
Para enviar, puedes llamar a htmlTemplate y pasar tu HTML personalizado
contenido del cuerpo para generar el contenido completo del correo electrónico.

Ejemplo 8-6 Agregar una función de plantilla HTML dinámica en
Plantillas de correo.js

```
const htmlTemplate = (contenido) => {           ❶
  devolver `<html>
    <cuerpo>
      ${contenido} ❷
    </cuerpo>
  </html>`;
};

exportar const welcomeMail = () => {           ❸
  const contenido = return ¡Bienvenido a Inn Box!
  htmlTemplate(contenido);
};
```

❶ Define htmlTemplate para tomar el contenido del cuerpo como argumento.

❷ Llene el contenido HTML en el medio de su estructura estándar.

❸ Defina y exporte welcomeMail para devolver un HTML simple

Correo electrónico de bienvenida.

Con la lógica de su correo trasladada a su propio servicio, es hora de construir el
Servidor que usarás para recopilar correos electrónicos e invocar el correo. Para esto
En este proyecto, utilizarás Fastify, un servidor ligero y de alto rendimiento.

Marco de trabajo. Instale Fastify ejecutando npm install

fastify@^5.4.0 en el nivel raíz de su proyecto en el comando

línea. Luego, crea una instancia de aplicación de Fastify para representar tu aplicación

y toda su lógica para manejar solicitudes externas, como se ve en [el Ejemplo 8-7](#) .

Si planea manejar entradas `application/x-www-form-urlencoded` (como envíos de formularios o comandos `curl` usando el indicador `-d`), también necesitará registrar el complemento `@fastify/formbody` .

Instálelo usando `npm install @fastify/formbody@^8.0.2` y regístrelo en su aplicación para permitir que Fastify analice los cuerpos codificados en URL.

Ejemplo 87. Agregar fastify en index.js

```
import Fastify from "fastify"; import ❶
formBody from "@fastify/formbody"; ❷
```

```
const app = Fastify(); await ❸
app.register(formBody); Importar ❹
```

^❶ Fastify.

^❷ Importe el complemento `@fastify/formbody` para manejar datos de formulario codificados en URL.

^❸ Crear una instancia de una aplicación desde Fastify como aplicación.

^❹ Registre el analizador del cuerpo del formulario para permitir solicitudes POST con datos `application/x-www-form-urlencoded` .

Tu aplicación ya está configurada y lista para definir rutas API para la comunicación externa. Agrega una ruta llamada `app.post("/subscribe")`, que gestionará las solicitudes HTTP POST entrantes a `/subscribe` con un correo electrónico en el cuerpo de la solicitud.

NOTA

Una ruta en Fastify permite que tu aplicación distinga las diferentes solicitudes. Una solicitud POST implica que se enviará contenido a la aplicación, como un formulario. Desde aquí, puedes consultar el contenido (el cuerpo) de la solicitud para recopilar los datos necesarios en tu aplicación.

Añade la ruta del **Ejemplo 8-8** para recibir una dirección de correo electrónico publicada y registrarla en la pantalla. Usa `res.json` para enviar contenido JSON al solicitante e indicar el final de la función. Este sencillo experimento te ayudará a determinar si tu aplicación es accesible mediante HTTP POST.

Ejemplo 88. Agregar una ruta POST en `index.js`

...

```
app.post("/subscribe", async (solicitud, respuesta) => { const { email } ❶
  = solicitud.body; console.log(`Recibido $ ❷
  {email}`); responder.send({ mensaje: "ok"}); }); ❸
  ❹
```

- ❶ Define una ruta POST para la ruta `/subscribe`.
- ❷ Desestructurar el correo electrónico del cuerpo de la solicitud.
- ❸ Registra el correo electrónico publicado en tu consola.
- ❹ Devuelve un mensaje JSON en la respuesta.

Por último, define un puerto en el que tu aplicación se ejecutará localmente en tu ordenador y configúrala para que escuche las solicitudes entrantes. Agrega el **Ejemplo 8-9** al final de `index.js` para usar un número de puerto predefinido en las variables de entorno de tu ordenador (`process.env.PORT`) o usar el puerto 3000 como predeterminado. `app.listen` iniciará tu aplicación y escuchará las solicitudes entrantes en `localhost:3000` o en el puerto que hayas definido. Esto

La versión usa `async/await` en el nivel superior, lo cual es compatible con los módulos ECMAScript.

Ejemplo 89. Agregar una función de escucha para su aplicación en `index.js`

```
...
const puerto = proceso.env.PUERTO || 3000; ❶

try
  { await app.listen({ puerto }); ❷
    console.log(`Servidor ejecutándose en http://localhost:${puerto}`);
  } ❸
  catch (err) {
    console.error('Error al iniciar el servidor:', err); proceso.exit(1); ❹
  }
}
```

❶ Asigne un número de puerto de su entorno o el predeterminado 3000.

❷ Inicie el servidor usando `await app.listen({ port })` en el nivel superior.

❸ Registra un mensaje cuando el servidor se ejecuta correctamente.

❹ Registra cualquier error si el servidor no puede iniciarse.

Con el servidor configurado, inicia la aplicación y observa el texto "Servidor ejecutándose en localhost:3000" registrado en la consola. Con el servidor en ejecución, puedes enviar una solicitud POST para probar la ruta de la aplicación. Abre una nueva ventana de línea de comandos y ejecuta `curl -X POST -d "email=jon@jonwexler.com" http://localhost:3000/subscribe`. Este comando enviará una solicitud

POST a la ruta `/subscribe`, con una dirección de correo electrónico como parte del cuerpo de la solicitud. Verás `{"message":"ok"}` como respuesta en la segunda ventana de línea de comandos. La primera ventana de línea de comandos, con el servidor en ejecución, mostrará "Recibido jon@jonwexler.com".

Ahora puede conectar su servicio de correo para enviar un correo electrónico al recibir una solicitud POST. Importe las funciones de plantilla welcomeMail y sendMail a index.js, como se muestra en el [Ejemplo 8-10](#). Estas dos funciones ahora pueden usarse en la función de devolución de llamada de ruta.

Ejemplo 810. Importación de módulos de correo en index.js

```
...
import { sendMail } from './services/mailer.js'; import {
  { welcomeMail } from './mailTemplates.js';
```

1 Importe la función sendMail y la función de plantilla HTML welcomeMail .

Para usar estas funciones, la llamada de retorno de ruta debe ser asíncrona para permitir la espera de correo para enviar. Agregue la palabra clave async antes de la función de llamada de retorno. Luego, agregue await sendMail(email, welcomeMail()) justo encima de la línea reply.send . Esta nueva línea invocará la función sendMail y pasará la dirección de correo electrónico recibida en la solicitud POST, así como el contenido HTML de welcomeMail . Vuelva a ejecutar la aplicación e intente ejecutar el comando curl para una solicitud POST en la línea de comandos. Observe el correo electrónico que recibe en su cliente de correo con el asunto " Correo electrónico de Inn Box!" y el contenido " ¡Bienvenido a Inn Box!".

Tu aplicación ahora puede recibir direcciones de correo electrónico a través de este punto final de la API y enviar correos electrónicos salientes dinámicamente. En la siguiente sección, conectarás una base de datos para empezar a guardar estas direcciones de correo electrónico.

Conexión de una base de datos.

Configurar una base de datos garantiza que el contenido recuperado pueda utilizarse y revisarse a medida que la empresa, Inn Box, crece. Para este proyecto, se utiliza SQLite3 y el mapeador relacional de objetos Sequelize.

(ORM) para estructurar y guardar sus datos. Instale estos dos paquetes Detener su aplicación en la línea de comandos y ejecutar el comando `npm instalar sqlite3@^5.1.7 sequelize@^6.37.7`.

Con estos dos paquetes instalados, cree un nuevo archivo llamado `dbjs` en el nivel raíz del directorio de su proyecto. Al final de esta sección, su La estructura del directorio del proyecto debe verse como [el Ejemplo 8-11](#).

Ejemplo 8-11. Estructura del directorio del proyecto con una base de datos

de `marketing_mailer`

```
|
- index.js
- servicios
  |
  -mailer.js
- db.js ❶
- base de datos.sqlite ❷
- paquete.json
- módulos_de_nodo
```

❶ Agregue el archivo de configuración de la base de datos, `dbjs`, en el nivel raíz.

❷ El archivo `databasesqlite` se generará automáticamente una vez que su La base de datos está configurada y conectada.

Dentro de `js`, importe el módulo `Sequelize` agregando `import { dbSequelize }`, desde `"sequelize"` hasta el principio del archivo. Luego, agregue Las configuraciones necesarias para que `Sequelize` se conecte a un `sqlite` Base de datos. Use `new Sequelize` para crear una nueva base de datos. conexión. El dialecto será `sqlite`, indicando el tipo de base de datos Se conectará a. El almacenamiento es `databasesqlite`; lo que resultará en un archivo llamado `databasesqlite` que se genera en el nivel raíz de su proyecto cuando se inicia la aplicación. Después, use `db.authenticate()` para Establezca una conexión con su base de datos. Agregue el código en [Ejemplo 8-12](#) para `dbjs`.

NOTA

Es importante envolver esta lógica en un try-catch para garantizar que cualquier problema de conexión a la base de datos se registre correctamente en la consola.

Ejemplo 812. Configurar la base de datos en db.js

```
...
const db = new Sequelize({ dialecto: ❶
  "sqlite", almacenamiento:
  "./database.sqlite", });

try
  { await db.authenticate(); ❷
    console.log("La conexión se ha establecido correctamente."); }
catch (error) {

  console.error("No se puede conectar a la base de datos:", error); }
  ❸
```

- ❶ Cree una instancia de una nueva base de datos Sequelize usando sqlite.
- ❷ Intente conectarse a su base de datos.
- ❸ Detecte y registre cualquier error que se produzca.

Con la base de datos configurada, aún necesita una tabla para almacenar el contenido de los datos que utiliza Inn Box. Cree un nuevo modelo de Sequelize llamado Lead para representar a los nuevos suscriptores que eventualmente podrían convertirse en miembros del gimnasio. Agregue el código del [Ejemplo 8-13](#) a dbjs para definir ese modelo con tres campos. El campo de correo electrónico está configurado para ser único, por lo que no se encontrarán direcciones de correo electrónico duplicadas en la base de datos. También agrega una regla de validación, `isEmail`, que viene con Sequelize para garantizar que la estructura del correo electrónico coincida con la estructura tradicional. Esto reducirá el ruido del texto que ni siquiera se procesará durante

La etapa de envío del correo electrónico. El campo lastCampaign es una cadena que registra la clave de campaña utilizada para ese cliente potencial. Si Inn Box envía correos electrónicos con un descuento del 30%, la clave de campaña podría ser 30promo (Ejemplo 8-13).

Ejemplo 813. Definir y exportar el modelo Lead en db.js

```
...
const Lead = db.define( "Lead", ❶
  { email:

    { tipo: ❷
      Sequelize.STRING, único:
      verdadero, validar:
      { isEmail:
        verdadero, }, },

    verificado: ❸
      { tipo: Sequelize.BOOLEAN, valor
        predeterminado: falso, },

    última campaña: ❹
      { tipo: Sequelize.STRING, }, },

  {} );

Plomo.sync(); ❺
```

Exportar valor predeterminado Lead; ❻

- ❶ Definir un nuevo modelo de Sequelize : Lead.
- ❷ Agregue un campo de correo electrónico único y conforme.
- ❸ Añade un campo verificado al valor predeterminado como falso.
- ❹ Añade un campo lastCampaign .

❺

Sincronice el modelo con su base de datos al conectarse.

- 6 Exporte el modelo Lead como el módulo de exportación predeterminado.

Con su modelo de cliente potencial en funcionamiento, está listo para comenzar a ahorrar correo electrónico. direcciones a medida que llegan a su ruta POST /subscribe . En su

archivo js , importe el modelo Lead en la parte superior de su archivo agregando Importación de índices desde ".db.js". Dentro de la ruta /subscribe , Agregue el código del **Ejemplo 8-14** para envolver su llamada sendMail en un bloque try-catch e intente guardar un registro de cliente potencial en su base de datos.

Con Lead.create({ email }). Esperando a que se guarde la base de datos.

Este registro pospondrá el envío del correo electrónico a esa dirección hasta después Esta guardado

Ejemplo 14.8 Guardar un nuevo cliente potencial en index.js

```
...
intentar { 1
  esperar Lead.crear({ email }); 2
  esperar sendMail(correo electrónico, welcomeMail());
} captura (e) {
  console.log(" No se pudo guardar el cliente potencial", e.message);
}
...
```

- 1 Envuelva su llamada a la base de datos en un bloque try/catch .

- 2 Cree un nuevo registro de cliente potencial con la dirección de correo electrónico del cuerpo de la solicitud.

Ahora, reinicia tu aplicación. Notarás que tu consola muestra más registros.

Esta vez para indicar la conexión a su base de datos. En un archivo separado

ventana de línea de comandos, ejecute su comando curl POST para ver su

Se guardó el registro y se envió un nuevo correo electrónico a su cliente de correo electrónico. Para probar el validación de correo electrónico, intente guardar la misma dirección de correo electrónico una segunda vez o enviar un texto malformado en lugar de una dirección de correo electrónico real.

Tenga en cuenta que el registro no se guardará en este caso, sino que registrará:
No se pudo guardar el error de validación de clientes potenciales.

CONSEJO

Limpiar su base de datos SQLite es tan fácil como eliminar el archivo databasesqlite en su carpeta de proyecto.

Con su base de datos conectada, agregue una nueva ruta para verificar su dirección de correo electrónico. Esta nueva ruta debe recibir solicitudes GET que contengan la dirección que desea verificar. Dentro de la ruta, la lógica comprobará si la dirección existe en su base de datos.

Luego, si el cliente potencial existe, modifica el campo verificado y guarda el registro .
Agrega dos líneas res.json diferentes , según si la verificación del correo electrónico es correcta (**Ejemplo 8-15**).

Ejemplo 815. Agregar una ruta de verificación en index.js

```
...
app.get("/verificar/:email", async (solicitud, respuesta) => { const { email } =      ❶
  solicitud.params; try { const lead = await      ❷

    Lead.findOne({ donde: { email } }); if (lead) { lead.verified = true; await lead.save();      ❸
      console.log(`      ❹
        {email} está verificado`);      ❺
      reply.send({ mensaje:      ❻
        "¡Verificado!" });      ❼

      ❽

    } } captura (e) {
      console.log(" No se pudo verificar el cliente potencial", e.message);

    } reply.send({ message: "No se puede verificar." }); });      ❸
...

```

❶ Agregue una nueva ruta para manejar direcciones de correo electrónico como un parámetro de URL.

❷

Desestructurar el correo electrónico a partir de los parámetros de la solicitud .

- ③ Busque un cliente potencial por la dirección de correo electrónico en la solicitud.
- ④ Compruebe si se encontró un cliente potencial .
- ⑤ Si se encuentra un cliente potencial , modifique su campo verificado .
- ⑥ Guarde la instancia de Lead modificada .
- ⑦ Confirmar al usuario que su correo electrónico fue verificado.
- ⑧ Notificar al usuario si no se puede verificar su correo electrónico.

Una forma de probar esta nueva ruta es modificando el welcomeMail Plantilla para incluir un enlace a esta ruta. Agregar una nueva función.

ConfirmationMail, en mailTemplates.js que se ajustará dinámicamente

El texto de tu correo electrónico en un enlace. Cuando el usuario haga clic en ese enlace, será... dirigido a su ruta /verify:email , con lo cual su correo electrónico Se verificará la dirección (Ejemplo 8-16).

Ejemplo 8-16. Agregar una nueva plantilla de confirmación dinámica en mailTemplates.js

```
...
exportar const confirmationMail = (url) => { ①
  const content = ` <a href="${url}"> <h1>Confirme su
correo electrónico</h1></a>`; ②
  devolver htmlTemplate(contenido);
};
```

① Agregue una nueva función de plantilla de correo para tomar un parámetro de URL .

② Crea un enlace al argumento de URL para confirmar el correo electrónico del usuario.

En el índice.js, importe la función de plantilla de confirmaciónMail y

Úselo en su ruta /subscribe en lugar de welcomeMail

Función de plantilla. La nueva función aceptará una URL. En este caso, `confirmationMail(http://localhost:3000/verify/${email})` apunta a tu aplicación local y pasa dinámicamente la dirección de correo electrónico.

NOTA

Para probar correctamente esta nueva ruta, deberá implementar su aplicación en internet. Una vez que su cliente de correo electrónico, como Gmail, recibe un correo electrónico, no necesariamente tiene forma de abrir un enlace a su servidor web local en `localhost:3000`. Una vez que la aplicación esté en producción, puede cambiar la URL para que refleje la ubicación pública. Al hacer clic en el enlace de confirmación, se realizará una verificación en su servidor y en su base de datos.

Ahora que tiene una forma de crear dinámicamente un enlace dentro de sus correos electrónicos salientes, puede comenzar a crear campañas de marketing para Inn Box y realizar un seguimiento de qué clientes potenciales están interesados.

Implementar un píxel de marketing para la interacción por correo electrónico. El

email marketing es un

negocio enorme que genera miles de millones de dólares al año en todos los sectores. Parte de su éxito reside en conocer mejor a los lectores de los correos electrónicos y si interactúan con tu producto. Dado que tu cliente de correo electrónico es un espacio personalizado y privado para tus correos, esto puede ser una tarea difícil.

Afortunadamente, la tecnología ofrece tanto las herramientas para llegar a tus clientes como la creatividad para comprender su interacción con tus correos electrónicos. Una forma en que los servicios de marketing rastrean si un cliente potencial abrió su correo electrónico es agregando un píxel de seguimiento, también conocido como píxel de marketing. Este píxel es una imagen literal de un solo píxel extraída del servicio de marketing.

Servidores. El truco está en que la URL de la imagen contenga suficiente información para que, cuando se solicite su carga desde el servidor, este también pueda registrar qué cliente potencial abrió el correo electrónico y qué campaña se promocionó en él.

De esta manera, las empresas pueden determinar qué correos electrónicos se están abriendo, lo que impulsa las decisiones sobre las campañas de marketing que están funcionando y, en última instancia, alimentando a la empresa con más clientes que pagan. Ya has establecido un modelo que permite guardar cadenas de campaña para cada cliente potencial. Siguiendo un enfoque similar al de la URL dinámica de la plantilla de confirmación, crearás una nueva ruta para mostrar el contenido cargado en los correos electrónicos de tu campaña de marketing.

Agregue el código del **Ejemplo 8-17** para definir una nueva ruta GET para /campaign/:campaignKey/user/:email/image.png. Esta ruta contiene una clave de campaña que identifica las promociones del correo electrónico enviado.

También se utiliza un parámetro de correo electrónico para asignar este enlace al correo electrónico del usuario objetivo en la base de datos. La ruta termina con image.png para simular que el servidor cargará una imagen. Dentro de la función de devolución de llamada, se extraen los valores de correo electrónico y de la clave de campaña de la solicitud.

Luego, se busca un usuario por la dirección de correo electrónico proporcionada y se actualiza el campo lastCampaign del registro correspondiente para que coincida con la clave de campaña del correo electrónico de marketing.

De esta manera, puede rastrear rápidamente qué usuarios abrieron los correos electrónicos de marketing más recientes y ayudar a la empresa a determinar el éxito de sus promociones por correo electrónico.

Ejemplo 817. Agregar una ruta de seguimiento de campañas en index.js

...

```
app.get("/campaign/:campaignKey/user/:email/image.png", async (solicitud,
respuesta) => { const { email, 1
  campaignKey } = request.params; try { const lead = await 2
```

```
    Lead.findOne({ donde: { email } }); if (cliente potencial) { lead.lastCampaign = 3
      campaignKey;
```

4

```

    esperar lead.save();
    console.log(`${email} abrió ${campaignKey}`);
  } } catch (e)
    { console.log(" Ocurrió un error", e.message);

  } responder.enviar({ mensaje: "ok" }); });

```

...

❶

Agregue una nueva ruta de seguimiento de campaña para tomar los parámetros de campaña y correo electrónico .

❷

Desestructurar el correo electrónico y la clave de campaña de los parámetros de la solicitud .

❸

Verifique si existe un cliente potencial con el correo electrónico proporcionado .

❹

Si el cliente potencial existe, modifique su última campaña para que sea la clave de campaña y guarde el registro.

❺

Registra si se vio el contenido del correo electrónico para la campaña determinada.

❻

Responda con un mensaje JSON simple.

En mailTemplatesjs, cree una nueva función llamada campaignMail para crear una nueva plantilla para una promoción por correo electrónico (Ejemplo 8-18).

Ejemplo 818. Agregar una plantilla de correo de campaña en mailTemplates.js

...

```

exportar const campaignMail = (campaignText, campaignKey, email) => { const content = <h1>$
{campaignText}</
h1> 

```

```
;
devolver htmlTemplate(contenido); };
```

- ❶ Agregue una nueva función de plantilla de correo con texto dinámico, una clave de campaña y el correo electrónico del usuario .
- ❷ Agregue una etiqueta de imagen para obtenerla de su ruta de seguimiento de campaña dinámica.

Para ejecutar esta función, importe `campaignMail` en la parte superior de `index.js` y agregue `sendMail("jon@jonwexler.com", campaignMail("Special Promotion", "promo1", "jon@jonwexler.com"))` encima del bloque `app.listen` en `index.js`. Asegúrese de probar el código con sus propias direcciones de correo electrónico. En cuanto reinicie la aplicación Node, se enviará un correo electrónico con un enlace oculto a una imagen de seguimiento. El correo electrónico que recibirá en su cliente de correo electrónico indicará " Special Promotion", aunque no podrá ver inmediatamente la clave de seguimiento de la campaña . Al igual que ocurre con la confirmación de una dirección de correo electrónico en un proyecto de desarrollo, primero deberá implementar esta aplicación para comprobar el funcionamiento de la lógica de seguimiento de correo electrónico y guardarla en su base de datos.

Mientras tanto, puede enviar una solicitud curl para imitar la llamada a la URL de la imagen abriendo una nueva ventana de línea de comandos y ejecutando `curl http://localhost:3000/campaign/promo1/user/jon@jonwexler.com/image.png`. En esa misma ventana, verá una respuesta de `{"message":"ok"}`. Su servidor registrará la apertura de `jon@jonwexler.com promo1` en su ventana de consola, junto con los registros de una consulta a la base de datos que indica un registro actualizado.

Con tu aplicación lista para verificar correos electrónicos y hacer seguimiento de campañas, el último paso es automatizar el proceso de envío. En la siguiente sección, usarás un paquete de programación para enviar correos electrónicos a horas específicas.

Integración de un programador de tareas

Su aplicación está configurada para gestionar la mayor parte de lo que una empresa necesita obtener. Empecé con el marketing por correo electrónico. La funcionalidad es básica, pero abre la puerta. puerta para correos electrónicos con formato creativo y un sistema de seguimiento para ayudar a identificar el mercado de pago de la empresa. Automatizar el envío de correos electrónicos no son obligatorios, pero en última instancia harán que el proceso sea más sencillo. Desde una perspectiva de gestión. Como todo en Node, Hay un paquete que ayuda a administrar tareas en un período de tiempo específico.

En el nivel raíz de la carpeta de su proyecto, en su línea de comandos, ejecute `npm install node-schedule@^2.1.1` para instalar node-schedule. Este paquete proporciona funciones con una variedad de sintaxis para programar tareas a ejecutar. En lugar de enviar el correo electrónico de la campaña como Tan pronto como se inicie la aplicación, puede moverla a un programador para mejorarla. Administrar cuándo se envía el correo electrónico.

Crea un archivo en tu carpeta de servicios llamado `scheduler.js`. Este archivo... Gestionar la programación y ejecución de tareas. Importar el `scheduleJob` directamente desde este paquete agregando importación `{ scheduleJob }` desde "node-schedule" hasta la parte superior de `Scheduler.js`. A continuación, agregue el código del **Ejemplo 8-19** para definir un función, programar, para iniciar una tarea programada. Por ahora, puede Haga que esa tarea sea una declaración registrada que se ejecuta de acuerdo con las Opciones de tiempo que usted proporcione.

Ejemplo 8-19. Agregar una función de programador de tareas en `scheduler.js`

```
...
export const horario = (opcionesDeTiempo) => { ❶
  ScheduleJob(opcionesDeTiempo, () => { ❷
    console.log("Es hora de enviar un correo electrónico."); ❸
  });
};
```

❶ Defina y exporte una función de programación que acepte un argumento `timeOptions`.

- ② Ejecute `scheduleJob` con las configuraciones de tiempo dadas.
- ③ Registre un mensaje en su consola durante la ejecución de su trabajo programado

Ahora importe la función de programación desde el módulo programador en `index.js` agregando `import {schedule} desde services/scheduler.js" al principio del archivo. Debajo, Agregue schedule({ segundos: 30 }) para ejecutar su consola programada Registra cada minuto en la marca de 30 segundos de ese minuto. Reinicia tu aplicación y observe cómo Hora de enviar un correo electrónico. se imprime en su consola una vez cada minuto.`

A continuación, copia las importaciones de `campaignMail` y `sendMail` a `Función` `js` y envía tu plantilla de campañaMail desde el `schedulerschedule` en lugar de `index.js`. Ahora, tu programación

La función de devolución de llamada se ha vuelto asíncrona, ya que maneja el envío de campañas. correos electrónicos. Cada vez que llame a la función de programación , los correos electrónicos no se enviarán. ser enviados inmediatamente, pero sólo a la hora programada (Ejemplo 8-20).

Ejemplo 820. Ejecutar una campaña de correo electrónico desde un trabajo programado en `scheduler.js`

```
...
import { scheduleJob } desde "node-schedule";
import { sendMail } desde "../mailer.js";
import { campaignMail } desde "../mailTemplates.js";

exportar const horario = (opcionesDeTiempo) => {
  ScheduleJob(opcionesDeTiempo, async () => {
    esperar sendMail(
      "jon@jonwexler.com",
      campaignMail(" Promoción especial ", "promo1",
        "jon@jonwexler.com")
    );
  });
}
```

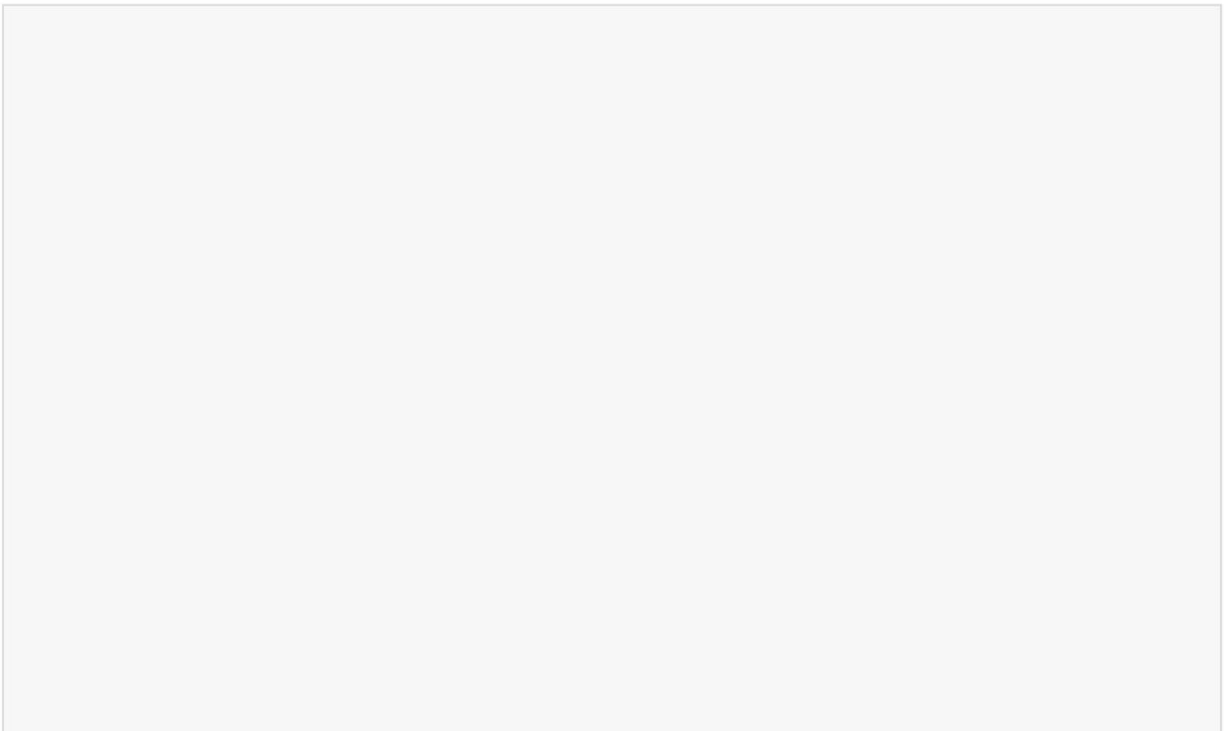
```
});};
```

- ❶ Ajuste las importaciones del módulo en relación con la ubicación del archivo schedulerjs .
- ❷ Cambie la función de devolución de llamada scheduleJob para que sea asincrónica.
- ❸ Enviar un correo electrónico a la hora programada.

Después de guardar este archivo y reiniciar la aplicación, verá que se envía un correo electrónico al destinatario especificado cada minuto. Puede cambiar el ejemplo de la hora de un archivo js modificando {second: 30}. Para trabajo programado en el índice , el correo electrónico podría programarse para enviarse todos los lunes a la 1 p. m., en cuyo caso, timeOptions sería {dayOfWeek: 1, hour: 13 }. Para obtener más información sobre node-schedule y sus configuraciones de programación, visite el [sitio web de npm](#).

Con su programador en funcionamiento, habrá creado una base integral para la aplicación de campaña de marketing de Inn Box.

Desde aquí podrás dinamizar algunas de las funciones o empezar a decorar el HTML de tus emails de campaña.



EJERCICIOS DEL CAPÍTULO

1. Añade la función de cancelación de suscripción a tus campañas de correo electrónico:

Para añadir la función de cancelación de suscripción a tus campañas de correo electrónico, empieza por crear una nueva ruta GET `/unsubscribe/:email` en tu servidor que actualice el registro de cliente potencial correspondiente estableciendo un nuevo campo de cancelación de suscripción como verdadero. Modifica tu modelo de cliente potencial para incluir este campo de cancelación de suscripción con un valor predeterminado de falso. A continuación, actualiza tus plantillas de correo electrónico para incluir un enlace de cancelación de suscripción visible que apunte a `/unsubscribe/:email` al final de cada mensaje. Por último, asegúrate de que tu aplicación revise este campo antes de enviar correos electrónicos, lo que evitará que los correos electrónicos programados o de resuscripción lleguen a los usuarios que se han dado de baja.

CONSEJO

Ofrecer a los usuarios la opción de cancelar la suscripción ayuda a evitar quejas de spam y mantiene su lista de correo saludable.

2. Seguimiento de clics en enlaces promocionales dentro de correos electrónicos:

Para rastrear los clics en enlaces promocionales, agregue una nueva ruta GET como `/click/:campaignKey/user/:email` que registre cuándo un usuario hace clic en un enlace en su correo electrónico. Actualice su modelo de cliente potencial para incluir un nuevo campo `"lastClickedCampaign"` que almacene esta información. Modifique una de las plantillas de correo electrónico de su campaña para incluir un botón o hipervínculo visible que dirija a la nueva ruta de seguimiento. Cuando se visite el enlace, actualice la información del usuario correspondiente.

campo `lastClickedCampaign` y registra un mensaje de confirmación en la consola.

CONSEJO

Si bien las tasas de apertura son útiles, las tasas de clics a menudo brindan una señal más clara del interés y la intención del usuario.

Resumen En este

capítulo, creó un correo de marketing completo utilizando Node.

Aprendiste a:

- Envíe correos electrónicos programáticamente con contenido HTML rico y dinámico
- Recopilar y validar direcciones de correo electrónico a través de un punto final de API
- Verificar cuentas de usuario mediante enlaces de confirmación en correos electrónicos
- Realice un seguimiento de la participación del usuario registrando las aperturas de correo electrónico y las interacciones de las campañas
- Programe y automatice los correos electrónicos salientes con un programador de tareas personalizable

Capítulo 9. Raspador web

Este capítulo cubre lo siguiente:

- Extraer contenido de un sitio web HTML
- Ejecutar un navegador sin cabeza
- Recopilación de datos extraídos y su oferta como API

En este capítulo, crearás un web scraper para recopilar contenido de sitios web HTML y procesarlo en tu servidor Node. El web scraping es el proceso de extraer datos de un sitio web y existe desde hace casi tanto tiempo como internet.

Antes de que las API se hicieran públicas y accesibles para las empresas, la única forma de obtener precios de productos actualizados, titulares de noticias inmediatos y contenido web estático era visitar manualmente una URL web y mirar directamente la página web resultante; algo similar a cómo todavía usamos Internet en gran medida hoy.

Ahora hay más maneras de procesar y usar datos que nunca, pero aún no hay suficientes API para alimentar esos sistemas de procesamiento. Incluso donde existen API, las restricciones en el tipo de datos a los que un usuario final tiene acceso pueden ser limitadas. Por ejemplo, un servicio de entrega de comida a domicilio puede proporcionar una API para los mejores restaurantes de un barrio, pero no permitir el filtrado por restricciones dietéticas. Para obtener esos datos, podría, alternativamente, visitar la URL de los resultados filtrados por restricción dietética y extraer el contenido resultante. En este capítulo, explorará las opciones disponibles en el ecosistema de Node para extraer datos de páginas web y usar esos datos en su propia aplicación.

HERRAMIENTAS Y APLICACIONES UTILIZADAS EN ESTE CAPÍTULO

Antes de comenzar, asegúrese de instalar y configurar las herramientas y aplicaciones necesarias para este proyecto. Las instrucciones de instalación de Node.js, Fastify y VS Code se encuentran en **el Capítulo 1**, mientras que los pasos de inicialización del proyecto, como la configuración de la estructura de directorios, la configuración de package.json y el uso de sintaxis moderna, se describen en **el Apéndice A**. Una vez completado, regrese aquí para continuar. Desarrollar un proyecto desde cero le ayuda a comprender mejor cada componente, brindándole mayor control y flexibilidad a medida que avanza.

Su mensaje Como

participante activo en un foro tecnológico en línea, Node Dojo, tiene la tarea de ayudar a recopilar artículos y tutoriales relacionados con Node.

Tras explorar internet, decides recopilar los mejores artículos de Node disponibles en **<https://medium.com>**. Aunque Medium ofrece una **API**, decides desarrollar una herramienta de web scraping para obtener los títulos y enlaces de los artículos más populares de Node.

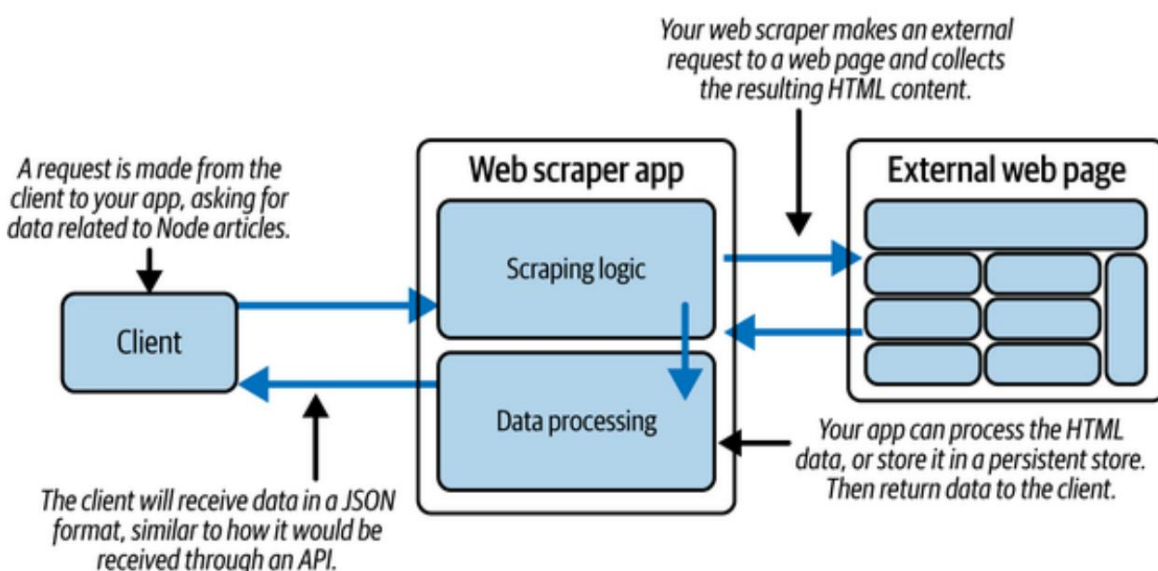
Planifica tu proyecto .

Planeas crear una aplicación que visite páginas web automáticamente y devuelva títulos y URLs de artículos. De esta forma, cuando se publiquen nuevos artículos en Node, podrás extraer datos del sitio de Medium y ofrecer a tu foro tecnológico los artículos y tutoriales más interesantes.

Para comenzar, apuntará a una URL específica y obtendrá su contenido HTML. Inicialmente, ese contenido podría analizarse como texto plano, donde encontrará el contenido de los artículos que busca. Más adelante, incorporará potentes paquetes npm diseñados para obtener el contenido de una página web y...

Te ayudará a analizar el elemento de esa página de forma más eficiente. Al final del proceso de desarrollo, tu aplicación incluso permitirá a los usuarios buscar palabras clave específicas y devolver artículos coincidentes del sitio de Medium.

Comienza dibujando un diagrama que detalle la relación entre la lógica de tu aplicación y cómo se extraen los datos de los sitios externos. En **la Figura 9-1**, estableces que un usuario iniciará tu aplicación, tras lo cual se obtendrá y extraerá el sitio externo de destino. El contenido HTML devuelto se procesa en tu aplicación para que puedas seleccionar solo los títulos y enlaces de los artículos. Finalmente, devuelves esos valores al cliente que inició la aplicación.



Cifra 91. Plano de proyecto para una aplicación Node de raspado web

Esta aplicación tiene dos partes:

Acceso y raspado

Dado que no existe necesariamente otra forma de acceder a los datos del sitio web, es importante que pueda acceder al sitio. Si el sitio está bloqueado por una capa de autenticación o es inaccesible por cualquier otro motivo, el scraping no funcionará.

Filtración

Después de ponerse en contacto con el sitio, recibirá el contenido del sitio. como datos HTML. HTML ofrece una estructura predecible que puede Aprovecha para enfocarte solo en los datos que necesitas. Si tu aplicación está diseñada Bueno, no debería haber necesidad de mucho mantenimiento en el futuro. camino.

Con esta estructura en su lugar, estás listo para comenzar a crear una aplicación.

Obtenga programación

Para comenzar a desarrollar su aplicación, cree una nueva carpeta llamada site_scraper. Navegue a la carpeta de su proyecto con su comando. y ejecute npm init. Este comando inicializa su aplicación Node. Puede presionar Enter durante los pasos de inicialización para aceptar el valores predeterminados. El resultado de estos pasos es la creación de un archivo llamado .json. Este archivo le indicará a su aplicación Node cualquier configuraciones de paquetes o scripts necesarios para ejecutarse correctamente.

A continuación, crea un archivo llamado index.js dentro de la carpeta de tu proyecto. Este archivo es El punto de entrada para su aplicación.

Con tu aplicación Node inicializada, puedes navegar al índice y comenzar a archivo js a codificar. Sin instalar ningún paquete adicional, estás...

Ya está equipado para obtener el contenido HTML de un sitio web mediante la API de búsqueda integrada . Agregue el código del **Ejemplo 9-1** para usar la función de búsqueda y crear un...

Solicitud GET a [**https://nodejs.org/en/**](https://nodejs.org/en/). Recibirás una

objeto de respuesta **de etiqueta** del sitio, en el que puede llamar al texto()

Método para convertir el HTML de respuesta a texto sin formato. Luego, registre el texto resultante usando console.log.

Este ejemplo utiliza await de nivel superior , una función moderna de JavaScript compatible con Node 18 y posteriores cuando su proyecto está configurado como un Módulo ECMAScript. Si su entorno no admite el nivel superior espera, puedes envolver tu lógica en una función asíncrona en su lugar.

Ejemplo 91. Obtener contenido HTML de una página externa en index.js

```
const URL = "https://medium.com/tag/nodejs";

try
{ const respuesta = await fetch(URL); const texto =
  await respuesta.text(); console.log(texto); } catch (e)
{ console.log("error",
e.message);
}
```

- ➊ Asignar la variable URL al enlace superior de los artículos del nodo mediano.
- ➋ Envuelva su código en un bloque try/catch para manejar posibles errores.
- ➌ Realice una solicitud GET a la URL especificada mediante fetch.
- ➍ Convierte la respuesta a texto sin formato.
- ➎ Envíe el HTML simple a su consola.
- ➏ Captura y registra cualquier error que ocurra durante la solicitud.

Accede a tu proyecto desde la línea de comandos y ejecuta `node index` para iniciar la aplicación. Dependiendo de tu conectividad, la llamada de búsqueda puede tardar unos segundos. Al finalizar, verás cientos de líneas de HTML en pantalla, similares al siguiente contenido:

```
<!doctype html> <html
lang="es"> <head> <title
data-
rh="true">Las historias más reveladoras sobre Node.js...</title>

<meta data-rh="true" charset="utf-8"/> <meta data-rh="true"
name="ventana gráfica"
contenido="ancho=ancho-del-dispositivo,..."/>
<meta data-rh="true" name="color del tema"
contenido="#000000"/>
```

```
...  
</  
cabeza> </html>
```

NOTA

Si no ve ningún contenido de salida, asegúrese de que la declaración del registro de la consola esté escrita correctamente. Si se produce algún error, verá el mensaje de error registrado en la pantalla y podrá depurar desde esa declaración impresa.

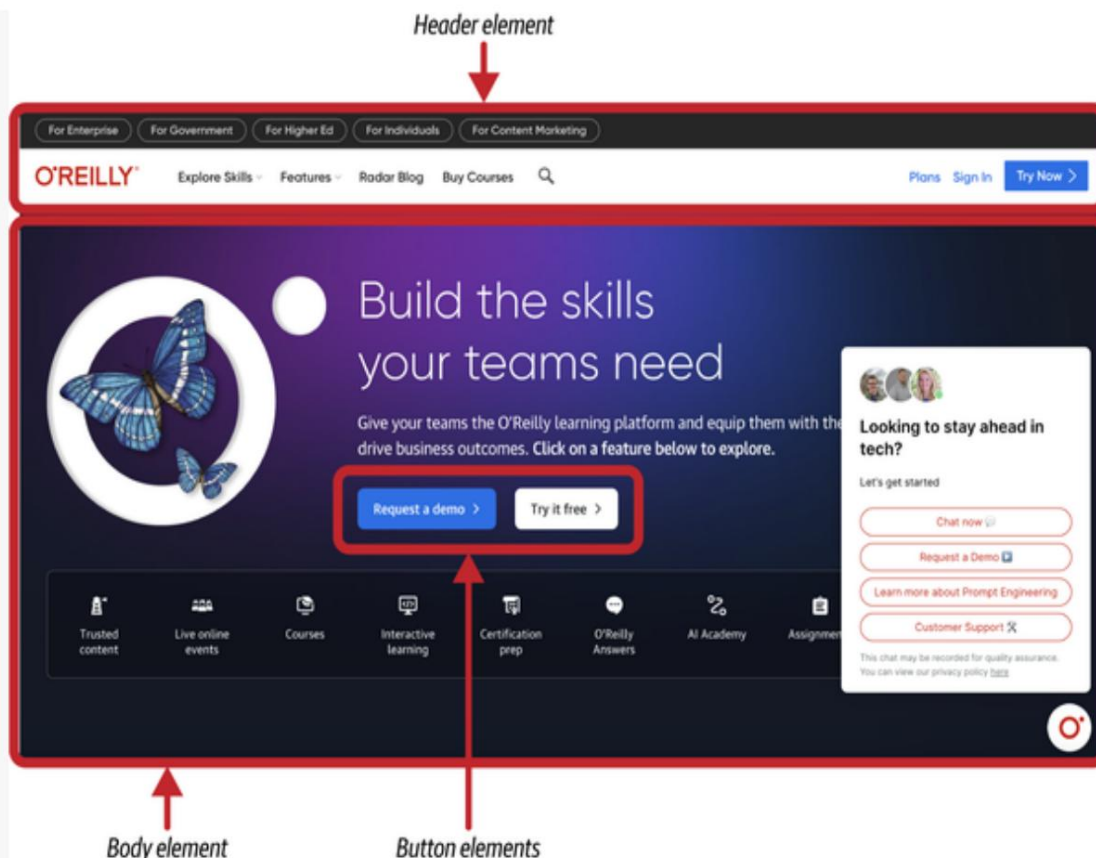
Esta salida es el código fuente HTML típico de una página web. Dentro de la respuesta, encontrará etiquetas HTML como `<div>` o `<a>` que marcan elementos individuales. Estos son los bloques de HTML que definen la estructura de la página web en su navegador.

UNA BREVE PALABRA SOBRE HTML

Si eres nuevo en el desarrollo frontend o en el desarrollo de software

En general, es posible que no esté familiarizado con el funcionamiento de las páginas web. construido. Así como el código JavaScript depende de una sintaxis determinada y estructura, la construcción visual de un sitio web tiene su propia normas.

El lenguaje de marcado de hipertexto (HTML) es la composición de casi cada página web o aplicación web que has visitado en internet. Cuando visitas una URL en tu navegador web, verás una imagen representación de bloques de datos conocidos como elementos HTML, en una formación accesible mediante JavaScript del lado del cliente, conocida como Modelo de Objetos de Documento (DOM). A través de esta relación, cada El elemento representa una parte de la página visual que ves. Algunos de Esos elementos contienen párrafos de texto. Otros pueden contener anuncios o enlaces. La Figura 9-2 muestra la página de inicio de <https://www.dailymail.com>. En este ejemplo, puede notar cómo Los aspectos visuales de la página son en realidad HTML autónomo. elementos bajo el capó.



Cifra 92. Ejemplo de desglose de página HTML para www.oreilly.com

Cada elemento HTML tiene una etiqueta HTML de apertura y una de cierre.

La etiqueta `<div>` es la etiqueta HTML más utilizada (aunque podría decirse que es generalmente mal utilizado). Es posible que el título de un artículo en línea

Se encuentra en una etiqueta `<div>`. Algunos elementos contienen un nombre de clase o id para ayudar a diferenciar entre otros elementos. Por ejemplo,

Un título de artículo puede aparecer en un elemento marcado con un título de clase como esta: `<div class="title"></div>`. Otras veces

Puede obtener el título implícitamente del tipo de etiqueta utilizada para ello.

elemento. Por ejemplo, un `<h1>` (o etiqueta de encabezado de nivel 1) es

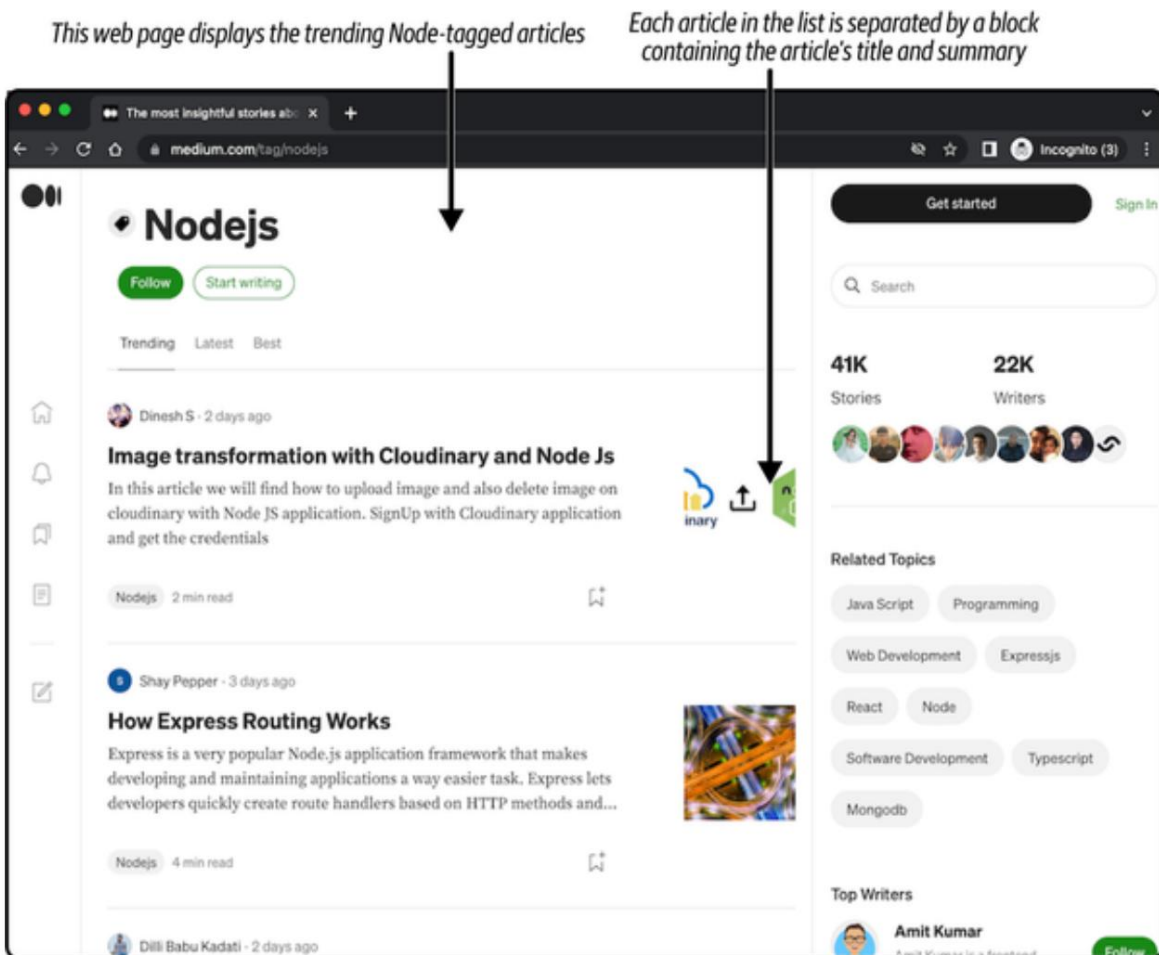
Se suelen usar para envolver el texto del título. Estas etiquetas facilitan... para apuntar cuando se busca a través de la salida HTML para recopilar la información específica que busca.

Más adelante en esta sección, explorará la respuesta HTML de una página obtenida externamente para analizar más a fondo una página web mientras

Raspando para encontrar contenido.

Para obtener más información sobre HTML, lea la [documentación de MDN](#).

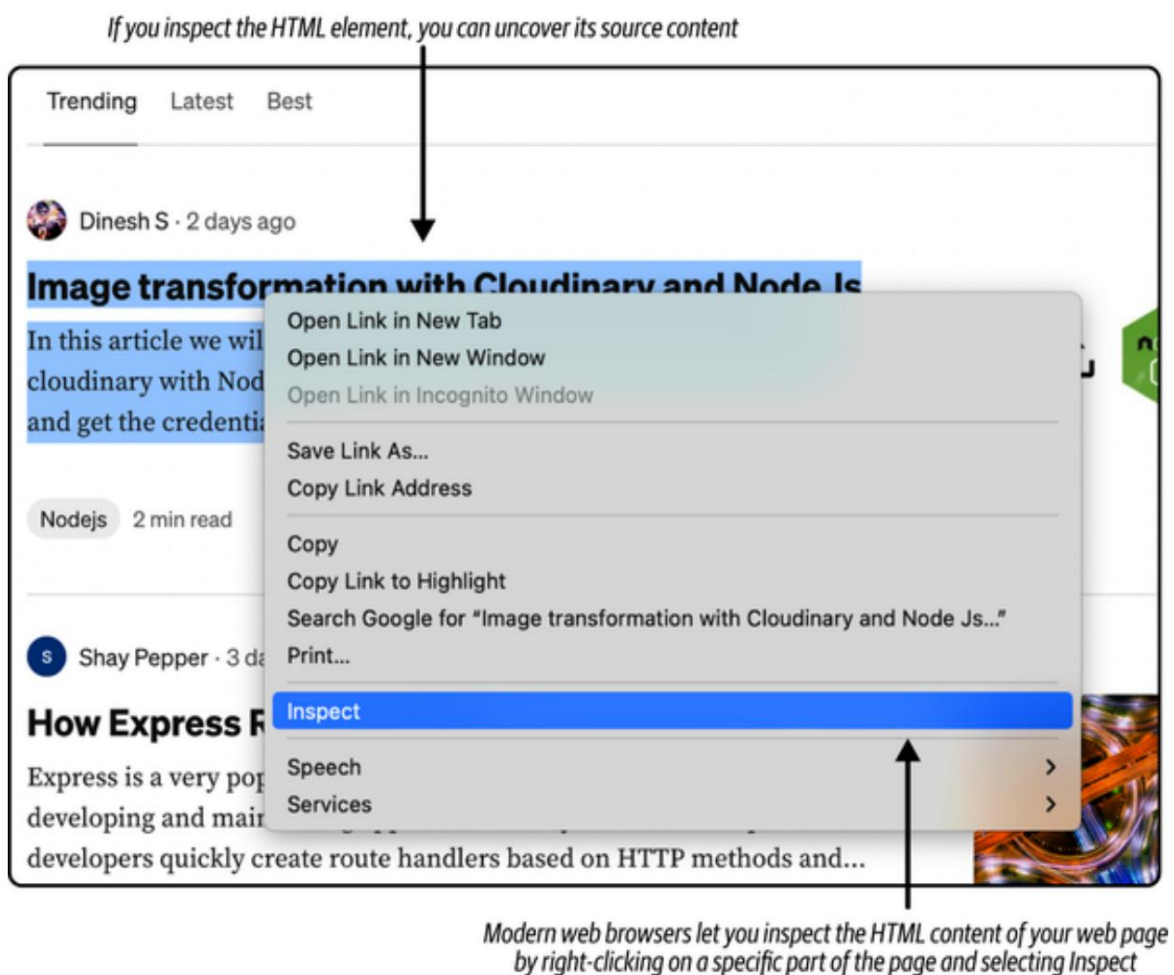
Después de confirmar que puede recuperar el contenido HTML del URL de origen, es hora de explorar la página web e identificar elementos que vale la pena raspar. En la [Figura 9-3](#) encontrará un ejemplo de lo que La página del navegador se parece a la de los artículos de Node en tendencia en Medium. Visualmente, es fácil separar el título en la parte superior de la página, Nodejs, de la lista de secciones del artículo a continuación. Estos serán indicadores que... puede utilizar para orientar sus elementos HTML relacionados.



Cifra 93. Ejemplo de desglose de página HTML

Para explorar los elementos HTML de uno de estos artículos enumerados, Puede hacer clic derecho en la sección del artículo y seleccionar Inspeccionar en la información sobre herramientas.

Menú. La Figura 9-4 muestra cómo se ve ese menú en el Navegador Chrome.

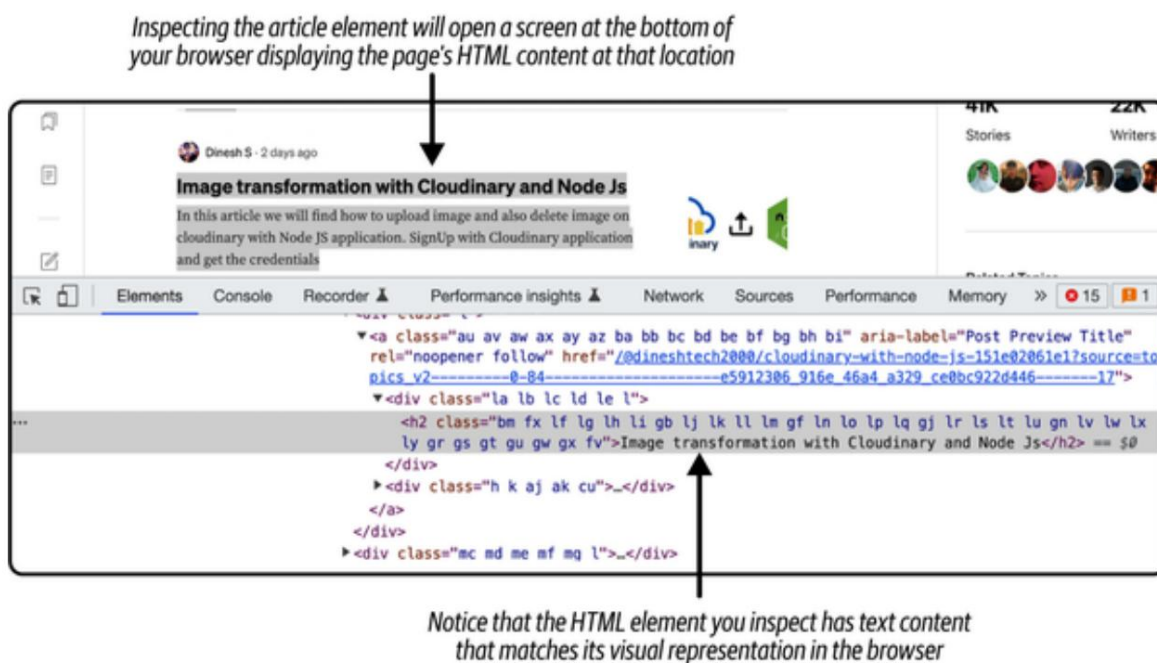


Cifra 94. Ejemplo de desglose de página HTML

Por último, verás una ventana en la parte inferior del navegador con información visible. HTML, como se ve en la Figura 9-5. Observe que el título del artículo aparece dentro de un Etiqueta `<h2>` . Esa etiqueta también aparece dentro de una etiqueta de anclaje, `<a>`, que es Se utiliza para enlazar al artículo. Estos son dos indicadores que puedes usar. en el lado del servidor al raspar esta página.

Solo queda un problema: analizar el HTML resultante de su llamada de búsqueda . A pesar de la estructura de árbol que ofrece HTML, solo te queda una versión de texto sin formato en la respuesta de tu llamada. Puedes Utilice una expresión regular elegante para seleccionar las etiquetas `<h2>` y `<a>` .

Sin embargo, existe una solución más sencilla utilizando los paquetes npm, explorados en las siguientes secciones.



Cifra 95. Ejemplo de desglose de página HTML

Análisis con herramientas compatibles con HTML

Has utilizado la API de búsqueda para recuperar contenido HTML de un sitio web. página y registrarla como texto sin formato. Puedes usar una cadena de JavaScript básica. funciones para inspeccionar ese texto y dar sentido a lo subyacente HTML. Por ejemplo, agregar `text.split("><")` a su código en `index.js` separará efectivamente los elementos HTML de su página en una matriz. Sin embargo, este proceso es rudimentario e ineficiente, lo que le deja Sólo que con más obstáculos en tu viaje de análisis.

Otra opción es explorar los paquetes npm que funcionan con HTML como Texto sin formato. Un paquete popular es Cheerio, que se puede instalar mediante ejecutando `npm install cheerio@^1.1.2` en el nivel raíz de su directorio del proyecto en la línea de comandos. El paquete cheerio es un Implementación del lado del servidor de la biblioteca jQuery. Mientras que jQuery La biblioteca se ha utilizado durante años para simplificar las interacciones de JavaScript.

Con el DOM HTML, Cheerio te ayuda a analizar HTML como texto plano y a identificar elementos específicos de la misma manera. Para obtener más información sobre el paquete Cheerio y su API, visita el sitio web [de Cheerio](#) .

el paquete. `js` , añade `import { load } de "cheerio"` a tu archivo `index start` usando `Luego`, añade el código del [Ejemplo 9-2](#), que usa la función `load()` para convertir tu contenido HTML sin formato en texto a un objeto cheerio . Este objeto se asigna a `$`, un nombre de variable usado en jQuery para referirse a todo el DOM HTML.

Ahora puede usar la variable `$` para localizar elementos específicos dentro de la página. Como se ve en [el Ejemplo 9-2](#), los títulos de los artículos se encuentran en las etiquetas `<h2>` . También puede ocurrir que estos títulos estén anidados dentro de las etiquetas `<article>` . Usar estos dos elementos juntos en `$("article h2")` permite identificar todos los títulos de los artículos de la página. Luego, se recorre cada elemento resultante para registrar el texto del título usando `$(element).text()`.

Ejemplo 92. Uso de cheerio para analizar contenido HTML en `index.js`

```
...
const $ = load(texto); const ❶
elementos = $("artículo h2"); elementos.each((i, ❷
elemento) => {                               ❸
  consola.log($(elemento).texto()); });      ❹
...
```

- ❶ Convierte tu texto simple HTML en un objeto cheerio .
- ❷ Orientar a todas las etiquetas `<h2>` dentro de una etiqueta `<article>` .
- ❸ Itera sobre cada elemento encontrado utilizando el método `.each()` de Cheerio .
- ❹ Registra el texto interno de las etiquetas `<h2>` en tu consola.

Ejecuta tu aplicación para ver una lista de los 10 títulos de artículos más populares en tu consola. En este ejemplo, se ejecuta la misma llamada de búsqueda .

Solo que esta vez, estás analizando el resultado para extraer solo los títulos. Intenta modificar tu lógica seleccionando solo la etiqueta `<article>` y luego usando la función `find()` para seleccionar las etiquetas `<h2>` y `<a>` anidadas (Ejemplo 9-3). Observa que, además del texto del título, también estás recopilando el atributo `href` de la etiqueta `<a>`. Aquí es donde se encuentra la URL del artículo.

Ejemplo 93. Orientación de títulos de artículos y URL en `index.js`

```
...
const elementos = $("artículo");           ❶
elementos.each((i, elemento) => { const    ❷
  título = $(elemento).find("h2").text();  ❸ const url = $
  (elemento).find("a").attr("href");      ❹ console.log(título, url); });
                                           ❺
```

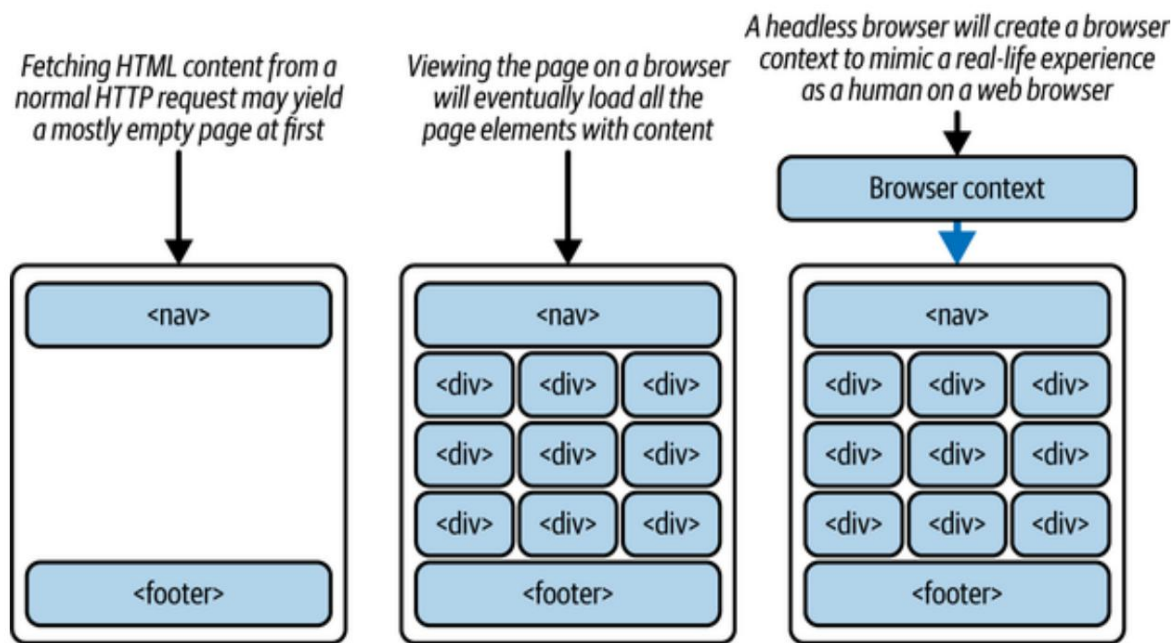
- ...
- ❶ Apunta a todos los elementos `<article>`.
 - ❷ Itere sobre cada elemento encontrado utilizando el método `.each()` de Cheerio.
 - ❸ Busque el título del artículo y asígnelo a la variable de título.
 - ❹ Busque la URL del artículo y asígnela a la variable URL.
 - ❺ Registra el título y la URL de cada artículo en tu consola.

Al ejecutar la aplicación, verás registrados el título y el enlace del artículo de los 10 artículos más populares. Sin embargo, las URL mostradas no están completas. Aunque este resultado te permita entender cómo construir la URL, no siempre puedes depender del código HTML de la página para tener una URL completamente generada y utilizable. Además, es posible que las aplicaciones web modernas y las aplicaciones de página única (SPA) ni siquiera generen la mayor parte de su HTML a menos que se cargue en un navegador. Por ello, necesitará un paquete adicional, que aprenderá en la siguiente sección.

Extracción de páginas web con un navegador headless. Hasta ahora, has podido extraer y analizar el HTML de una página web. Este proceso es suficiente para la mayoría de los sitios web, aunque la mayoría de las aplicaciones modernas y sitios web progresivos están adoptando un estilo diferente de carga de contenido.

Con la llegada de componentes web y bibliotecas frontend como React, Vue y Angular, no se garantiza que la primera impresión de una página web cargada contenga todo su contenido. Esto significa que una llamada de búsqueda puede generar una respuesta bastante vacía, en comparación con la que se vería en un navegador web. Esta es, en parte, la razón por la que se crearon los navegadores headless .

Un navegador headless es un navegador web sin componente visual. Se crea una instancia del navegador y puede cargar una página web, sus cookies y la ubicación de los documentos. Incluso puede mantener el estado y el historial del navegador. Lo mejor es que puede ejecutarse en el servidor. **La Figura 9-6** muestra la diferencia entre el contenido obtenido mediante una solicitud HTTP tradicional y un navegador headless. Para páginas web que cargan su contenido de forma diferida o tienen alguna restricción inmediata para mostrar contenido visual, una llamada de búsqueda puede devolver solo una capa de la página HTML, mientras que un navegador headless recupera contenido que refleja la experiencia humana real sin la verificación manual de una persona real.



Cifra 96. Obtener el código fuente de una página HTML en comparación con usar un sistema sin interfaz gráfica navegador

En su proyecto, utilizará la biblioteca Node de puppeteer para crear un Instancia del navegador y recuperación de títulos y URL de artículos. Instalar npm paquete ejecutando `npm install puppeteer@^24.15.0` en su nivel raíz del proyecto en la línea de comandos.

A partir de la versión 22, Puppeteer ya no descarga el El navegador Chromium se instala automáticamente durante la instalación. Para instalarlo Para corregir la versión del navegador manualmente, agregue el siguiente script a su

·Archivo json : "install-browser": "puppeteer"

"packagebrowsers install chrome". Una vez agregado, ejecute el script

Ejecutar `npm run install-browser` en la línea de comandos. Esto descargará la versión correcta de Chromium que Puppeteer usa para Ejecute su navegador sin cabeza.

TRABAJANDO CON TITRITERISTA

Cuando un sitio web es la única fuente para visualizar y acceder a ciertos conjuntos de datos, puede resultar complicado confiar en ellos para tu propia aplicación. Aunque cada vez es más común ver APIs dedicadas disponibles para ingenieros de software, la mayor parte del contenido de internet todavía solo es accesible a través de tu navegador web. Esta limitación es especialmente difícil para crear y probar tus propias aplicaciones web. ¿Cómo puedes probar el resultado visual de tu sitio sin una herramienta que pueda "ver" el resultado generado?

Afortunadamente, este problema es una preocupación compartida por toda la comunidad tecnológica, y se están diseñando nuevas soluciones para solucionarlo. Una de estas herramientas es Puppeteer, una biblioteca de Node que crea un contexto de navegador usando Chromium (derivado del navegador web Chrome). Desde el servidor de Node, el paquete npm de puppeteer utiliza las herramientas de desarrollo del navegador para recrear la experiencia de navegación de una página web. La biblioteca carga una nueva instancia del navegador, abre virtualmente una nueva ventana y visita la URL que elijas. En efecto, hay cuatro pasos principales:

1. `const navegador = await puppeteer.launch()`

 Inicia una instancia del navegador y asigna su valor a una variable

2. `const page = await navegador.newPage()`

 Crea una nueva página, desde la cual cargar una página web

3. `esperar página.goto(url)`

 Visita esa página web, como lo harías en un navegador normal.

4. `navegador.close()`

 Completa la tarea y apaga la instancia del navegador.

Antes de cerrar el navegador, puede usar la API de Puppeteer para evaluar y navegar por la página visitada. Por ejemplo, puede seleccionar elementos según sus nombres de clase o atributos ID, esperar a que se carguen con `page.waitForSelector` y hacer clic en ellos con `page.click`, como lo haría en una representación gráfica.

Puppeteer también te permite tomar capturas de pantalla y guardar las imágenes resultantes en tu sistema de archivos.

La flexibilidad de esta herramienta para automatizar tareas frontend la convierte en una de las bibliotecas líderes en pruebas integrales para aplicaciones web. Puede obtener más información sobre Puppeteer en su [sitio web](#).

Con puppeteer instalado, ahora tiene acceso a un navegador web completo en su servidor Node. Añada `"import puppeteer from 'puppeteer'"` al principio de su archivo `index.js` . Luego, reemplace el código con el del [Ejemplo 9-4](#). En este bloque de código, inicializa un navegador puppeteer y crea una nueva página.

Estos objetos se utilizan para completar una solicitud web iniciada por el navegador. La solicitud se realiza en `page.goto(URL)`, donde se solicita ver el contenido completo de la URL solicitada. Se añade una cadena de agente de usuario para garantizar que la respuesta coincida con la que recibiría un navegador típico. Un breve retraso da tiempo para que aparezca el contenido renderizado en JavaScript. La sintaxis `page.$$` se utiliza para ejecutar `document.querySelectorAll`, que recopila todos los elementos `<article>` del DOM.

Se itera sobre cada el que contiene el contenido de un elemento `<article>` . La función `$eval` se utiliza en la selección para extraer el contenido textual del elemento `<h2>` y el atributo `href` del elemento `<a>` .

Dado que el navegador ha renderizado la página por completo, el `href` contiene un enlace completo y utilizable. Tras cada iteración, los valores de título y URL extraídos se registran en la consola.

Ejemplo 94. Uso de puppeteer para extraer contenido HTML en index.js

```
const URL = "https://medium.com/tag/nodejs";
```

```
const navegador = await puppeteer.launch(); const página = await navegador.newPage();
```

```
espere página.setUserAgent( "Mozilla/5.0 (Macintosh; Intel Mac OS X) AppleWebKit/537.36 "(KHTML, como Gecko) Chrome/118 Safari/537.36" );
```

```
esperar página.goto(URL, { esperarHasta: "networkidle2" });
```

```
(aguardar página.waitForTimeout?.(3000)) ?? (aguardar nueva Promesa((r => setTimeout(r, 3000)));
```

```
const articulos = await página.$$("artículo");
```

```
para (const el de articulos) { const título = await el.$eval("h2", (el) => el.textContent.trim()) .catch(() => null);
```

```
constante url = esperar el.$eval("a", (el) => el.href) .catch(() => null);
```

```
si (título y url) { console.log(título, url); } }
```

```
await browser.close();
```

1 nueva instancia del navegador Puppeteer.

2 Abra una nueva página (pestaña) en el navegador.

3 Establezca un agente de usuario realista para evitar respuestas bloqueadas o eliminadas.

4 Cargue la URL de la etiqueta Medium y espere hasta que la red esté inactiva.

- 5 Espere brevemente para permitir que aparezca el contenido renderizado con JavaScript.
- 6 Seleccionar todos los elementos `<article>` de la página.
- 7 Recorra cada elemento del artículo para extraer el título y la URL.
- 8 Registra el resultado en la consola solo si se encuentran ambos valores.

CONSEJO

puppeteer se ejecuta por defecto como un navegador headless. Esto es útil para ejecutar pruebas y lógica masiva automáticamente. Dado que un navegador web completo se carga en segundo plano, puede optar por que se ejecute gráficamente. Para ello, añada `{headless: false}` como argumento en la función `puppeteer.launch`.

Al ejecutar tu aplicación esta vez, notarás que tarda un poco más que un comando de búsqueda estándar. Esto se debe a que se está iniciando un navegador Chromium completo y cargando la página web deseada. Verás aparecer en tu consola los mismos 10 títulos de artículos y enlaces más populares. Los enlaces funcionarán correctamente, incluyendo el dominio medio .con

Con este scraper instalado, ya has dado tus primeros pasos en la extracción y el uso de datos estructurados de sitios web complejos basados en JavaScript. Ya sea que estés creando herramientas para la agregación de contenido, probando diseños visuales o simplemente explorando cómo los sitios web modernos presentan contenido, Puppeteer te ofrece la flexibilidad de un navegador completo, directamente en tu aplicación Node. En el siguiente capítulo, ampliarás esta base convirtiendo tu scraper en un servicio de API reutilizable que puede impulsar interfaces frontend o automatizaciones posteriores.

EJERCICIOS DEL CAPÍTULO

1. Automatizar el filtrado de palabras clave de los artículos:

Escriba una función que acepte una palabra clave (p. ej., "rendimiento") como entrada y filtre los títulos de artículos que no la contengan. Después de extraer la lista de títulos y URL de los artículos, aplique esta función para mostrar solo los artículos que coincidan en su consola. Esto anima a los usuarios a adaptar sus resultados a sus objetivos de aprendizaje o preferencias temáticas.

CONSEJO

Utilice `.includes()` en la cadena de título para la coincidencia básica de palabras clave. Asegúrese de normalizar a minúsculas para realizar una comparación sin distinguir entre mayúsculas y minúsculas.

2. Escribe los resultados extraídos en una API JSON:

En lugar de registrar los títulos y URL de los artículos extraídos en la consola, crea una ruta como `/api/articles` que devuelva los resultados extraídos como un objeto JSON. Esto permitirá que tu extraídor funcione como un servicio de backend para otras aplicaciones o como una interfaz de usuario frontend.

CONSEJO

Almacene los resultados en una matriz local y devuelva esa matriz como el cuerpo de respuesta de su nueva ruta.

Resumen

En este capítulo aprendiste a:

- Acceder al contenido de una página HTML desde un servidor Node
- Elementos de datos de destino de un sitio web
- Recopilar y procesar datos desde un navegador sin cabeza

Capítulo 10. Autenticación de aplicaciones

Este capítulo cubre lo siguiente:

- Diseño de la lógica de autenticación de inicio de sesión
- Uso de la biblioteca Passport.js para autenticar usuarios y administrar estrategias basadas en sesiones o tokens
- Uso de tokens web JSON para autenticarse en las API

En este capítulo, creará la lógica de autenticación para una aplicación Node. Independientemente del tipo de aplicación principal que decida crear, la autenticación de usuarios sigue siendo un componente vital para proteger los datos de su aplicación. Con la expansión de la accesibilidad y la disponibilidad de internet, las aplicaciones también se han vuelto más vulnerables a los ataques.

Las aplicaciones han avanzado mucho desde la verificación de identidad mediante una dirección de correo electrónico y una contraseña de texto plano. La mayoría ha implementado un cifrado básico o una función hash para guardar solo versiones de texto desordenado de las contraseñas. Otras han llevado la seguridad a un nuevo nivel con la autenticación multifactor (MFA), que garantiza que un usuario solo pueda iniciar sesión con su contraseña si también verifica su cuenta con un código adicional enviado a su teléfono o correo electrónico.

Cada año, la comunidad tecnológica se enfrenta a nuevos problemas de seguridad y autenticación de usuarios, y muchas empresas invierten en equipos dedicados a resolverlos. Afortunadamente, la mayoría de las empresas comparten el interés de proteger los datos de las cuentas de sus clientes, lo que ha dado lugar a estándares del sector para la creación de nuevas cuentas y el procesamiento de solicitudes entrantes. Estas prácticas recomendadas se extienden más allá de la página web estándar, a clientes móviles e interfaces de programación de aplicaciones (API), que pueden no tener un formulario de inicio de sesión. En las siguientes secciones, creará un sistema de autenticación y lo iterará sobre mejoras.

versiones para poner en práctica algunas de estas estrategias de autenticación comunes.

HERRAMIENTAS Y APLICACIONES UTILIZADAS EN ESTE CAPÍTULO

Antes de comenzar, asegúrese de instalar y configurar las herramientas y aplicaciones necesarias para este proyecto. Las instrucciones de instalación de Node.js, Fastify y VS Code se encuentran en [el Capítulo 1](#), mientras que los pasos de inicialización del proyecto, como la configuración de la estructura de directorios, la configuración de package.json y el uso de sintaxis moderna, se describen en [el Apéndice A](#). Las instrucciones para instalar y usar Postman se encuentran en [el Apéndice B](#). Para una explicación más detallada de los conceptos de SQLite, consulte [el Apéndice C](#). Una vez completado, vuelva aquí para continuar. Desarrollar un proyecto desde cero le ayuda a comprender mejor cada componente, lo que le brinda mayor control y flexibilidad a medida que avanza.

Tu mensaje: Has

creado una app de planificación de viajes exitosa, Journey Doc, y su base de usuarios crece constantemente. El único problema es que, si bien los usuarios pueden crear nuevos itinerarios, nunca se les pidió que crearan una cuenta con una contraseña segura. Esto representa un riesgo de seguridad y un inconveniente, ya que sus itinerarios guardados son de acceso público, lo que genera dudas sobre si pueden confiar en tu plataforma. Como proyecto de remediación y desarrollo, decides iterar en varias medidas de autenticación, requiriendo que los usuarios inicien sesión y autenticuen su cuenta antes de crear nuevos planes de viaje o acceder a los datos de viaje existentes.

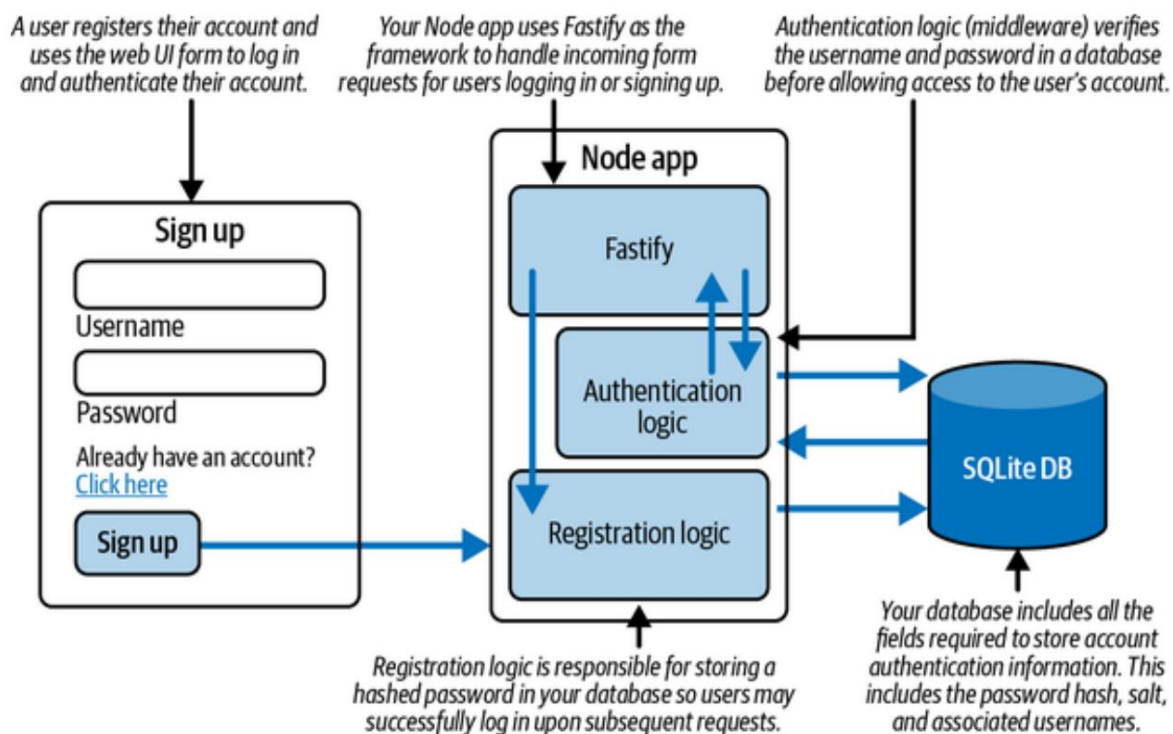
Obtener planificación

Aunque gran parte de tu aplicación ya está desarrollada y funcionando, decides abordar el reto de la autenticación desde cero. Al finalizar este proyecto, tendrás una aplicación y una API dedicadas para crear cuentas de usuario con nombre de usuario y contraseña cifrada.

El objetivo del proyecto es usar las herramientas disponibles para diseñar la lógica más eficiente y segura para gestionar numerosas solicitudes de cuentas nuevas y frecuentes solicitudes de verificación de contraseña. Por ello, se usará Node y Fastify para crear un servidor de aplicaciones que gestione las solicitudes HTTP y una base de datos SQLite para almacenar la información relevante de las cuentas.

La Figura 10-1 detalla el flujo de información de la interacción del usuario en un navegador web. Primero, el usuario se registra con la información de su cuenta, registrándola y almacenando su nombre de usuario y contraseña en la base de datos.

Tu lógica de registro gestiona el proceso de proteger las contraseñas de las cuentas, de modo que solo se almacena la información necesaria. Posteriormente, cuando los usuarios inicien sesión en tu página web, se les mostrará una pantalla de inicio de sesión similar. Al enviar el formulario de inicio de sesión, la solicitud del usuario es procesada por un controlador de rutas de Fastify que invoca la lógica de autenticación. Esta lógica compara la combinación de nombre de usuario y contraseña entrantes para determinar si se puede acceder a la cuenta.



Cifra 101. Plano de proyecto para una aplicación Node de autenticación de inicio de sesión

Para comenzar, creará un servidor con los puntos finales de API mínimos necesarios para crear y verificar nuevas cuentas de usuario, así como una página de inicio de sesión única como referencia visual para la actividad de inicio de sesión de los usuarios.

Conectará una base de datos para almacenar la información de nombre de usuario y contraseña, y usará las contraseñas de esa base de datos para verificar si los intentos de inicio de sesión de los usuarios deben autenticarse. Trabjará con la biblioteca Passport.js, que contiene muchos de los protocolos de autenticación estándar utilizados en las principales aplicaciones web, empaquetados en un conjunto de clases y funciones fáciles de usar.

Una vez que tenga un proceso de autenticación que funcione, deberá abordar la protección de las API que no necesariamente tengan un cliente de navegador web.

Estas API pueden ser utilizadas por una amplia gama de clientes, desde aplicaciones web progresivas hasta dispositivos móviles. Una estrategia común consiste en generar tokens de autenticación que se transfieren entre el cliente y el servidor para verificar la identidad de los usuarios.

Una vez que el diseño de su aplicación esté esbozado, pasará a desarrollar la lógica del marco del servidor y los puntos finales de API en la siguiente sección.

Obtenga programación

Para comenzar a desarrollar su aplicación, cree una nueva carpeta llamada `app_authentication`. Navegue a la carpeta de su proyecto en su Línea de comandos y ejecute `npm init`. Este comando inicializa su Aplicación de nodo. Puede presionar Enter durante los pasos de inicialización para Acepte los valores predeterminados. El resultado de estos pasos es la creación de un archivo. llamadas `package.json`. Este archivo le indicará a su aplicación Node cualquier configuraciones de paquetes o scripts necesarios para ejecutarse correctamente.

A continuación, crea un archivo llamado `index.js` dentro de la carpeta de tu proyecto. Este archivo es El punto de entrada para su aplicación.

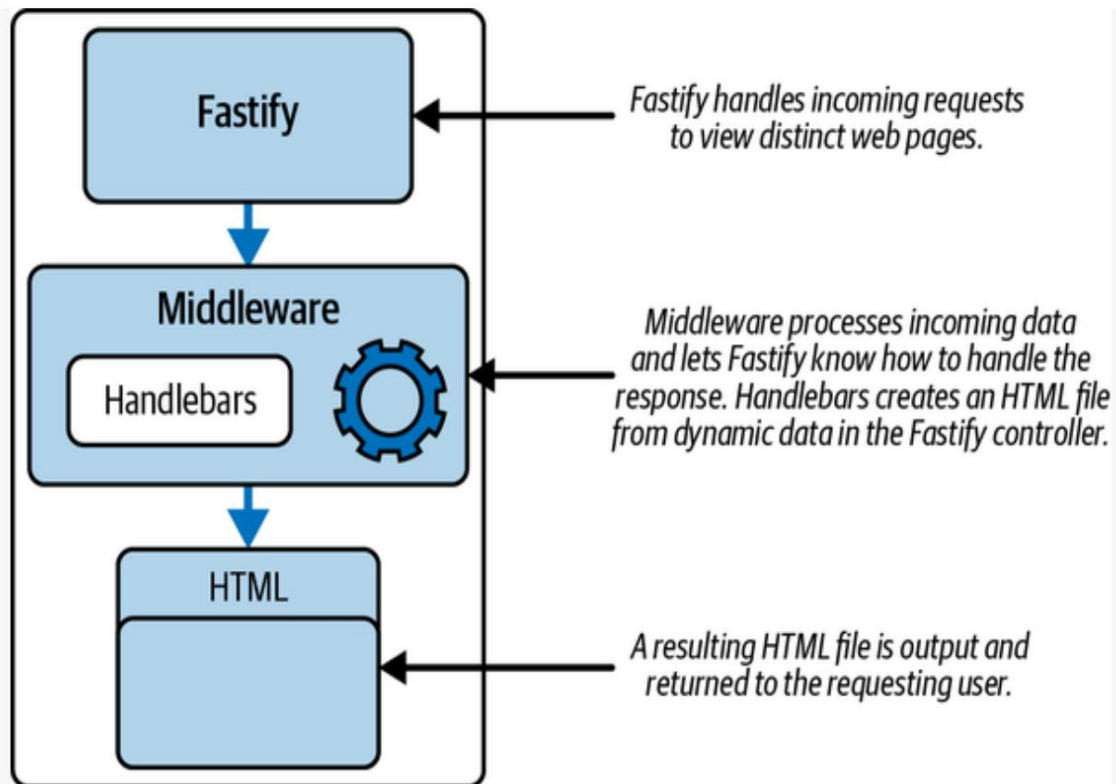
Con su marco de `Express` Archivo js listo, puedes instalar el servidor principal índice , motor de plantillas y complementos de soporte utilizados en este Proyecto. Ejecute `npm install fastify@^5.0.0 @fastify/formbody@^8.0.0 @fastify/view@^10.0.0 handlebars@^4.7.8` para instalar Fastify, la plantilla Handlebars motor y los complementos Fastify necesarios para manejar datos de formularios y Renderizado del lado del servidor. Estos paquetes permiten que su aplicación analice Envíos de formularios entrantes y renderización dinámica de plantillas HTML basado en datos específicos del usuario.

CONSTRUCCIÓN DE PLANTILLAS CON MANILLARES

Además del enrutamiento y la estructura de la API, Fastify ofrece renderizado del lado del servidor (SSR) a través del complemento `@fastify/view`. SSR consiste en crear una página HTML a partir de datos dinámicos en el servidor y entregar ese archivo a un usuario a través de internet. Un lenguaje de plantillas permite que las variables actúen como marcadores de posición dentro de una plantilla HTML y posteriormente se compilen en un archivo HTML estático. Este proceso permite que las aplicaciones muestren páginas web con información específica para cada usuario. Por ejemplo, cada usuario debería recibir la misma "página de bienvenida", pero solo con su nombre de usuario insertado: "¡Bienvenido, Jon!".

La Figura 10-2 muestra cómo Fastify pasa los datos de la solicitud a un controlador de ruta que renderiza la plantilla Handlebars. Cuando Fastify está listo para devolver una página web al usuario, pasa datos, como su nombre o correo electrónico, al archivo Handlebars, que compila el archivo en una página HTML estática.

El archivo HTML resultante se entrega desde el servidor backend. Esto significa que el usuario recibirá una respuesta con una página HTML completa. Sin embargo, cada usuario puede tener la seguridad de que su información personal solo aparece para él, ya que la información de su cuenta se autentica y se procesa en una plantilla de Handlebars.



Cifra 102. Creación de un archivo HTML con plantillas Handlebars

Handlebars.js es una biblioteca de JavaScript popular que proporciona una forma sencilla y flexible de generar plantillas HTML y crear aplicaciones web dinámicas e interactivas. La biblioteca se utiliza a menudo en combinación con un marco web, como Fastify.js, para construir aplicaciones web renderizadas por servidor.

La biblioteca Handlebars contiene algunas expresiones sintácticas para usarlas en sus plantillas. Para mostrar una variable dentro de un div HTML, simplemente envuelve la variable entre llaves dobles: `<div> {{nameVar}} </div>`. Además, puedes envolver un bloque de HTML dentro de una condición. Esa condición puede estar representada entre llaves dobles y la adición de un símbolo de almohadilla: `{{#if boolStatement}}`. De esta manera, Handlebars permite su archivo de plantilla para tomar decisiones dinámicas sobre qué información mostrar antes de renderizar el resultado HTML final.

Para obtener más información sobre la biblioteca Handlebars, visite su [sitio web](#). Para obtener más información sobre los lenguajes de plantilla compatibles con Fastify, consulte la [documentación de Fastify](#).

Comienza a desarrollar la lógica principal de tu aplicación importando Fastify a tu proyecto y configurando la aplicación para que reciba las solicitudes entrantes correctamente ([Ejemplo 10-1](#)). Después de importar Fastify en la parte superior de `index.js`, instancia el objeto de la aplicación desde el módulo Fastify . Define una constante llamada `PORT` y le asigna el valor 3000, que se utilizará para iniciar tu servidor localmente.

NOTA

Este objeto de aplicación actúa como su servidor principal. Es responsable de iniciar su servidor, interpretar las solicitudes entrantes y gestionar todas las operaciones relacionadas con la autenticación.

En Fastify, las rutas se definen directamente en la instancia de la aplicación mediante métodos como `app.get(...)` y `app.post(...)`. Para gestionar los datos entrantes del formulario, se registra el complemento `@fastify/formbody` , que analiza los datos de `application/x-www-form-urlencoded` .

Fastify también maneja automáticamente el análisis del cuerpo de la aplicación/`json` para solicitudes con formato JSON.

CONSEJO

Fastify gestiona el procesamiento de solicitudes mediante su arquitectura de plugins y ganchos de ciclo de vida de ruta. Estos ganchos permiten ejecutar código en diversas etapas, como antes de la validación, antes de la ejecución del controlador o después del envío de la respuesta, lo que facilita la ampliación de funcionalidades y la aplicación de una lógica coherente en todas las rutas.

Para garantizar que Fastify pueda localizar sus archivos de plantilla Handlebars, configurar la opción `root` al registrar `@fastify/view` complemento. Esto apunta a la carpeta que contiene sus archivos `hbs`, normalmente vistas nombradas

Ejemplo 101. Configurando su aplicación en `index.js`

```
de importación Fastify desde "fastify"; ❶
importar fastifyFormbody desde "@fastify/formbody"; ❷
importar fastifyView desde "@fastify/view"; ❸
importar manillares desde "manillar"; ❹

constante aplicación = Fastify(); ❺
constante PUERTO = 3000; ❻

esperar aplicación.register(fastifyFormbody); ❼
esperar aplicación.register(fastifyView, { ❽
  motor: { manillar },
  raíz: "vistas",
});
```

- ❶ Importa el módulo `fastify` para crear tu servidor web.
- ❷ Importe el complemento `@fastify/formbody` para admitir la codificación URL de datos del formulario.
- ❸ Importe el complemento `@fastify/view` para admitir motores de plantillas.
- ❹ Importe los manillares como implementación de su motor de plantillas.
- ❺ Crea una instancia de la aplicación desde `Fastify`.
- ❻ Define el puerto que utilizarás para llegar a tu servidor web como `3000`.
- ❼ Registre el complemento `formbody` para analizar las cargas útiles de `application/x-www-form-urlencoded`.
- ❽ Registre el complemento de vista y configure `Handlebars` como su motor de renderizado.

Con su aplicación configurada para crear rutas y usar SSR, usted define
 Tu primera ruta para la ruta de inicio, /. Para comprobar que tu aplicación funciona,
 Simplemente responde con un mensaje de "¡Bienvenido!" usando
 reply.send. Por último, configura app.listen para que esté atento a las llamadas entrantes.
 solicitudes en el puerto 3000. Agregue el código del **Ejemplo 10-2** al final de
 su archivo index.js .

Ejemplo 102. Agregar una ruta e iniciar el servidor en index.js

```
...
app.get("/", async (solicitud, respuesta) => {           ❶
  responder.enviar("¡Bienvenido!"); ❷
});

intentar {
  constante dirección = await app.listen({ puerto: PUERTO, host:
"127.0.0.1" }); ❸
  console.log(`Aplicación escuchando en ${address}`);
} atrapar (err) {
  consola.error(err);
  proceso.salir(1);
}
```

- ❶ Define una ruta para recibir solicitudes GET a tu ruta predeterminada usando Método app.get de Fastify .
- ❷ Responda con un valor de texto sin formato utilizando reply.send.
- ❸ Inicie el servidor usando await app.listen(...) y registre el Dirección cuando esté lista.

Tu aplicación está lista para ejecutarse y aceptar solicitudes. Navega a tu carpeta del proyecto en la línea de comandos e inicia la aplicación. En tu sitio web Navegador, navega a **httplocalhost. :3000** , donde verás el
 Aparecerá el mensaje ¡Bienvenido!

NOTA

Si no ve aparecer ningún mensaje, primero asegúrese de que su servidor esté funcionando correctamente. A continuación, asegúrese de haber añadido la ruta necesaria. y que todos los archivos han sido guardados.

Es genial ver que tu aplicación funciona, pero los mensajes de texto sin formato no ofrecen mucho para apoyar la autenticación de la aplicación. En la siguiente sección, creará una

Página de interfaz de usuario para brindar información a sus usuarios.

Creación de un formulario de inicio de sesión

Para crear una interfaz de usuario para su aplicación, configure una carpeta llamada vistas en la nivel raíz de su proyecto y agregue un archivo llamado indexhbs. El archivo hbs La extensión significa que es un archivo Handlebars que contiene tanto HTML como Sintaxis de manillares.

NOTA

Fastify espera que los archivos de plantilla se encuentren en una carpeta llamada vistas. De esta manera, El marco le ayuda a organizar sus archivos y facilita su localización. archivos relevantes que requieren compilación en HTML.

Dentro del índice Archivo hbs , agregue el contenido HTML repetitivo en **Ejemplo 10-3**. Este contenido importa la biblioteca CSS de Bootstrap para Resultados de estilo inmediatos y crea el HTML estándar <head> etiqueta, así como estilos personalizados.

Ejemplo 103. Añadiendo la estructura fundacional en index.hbs

```
<html lang="es">
  <cabeza>
    <meta charset="UTF-8" />
    <title>Inicio de sesión en Journey Doc</title>
    <enlace
```

```
rel="hoja de estilo"

href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.3/dist/css/ bootstrap.min.css"
crossorigin="anonymous"
referrerpolicy="no-referrer" />
<style> cuerpo { relleno: 50px; visualización:
❷
flex;
elementos de alineación: centrado; contenido justificado: centrado; altura:
100vh; } formulario { ancho: 500px;
margen: 0 automático; } </style> </
head> </html>
❸
```

❶ Agregue una etiqueta de título para la página de inicio de sesión que está representando.

❷ Agregue una etiqueta de enlace para cargar CSS de Bootstrap.

❸ Define estilos personalizados para tu página de inicio de sesión.

Con esta base establecida, ahora puede agregar el cuerpo de la página con variables dinámicas. Agregue el cuerpo definido en el **Ejemplo 10-4** después de la etiqueta de cierre `</head>` . El contenido de este bloque de código crea un formulario para enviar información de autenticación. El método de la mayoría de los formularios es POST, ya que la información se envía al servidor en el cuerpo de la solicitud. La acción del formulario se refiere a la ruta de la aplicación a la que se dirige.

Esa ruta puede ser la de autenticación o la de creación de cuenta, dependiendo de si ya se ha registrado una cuenta. Por ello, la acción se vinculará dinámicamente al valor de la variable de ruta . Esta variable es una de las pocas cuyos valores se determinarán en el servidor.

El título del formulario se muestra mostrando la variable "title" entre llaves. A continuación, se generan dos entradas de formulario y sus etiquetas.

Una entrada es para el nombre de usuario del usuario y la otra es para su contraseña.

NOTA

HTML ofrece tipos de entrada específicos para cumplir con las expectativas del contenido de entrada. Por ejemplo, el tipo de entrada de correo electrónico espera un valor con el formato "abc@xyz.com". y el tipo de ingreso de contraseña reemplazará los caracteres de la contraseña con viñetas en la interfaz de usuario de manera predeterminada.

Al final del formulario, hay una sección para mostrar un mensaje del servidor, como un error de inicio de sesión o una confirmación, y un enlace que el usuario puede seguir para cambiar de página. Este enlace añade un parámetro de consulta (? page=...) a la URL de la página de inicio, lo que permite al servidor determinar el contenido que se mostrará.

El valor switchPage ayudará al servidor a determinar lo que el usuario desea ver. Por último, se añade la variable title como texto para el botón de envío. De esta forma, aparecerá el título "Iniciar sesión" tanto en la parte superior del formulario como en el botón de envío. Todas estas variables actúan como marcadores de posición para los valores que se determinan en el servidor. Antes de mostrar esta página como HTML, las variables se reemplazan con sus valores asociados.

Ejemplo 104. Agregar un formulario de autenticación en index.hbs

...

```
<cuero>
  <formulario método="publicación" acción="{{ruta}}"> ❶
    <h2>
      {{título}} </h2> ❷
    <div
      class="form-outline mb-4"> <input type="text" ❸

      name="nombre de
        usuario" id="nombre de
          usuariInput" class="form-
            control" /> <label class="form-

            label" for="nombre de usuariInput">
              Nombre de usuario
```

```

        </label> </
div> <div
class="form-outline mb-4"> <input type="contraseña" ❷

        name="contraseña"
        id="passwordInput"
        class="form-control" /> <label
        class="form-label"

        for="passwordInput">
        Contraseña
        </label> </
div> <div
class="row mb-4"> <div class="col">
    {{message}} <a href="/?
                                ❸
                                page={{switchPage}}"> Haga clic aquí</a> </div> </div> <button type="submit" ❹
                                class="btn
                                btn-
                                primary btn-block mb-4"> {{title}} </button> </form> </body> Cree un formulario HTML
con una
    acción dinámica ❺
    definida en la
    variable de
    ruta .
❶

```

❷ Mostrar la variable de título dentro de una etiqueta de encabezado.

❸ Crea un formulario de entrada para el nombre de usuario del usuario.

❹ Crea un formulario de entrada para la contraseña del usuario.

❺ Mostrar el contenido de la variable del mensaje .

❻ Agregue la variable switchPage como un parámetro de consulta en el valor href de la etiqueta de anclaje .

❼ Muestra la variable de título como texto en tu botón de envío.

La salida de este archivo HTML generado genera un formulario con el nombre de usuario y la contraseña. Dado que este formulario se utiliza tanto para el registro de la cuenta como para la autenticación del inicio de sesión, se añade una entrada opcional para la confirmación de la contraseña. De esta forma, al crear la cuenta, el usuario puede escribir la contraseña dos veces para garantizar que ambas coincidan al crearla y guardarla inicialmente.

CONSEJO

Muchos pasos de autenticación pueden ejecutarse en el cliente web antes de llegar al servidor. HTML proporciona herramientas útiles para garantizar que el texto tenga el formato correcto. JavaScript del lado del cliente se utiliza a menudo para verificar que la contraseña y otras entradas de formulario se escriban correctamente, evitando así que el servidor tenga que procesar una solicitud mal formada.

Agregue el código del **Ejemplo 10-5** para añadir un campo de entrada condicional debajo del campo de contraseña. La instrucción `{{#if showExtraFields}}` utiliza la sintaxis condicional de Handlebars para comprobar si la variable `showExtraFields` es verdadera y se debe mostrar un campo de contraseña adicional.

Ejemplo 105. Agregar una entrada de contraseña condicional en `index.hbs`

...

```

{{#if showExtraFields}} <div
    class="form-outline mb-4"> <input type="contraseña"

    name="confirmarContraseña"
    id="entradaConfirmarContraseña"

    class="control-form" /> <label
    class="etiqueta-formulario"

    for="entradaConfirmarContraseña">
        Confirmar contraseña
    </label> </
div>
  
```

`{{/si}}``...`**1**

Agregue un campo para mostrar condicionalmente el campo “Confirmar contraseña”.

Ahora puede definir las variables requeridas antes de su primera ruta definida en `index.js`, como se muestra en el **Ejemplo 10-6**. Las variables residen en el

El objeto `loginFormVars` contiene valores diferentes para las páginas de inicio de sesión y registro. Observe la diferencia en los títulos y mensajes que se muestran al usuario. Observe también que la ruta de registro envía los envíos de formularios a `/account`, mientras que el valor de la ruta de inicio de sesión es `/auth`. Estas dos rutas deben crearse para gestionar las solicitudes de formulario.

Ejemplo 106. Definición de las variables de la página de autenticación en `index.js`

`...`

```
const loginFormVars = { signup:
```

```
  { title:
```

1

```
    "Regístrate", message:
```

```
    "¿Ya tienes una cuenta?", route: "/account",
```

```
    switchPage: "login",
```

```
    showExtraFields: true, },
```

```
    login: { title: "Iniciar sesión",
```

```
  message:
```

2

```
    "¿Necesitas crear
```

```
    una cuenta?", route: "/auth", switchPage: "signup",
```

```
    showExtraFields:
```

```
    false, }, };
```

`...`**1**

La clave de registro se asigna a todas las variables de página necesarias para mostrarse en la página de registro.

2

La clave de inicio de sesión se asigna a todas las variables de página necesarias para mostrarse en la página de inicio de sesión.

Para representar la ruta .Página hbs con estas variables, actualice el valor predeterminado index/ con el contenido del **Ejemplo 10-7**, comience por comprobar Si se pasó un parámetro de consulta en la URL. Verifica el El campo de consulta de la solicitud y la desestructuración para extraer el valor de la página . Esto El valor (switchPage) solo se pasará al servidor cuando alternar entre las páginas de registro e inicio de sesión. A continuación, define tu formVars para que coincida con el conjunto de variables de página para la página que está visitando Mostrar o usar por defecto las variables de registro . Por último, se utiliza res.render para pasar estas variables a la página indexhbs y . compilarlo en un archivo HTML para enviarlo de vuelta al navegador web del usuario.

Ejemplo 107. Representar la página index.hbs desde index.js

```
...
const { página } = solicitud.consulta;    ❶
const formVars = loginFormVars[página] ||
loginFormVars.signup;                    ❷
devolver respuesta.view("índice", formVars);  ❸
...
```

- ❶ Desestructura la clave de la página a partir de los parámetros de consulta de tu solicitud.
- ❷ Define tus formVars como los valores que coinciden con tu consulta parámetro para la página, o valor predeterminado para los valores de la página de registro .
- ❸ Representa la página indexhbs con los valores dinámicos seleccionados.

Con estos cambios implementados, puede volver a ejecutar su aplicación y navegar a <http://host.local:3000> donde verá una página similar a la **Figura 10-3**.

Este formulario muestra los valores de las variables para el título, el mensaje y Botón de envío. Al enviarlo, el formulario también publicará datos en el Ruta /cuenta . Esa ruta aún no se ha configurado, así que por ahora Verías un error en la pantalla.

Sign up

Username

Password

Confirm Password

Already have an account? [Click here](#)

Sign up

The sign-up form contains username, password, and password confirmation fields

Cifra 103. Visualizar el formulario de registro

El paso final para hacer uso de este formulario es definir la /cuenta y Rutas /auth . Estas rutas gestionarán el registro y la creación de cuentas. y autenticación de inicio de sesión, respectivamente. Porque vas a estar Para guardar nuevos usuarios y sus contraseñas, necesitará un lugar para almacenarlas Por ahora, los almacenas en un objeto en la memoria. Eso significa Guardará nuevos usuarios mientras se ejecuta su aplicación y borrará la lista de usuarios cada vez que cierras tu aplicación. Agrega `const users = {};` encima de sus rutas en el índicejs como un almacén en memoria para usuarios guardados. Luego agregue el código del **Ejemplo 10-8** para definir las rutas restantes.

Ambas rutas nuevas usan `app.post` y son asincrónicas porque aceptan información publicada y eventualmente ejecutará operaciones de E/S. Para el Ruta /cuenta , recopilará los campos de nombre de usuario y contraseña desde el cuerpo de la solicitud y guarde la cuenta de usuario agregando una clave mediante ese nombre de usuario en el objeto de usuarios y asignarlo a la contraseña valor.

NOTA

De inmediato, este enfoque tiene muchos fallos, ya que almacena una contraseña de texto simple que puede ser anulada por la misma acción de registro del nombre de usuario. Sin embargo, esta lógica es un paso intermedio hacia guardar tus contraseñas de una manera más práctica.

En la primera ruta donde se configura el nombre de usuario y la contraseña, el objeto "usuarios" representa la creación de una nueva cuenta. Después, se usa "reply.send" para enviar un mensaje de validación al usuario. La ruta "/auth" funciona de forma similar. Sin embargo, esta ruta no guarda una nueva cuenta de usuario, sino que comprueba si se encuentra una clave de nombre de usuario en el objeto "usuarios". Si se encuentra el nombre de usuario y la contraseña almacenada coincide con la contraseña del cuerpo de la solicitud, consideramos que la cuenta está autenticada y respondemos con un mensaje indicando que el inicio de sesión se ha realizado correctamente. De lo contrario, redirigimos al usuario a la página de inicio de sesión para que lo intente de nuevo.

Ejemplo 108. Define las rutas /account y /auth en index.js

```
...
const usuarios = {};

app.post("/cuenta", async (solicitud, respuesta) => { const ❶
  { nombre de usuario, contraseña } = solicitud.cuerpo; ❷
  usuarios[nombre de usuario] = ❸
  contraseña; devolver respuesta.enviar({ mensaje: "Cuenta ❹
creada." }); });

app.post("/auth", async (solicitud, respuesta) => { const ❺
  { nombre de usuario, contraseña } = solicitud.cuerpo; si ❻
  (usuarios[nombre de usuario] y usuarios[nombre de usuario] ===
  contraseña) { devolver responder.enviar({ mensaje: "Iniciado sesión." });
} devolver respuesta.redirect("/?page=login"); }); ❽
...

```

- ❶ Cree un controlador de ruta POST para /cuenta para aceptar nuevos datos de usuario.
- ❷ Extraiga el nombre de usuario y la contraseña del cuerpo de la solicitud entrante.
- ❸ Almacene la contraseña proporcionada en el objeto de usuarios utilizando el nombre de usuario como clave.
- ❹ Envíe una respuesta JSON confirmando la creación de la cuenta.
- ❺ Cree un controlador de ruta POST para /auth para autenticar los existentes usuarios.
- ❻ Verifique que el nombre de usuario exista y que la contraseña coincida con el valor almacenado.
- ❼ Responda con un mensaje de éxito si el inicio de sesión es válido.
- ❽ Redirigir al usuario a la página de inicio de sesión si falla la autenticación.

Ahora tu aplicación está lista para procesar los formularios de registro e inicio de sesión y aceptar los envíos. Inicia la aplicación e introduce tu nombre de usuario y contraseña en los campos del formulario de registro. Haz clic en "Registrarse". Accederás a una página con un mensaje indicando que la cuenta se ha creado correctamente. Intenta iniciar sesión.

Puede retroceder en el historial de su navegador e intentar hacer clic en el enlace "Haga clic aquí" para acceder a la página de inicio de sesión. Al hacer clic en el enlace, se añadirá el parámetro de consulta de la página a la solicitud de ruta, especificando las variables de la página de inicio de sesión . Estas variables se utilizan para volver a rellenar la página en el servidor y volver a mostrar el formulario, listo para enviar datos a la ruta /auth (Figura 10-4). Introduzca el mismo nombre de usuario y contraseña y haga clic en "Iniciar sesión".

Log in

Username

Password

Need to create an account? [Click here](#)

Log in

← The login form contains username and password fields

Cifra 104. Visualización del formulario de inicio de sesión

Si tu información de inicio de sesión es correcta, recibirás una respuesta JSON con el mensaje "Iniciado sesión". De lo contrario, serás redirigido a la página de inicio de sesión para volver a intentarlo.

Esta aplicación es un buen punto de partida, pero requiere un poco más de trabajo para proteger la privacidad y las identidades de inicio de sesión de los usuarios. En la siguiente sección, incorporarás la biblioteca Passport.js para facilitar este proceso.

Guardado y protección de cuentas de usuario. Has creado un sistema de autenticación desde cero. La mala noticia es que no es muy seguro y borra toda la información de tus usuarios al reiniciar la aplicación. Esto requiere una estrategia de seguridad de contraseñas y una base de datos para conservar tus cuentas de usuario. Afortunadamente, estos dos pasos se pueden integrar al definir un modelo de cuenta en tu aplicación.

Su modelo de cuenta define los campos necesarios para registrar una cuenta, autenticarla con Passport.js y guardar los valores de la cuenta de usuario en una base de datos SQLite a través de la biblioteca Sequelize.

NOTA

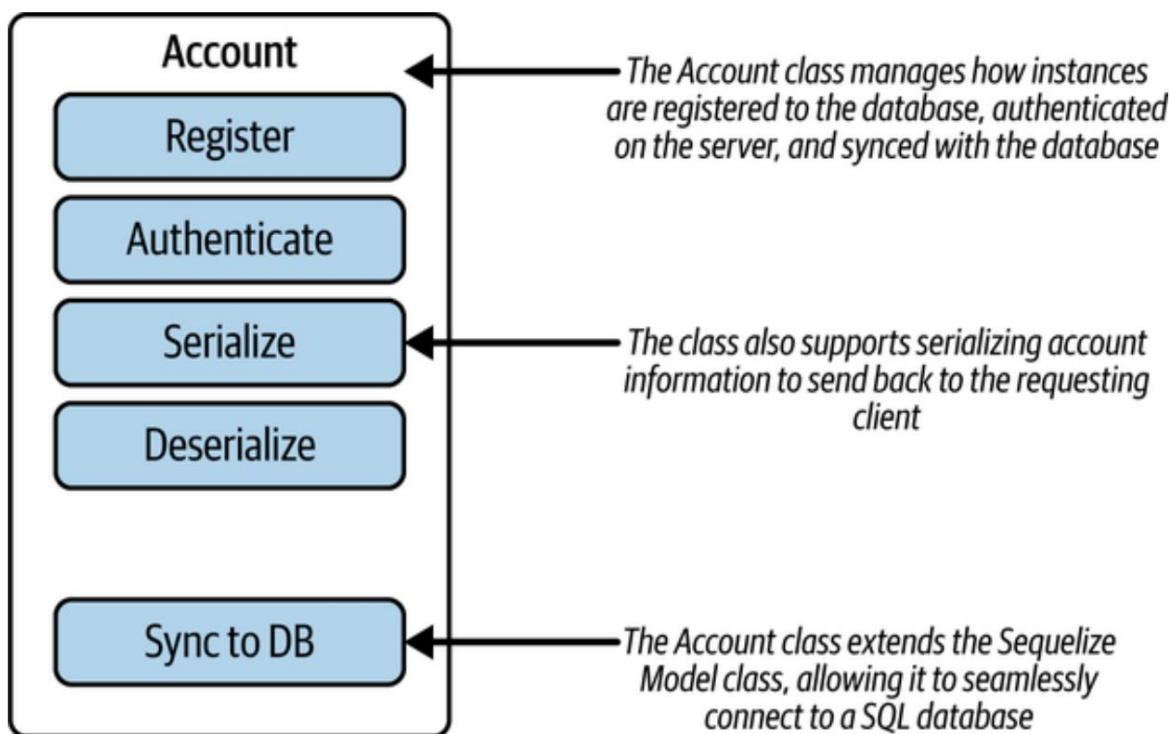
Este libro utiliza el complemento `@fastify/passport` para integrarlo en tu aplicación Fastify. Ofrece un sistema de autenticación directo similar a Express, pero adaptado al ciclo de vida de Fastify.

La Figura 10-5 muestra algunas de las funciones que gestionará la clase Cuenta . Además de registrar y autenticar, también incorporará la serialización y deserialización de datos de cuentas. La clase extenderá una clase de modelo Sequelize , lo que le permitirá definir campos persistentes y ejecutar funciones de consulta en una base de datos SQL.

NOTA

La serialización es el proceso de tomar un objeto de datos y convertirlo a un formato compatible con un objeto de respuesta. Esto suele implicar convertir un objeto complejo en una cadena grande. La deserialización es el proceso inverso: una cadena grande o codificada se transforma en un objeto JavaScript.

Para comenzar, instale los paquetes `sequelize`, `sqlite3`, `passport` y `passport-local` . Vaya a la raíz de la carpeta de su proyecto en la línea de comandos y ejecute `npm i sequelize@^6.37.7 sqlite3@^5.1.7 passport@^0.7.0 passport-local@^1.0.0`. El paquete `passport-local` funciona con el paquete `passport` para ayudarle a definir una estrategia de autenticación personalizada (en lugar de usar un servicio externo como Facebook o Google).



Cifra 105. Descripción general de la clase de modelo Cuenta

Para comenzar, instale sequelize, sqlite3, passport y

Paquetes de Passport-Local . Navegue hasta el nivel raíz de su...

carpeta del proyecto en su línea de comandos y ejecute `npm install`

`sequelize@^6.37.7 sqlite3@^5.1.7 passport@^0.7.0`

`passport-local@^1.0.0`. El paquete `passport-local` funciona

Con el paquete de `passport` para ayudarle a definir una costumbre estrategia de autenticación (a diferencia de la autenticación con un servicio de terceros como Facebook o Google).

Después de instalar estos paquetes, crea un nuevo archivo, `dbjs`, en el

Nivel raíz de su proyecto para almacenar las configuraciones de su base de datos. Agregar

El código del **Ejemplo 10-9** se transfiere a `dbjs`. Este archivo importa el Sequelize.

biblioteca e instancia un nuevo objeto de base de datos, `db`, para una SQLite

base de datos almacenada dentro de una carpeta `db` en el nivel raíz de su proyecto.

Luego, ejecuta `db.authenticate` para conectarte a la base de datos. En el

Al final del archivo se exporta la instancia de base de datos y se Sequelize

Módulo para utilizar en otras partes de la aplicación.

Ejemplo 109. Configurando sus configuraciones de SQLite en db.js

```

de importación { Sequelize } desde "sequelize"; ❶

constante db = nueva Sequelize({ ❷
  dialecto: "sqlite",
  almacenamiento: "./db/database.sqlite",
});

intentar {
  esperar db.authenticate(); ❸
  console.log(" Se ha establecido la conexión
exitosamente.");
} captura (error) {
  console.error("No se puede conectar a la base de datos:",
error);
}

exportar predeterminado { ❹
  Secuela,
  base de datos,
};

```

- ❶ Importe Sequelize para crear una nueva instancia de base de datos.
- ❷ Define db para conectarse a una base de datos SQLite almacenada en un archivo llamado base de datossqlite.
- ❸ Conéctese a la base de datos con db.authenticate.
- ❹ Exportar la base de datos y el módulo Sequelize .

Después de configurar la base de datos, puede trabajar en el modelo que Permanecerá en la base de datos. Comience creando una carpeta llamada modelos. y agregando un archivo llamado Accountjs (Ejemplo 10-10). Este archivo importa Las configuraciones de la base de datos donde puedes acceder a todas las funciones y clases dentro de Sequelize. Configurar una clase de JavaScript para Cuenta que extiende Sequelize.Model. También puede definir cualquier Otras funciones que desee dentro de esta clase. Agregue una función de clase a

Busque una cuenta por su nombre de usuario, `findByUsername`. Esto
 La función modifica el parámetro de función a minúsculas y utiliza el
 Selecciona la función `findOne` para buscar el retorno de un único
 cuenta con ese nombre de usuario. La palabra clave estática de la función significa
 que es una función de nivel de clase (no un método de instancia). Más adelante, puede
 Se puede acceder llamando a `Account.findByUsername` y pasando un
 cadena de nombre de usuario .

Ejemplo de creación de su clase Cuenta en Account.js

```
importar dbConfig desde "../db.js";           ❶
const { Sequelize, db } = dbConfig;           ❷

class Cuenta extiende Sequelize.Model {        ❸
  estática asíncrona findByUsername(nombre de usuario) { ❹
    const lowerCaseUsername = nombredeusuario.toLowerCase();
    devolver esperar esto.findOne({ donde: { nombre de usuario:
nombreDeUsuarioenminúsculas } }); ❺
  }
}
```

- ❶ Importar las configuraciones de la base de datos desde `db.js`.
- ❷ Desestructura `dbConfig` para acceder a `Sequelize` y a tu base de datos instancia.
- ❸ Define tu cuenta, que extiende `Sequelize.Model`, dando
Las funciones de consulta de la clase `Sequelize` .
- ❹ Agregue una función de clase para buscar cuentas por nombre de usuario.
- ❺ Ejecute una consulta `findOne` de `Sequelize` para encontrar un solo registro de cuenta en la base de datos.

Esta clase está lista para la integración completa con `Sequelize`. Para lograrlo

Para esto, agregarás el código del **Ejemplo 10-11** al final de `Account.js`. En

Este código se ejecuta `Account.init` para inicializar los campos para el

Cree una cuenta y registre el modelo en su base de datos. Aquí...

Defina el campo de nombre de usuario como una cadena que debe tener un valor válido y no estar duplicado en la base de datos. A continuación, agregue los campos para un hash y una sal, ambos como cadenas que no deben ser nulas.

Contraseñas hash

Almacenar contraseñas en texto plano es una forma segura de que hackeen cuentas y roben información confidencial. Por ello, es un estándar en la industria no almacenar nunca una contraseña real en la base de datos ni en el servidor. Si bien una contraseña en texto plano debe pasar por el servidor con la solicitud entrante, al persistir o verificar la información de la cuenta, es mejor usar otras técnicas.

Un enfoque moderno para almacenar información de contraseñas consiste en aplicar un hash a la contraseña y almacenar el valor hash y la cadena de sal . El hash se genera ejecutando una función hash sobre la contraseña original en texto plano. Las funciones hash utilizan un algoritmo para convertir la contraseña en una cadena ininteligible. La clave reside en crear una sal criptográfica personalizada, o una combinación aleatoria de bits para cada hash generado. La combinación de una sal y un algoritmo hash da como resultado una cadena que no se puede convertir de nuevo a la contraseña original. Entonces, ¿por qué almacenamos el hash y la sal?

La Figura 10-6 muestra cómo se pasa una contraseña entrante en texto plano a la función de hash, junto con la sal personalizada de la cuenta. El resultado final es un hash de contraseña. La autenticación de cuenta verifica la información del nombre de usuario y la contraseña comprobando que la contraseña del cuerpo de la solicitud, al combinarse con la sal de la cuenta del nombre de usuario, genere un hash que coincida con el valor hash de la cuenta . Si ambos hashes coinciden, la cuenta está verificada.

De esta forma, la aplicación nunca conoce con exactitud las contraseñas de sus usuarios. En caso de piratería de una base de datos, los hackers solo obtendrían una serie de valores hash y sal , sin poder recuperar las contraseñas originales en texto plano.